

CSE 213: Software Engineering

Behavioral Design Patterns

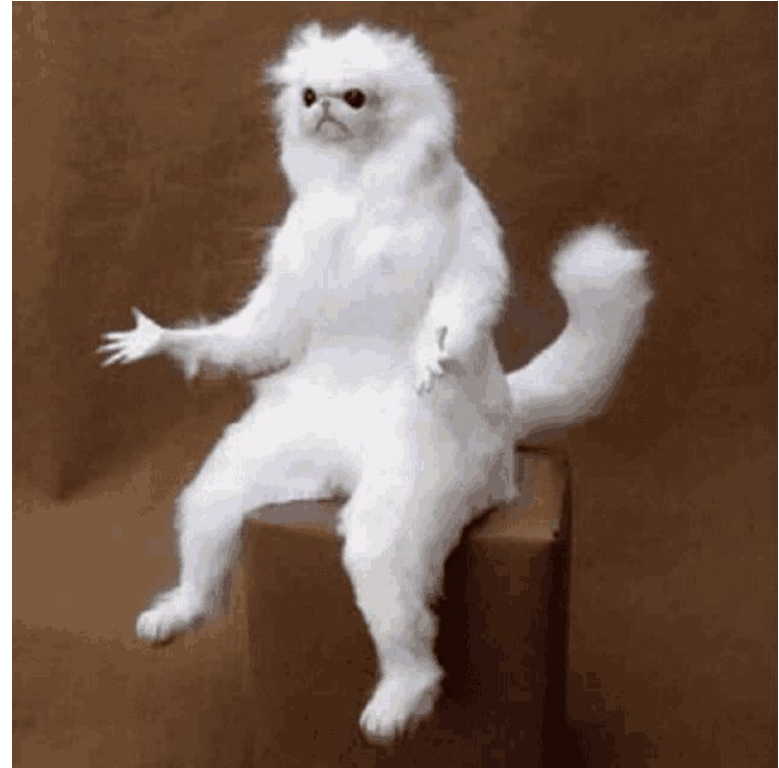
Nafis Tahmid
Lecturer, CSE, BUET

What are behavioral design patterns?

- ❑ Behavioral patterns deal with how objects **behave and interact**; defining *communication*, *responsibility flow* and how tasks are carried out between objects etc.
- ❑ Concerned with **algorithms** and *assignment of responsibilities* between objects.

What are behavioral design patterns?

- ❑ Behavioral patterns deal with how objects **behave and interact**; defining *communication*, *responsibility flow* and how tasks are carried out between objects etc.
- ❑ Concerned with **algorithms** and *assignment of responsibilities* between objects.



Simple Duck Simulator

Duck species can -

- Swim
- Quack

They quack & swim the same way!

But look different!!

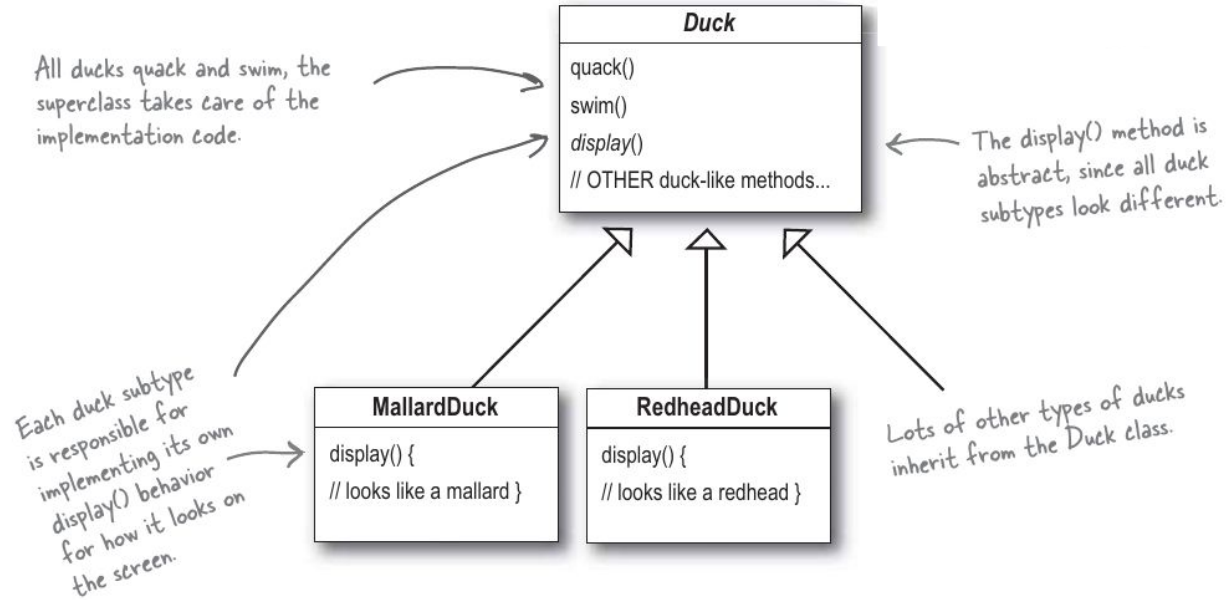


Simple Duck Simulator

Duck species can -

- Swim
- Quack

They quack & swim the same way!
But look different!!



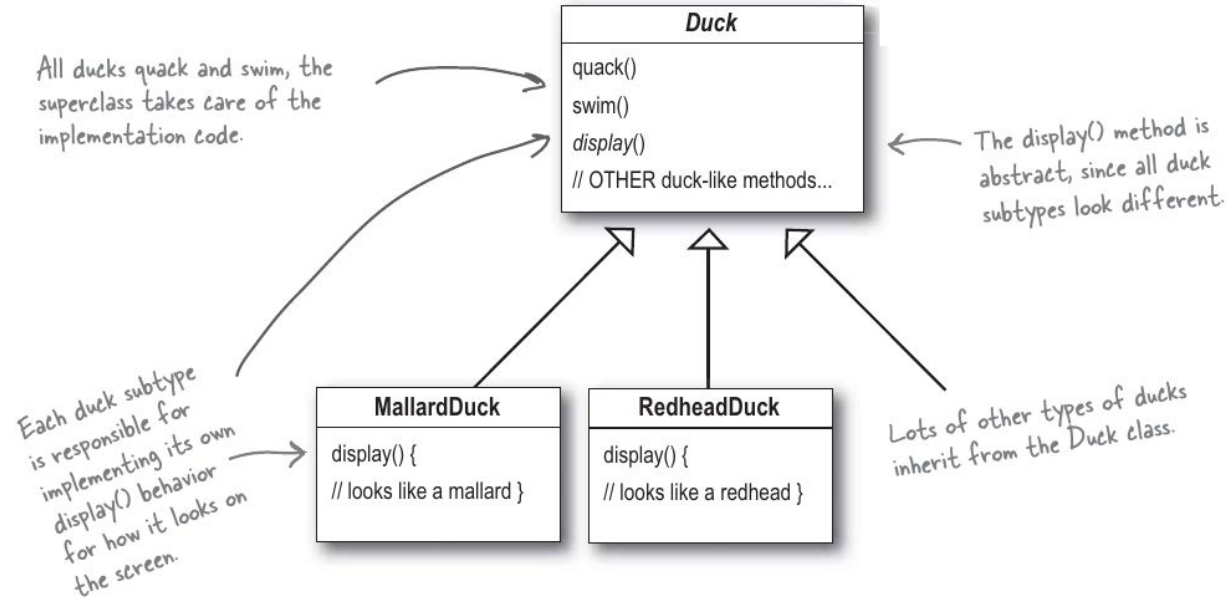
Simple Duck Simulator

Duck species can -

- Swim
- Quack

They quack & swim the same way!
But look different!!

Now we need to make the ducks fly!



Simple Duck Simulator

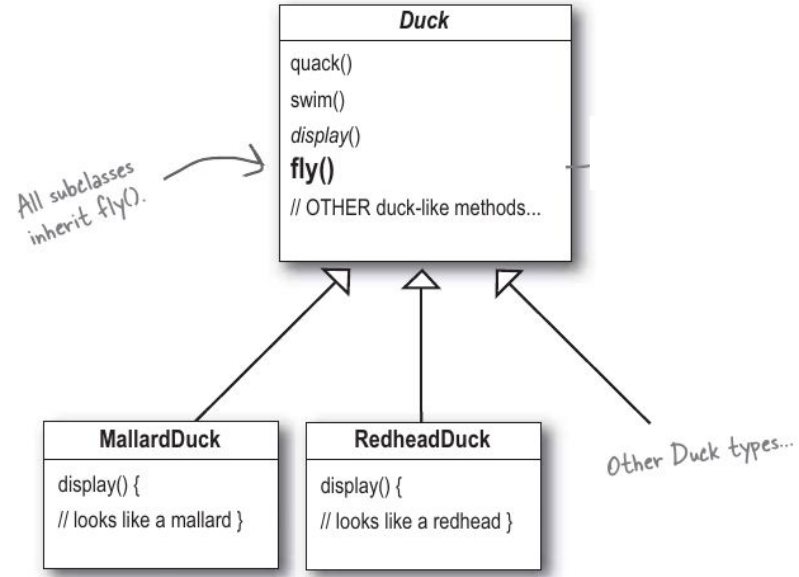
Duck species can -

- Swim
- Quack

They quack & swim the same way!

But look different!!

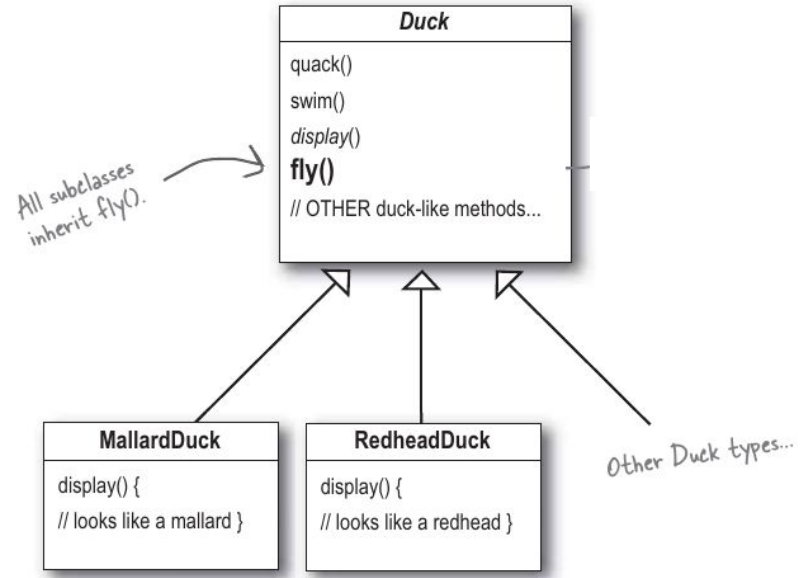
Now we need to make the ducks fly!



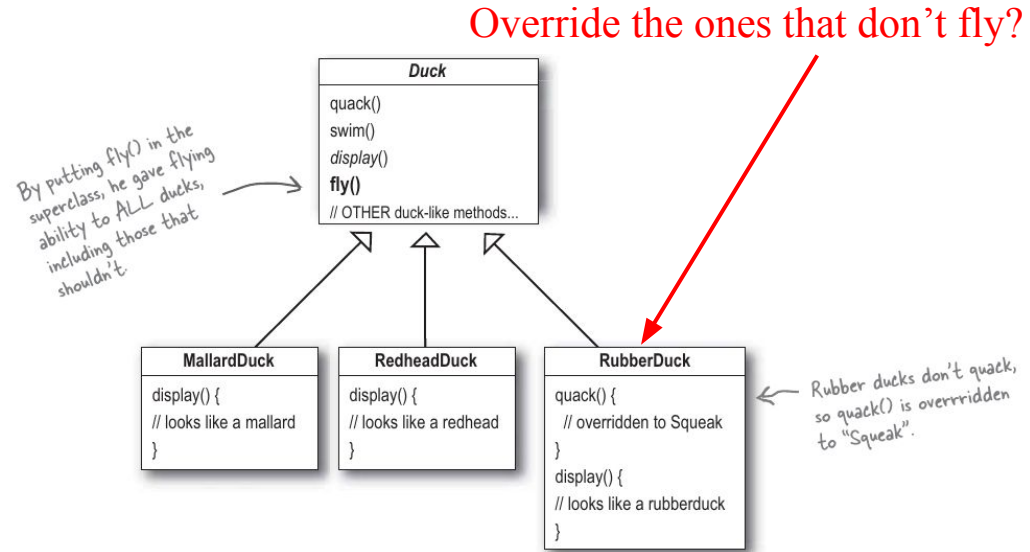
Simple Duck Simulator



At the project show, **RUBBER DUCK** starts flying!



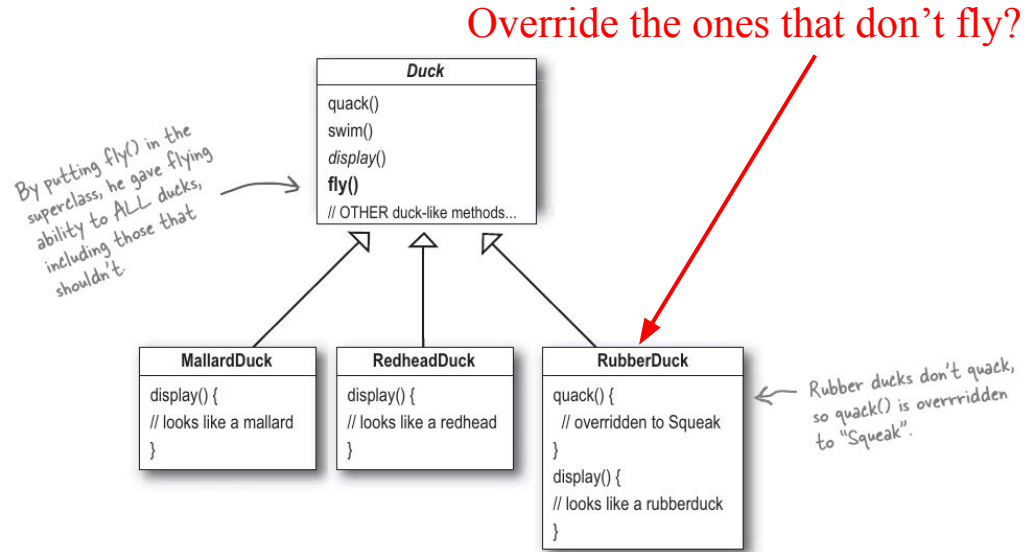
Solution?



Solution?



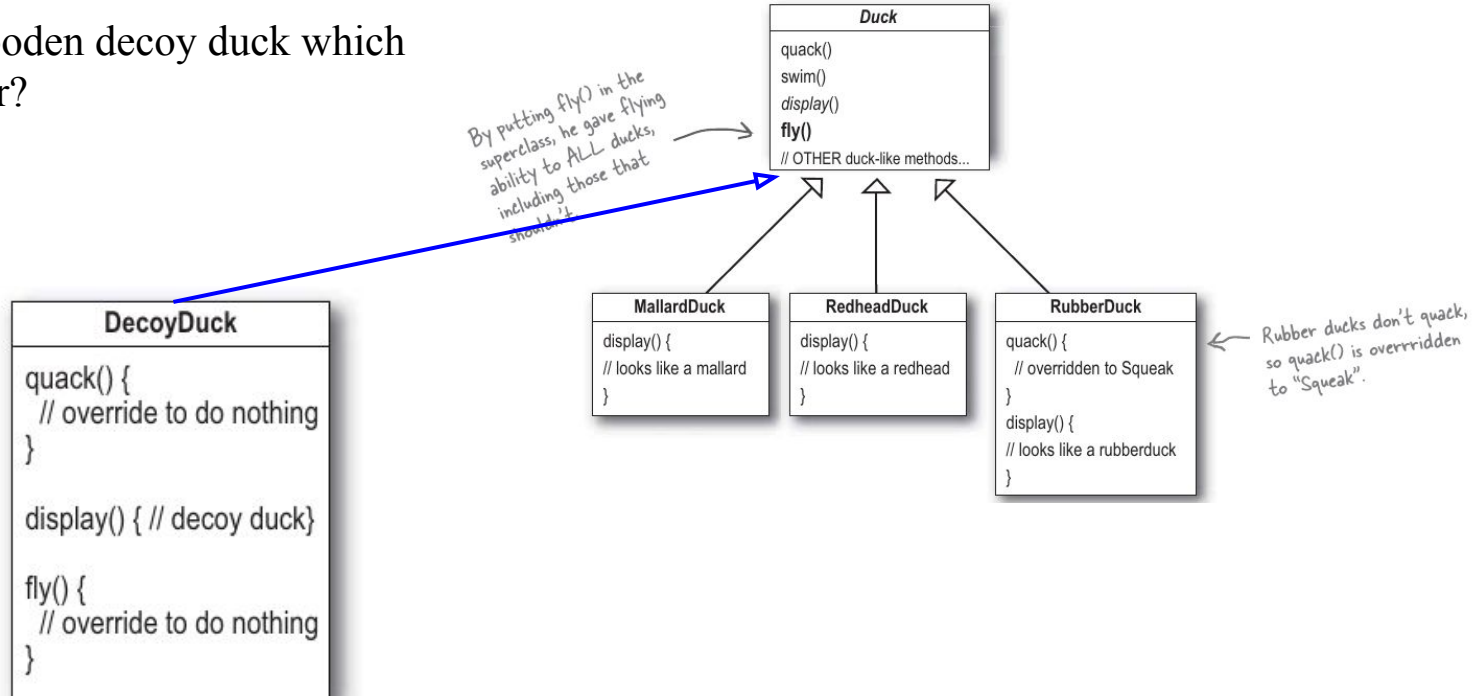
What if we add a wooden decoy duck which **doesn't squeak** either?



Solution?



What if we add a wooden decoy duck which **doesn't squeak** either?



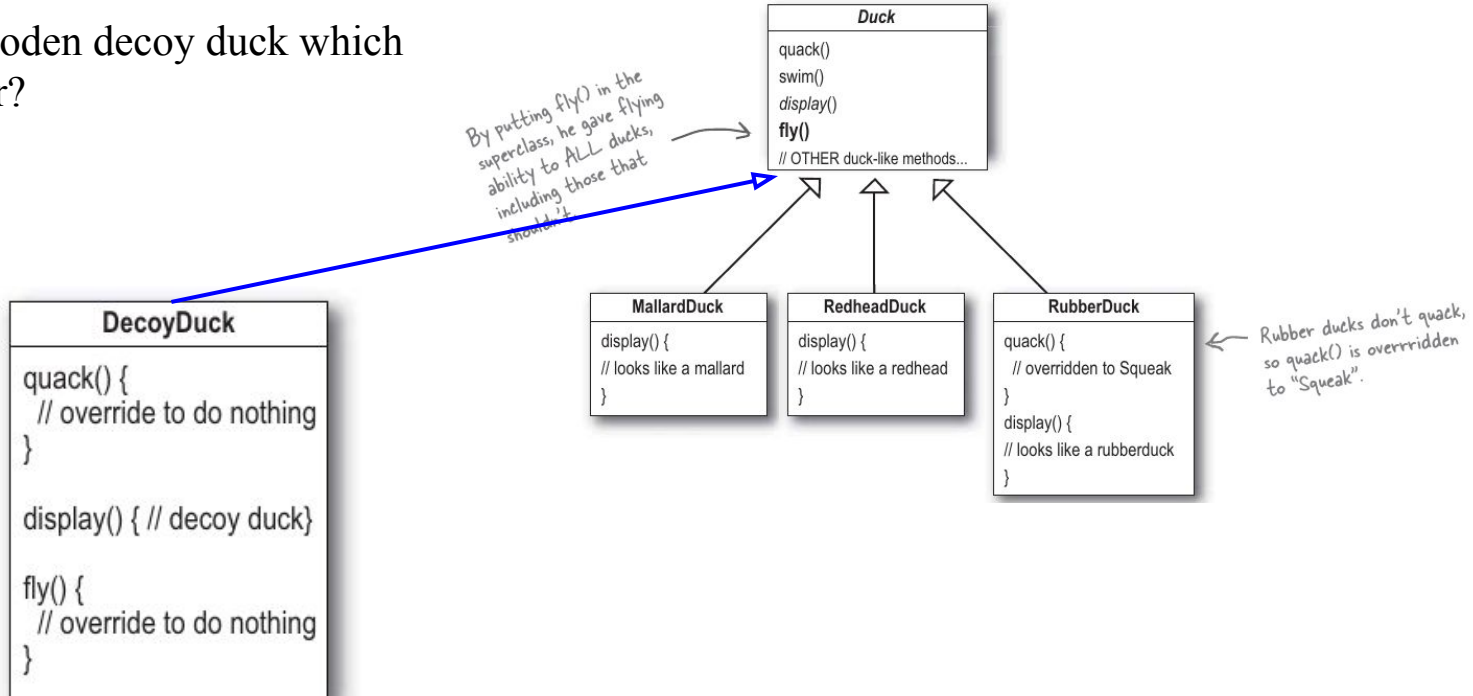
Solution?



What if we add a wooden decoy duck which **doesn't squeak** either?



Bad design



Another Approach?

Now we know that new features will be **added/updated** every **six** months

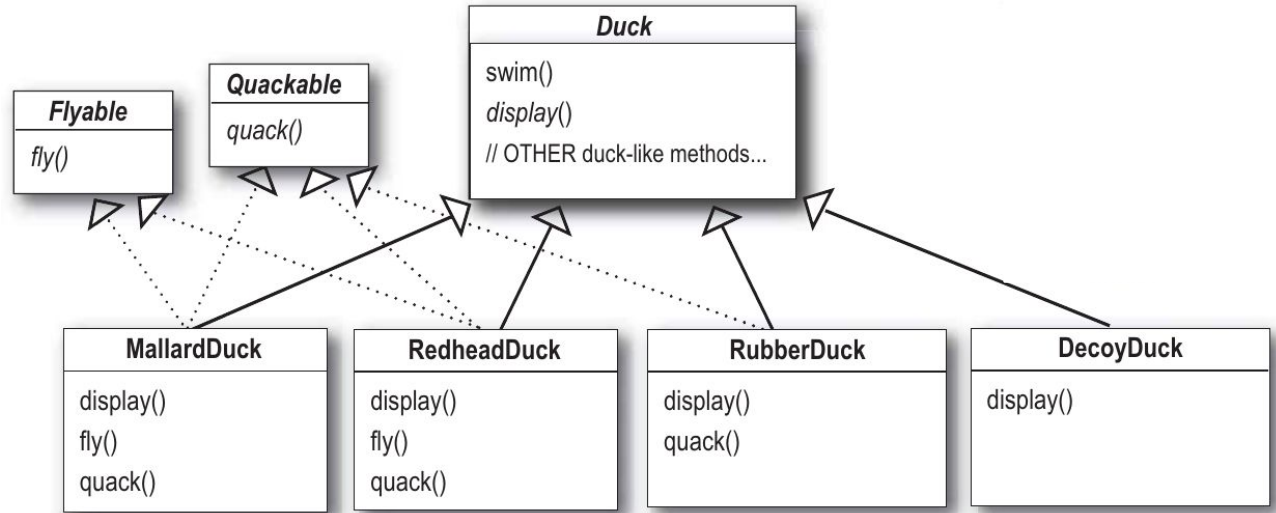
Obviously, adding new ducks and **overriding** *fly()*, *quack()* for each is **not** an option

What to do...

Another Approach - Using Interfaces?

Flyable and **Quackable** are **interfaces**

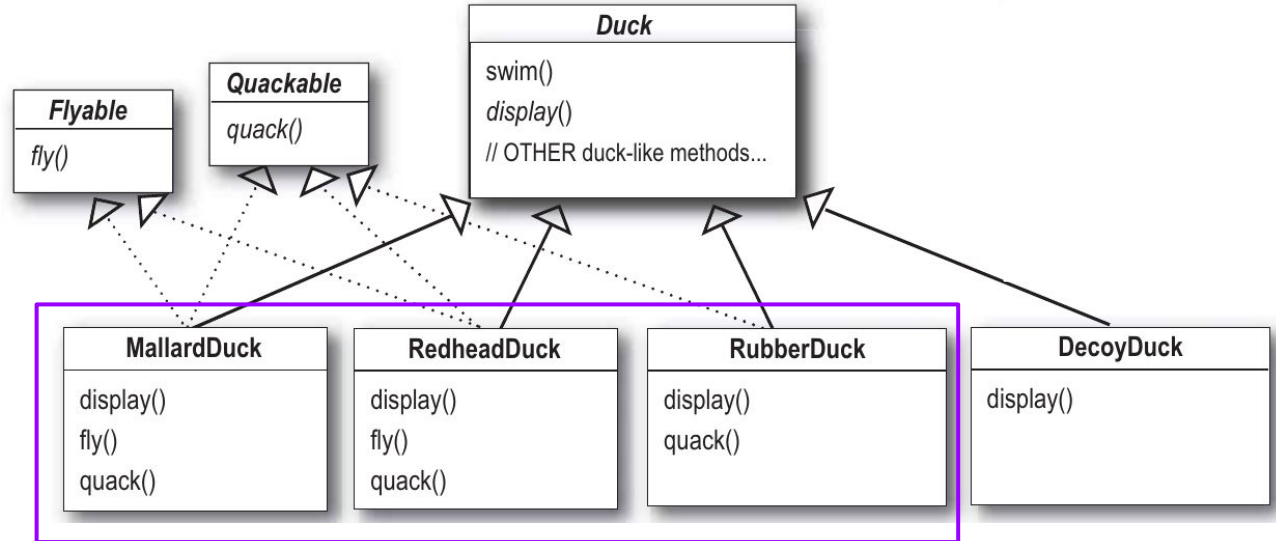
Those who **fly and/or quack**, *implement* those.



Another Approach - Using Interfaces?

Flyable and **Quackable** are **interfaces**

Those who **fly and/or quack**, *implement* those.

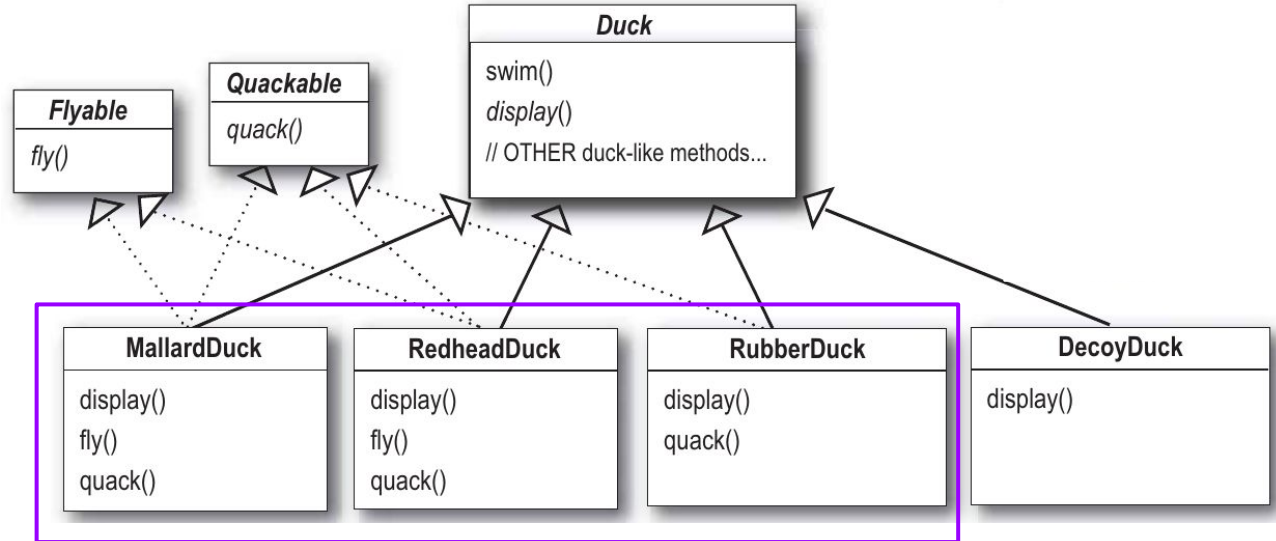


Another Approach - Using Interfaces?

Flyable and **Quackable** are **interfaces**

Those who **fly and/or quack**, *implement* those.

Is it **good** enough?



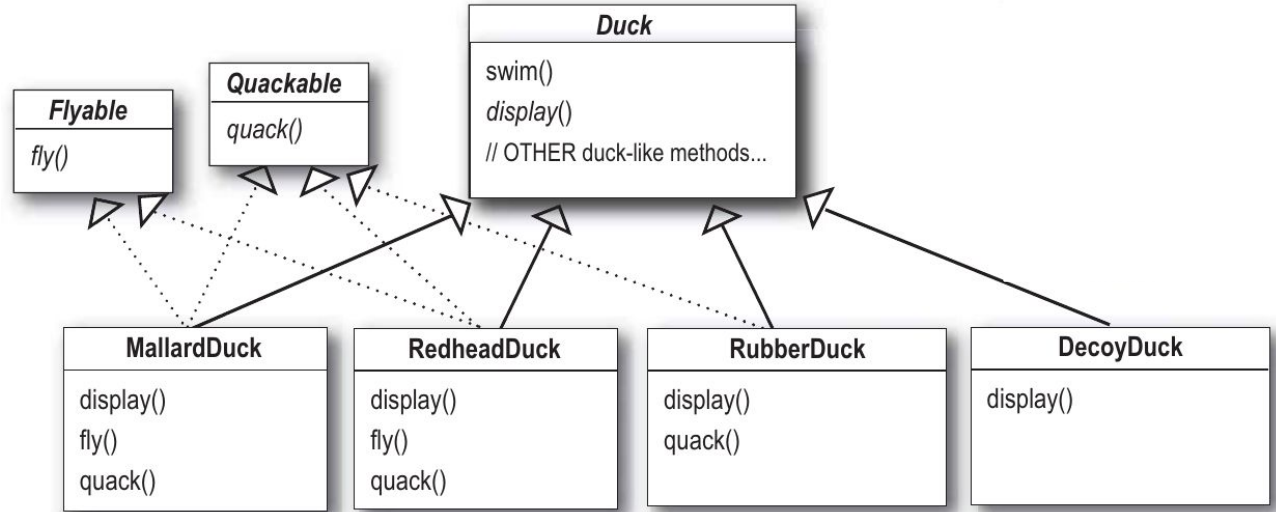
Another Approach - Using Interfaces?

Flyable and **Quackable** are **interfaces**

Those who **fly and/or quack**, *implement* those.

What if we have to **change fly** behavior slightly?

Say wingbeat rate, angle etc.



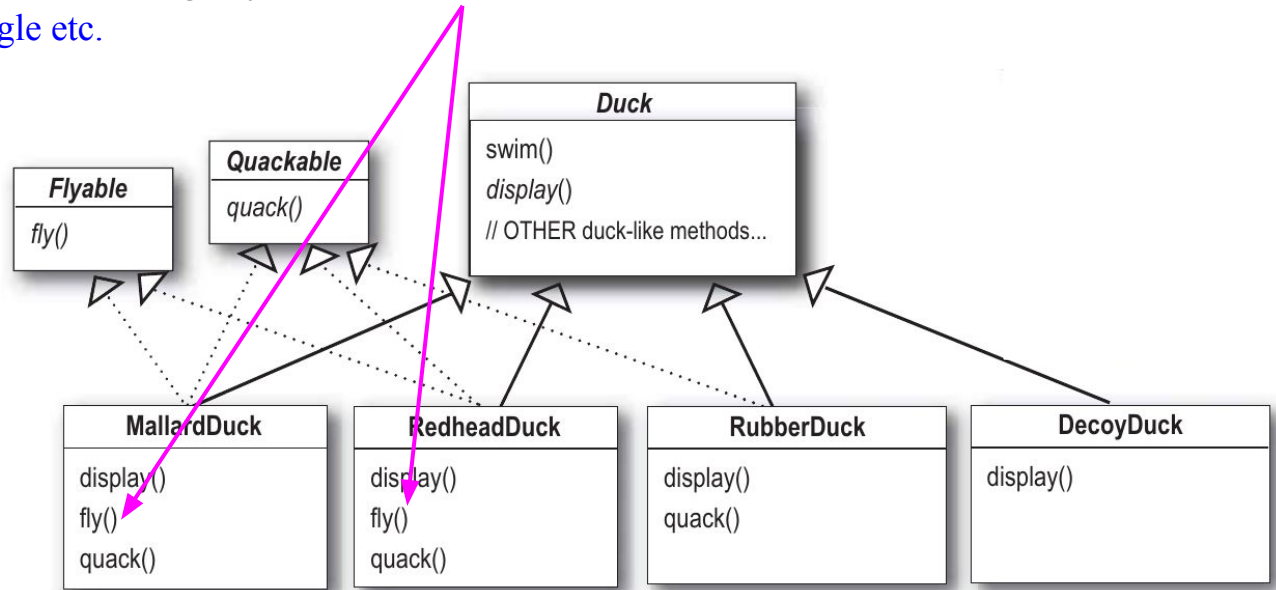
Another Approach - Using Interfaces?

Flyable and **Quackable** are **interfaces**

Those who **fly** and/or **quack**, *implement* those.

What if we have to **change fly** behavior slightly? **CHANGE ALL IMPLEMENTATIONS!!!**

Say wingbeat rate, angle etc.



Another Principle



Design Principle

Identify the aspects of your application that vary and separate them from what stays the same.

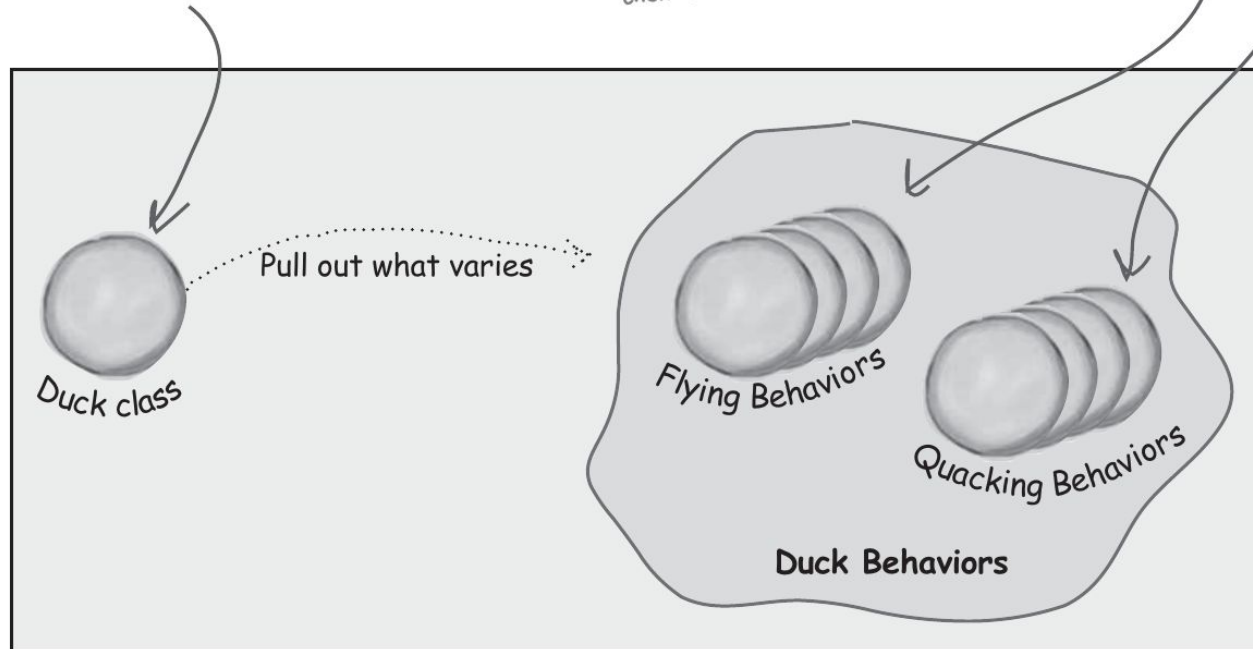
**Take what varies and
“encapsulate” it so it won’t
affect the rest of your code.**

Let's follow the principle

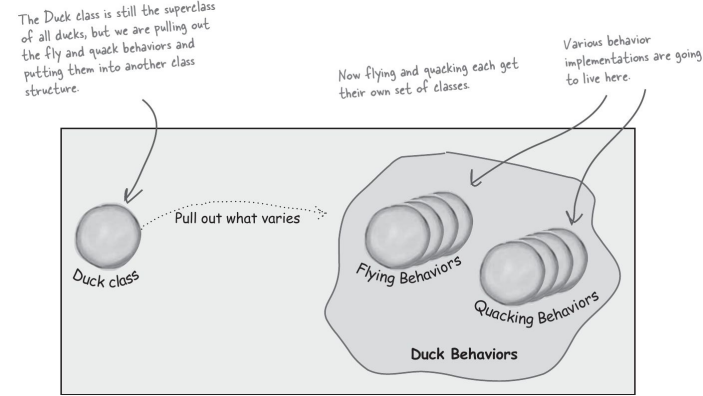
The Duck class is still the superclass of all ducks, but we are pulling out the fly and quack behaviors and putting them into another class structure.

Now flying and quacking each get their own set of classes.

Various behavior implementations are going to live here.



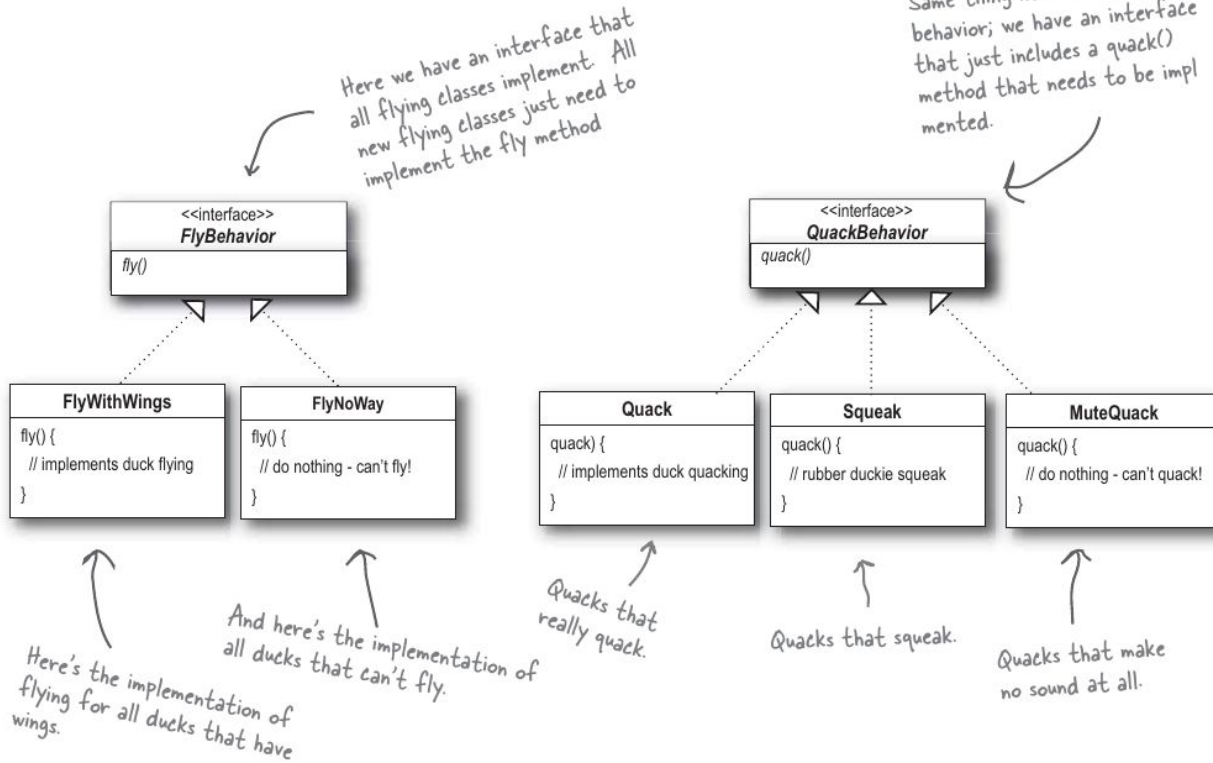
Let's follow the principle



Design Principle

Program to an interface, not an implementation.

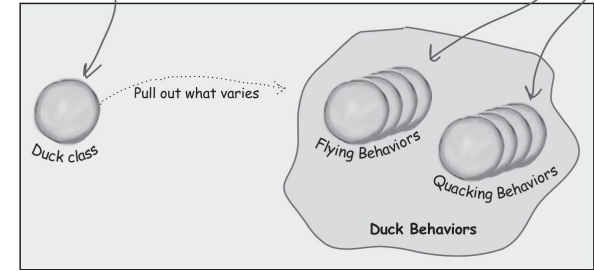
Let's follow the principle



The Duck class is still the superclass of all ducks, but we are pulling out the fly and quack behaviors and putting them into another class structure.

Now flying and quacking each get their own set of classes.

Various behavior implementations are going to live here.



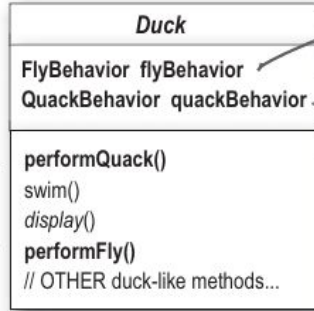
Design Principle

Program to an interface, not an implementation.

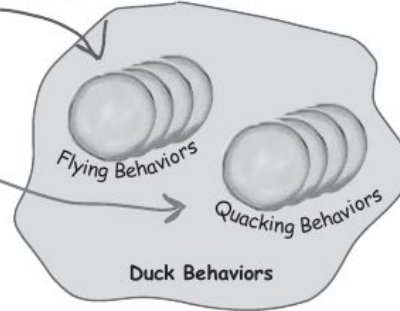
Let's follow the principle

The behavior variables are declared as the behavior INTERFACE type.

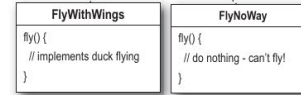
These methods replace fly() and quack().



Instance variables hold a reference to a specific behavior at runtime.



Here we have an interface that all flying classes implement. All new flying classes just need to implement the fly method

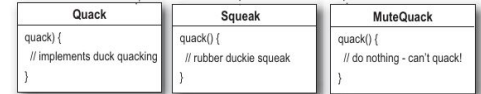


Here's the implementation of flying for all ducks that have wings.

And here's the implementation of all ducks that can't fly.



Same thing here for the quack behavior; we have an interface that just includes a quack() method that needs to be implemented.



Quacks that really quack.

Quacks that squeak.

Quacks that make no sound at all.

Let's see what we have done

```
public abstract class Duck {
```

```
    FlyBehavior flyBehavior;  
    QuackBehavior quackBehavior;  
    public Duck() {  
    }
```

Declare two reference variables
for the behavior interface types.
All duck subclasses (in the same
package) inherit these.

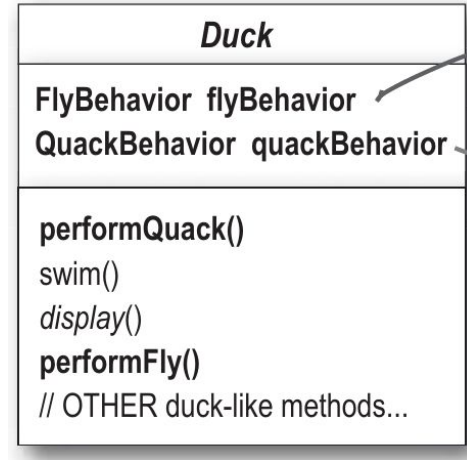
```
    public abstract void display();
```

```
    public void performFly() {  
        flyBehavior.fly();
```

Delegate to the behavior class.

```
    public void performQuack() {  
        quackBehavior.quack();  
    }
```

```
    public void swim() {  
        System.out.println("All ducks float, even decoys!");  
    }
```



Let's see what we have done

```
public abstract class Duck {  
  
    FlyBehavior flyBehavior;  
    QuackBehavior quackBehavior;  
    public Duck() {  
    }  
  
    public abstract void display();  
  
    public void performFly() {  
        flyBehavior.fly();  
    }  
  
    public void performQuack() {  
        quackBehavior.quack();  
    }  
  
    public void swim() {  
        System.out.println("All ducks float, even decoys!");  
    }  
}
```

Declare two reference variables for the behavior interface types. All duck subclasses (in the same package) inherit these.

Delegate to the behavior class.

```
public interface FlyBehavior {  
    public void fly();  
}
```

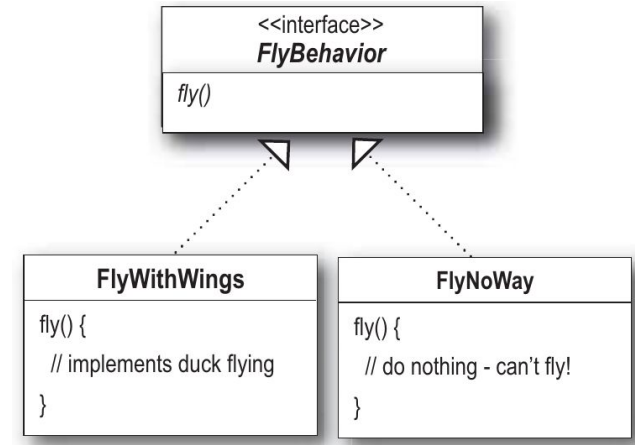
The interface that all flying behavior classes implement.

```
public class FlyWithWings implements FlyBehavior {  
    public void fly() {  
        System.out.println("I'm flying!!");  
    }  
}
```

Flying behavior implementation for ducks that DO fly...

```
public class FlyNoWay implements FlyBehavior {  
    public void fly() {  
        System.out.println("I can't fly");  
    }  
}
```

Flying behavior implementation for ducks that do NOT fly (like rubber ducks and decoy ducks).



Let's see what we have done

```
public abstract class Duck {  
    FlyBehavior flyBehavior;  
    QuackBehavior quackBehavior;  
    public Duck() {  
    }  
}
```

Declare two reference variables
for the behavior interface types.
All duck subclasses (in the same
package) inherit these.

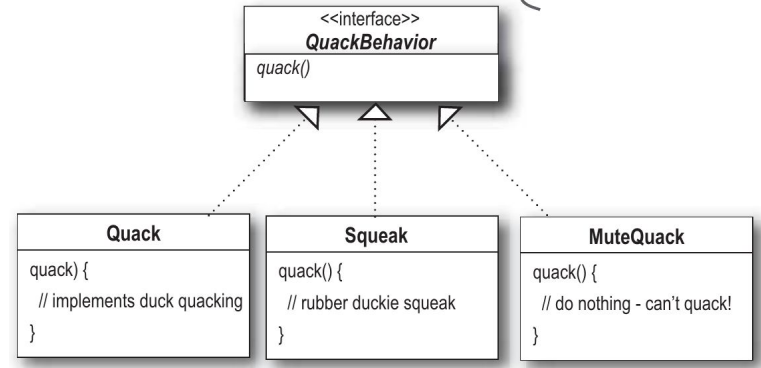
```
public abstract void display();
```

```
public void performFly() {  
    flyBehavior.fly();  
}
```

Delegate to the behavior class.

```
public void performQuack() {  
    quackBehavior.quack();  
}
```

```
public void swim() {  
    System.out.println("All ducks float, even decoys!");  
}
```



```
public interface QuackBehavior {  
    public void quack();  
}
```

```
public class Quack implements QuackBehavior {  
    public void quack() {  
        System.out.println("Quack");  
    }  
}
```

```
public class MuteQuack implements QuackBehavior {  
    public void quack() {  
        System.out.println("<< Silence >>");  
    }  
}
```

Let's see what we have done

```
public abstract class Duck {
```

```
    FlyBehavior flyBehavior;  
    QuackBehavior quackBehavior;  
    public Duck() {  
    }
```

```
    public abstract void display();
```

```
    public void performFly() {  
        flyBehavior.fly();  
    }
```

```
    public void performQuack() {  
        quackBehavior.quack();  
    }
```

```
    public void swim() {  
        System.out.println("All ducks float, even decoys!");  
    }  
}
```

← Declare two reference variables
for the behavior interface types.
All duck subclasses (in the same
package) inherit these.

← Delegate to the behavior class.

```
public interface FlyBehavior {  
    public void fly();  
}
```

The interface that all flying
behavior classes implement

```
public class FlyWithWings implements FlyBehavior {  
    public void fly() {  
        System.out.println("I'm flying!!");  
    }  
}
```

Flying behavior implementation
for ducks that DO fly...

```
public class FlyNoWay implements FlyBehavior {  
    public void fly() {  
        System.out.println("I can't fly");  
    }  
}
```

Flying behavior implementation
for ducks that do NOT fly (like
rubber ducks and decoy ducks).

```
public interface QuackBehavior {  
    public void quack();  
}
```

```
public class Quack implements QuackBehavior {  
    public void quack() {  
        System.out.println("Quack");  
    }  
}
```

```
public class MuteQuack implements QuackBehavior {  
    public void quack() {  
        System.out.println("<< Silence >>");  
    }  
}
```

Let's see what we have done

```
public class MallardDuck extends Duck {  
  
    public MallardDuck() {  
        quackBehavior = new Quack();  
        flyBehavior = new FlyWithWings();  
    }  
  
    public void display() {  
        System.out.println("I'm a real Mallard duck");  
    }  
}
```

Remember, MallardDuck inherits the quackBehavior and flyBehavior instance variables from class Duck.

A MallardDuck uses the Quack class to handle its quack, so when performQuack is called, the responsibility for the quack is delegated to the Quack object and we get a real quack.

And it uses FlyWithWings as its FlyBehavior type.

```
public abstract class Duck {
```

```
    FlyBehavior flyBehavior;  
    QuackBehavior quackBehavior;  
    public Duck() {  
    }
```

Declare two reference variables for the behavior interface types. All duck subclasses (in the same package) inherit these.

```
    public abstract void display();
```

```
    public void performFly() {  
        flyBehavior.fly();  
    }
```

Delegate to the behavior class.

```
    public void performQuack() {  
        quackBehavior.quack();  
    }
```

```
    public void swim() {  
        System.out.println("All ducks float, even decoys!");  
    }  
}
```

Let's see what we have done

```
public class MallardDuck extends Duck {  
  
    public MallardDuck() {  
        quackBehavior = new Quack();  
        flyBehavior = new FlyWithWings();  
    }  
  
    public void display() {  
        System.out.println("I'm a real Mallard duck");  
    }  
}
```

Remember, MallardDuck inherits the quackBehavior and flyBehavior instance variables from class Duck.

A MallardDuck uses the Quack class to handle its quack, so when performQuack is called, the responsibility for the quack is delegated to the Quack object and we get a real quack.

And it uses FlyWithWings as its FlyBehavior type.

```
public abstract class Duck {  
    FlyBehavior flyBehavior;  
    QuackBehavior quackBehavior;  
    public Duck() {  
    }  
  
    public abstract void display();  
  
    public void performFly() {  
        flyBehavior.fly();  
    }  
  
    public void performQuack() {  
        quackBehavior.quack();  
    }  
  
    public void swim() {  
        System.out.println("All ducks float, even decoys!");  
    }  
}
```

Declare two reference variables for the behavior interface types. All duck subclasses (in the same package) inherit these.

Delegate to the behavior class.

Let's see what we have done

```
public class MallardDuck extends Duck {  
  
    public MallardDuck() {  
        quackBehavior = new Quack();  
        flyBehavior = new FlyWithWings();  
    }  
  
    public void display() {  
        System.out.println("I'm a real Mallard duck");  
    }  
}
```

Remember, `MallardDuck` inherits the `quackBehavior` and `flyBehavior` instance variables from class `Duck`.

A `MallardDuck` uses the `Quack` class to handle its quack, so when `performQuack` is called, the responsibility for the quack is delegated to the `Quack` object and we get a real quack.

And it uses `FlyWithWings` as its `FlyBehavior` type.

```
public void display() {  
    System.out.println("I'm a real Mallard duck");  
}  
}  
  
public class MiniDuckSimulator {  
    public static void main(String[] args) {  
        Duck mallard = new MallardDuck();  
        mallard.performQuack();  
        mallard.performFly();  
    }  
}
```

This calls the `MallardDuck`'s inherited `performQuack()` method, which then delegates to the object's `QuackBehavior` (i.e. calls `quack()` on the duck's inherited `quackBehavior` reference).

Then we do the same thing with `MallardDuck`'s inherited `performFly()` method.

Let's see what we have done

```
public class MallardDuck extends Duck {  
  
    public MallardDuck() {  
        quackBehavior = new Quack();  
        flyBehavior = new FlyWithWings();  
    }  
  
    public void display() {  
        System.out.println("I'm a real Mallard duck");  
    }  
}
```

Remember, MallardDuck inherits the quackBehavior and flyBehavior instance variables from class Duck.

A MallardDuck uses the Quack class to handle its quack, so when performQuack is called, the responsibility for the quack is delegated to the Quack object and we get a real quack.

And it uses FlyWithWings as its FlyBehavior type.

```
public class MiniDuckSimulator {  
    public static void main(String[] args) {  
        Duck mallard = new MallardDuck();  
        mallard.performQuack();  
        mallard.performFly();  
    }  
}
```

This calls the MallardDuck's inherited performQuack() method, which then delegates to the object's QuackBehavior (i.e. calls quack() on the duck's inherited quackBehavior reference).

Then we do the same thing with MallardDuck's inherited performFly() method.

```
%java MiniDuckSimulator  
Quack  
I'm flying!!
```

Let's make behavior "DYNAMIC"

```
public void setFlyBehavior(FlyBehavior fb) {  
    flyBehavior = fb;  
}  
  
public void setQuackBehavior(QuackBehavior qb) {  
    quackBehavior = qb;  
}
```

Duck
FlyBehavior flyBehavior; QuackBehavior quackBehavior;
swim() display() performQuack() performFly() setFlyBehavior() setQuackBehavior() // OTHER duck-like methods...

We can call these methods anytime we want to change the behavior of a duck on the fly.

gratuitous pun

Let's make behavior "DYNAMIC"

```
public void setFlyBehavior(FlyBehavior fb) {  
    flyBehavior = fb;  
}  
  
public void setQuackBehavior(QuackBehavior qb) {  
    quackBehavior = qb;  
}
```

Duck
FlyBehavior flyBehavior; QuackBehavior quackBehavior;
swim() display() performQuack() performFly() setFlyBehavior() setQuackBehavior() // OTHER duck-like methods...

We can call these methods anytime we want to change the behavior of a duck on the fly.

gratuitous pun

new duck

```
public class ModelDuck extends Duck {  
    public ModelDuck() {  
        flyBehavior = new FlyNoWay();  
        quackBehavior = new Quack();  
    }  
  
    public void display() {  
        System.out.println("I'm a model duck");  
    }  
}
```

can't fly

Let's make behavior "DYNAMIC"

```
public void setFlyBehavior(FlyBehavior fb) {  
    flyBehavior = fb;  
}  
  
public void setQuackBehavior(QuackBehavior qb) {  
    quackBehavior = qb;  
}
```

Duck
FlyBehavior flyBehavior; QuackBehavior quackBehavior;
swim() display() performQuack() performFly() setFlyBehavior() setQuackBehavior() // OTHER duck-like methods...

We can call these methods anytime we want to change the behavior of a duck on the fly.

gratuitous pun

```
public class ModelDuck extends Duck {  
    public ModelDuck() {  
        flyBehavior = new FlyNoWay();  
        quackBehavior = new Quack();  
    }  
  
    public void display() {  
        System.out.println("I'm a model duck");  
    }  
}
```

```
public class FlyRocketPowered implements FlyBehavior {  
    public void fly() {  
        System.out.println("I'm flying with a rocket!");  
    }  
}
```



Let's make behavior "DYNAMIC"

```
Duck mallard = new MallardDuck();  
mallard.performQuack();  
mallard.performFly();
```

```
Duck model = new ModelDuck()  
model.performFly();  
model.setFlyBehavior(new FlyRocketPowered());  
model.performFly();
```

```
public class ModelDuck extends Duck {  
    public ModelDuck() {  
        flyBehavior = new FlyNoWay();  
        quackBehavior = new Quack();  
    }  
  
    public void display() {  
        System.out.println("I'm a model duck");  
    }  
}
```

```
public class FlyRocketPowered implements FlyBehavior {  
    public void fly() {  
        System.out.println("I'm flying with a rocket!");  
    }  
}
```



Let's make behavior "DYNAMIC"

```
Duck mallard = new MallardDuck();  
mallard.performQuack();  
mallard.performFly();
```

```
Duck model = new ModelDuck()  
model.performFly();  
model.setFlyBehavior(new FlyRocketPowered());  
model.performFly();
```

```
public class ModelDuck extends Duck {  
    public ModelDuck() {  
        flyBehavior = new FlyNoWay();  
        quackBehavior = new Quack();  
    }  
  
    public void display() {  
        System.out.println("I'm a model duck");  
    }  
}
```

Quack

I'm flying!!

I can't fly

I'm flying with a rocket



```
public class FlyRocketPowered implements FlyBehavior {  
    public void fly() {  
        System.out.println("I'm flying with a rocket!");  
    }  
}
```

Let's make behavior "DYNAMIC"

```
Duck mallard = new MallardDuck();  
mallard.performQuack();  
mallard.performFly();
```

```
Duck model = new ModelDuck()  
model.performFly();  
model.setFlyBehavior(new FlyRocketPowered());  
model.performFly();
```

```
public class ModelDuck extends Duck {  
    public ModelDuck() {  
        flyBehavior = new FlyNoWay();  
        quackBehavior = new Quack();  
    }  
  
    public void display() {  
        System.out.println("I'm a model duck");  
    }  
}
```

Quack
I'm flying!!
I can't fly
I'm flying with a rocket



```
public class FlyRocketPowered implements FlyBehavior {  
    public void fly() {  
        System.out.println("I'm flying with a rocket!");  
    }  
}
```

Let's make behavior "DYNAMIC"

```
Duck mallard = new MallardDuck();  
mallard.performQuack();  
mallard.performFly();
```

```
Duck model = new ModelDuck()  
model.performFly();  
model.setFlyBehavior(new FlyRocketPowered());  
model.performFly();
```

Quack

I'm flying!!

I can't fly

I'm flying with a rocket



```
public class ModelDuck extends Duck {  
    public ModelDuck() {  
        flyBehavior = new FlyNoWay();  
        quackBehavior = new Quack();  
    }  
  
    public void display() {  
        System.out.println("I'm a model duck");  
    }  
}
```

```
public class FlyRocketPowered implements FlyBehavior {  
    public void fly() {  
        System.out.println("I'm flying with a rocket!");  
    }  
}
```

Let's make behavior "DYNAMIC"

```
Duck mallard = new MallardDuck();  
mallard.performQuack();  
mallard.performFly();
```

```
Duck model = new ModelDuck()  
model.performFly();  
model.setFlyBehavior(new FlyRocketPowered()),  
model.performFly();
```

Quack

I'm flying!!

I can't fly

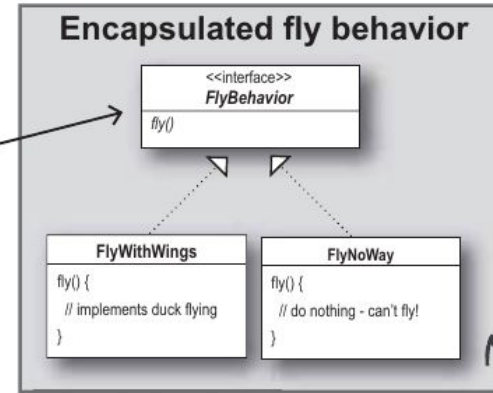
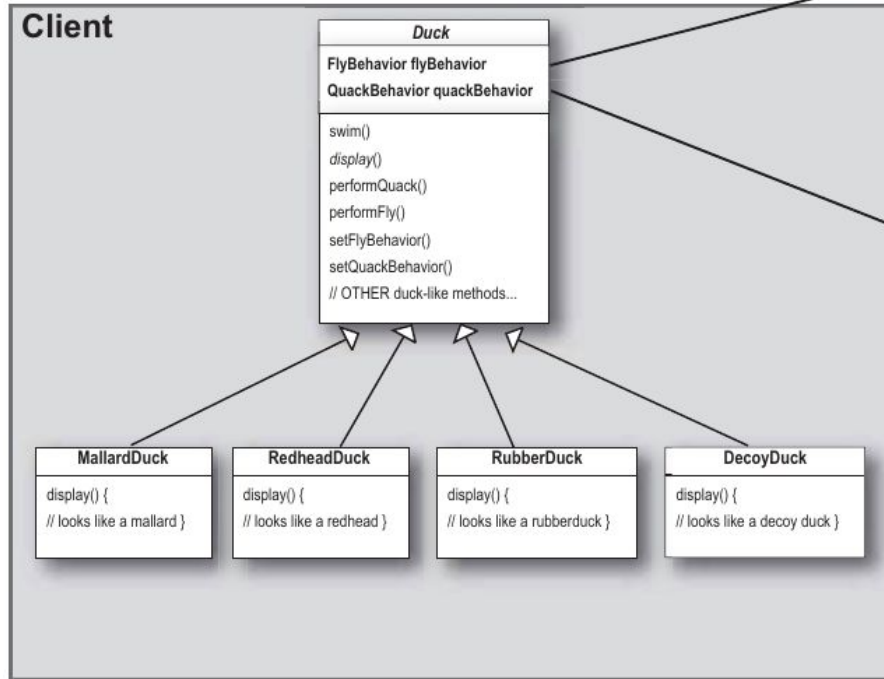
I'm flying with a rocket



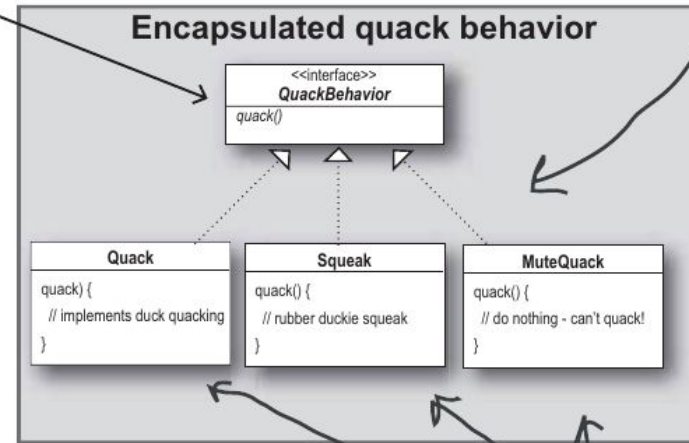
```
public class ModelDuck extends Duck {  
    public ModelDuck() {  
        flyBehavior = new FlyNoWay();  
        quackBehavior = new Quack();  
    }  
  
    public void display() {  
        System.out.println("I'm a model duck");  
    }  
}
```

```
public class FlyRocketPowered implements FlyBehavior {  
    public void fly() {  
        System.out.println("I'm flying with a rocket!");  
    }  
}
```

Client makes use of an encapsulated family of algorithms for both flying and quacking.



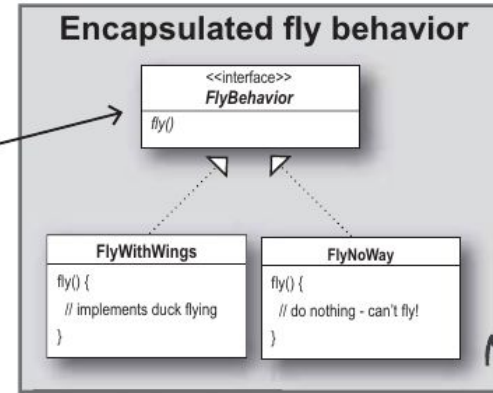
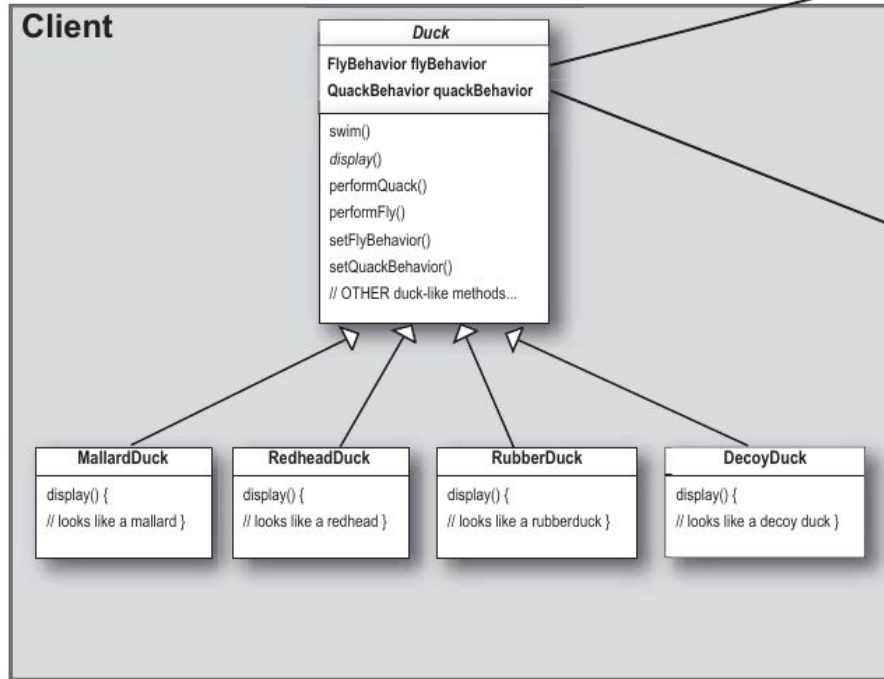
Think of each set of behaviors as a family of algorithms.



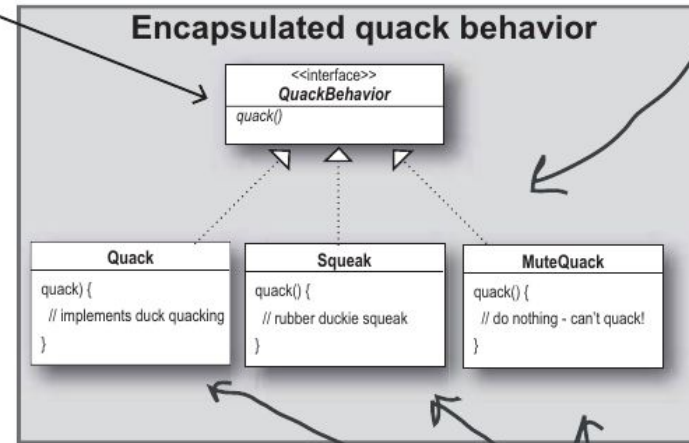
These behaviors "algorithms" are interchangeable.

Birds Eye View

Client makes use of an encapsulated family of algorithms for both flying and quacking.



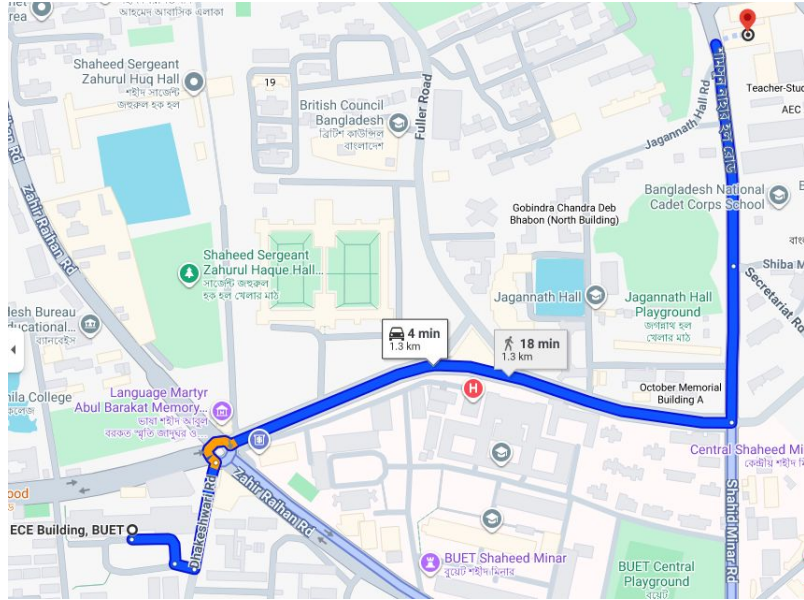
Think of each set of behaviors as a family of algorithms.



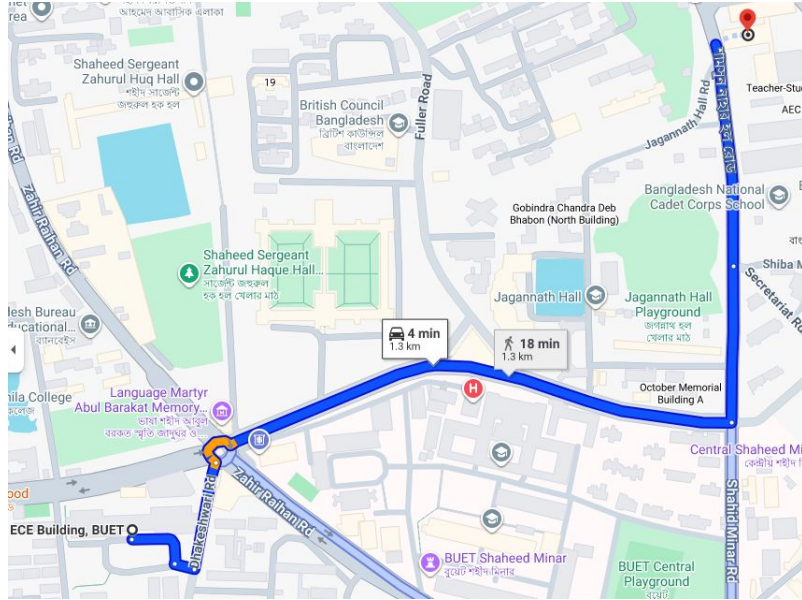
Birds Eye View

These behaviors "algorithms" are interchangeable.

Yet another example

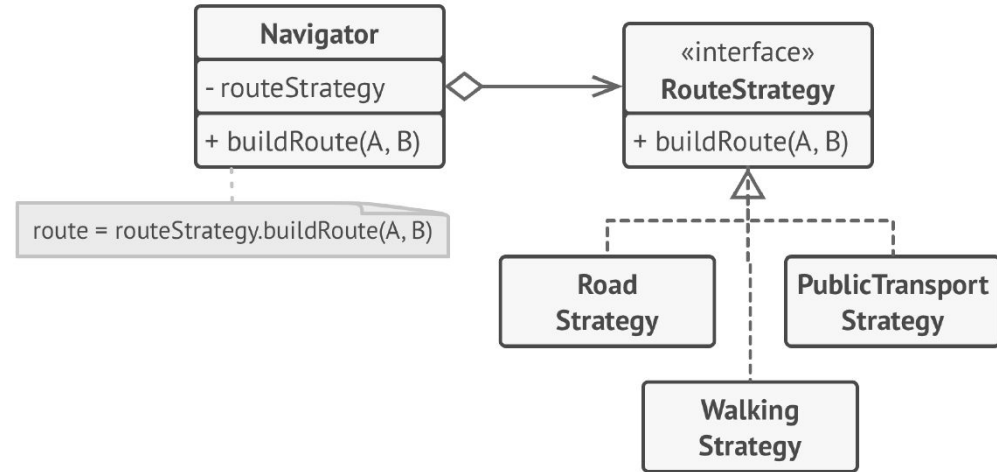
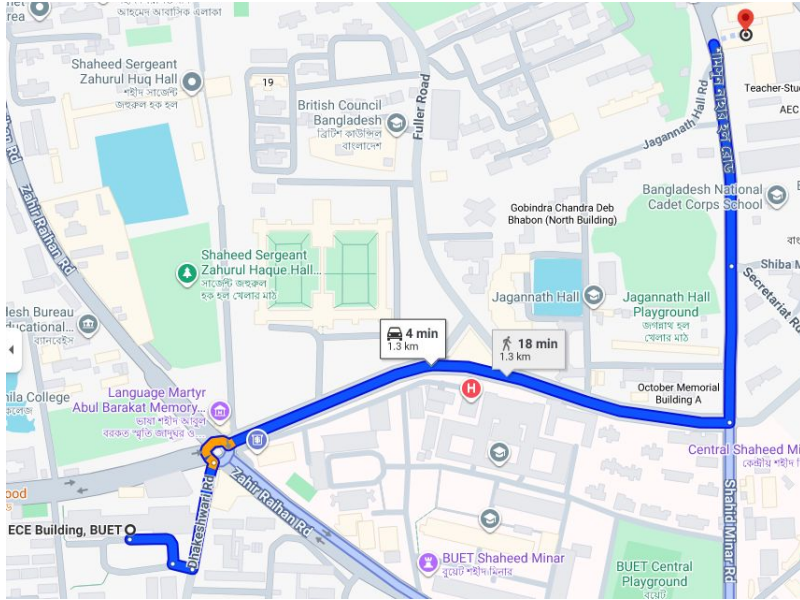


Yet another example



In map app, you can change different *strategies* in real-time between two different points!

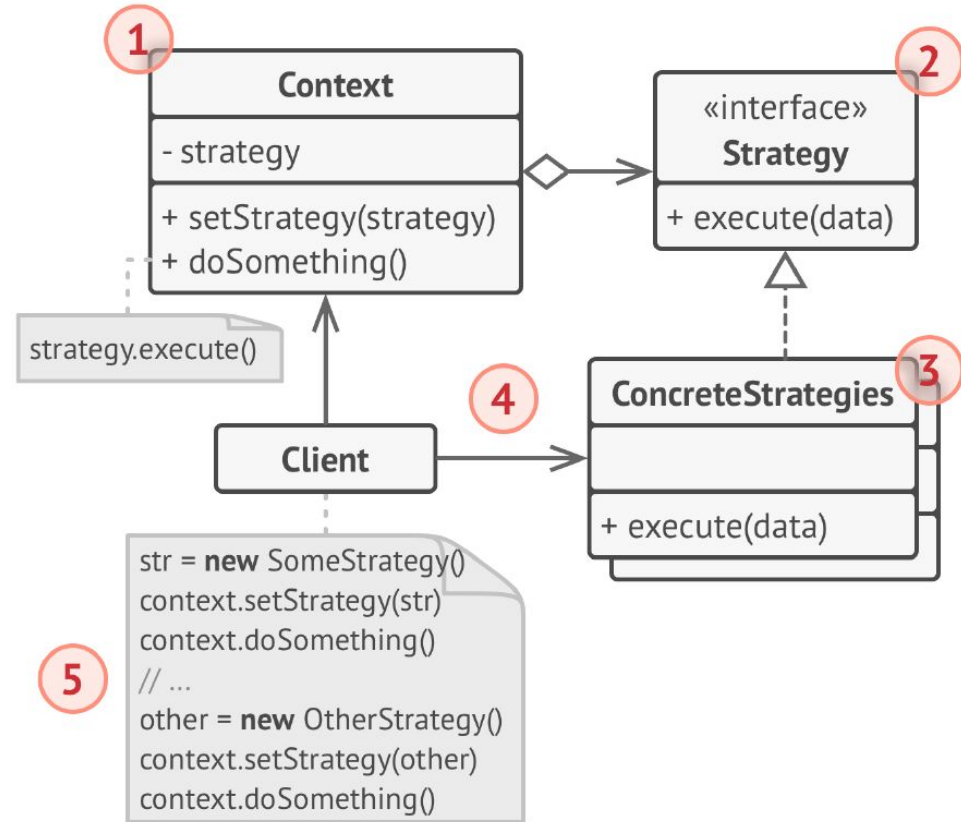
Yet another example



In map app, you can change different *strategies* in real-time between two different points!

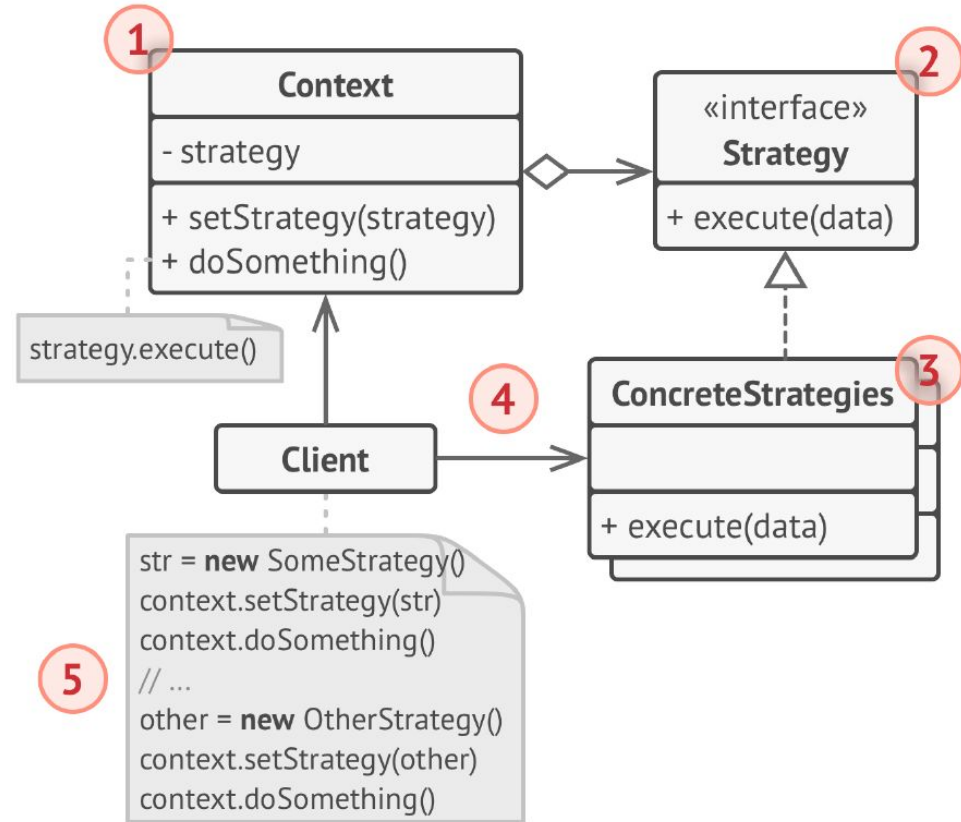
Strategy Pattern Defined

Define a **family of algorithms**, encapsulate each one, and make them **interchangeable**. Strategy lets the algorithm vary *independently* from the clients that use it.



Strategy Pattern Defined

1. The **Context** maintains a reference to one of the concrete strategies and communicates with this object only via the strategy interface.
2. The **Strategy** interface is common to all concrete strategies. It declares a method the context uses to execute a strategy.
3. **Concrete Strategies** implement different variations of an algorithm the context uses.
4. The context calls the execution method on the linked strategy object each time it needs to run the algorithm. The context doesn't know what type of strategy it works with or how the algorithm is executed.
5. The **Client** creates a specific strategy object and passes it to the context. The context exposes a setter which lets clients replace the strategy associated with the context at runtime.





There are no DUMB Questions

Q: It feels a little weird to have a class that's just a behavior. Aren't classes supposed to represent *things*? Aren't classes supposed to have both state **AND** behavior?

A: In an OO system, yes, classes represent things that generally have both state (instance variables) and methods. And in this case, the *thing* happens to be a behavior. But even a behavior can still have state and methods; a flying behavior might have instance variables representing the attributes for the flying (wing beats per minute, max altitude and speed, etc.) behavior.

It's time for some caffeine

Starbuzz Coffee Barista Training Manual

Baristas! Please follow these recipes precisely when preparing Starbuzz beverages.

Starbuzz Coffee Recipe

- (1) Boil some water
- (2) Brew coffee in boiling water
- (3) Pour coffee in cup
- (4) Add sugar and milk

Starbuzz Tea Recipe

- (1) Boil some water
- (2) Steep tea in boiling water
- (3) Pour tea in cup
- (4) Add lemon

All recipes are Starbuzz Coffee trade secrets and should be kept strictly confidential.

← The recipe for coffee looks a lot like the recipe for tea, doesn't it?

Let's see some details...

```
public class Coffee {  
  
    void prepareRecipe() {  
        boilWater();  
        brewCoffeeGrinds();  
        pourInCup();  
        addSugarAndMilk();  
    }  
  
    public void boilWater() {  
        System.out.println("Boiling water");  
    }  
  
    public void brewCoffeeGrinds() {  
        System.out.println("Dripping Coffee through filter");  
    }  
  
    public void pourInCup() {  
        System.out.println("Pouring into cup");  
    }  
  
    public void addSugarAndMilk() {  
        System.out.println("Adding Sugar and Milk");  
    }  
}
```

```
public class Tea {  
  
    void prepareRecipe() {  
        boilWater();  
        steepTeaBag();  
        pourInCup();  
        addLemon();  
    }  
  
    public void boilWater() {  
        System.out.println("Boiling water");  
    }  
  
    public void steepTeaBag() {  
        System.out.println("Steeping the tea");  
    }  
  
    public void addLemon() {  
        System.out.println("Adding Lemon");  
    }  
  
    public void pourInCup() {  
        System.out.println("Pouring into cup");  
    }  
}
```

Let's see some details...

```
public class Coffee {  
  
    void prepareRecipe() {  
        boilWater();  
        brewCoffeeGrinds();  
        pourInCup();  
        addSugarAndMilk();  
    }  
  
    public void boilWater() {  
        System.out.println("Boiling water");  
    }  
  
    public void brewCoffeeGrinds() {  
        System.out.println("Dripping Coffee through filter");  
    }  
  
    public void pourInCup() {  
        System.out.println("Pouring into cup");  
    }  
  
    public void addSugarAndMilk() {  
        System.out.println("Adding Sugar and Milk");  
    }  
}  
  
public class Tea {  
  
    void prepareRecipe() {  
        boilWater();  
        steepTeaBag();  
        pourInCup();  
        addLemon();  
    }  
  
    public void boilWater() {  
        System.out.println("Boiling water");  
    }  
  
    public void steepTeaBag() {  
        System.out.println("Steeping the tea");  
    }  
  
    public void addLemon() {  
        System.out.println("Adding Lemon");  
    }  
  
    public void pourInCup() {  
        System.out.println("Pouring into cup");  
    }  
}
```

Let's see some details...

```
public class Coffee {
```

```
    void prepareRecipe() {  
        boilWater();  
        brewCoffeeGrinds();  
        pourInCup();  
        addSugarAndMilk();  
    }
```

```
    public void boilWater() {  
        System.out.println("Boiling water");  
    }
```

```
    public void brewCoffeeGrinds() {  
        System.out.println("Dripping Coffee through filter");  
    }
```

```
    public void pourInCup() {  
        System.out.println("Pouring into cup");  
    }
```

```
    public void addSugarAndMilk() {  
        System.out.println("Adding Sugar and Milk");  
    }
```

```
}
```

```
public class Tea {
```

```
    void prepareRecipe() {  
        boilWater();  
        steepTeaBag();  
        pourInCup();  
        addLemon();  
    }
```

```
    public void boilWater() {  
        System.out.println("Boiling water");  
    }
```

```
    public void steepTeaBag() {  
        System.out.println("Steeping the tea");  
    }
```

```
    public void addLemon() {  
        System.out.println("Adding Lemon");  
    }
```

```
    public void pourInCup() {  
        System.out.println("Pouring into cup");  
    }
```

```
}
```

exactly same!

Let's see some details...

```
public class Coffee {
```

```
    void prepareRecipe() {  
        boilWater();  
        brewCoffeeGrinds();  
        pourInCup();  
        addSugarAndMilk();  
    }
```

```
    public void boilWater() {  
        System.out.println("Boiling water");  
    }
```

```
    public void brewCoffeeGrinds() {  
        System.out.println("Dripping Coffee through filter");  
    }
```

```
    public void pourInCup() {  
        System.out.println("Pouring into cup");  
    }
```

```
    public void addSugarAndMilk() {  
        System.out.println("Adding Sugar and Milk");  
    }
```

```
public class Tea {
```

```
    void prepareRecipe() {  
        boilWater();  
        steepTeaBag();  
        pourInCup();  
        addLemon();  
    }
```

```
    public void boilWater() {  
        System.out.println("Boiling water");  
    }
```

```
    public void steepTeaBag() {  
        System.out.println("Steeping the tea");  
    }
```

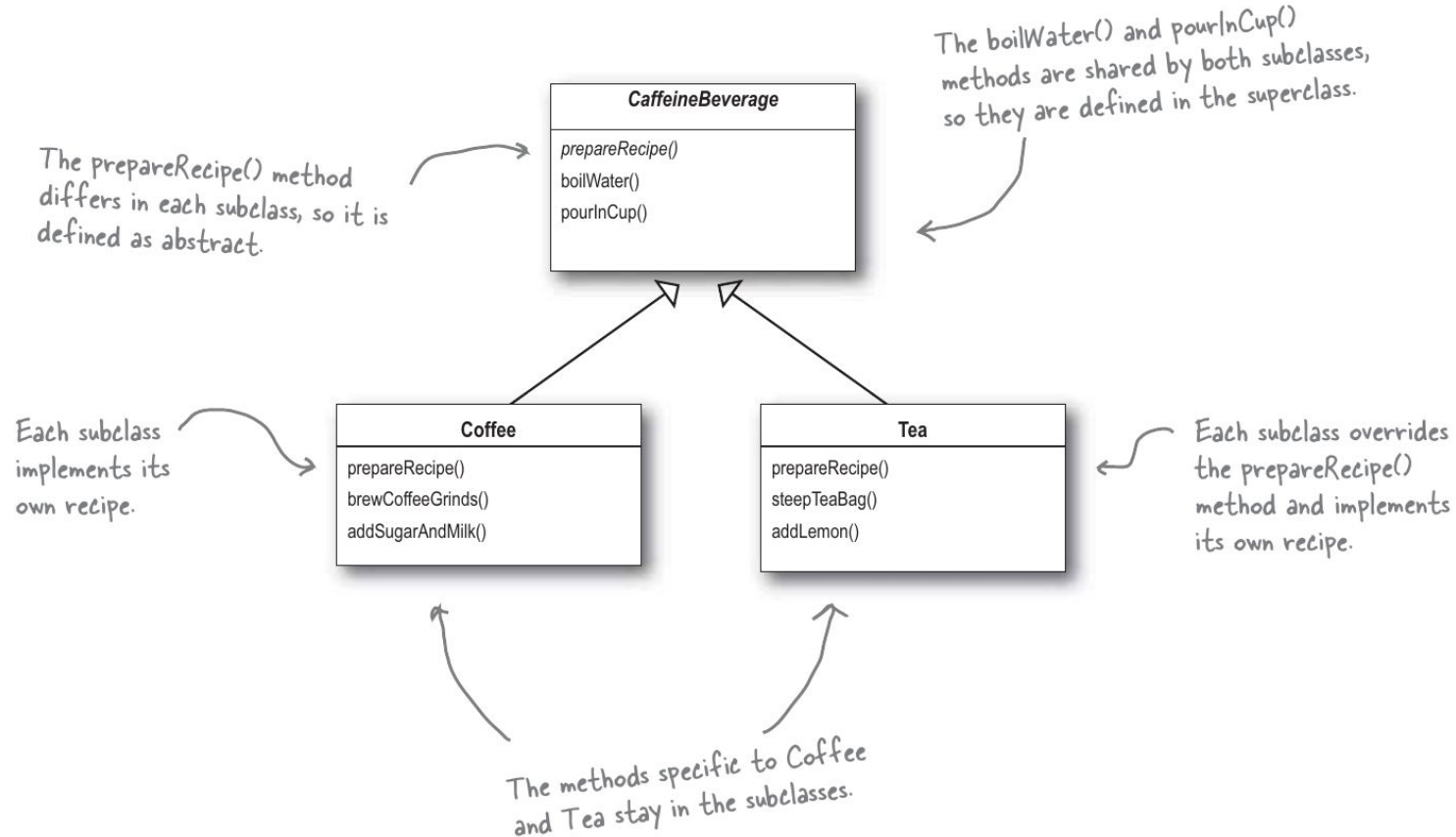
```
    public void addLemon() {  
        System.out.println("Adding Lemon");  
    }
```

```
    public void pourInCup() {  
        System.out.println("Pouring into cup");  
    }
```

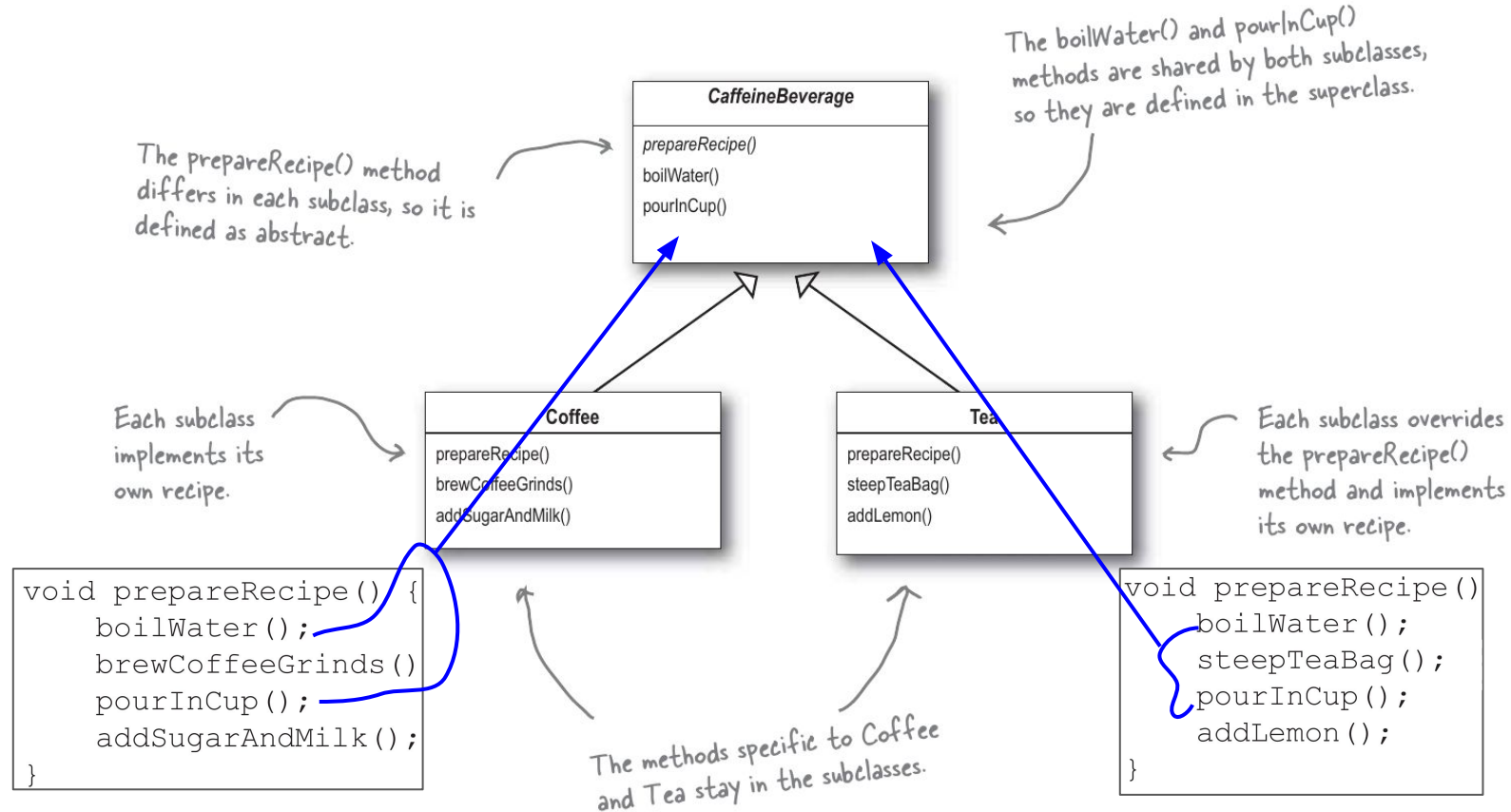
```
}
```

Some steps vary!

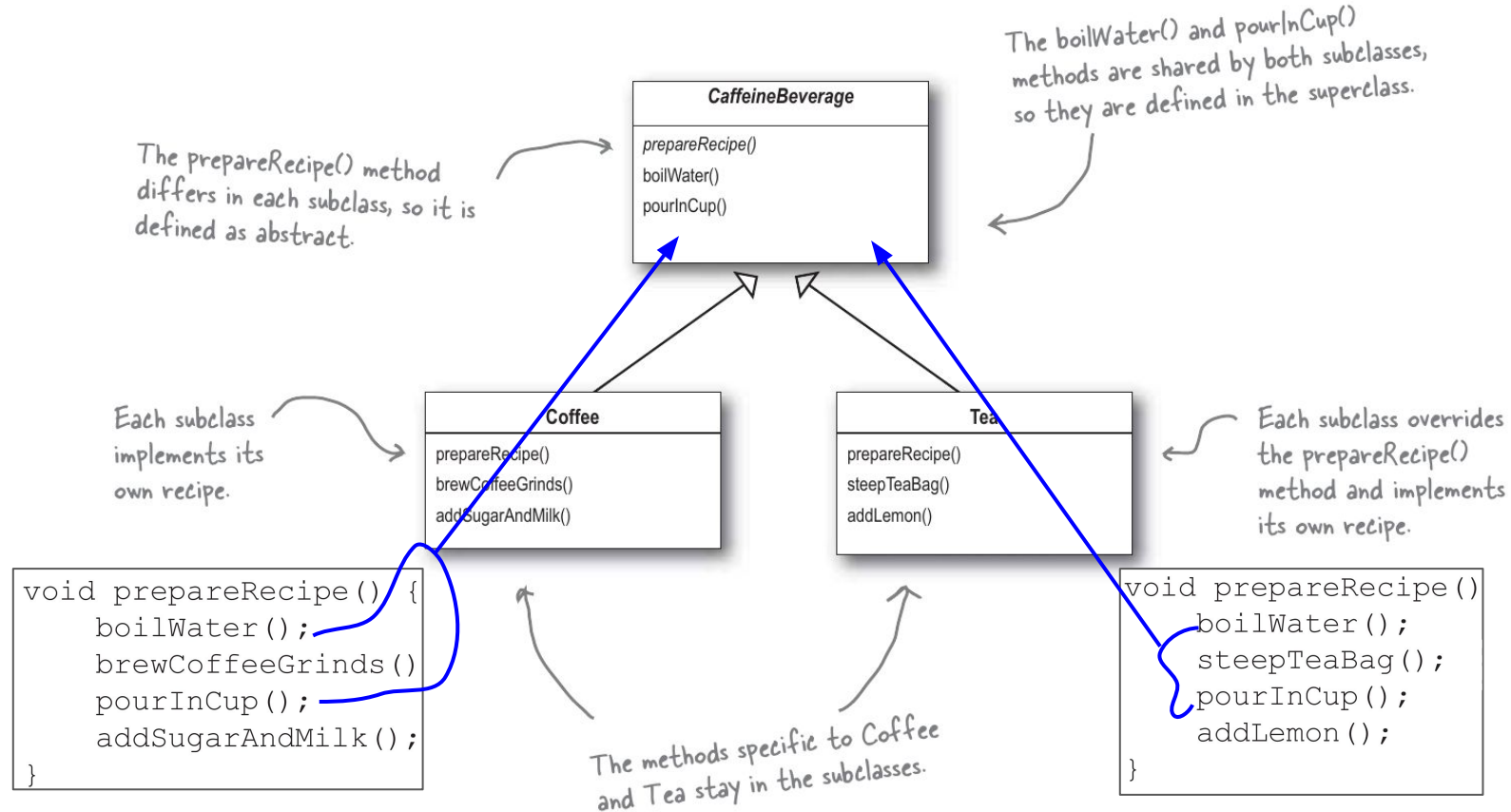
Doing simple abstractions? [keeping what is unchanged in abstract class]



Doing simple abstractions? [keeping what is unchanged in abstract class]



Doing simple abstractions...; is it a good design?

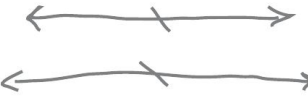


Don't they have the same “skeleton”?

- 1 Boil some water.
- 2 Use the hot water to extract the coffee or tea.
- 3 Pour the resulting beverage into a cup.
- 4 Add the appropriate condiments to the beverage.

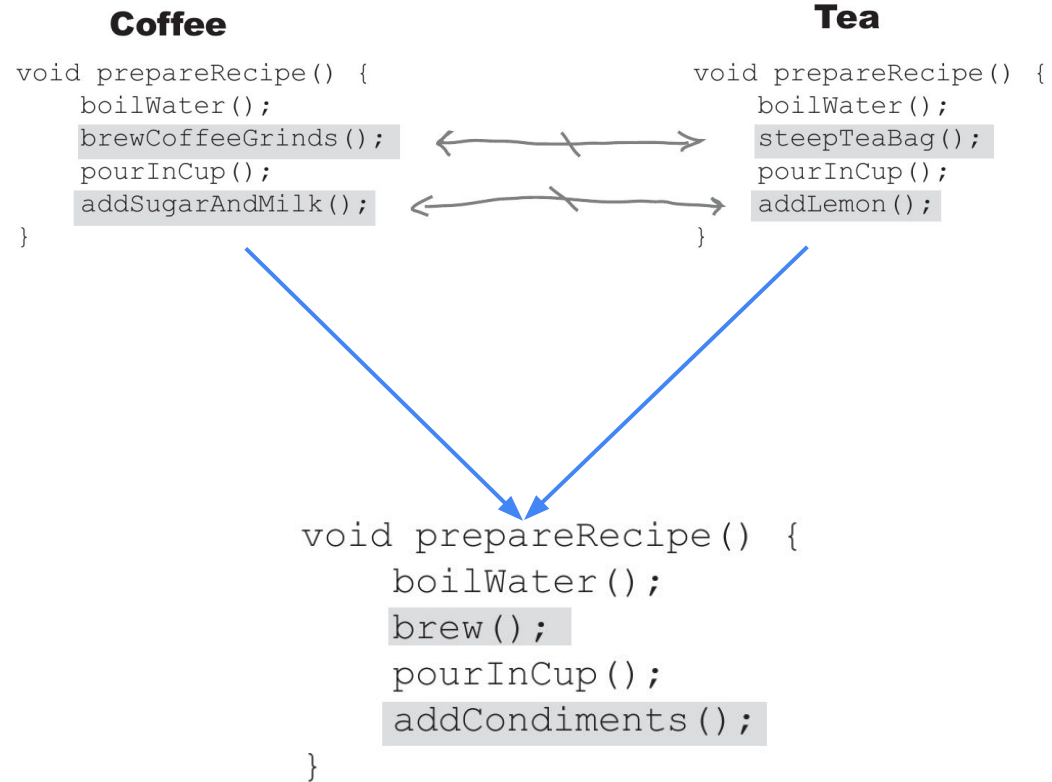
Don't they have the same "skeleton"?

- 1 Boil some water.
- 2 Use the hot water to extract the coffee or tea.
- 3 Pour the resulting beverage into a cup.
- 4 Add the appropriate condiments to the beverage.

Coffee		Tea
<pre>void prepareRecipe() { boilWater(); brewCoffeeGrinds(); pourInCup(); addSugarAndMilk(); }</pre>		<pre>void prepareRecipe() { boilWater(); steepTeaBag(); pourInCup(); addLemon(); }</pre>

Don't they have the same "skeleton"?

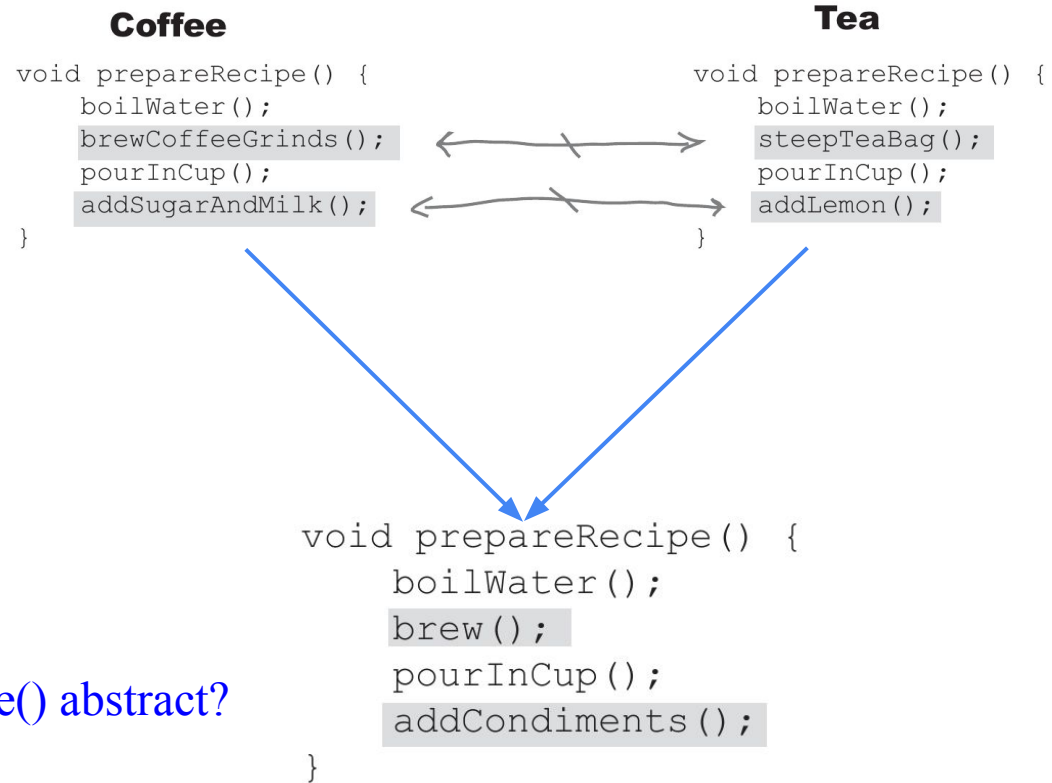
- 1 Boil some water.
- 2 Use the hot water to extract the coffee or tea.
- 3 Pour the resulting beverage into a cup.
- 4 Add the appropriate condiments to the beverage.



Don't they have the same "skeleton"?

- 1 Boil some water.
- 2 Use the hot water to extract the coffee or tea.
- 3 Pour the resulting beverage into a cup.
- 4 Add the appropriate condiments to the beverage.

Make prepareRecipe() abstract?



Keep the common steps in abstract class...

CaffeineBeverage is abstract, just like in the class design.

Now, the same prepareRecipe() method will be used to make both Tea and Coffee. prepareRecipe() is declared final because we don't want our subclasses to be able to override this method and change the recipe! We've generalized steps 2 and 4 to brew() the beverage and addCondiments().

Because Coffee and Tea handle these methods in different ways, they're going to have to be declared as abstract. Let the subclasses worry about that stuff!

Remember, we moved these into the CaffeineBeverage class (back in our class diagram).

```
public abstract class CaffeineBeverage {  
  
    final void prepareRecipe() {  
        boilWater();  
        brew();  
        pourInCup();  
        addCondiments();  
    }  
  
    abstract void brew();  
  
    abstract void addCondiments();  
  
    void boilWater() {  
        System.out.println("Boiling water");  
    }  
  
    void pourInCup() {  
        System.out.println("Pouring into cup");  
    }  
}
```

Let subclasses worry about the steps that differ...

CaffeineBeverage is abstract, just like in the class design.

```
public abstract class CaffeineBeverage {  
  
    final void prepareRecipe() {  
        boilWater();  
        brew();  
        pourInCup();  
        addCondiments();  
    }  
  
    abstract void brew();  
  
    abstract void addCondiments();  
  
    void boilWater() {  
        System.out.println("Boiling water");  
    }  
  
    void pourInCup() {  
        System.out.println("Pouring into cup");  
    }  
}
```

```
public class Tea extends CaffeineBeverage {  
    public void brew() {  
        System.out.println("Steeping the tea");  
    }  
    public void addCondiments() {  
        System.out.println("Adding Lemon");  
    }  
}
```

now extend CaffeineBeverage.

Tea needs to define brew() and addCondiments() – the two abstract methods from Beverage.

Same for Coffee, except Coffee deals with coffee, and sugar and milk instead of tea bags and lemon.

```
public class Coffee extends CaffeineBeverage {  
    public void brew() {  
        System.out.println("Dripping Coffee through filter");  
    }  
    public void addCondiments() {  
        System.out.println("Adding Sugar and Milk");  
    }  
}
```

What have we done?

Tea

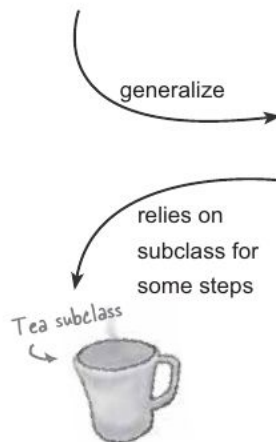
- 1 Boil some water
- 2 Steep the teabag in the water
- 3 Pour tea in a cup
- 4 Add lemon

Coffee

- 1 Boil some water
- 2 Brew the coffee grinds
- 3 Pour coffee in a cup
- 4 Add sugar and milk

Caffeine Beverage

- 1 Boil some water
- 2 Brew
- 3 Pour beverage in a cup
- 4 Add condiments



- 2 Steep the teabag in the water
- 4 Add lemon

- 2 Brew the coffee grinds
- 4 Add sugar and milk

Caffeine Beverage knows and controls the steps of the recipe, and performs steps 1 and 3 itself, but relies on Tea or Coffee to do steps 2 and 4.

Template Method

```
public abstract class CaffeineBeverage {  
    void final prepareRecipe() {  
        boilWater();  
        brew();  
        pourInCup();  
        addCondiments();  
    }  
  
    abstract void brew();  
    abstract void addCondiments();  
    void boilWater() {  
        // implementation  
    }  
    void pourInCup() {  
        // implementation  
    }  
}
```

prepareRecipe() is our template method. Why?

Because:

(1) It is a method, after all.

(2) It serves as a template for an algorithm, in this case, an algorithm for making caffeinated beverages.

In the template, each step of the algorithm is represented by a method.

Some methods are handled by this class...

...and some are handled by the subclass.

The methods that need to be supplied by a subclass are declared abstract.

Template Method

```
public abstract class CaffeineBeverage {  
  
    void final prepareRecipe() {  
  
        boilWater();  
  
        brew();  
  
        pourInCup();  
  
        addCondiments();  
  
    }  
  
    abstract void brew();  
  
    abstract void addCondiments();  
  
    void boilWater() {  
        // implementation  
    }  
  
    void pourInCup() {  
        // implementation  
    }  
  
}
```

prepareRecipe() is our template method. Why?

Because:

- (1) It is a method, after all.
- (2) It serves as a template for an algorithm, in this case, an algorithm for making caffeinated beverages.

In the template, each step of the algorithm is represented by a method.

Some methods are handled by this class...

...and some are handled by the subclass.

The methods that need to be supplied by a subclass are declared abstract.

Let's make some tea!

1

Okay, first we need a Tea object...

```
Tea myTea = new Tea();
```

2

Then we call the template method:

```
myTea.prepareRecipe();
```

which follows the algorithm for making caffeine beverages...

3

First we boil water:

```
boilWater();
```

which happens in CaffeineBeverage.

4

Next we need to brew the tea, which only the subclass knows how to do:

```
brew();
```

5

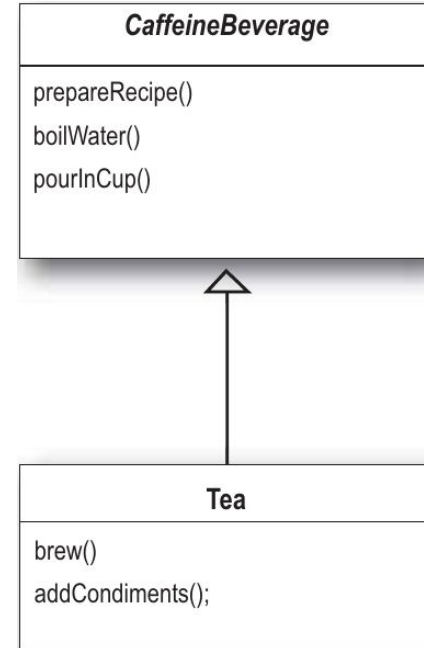
Now we pour the tea in the cup; this is the same for all beverages so it happens in CaffeineBeverage:

```
pourInCup();
```

6

Finally, we add the condiments, which are specific to each beverage, so the subclass implements this:

```
addCondiments();
```



Let's make some tea!

Common
Different

1

Okay, first we need a Tea object...

```
Tea myTea = new Tea();
```

2

Then we call the template method:

```
myTea.prepareRecipe();
```

which follows the algorithm for making caffeine beverages...

3

First we boil water:

```
boilWater();
```

which happens in CaffeineBeverage.

4

Next we need to brew the tea, which only the subclass knows how to do:

```
brew();
```

5

Now we pour the tea in the cup; this is the same for all beverages so it happens in CaffeineBeverage:

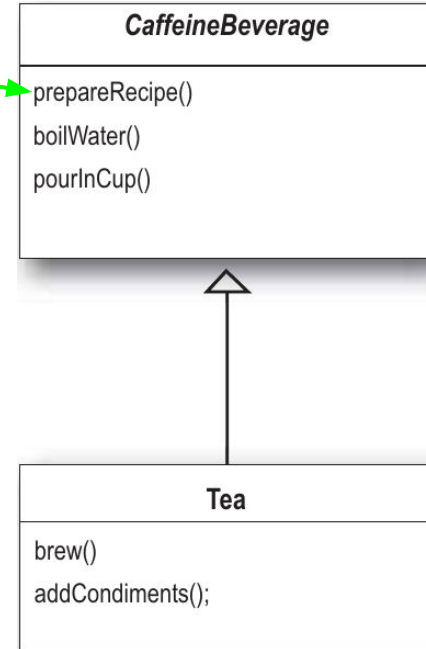
```
pourInCup();
```

6

Finally, we add the condiments, which are specific to each beverage, so the subclass implements this:

```
addCondiments();
```

```
void prepareRecipe() {  
    boilWater();  
    brew();  
    pourInCup();  
    addCondiments();  
}
```



Let's make some tea!

Common
Different

1

Okay, first we need a Tea object...

```
Tea myTea = new Tea();
```

2

Then we call the template method:

```
myTea.prepareRecipe();
```

which follows the algorithm for making caffeine beverages...

3

First we boil water:

```
boilWater();
```

which happens in CaffeineBeverage.

4

Next we need to brew the tea, which only the subclass knows how to do:

```
brew();
```

5

Now we pour the tea in the cup; this is the same for all beverages so it happens in CaffeineBeverage:

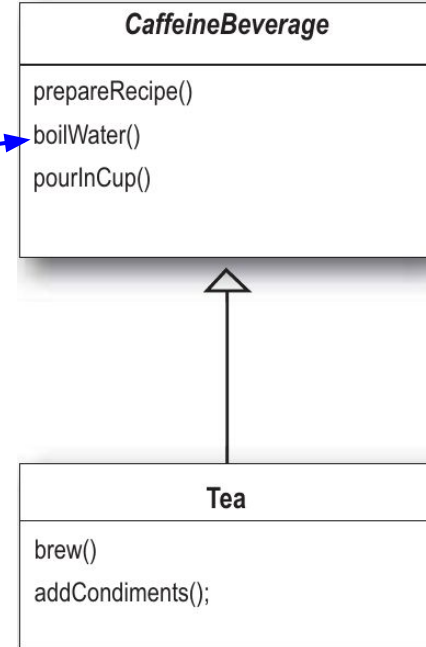
```
pourInCup();
```

6

Finally, we add the condiments, which are specific to each beverage, so the subclass implements this:

```
addCondiments();
```

```
void prepareRecipe() {  
    boilWater();  
    brew();  
    pourInCup();  
    addCondiments();  
}
```



Let's make some tea!

Common
Different

1 Okay, first we need a Tea object...

```
Tea myTea = new Tea();
```

2 Then we call the template method:

```
myTea.prepareRecipe();
```

which follows the algorithm for making caffeine beverages...

3 First we boil water:

```
boilWater();
```

which happens in CaffeineBeverage.

4 Next we need to brew the tea, which only the subclass knows how to do:

```
brew();
```

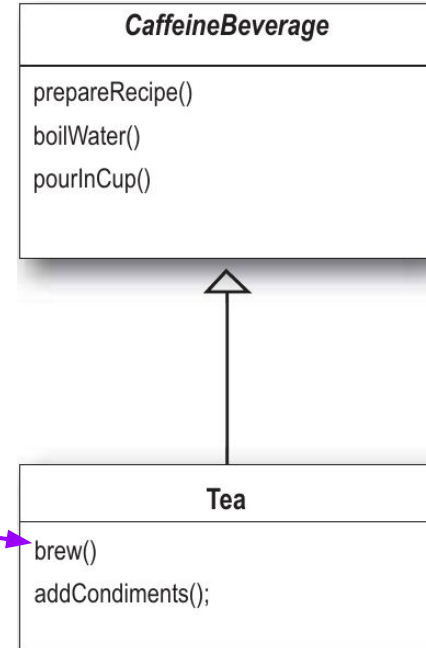
5 Now we pour the tea in the cup; this is the same for all beverages so it happens in CaffeineBeverage:

```
pourInCup();
```

6 Finally, we add the condiments, which are specific to each beverage, so the subclass implements this:

```
addCondiments();
```

```
void prepareRecipe() {  
    boilWater();  
    brew();  
    pourInCup();  
    addCondiments();  
}
```



Let's make some tea!

Common
Different

1 Okay, first we need a Tea object...

```
Tea myTea = new Tea();
```

2 Then we call the template method:

```
myTea.prepareRecipe();
```

which follows the algorithm for making caffeine beverages...

3 First we boil water:

```
boilWater();
```

which happens in CaffeineBeverage.

4 Next we need to brew the tea, which only the subclass knows how to do:

```
brew();
```

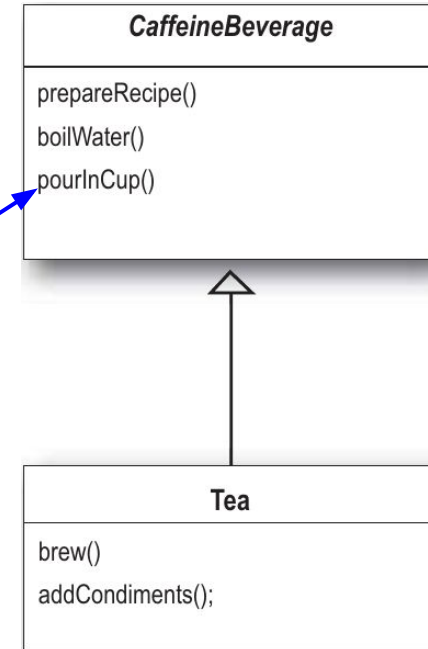
5 Now we pour the tea in the cup; this is the same for all beverages so it happens in CaffeineBeverage:

```
pourInCup();
```

6 Finally, we add the condiments, which are specific to each beverage, so the subclass implements this:

```
addCondiments();
```

```
void prepareRecipe() {  
    boilWater();  
    brew();  
    pourInCup();  
    addCondiments();  
}
```



Let's make some tea!

Common Different

1 Okay, first we need a Tea object...

```
Tea myTea = new Tea();
```

2 Then we call the template method:

```
myTea.prepareRecipe();
```

which follows the algorithm for making caffeine beverages...

3 First we boil water:

```
boilWater();
```

which happens in CaffeineBeverage.

4 Next we need to brew the tea, which only the subclass knows how to do:

```
brew();
```

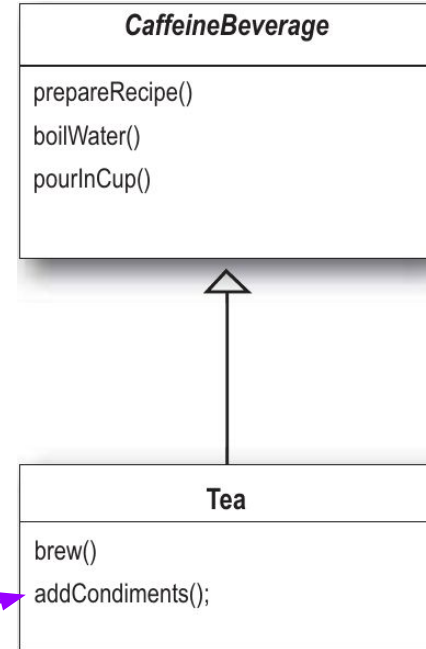
5 Now we pour the tea in the cup; this is the same for all beverages so it happens in CaffeineBeverage:

```
pourInCup();
```

6 Finally, we add the condiments, which are specific to each beverage, so the subclass implements this:

```
addCondiments();
```

```
void prepareRecipe() {  
    boilWater();  
    brew();  
    pourInCup();  
    addCondiments();  
}
```



Adding conditional executions with if-else

```
public abstract class CaffeineBeverage
```

```
void prepareRecipe() {  
    boilWater();  
    brew();  
    pourInCup();  
    if (customerWantsCondiments()) {  
        addCondiments();  
    }  
}
```

↖ We've added a little conditional statement that bases its success on a concrete method, `customerWantsCondiments()`. If the customer WANTS a condiments, only then do we call `addCondiments()`.

```
abstract void brew();
```

```
abstract void addCondiments();
```

```
void boilWater() {  
    System.out.println("Boiling water");  
}
```

```
void pourInCup() {  
    System.out.println("Pouring into cup");  
}
```

↖ Here we've defined a method with a (mostly) empty default implementation. This method just returns true and does nothing else.

```
boolean customerWantsCondiments() {  
    return true;  
}
```

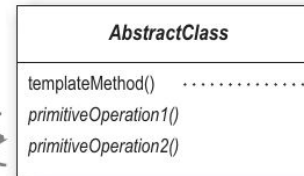
```
}
```

Template Method Pattern Defined

Template Method pattern defines the **skeleton** of an algorithm in the **superclass** but lets subclasses **override specific steps** of the algorithm without changing its **structure**.

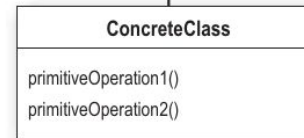
The template method makes use of the `primitiveOperations` to implement an algorithm. It is decoupled from the actual implementation of these operations.

The `AbstractClass` contains the template method.
...and abstract versions of the operations used in the template method.



`primitiveOperation1();`
`primitiveOperation2();`

There may be many `ConcreteClasses`, each implementing the full set of operations required by the template method.

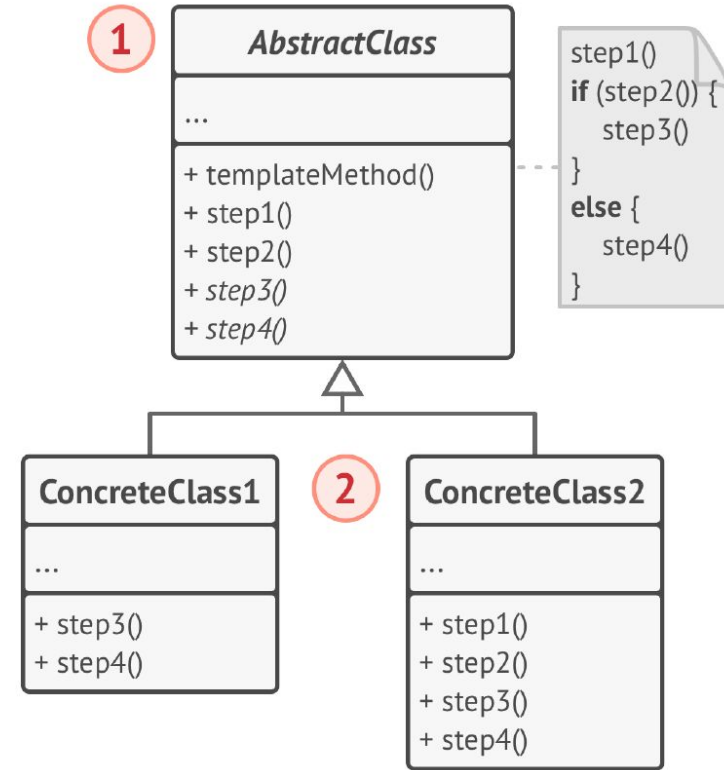


The `ConcreteClass` implements the abstract operations, which are called when the `templateMethod()` needs them.

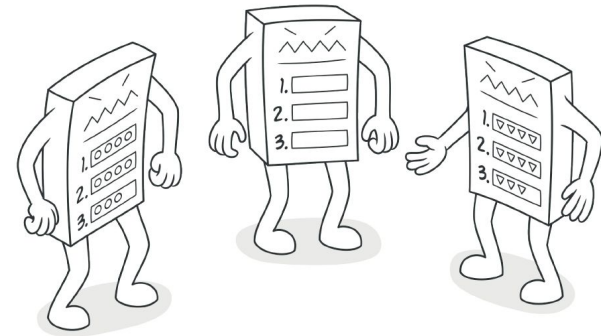
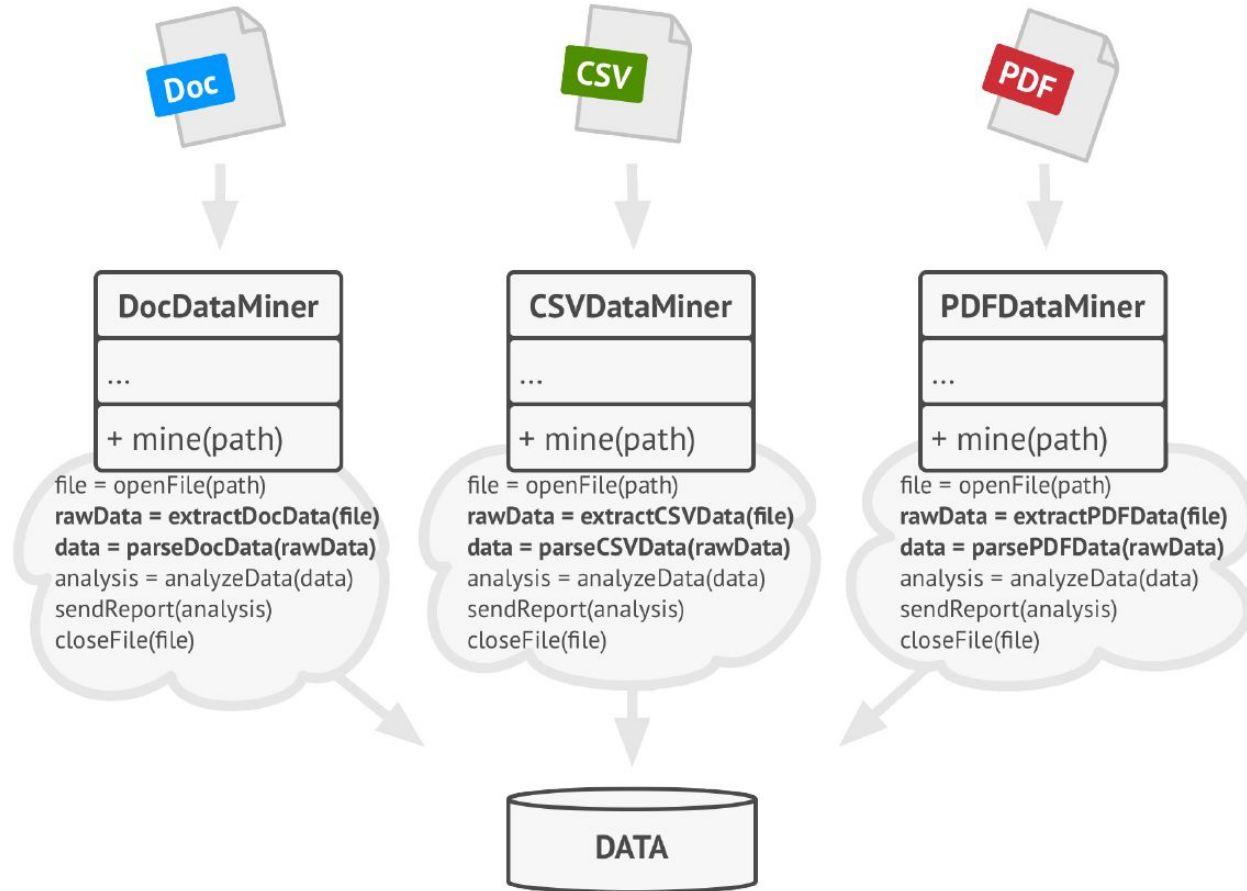
The generalized structure

1. The **Abstract Class** declares methods that act as steps of an algorithm, as well as the actual template method which calls these methods in a specific order. The steps may either be declared `abstract` or have some default implementation.
2. **Concrete Classes** can override all of the steps, but not the template method itself.

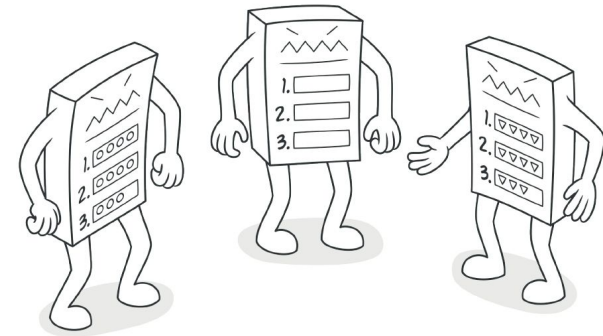
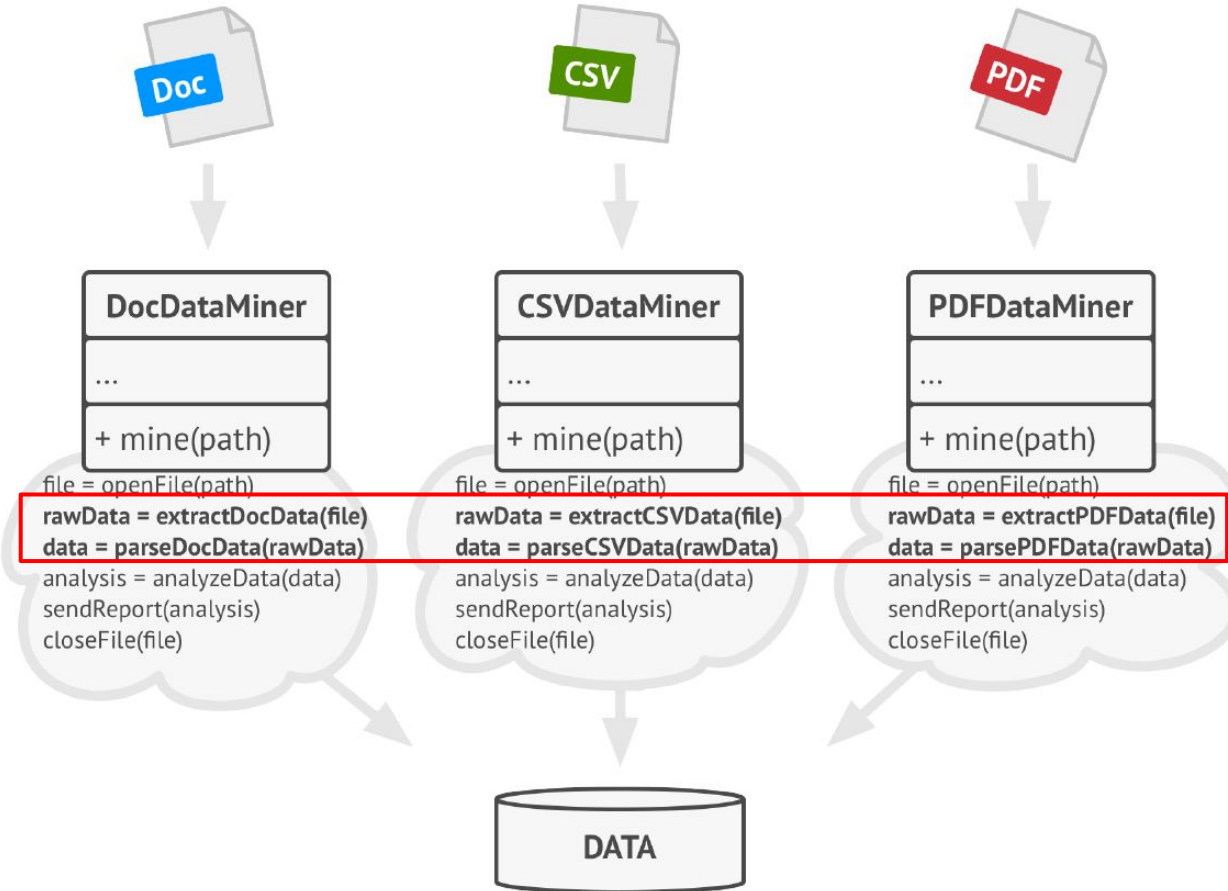
It is *final* method



Yet another example: Data Mining

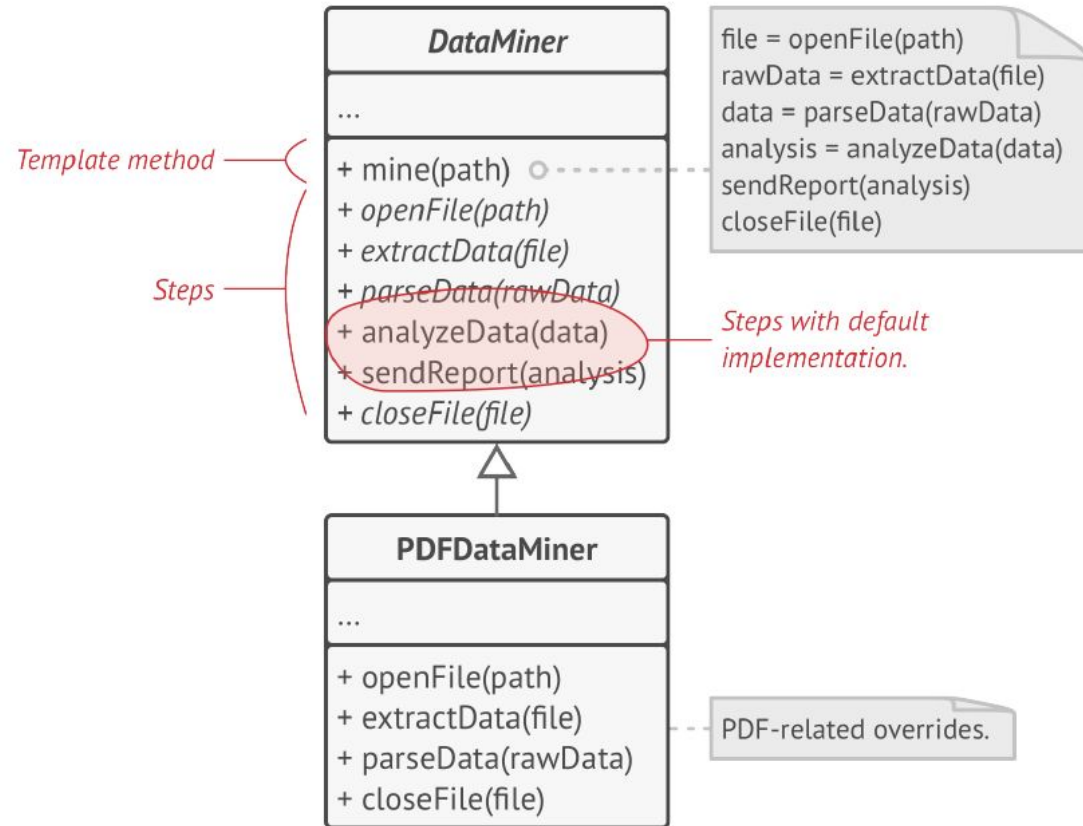


Yet another example: Data Mining



only these vary

Solution...



Template method breaks the algorithm into steps, allowing subclasses to override these steps but not the actual method.

Difference between Strategy and Template

- Template Method based on **inheritance**
 - Alter *parts of an algorithm* by extending those parts in subclasses.
- Strategy is based on **composition**
 - Alter *parts of the object's behavior* by supplying it with different strategies that correspond to that behavior.
- Template Method works at the **class level**, so it's **static**.
- Strategy works on the **object level**, letting you switch behaviors at **runtime**.

Template Method

Hey Strategy, what are you doing in my chapter? I figured I'd get stuck with someone boring like Factory Method

I was just kidding! But seriously, what are you doing here? We haven't heard from you in eight chapters!

You might want to remind the reader what you're all about, since it's been so long.

Strategy



Nope, it's me, although be careful – you and Factory Method are related, aren't you?

I'd heard you were on the final draft of your chapter and I thought I'd swing by to see how it was going. We have a lot in common, so I thought I might be able to help...

I don't know, since Chapter 1, people have been stopping me in the street saying, "Aren't you that pattern..." So I think they know who I am. But for your sake: I define a family of algorithms and make them interchangeable. Since each algorithm is encapsulated, the client can use different algorithms easily.



Template Method

Hey, that does sound a lot like what I do. But my intent's a little different from yours; my job is to define the outline of an algorithm, but let my subclasses do some of the work. That way, I can have different implementations of an algorithm's individual steps, but keep control over the algorithm's structure. Seems like you have to give up control of your algorithms.

I remember that. But I have more control over my algorithm and I don't duplicate code. In fact, if every part of my algorithm is the same except for, say, one line, then my classes are much more efficient than yours. All my duplicated code gets put into the superclass, so all the subclasses can share it.

Strategy

I'm not sure I'd put it quite like *that...* and anyway, I'm not stuck using inheritance for algorithm implementations. I offer clients a choice of algorithm implementation through object composition.

Template Method

Yeah, well, I'm *real* happy for ya, but don't forget I'm the most used pattern around. Why? Because I provide a fundamental method for code reuse that allows subclasses to specify behavior. I'm sure you can see that this is perfect for creating frameworks.

How's that? My superclass is abstract.

Like I said Strategy, I'm *real* happy for you. Thanks for stopping by, but I've got to get the rest of this chapter done.

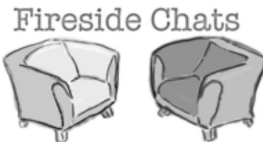
Strategy

You *might* be a little more efficient (just a little) and require fewer objects. *And* you might also be a little less complicated in comparison to my delegation model, but I'm more flexible because I use object composition. With me, clients can change their algorithms at runtime simply by using a different strategy object. Come on, they didn't choose *me* for Chapter 1 for nothing!

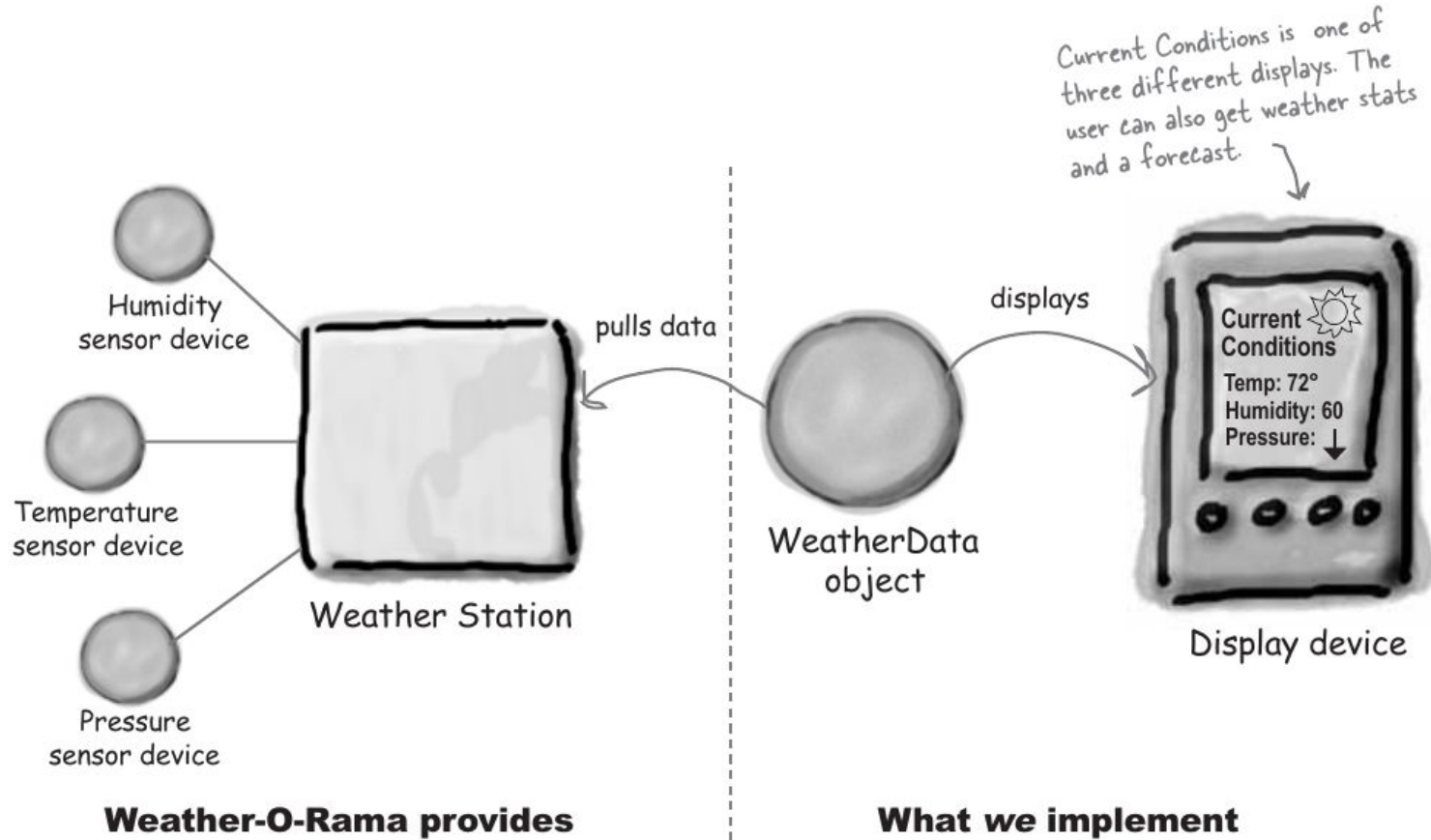
Yeah, I guess... but, what about dependency? You're way more dependent than me.

But you have to depend on methods implemented in your superclass, which are part of your algorithm. I don't depend on anyone; I can do the entire algorithm myself!

Okay, okay, don't get touchy. I'll let you work, but let me know if you need my special techniques anyway, I'm always glad to help.

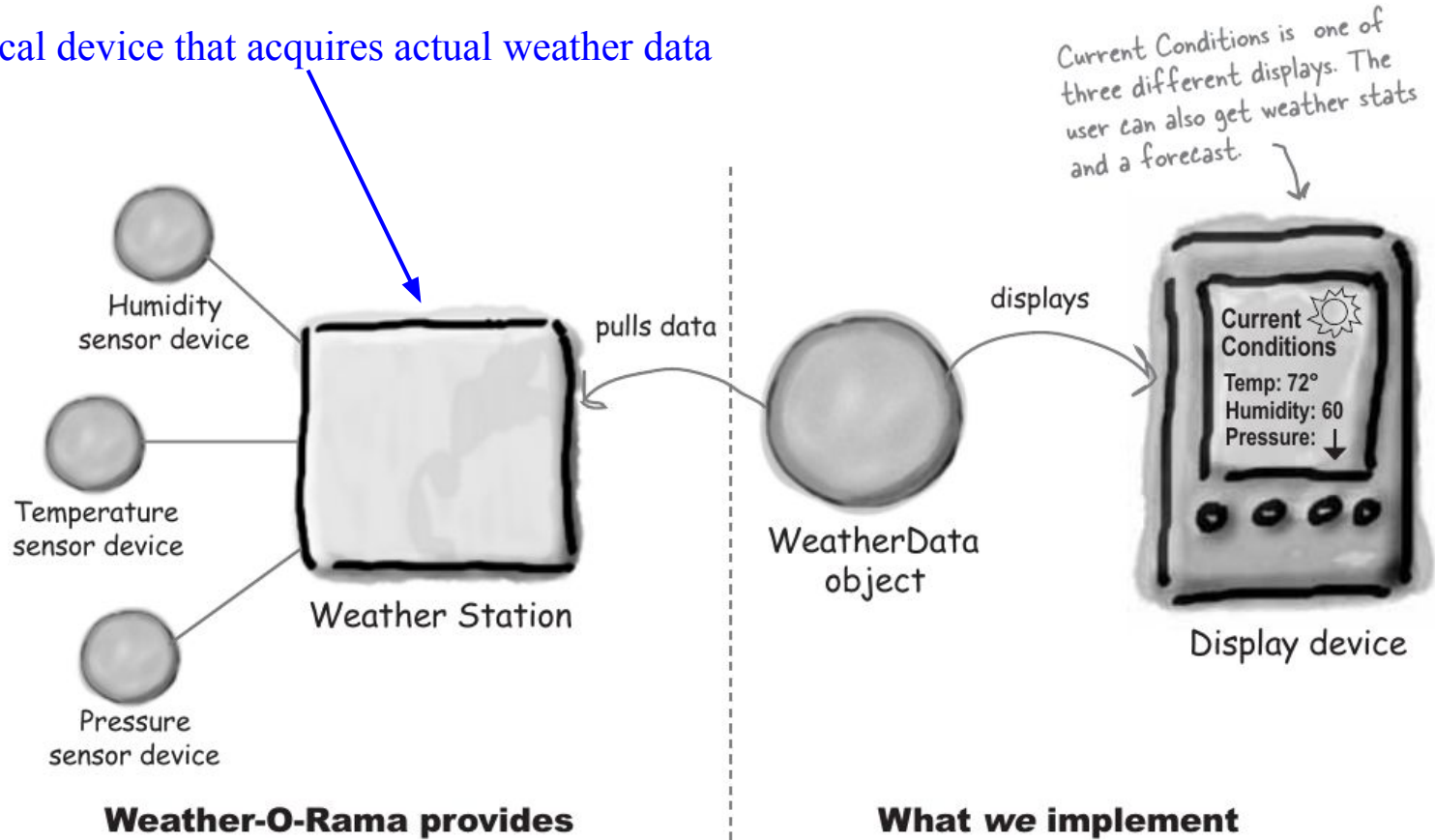


Now let's talk about weather monitoring system



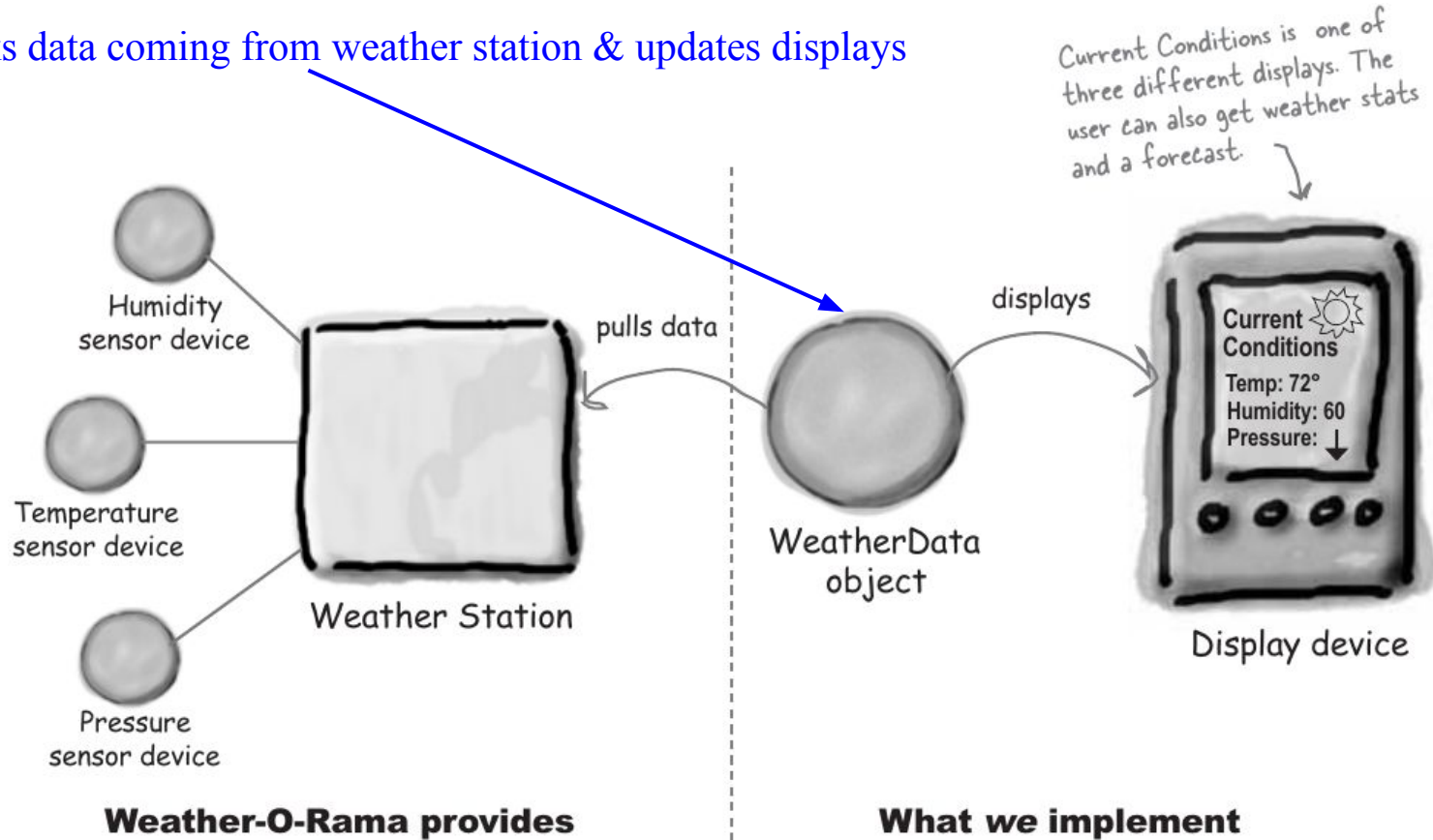
Now let's talk about weather monitoring system

Physical device that acquires actual weather data



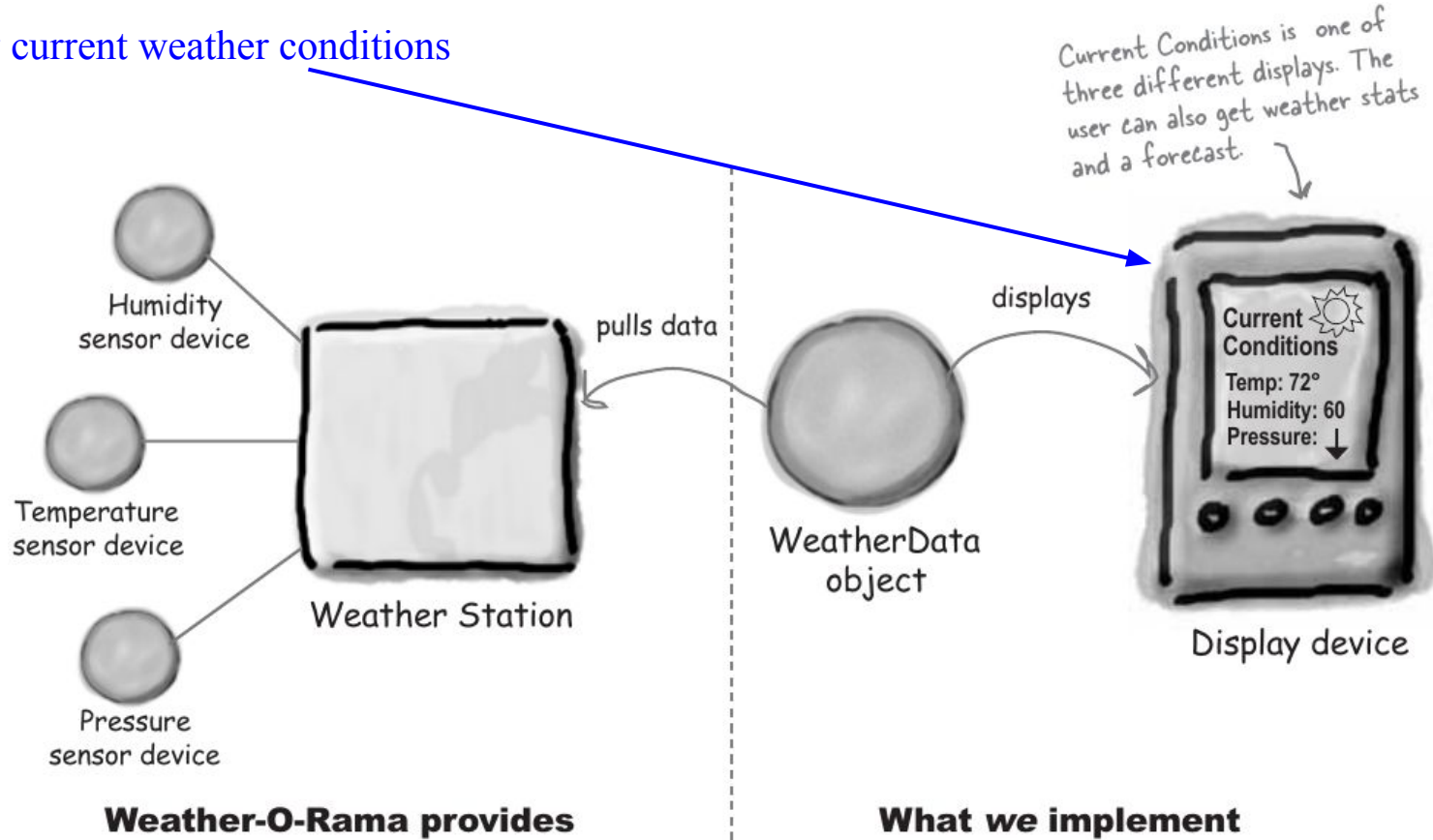
Now let's talk about weather monitoring system

Tracks data coming from weather station & updates displays

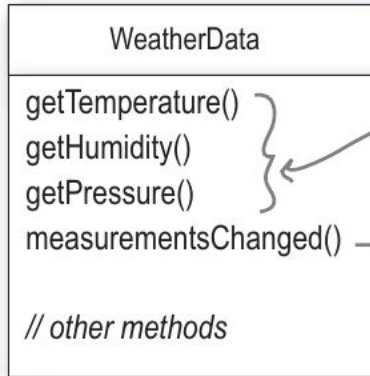


Now let's talk about weather monitoring system

Show current weather conditions



Let's see what we have got from Weather-o-Rama



These three methods return the most recent weather measurements for temperature, humidity and barometric pressure respectively.

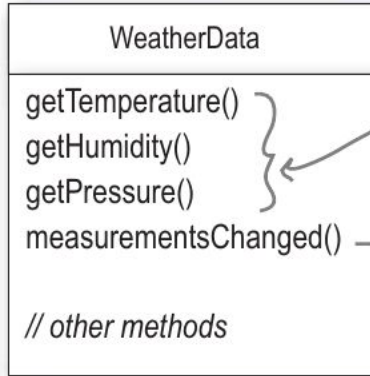
We don't care HOW these variables are set; the `WeatherData` object knows how to get updated info from the Weather Station.

The developers of the `WeatherData` object left us a clue about what we need to add...

```
/*
 * This method gets called
 * whenever the weather measurements
 * have been updated
 *
 */
public void measurementsChanged() {
    // Your code goes here
}
```

WeatherData.java

Let's see what we have got from Weather-o-Rama



These three methods return the most recent weather measurements for temperature, humidity and barometric pressure respectively.

We don't care HOW these variables are set; the `WeatherData` object knows how to get updated info from the Weather Station.

The developers of the `WeatherData` object left us a clue about what we need to add...

```
/*
 * This method gets called
 * whenever the weather measurements
 * have been updated
 *
 */
public void measurementsChanged() {
    // Your code goes here
}
```

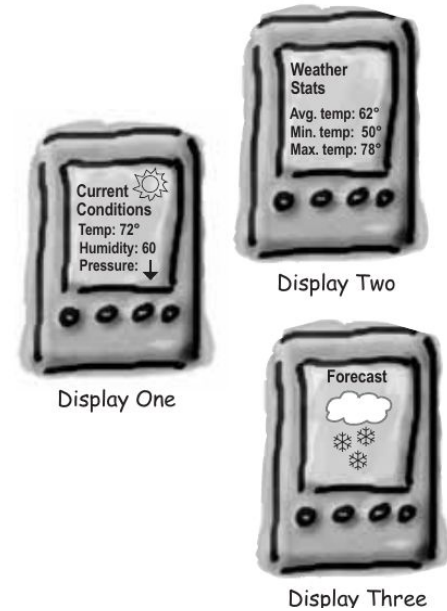
Change triggers it

WeatherData.java

What we need to do?

Implement **three** displays

- ❑ **One:** current conditions
- ❑ **Two:** statistics
- ❑ **Three:** forecast

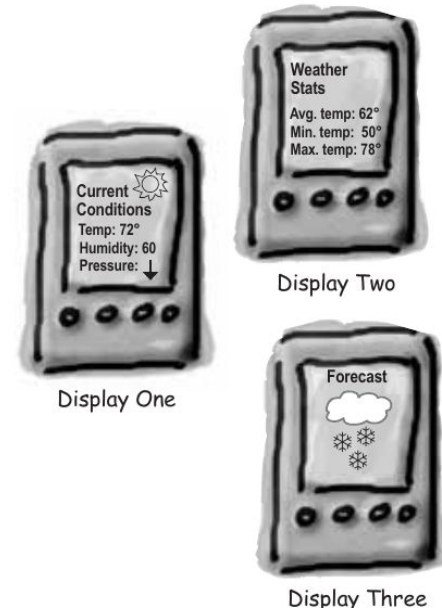


What we need to do?

Implement **three** displays

- ❑ **One:** current conditions
- ❑ **Two:** statistics
- ❑ **Three:** forecast

Remember *expandability*; display can be added/removed



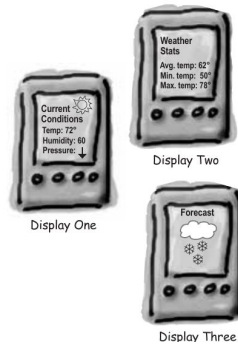
Misguided implementation...

```
public class WeatherData {  
  
    // instance variable declarations  
  
    public void measurementsChanged() {  
  
        float temp = getTemperature();  
        float humidity = getHumidity();  
        float pressure = getPressure();  
  
        currentConditionsDisplay.update(temp, humidity, pressure);  
        statisticsDisplay.update(temp, humidity, pressure);  
        forecastDisplay.update(temp, humidity, pressure);  
    }  
  
    // other WeatherData methods here  
}
```

Grab the most recent measurements by calling the WeatherData's getter methods (already implemented).

Now update the displays...

Call each display element to update its display, passing it the most recent measurements.



Misguided implementation...

```
public class WeatherData {
```

```
    // instance variable declarations
```

```
    public void measurementsChanged() {
```

```
        float temp = getTemperature();  
        float humidity = getHumidity();  
        float pressure = getPressure();
```

get results

Grab the most recent measurements by calling the WeatherData's getter methods (already implemented).

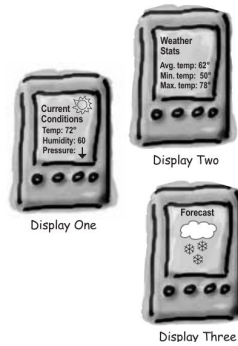
```
        currentConditionsDisplay.update(temp, humidity, pressure);  
        statisticsDisplay.update(temp, humidity, pressure);  
        forecastDisplay.update(temp, humidity, pressure);
```

Now update the displays...

```
    // other WeatherData methods here
```

```
}
```

Call each display element to update its display, passing it the most recent measurements.



Misguided implementation...

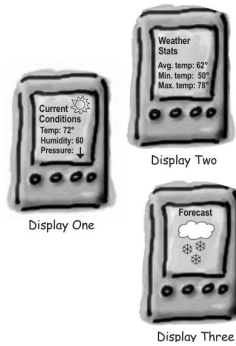
```
public class WeatherData {  
  
    // instance variable declarations  
  
    public void measurementsChanged() {  
  
        float temp = getTemperature();  
        float humidity = getHumidity();  
        float pressure = getPressure();  
  
        currentConditionsDisplay.update(temp, humidity, pressure);  
        statisticsDisplay.update(temp, humidity, pressure);  
        forecastDisplay.update(temp, humidity, pressure);  
  
    }  
  
    // other WeatherData methods here  
}
```

Grab the most recent measurements by calling the WeatherData's getter methods (already implemented).

update displays

Now update the displays...

Call each display element to update its display, passing it the most recent measurements.



Misguided implementation...

```
public void measurementsChanged() {
```

```
    float temp = getTemperature();  
    float humidity = getHumidity();  
    float pressure = getPressure();
```

```
    currentConditionsDisplay.update(temp, humidity, pressure);  
    statisticsDisplay.update(temp, humidity, pressure);  
    forecastDisplay.update(temp, humidity, pressure);  
}
```

Area of change, we need
to encapsulate this.

By coding to concrete implementations
we have no way to add or remove
other display elements without making
changes to the program.

At least we seem to be using a
common interface to talk to the
display elements... they all have an
update() method takes the temp,
humidity, and pressure values.



Display One



Display Two



Display Three

Misguided implementation...

```
public void measurementsChanged() {
```

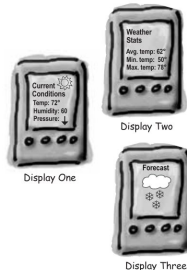
```
    float temp = getTemperature();  
    float humidity = getHumidity();  
    float pressure = getPressure();
```

```
    currentConditionsDisplay.update(temp, humidity, pressure);  
    statisticsDisplay.update(temp, humidity, pressure);  
    forecastDisplay.update(temp, humidity, pressure);  
}
```

Area of change, we need
to encapsulate this.

By coding to concrete implementations
we have no way to add or remove
other display elements without making
changes to the program.

At least we seem to be using a
common interface to talk to the
display elements... they all have an
update() method takes the temp,
humidity, and pressure values.

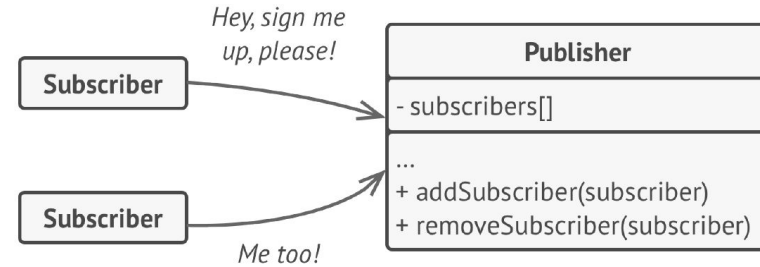


How magazine subscriptions work?

- 1** A newspaper publisher goes into business and begins publishing newspapers.
- 2** You subscribe to a particular publisher, and every time there's a new edition it gets delivered to you. As long as you remain a subscriber, you get new newspapers.
- 3** You unsubscribe when you don't want papers anymore, and they stop being delivered.
- 4** While the publisher remains in business, people, hotels, airlines and other businesses constantly subscribe and unsubscribe to the newspaper.

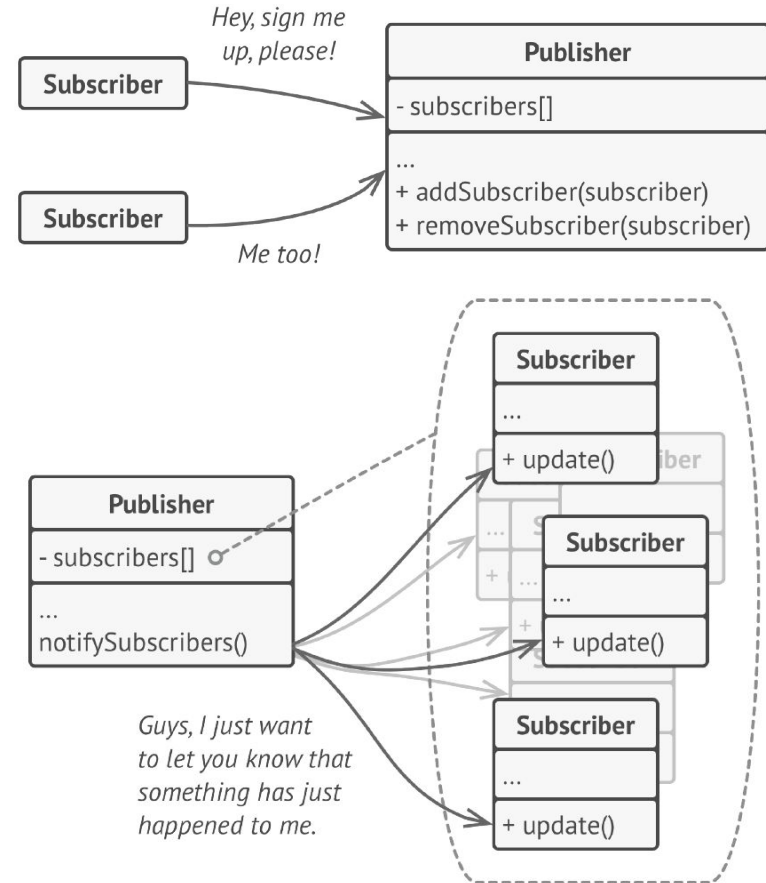
How magazine subscriptions work?

- 1 A newspaper publisher goes into business and begins publishing newspapers.
- 2 You subscribe to a particular publisher, and every time there's a new edition it gets delivered to you. As long as you remain a subscriber, you get new newspapers.
- 3 You unsubscribe when you don't want papers anymore, and they stop being delivered.
- 4 While the publisher remains in business, people, hotels, airlines and other businesses constantly subscribe and unsubscribe to the newspaper.



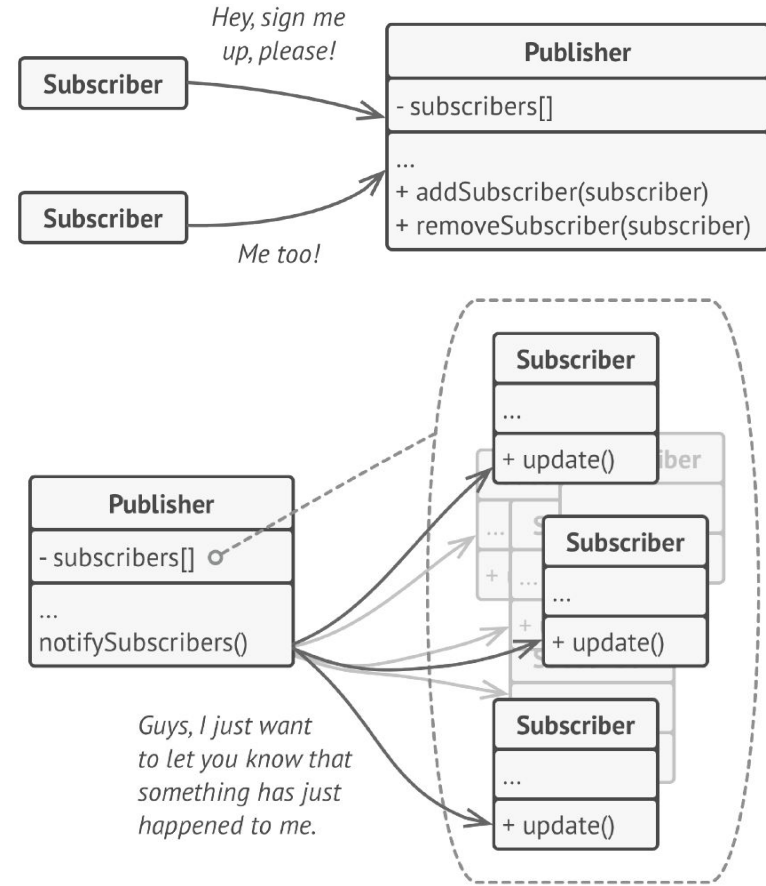
How magazine subscriptions work?

- 1 A newspaper publisher goes into business and begins publishing newspapers.
- 2 You subscribe to a particular publisher, and every time there's a new edition it gets delivered to you. As long as you remain a subscriber, you get new newspapers.
- 3 You unsubscribe when you don't want papers anymore, and they stop being delivered.
- 4 While the publisher remains in business, people, hotels, airlines and other businesses constantly subscribe and unsubscribe to the newspaper.

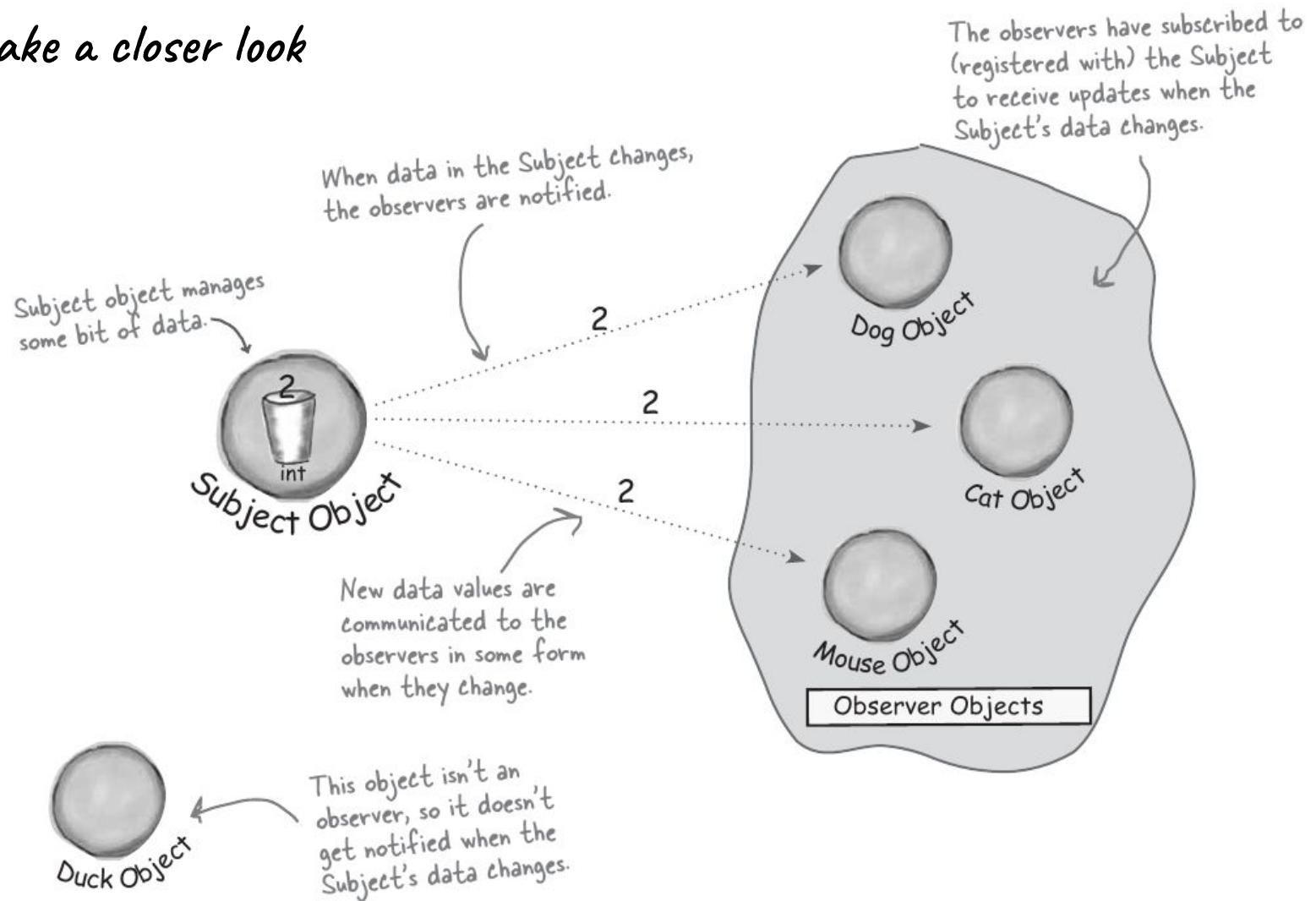


How magazine subscriptions work?

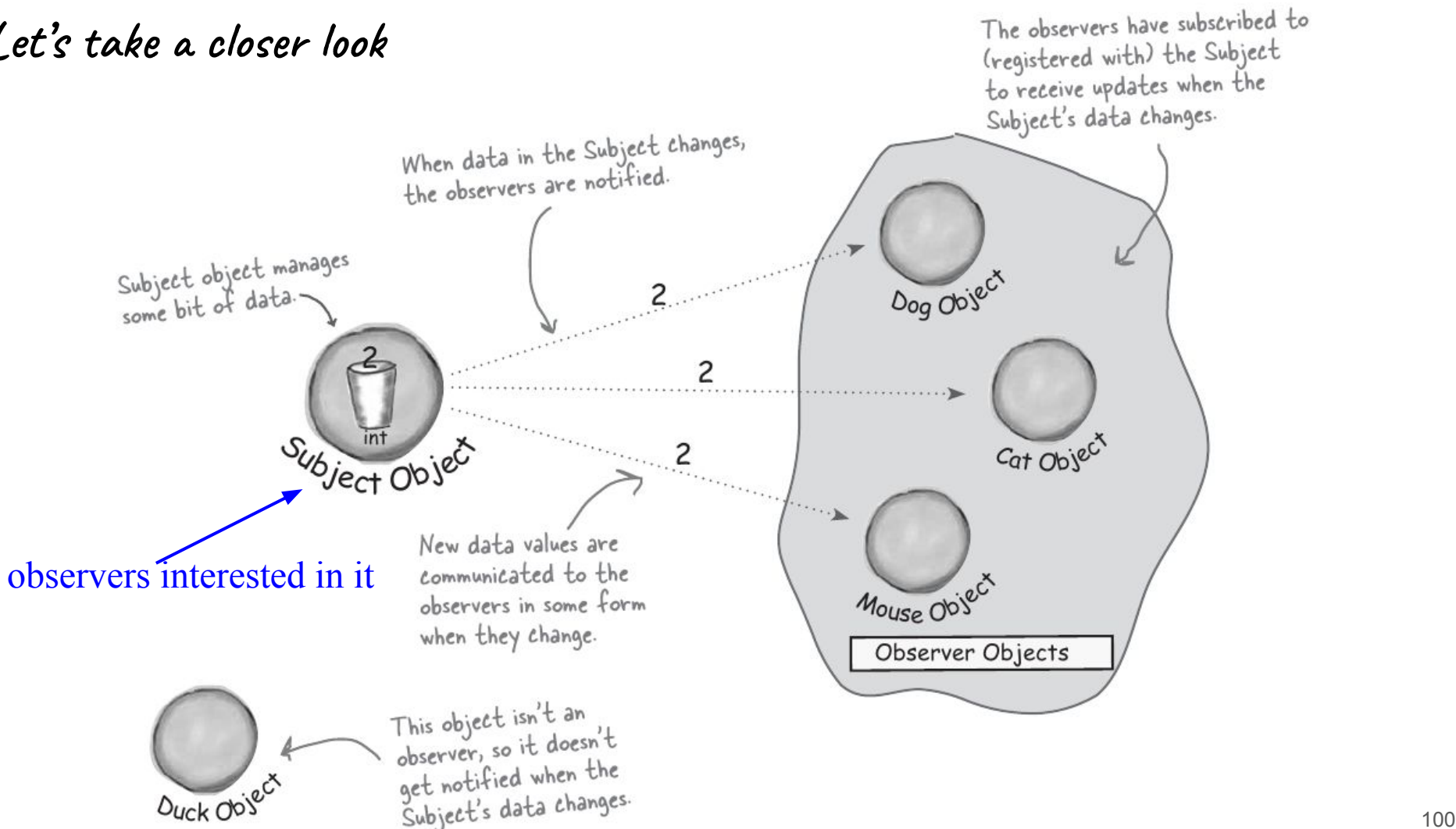
- 1 A newspaper publisher goes into business and begins publishing newspapers.
- 2 You subscribe to a particular publisher, and every time there's a new edition it gets delivered to you. As long as you remain a subscriber, you get new newspapers.
- 3 You unsubscribe when you don't want papers anymore, and they stop being delivered.
- 4 While the publisher remains in business, people, hotels, airlines and other businesses constantly subscribe and unsubscribe to the newspaper.



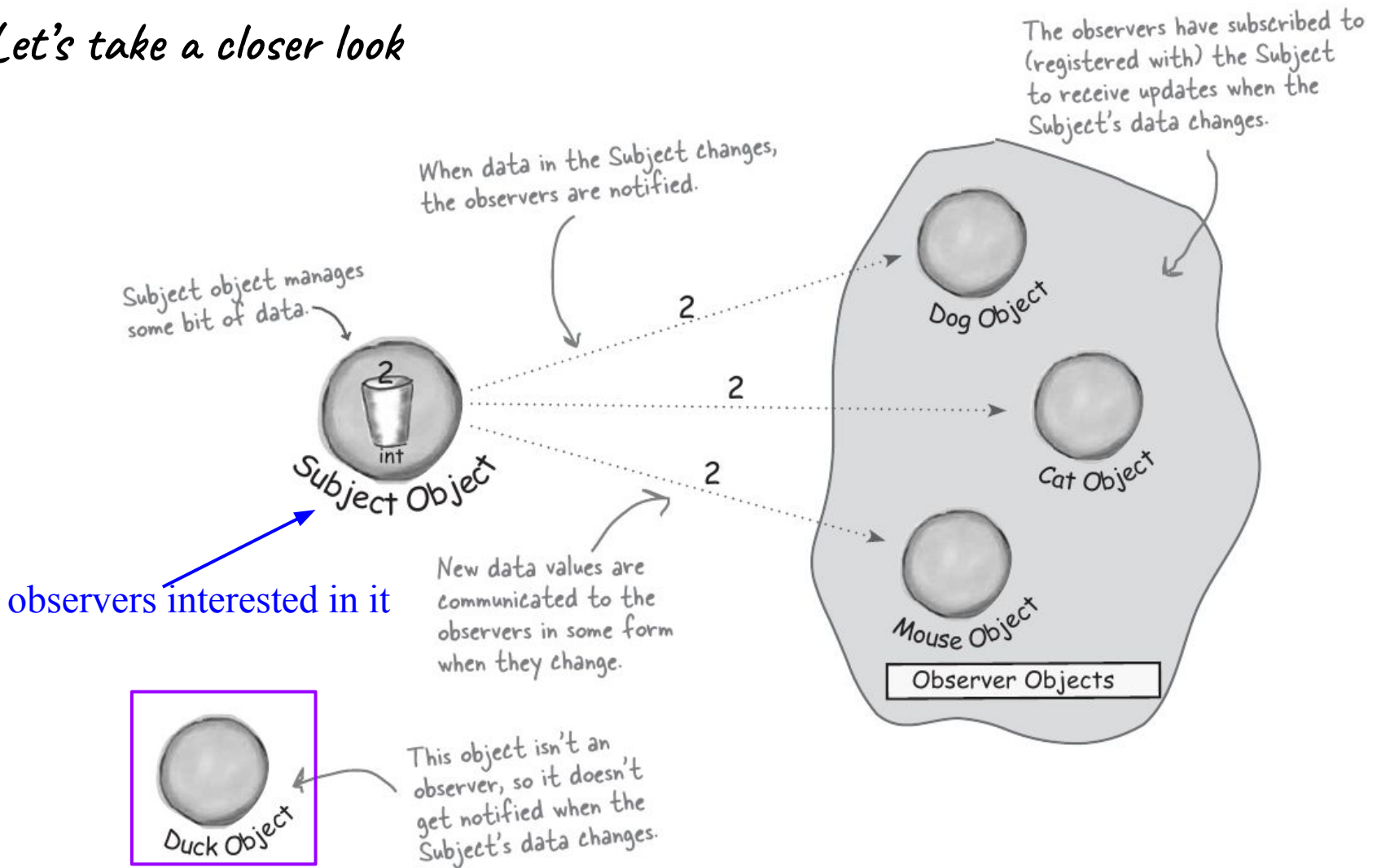
Let's take a closer look



Let's take a closer look



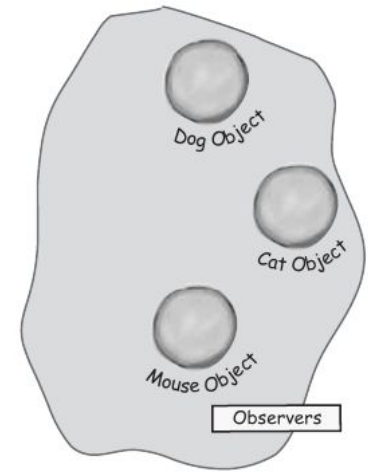
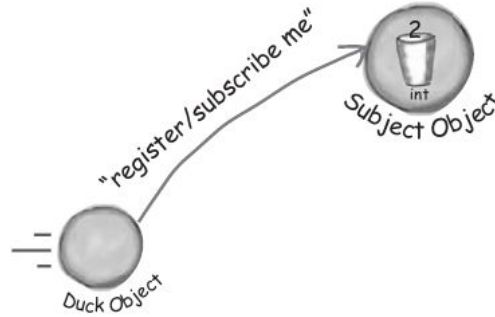
Let's take a closer look



Let's take a closer look

A Duck object comes along and tells the Subject that it wants to become an observer.

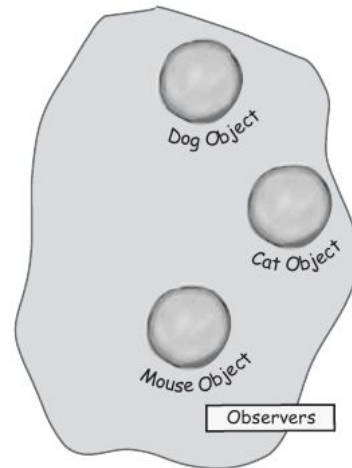
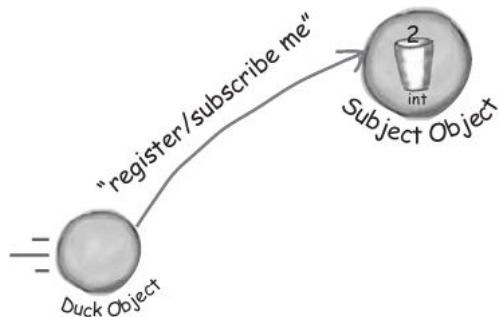
Duck really wants in on the action; those ints Subject is sending out whenever its state changes look pretty interesting...



Let's take a closer look

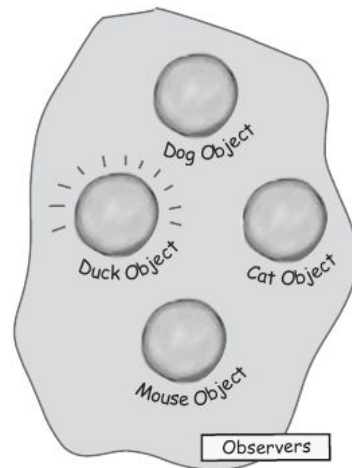
A Duck object comes along and tells the Subject that it wants to become an observer.

Duck really wants in on the action; those ints Subject is sending out whenever its state changes look pretty interesting...



The Duck object is now an official observer.

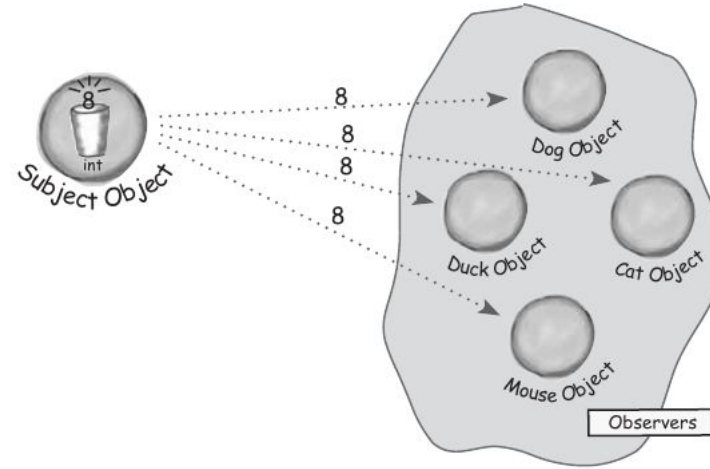
Duck is psyched... he's on the list and is waiting with great anticipation for the next notification so he can get an int.



Let's take a closer look

The Subject gets a new data value!

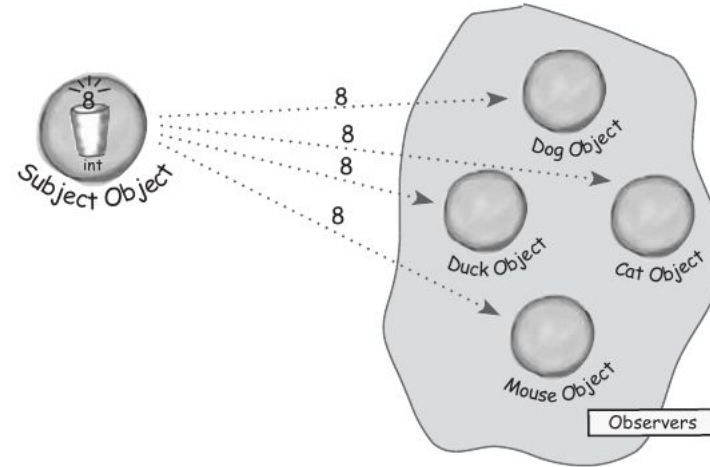
Now Duck and all the rest of the observers get a notification that the Subject has changed.



Let's take a closer look

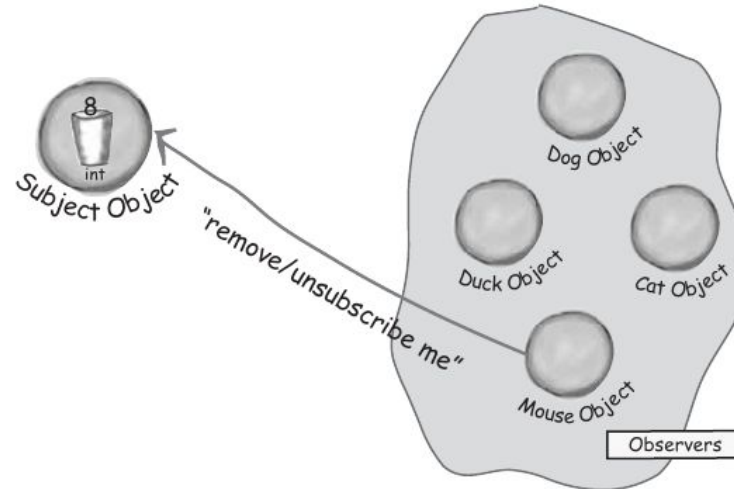
The Subject gets a new data value!

Now Duck and all the rest of the observers get a notification that the Subject has changed.



The Mouse object asks to be removed as an observer.

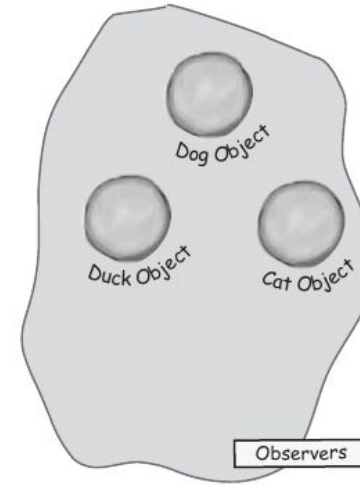
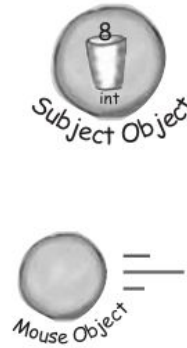
The Mouse object has been getting ints for ages and is tired of it, so it decides it's time to stop being an observer.



Let's take a closer look

Mouse is outta here!

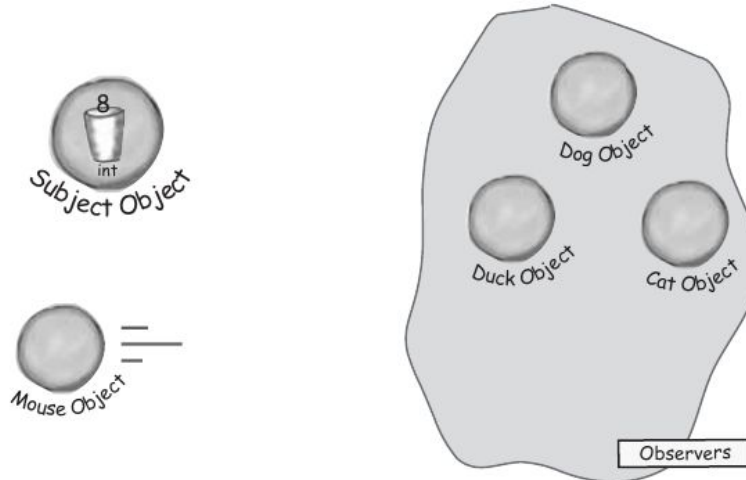
The Subject acknowledges the Mouse's request and removes it from the set of observers.



Let's take a closer look

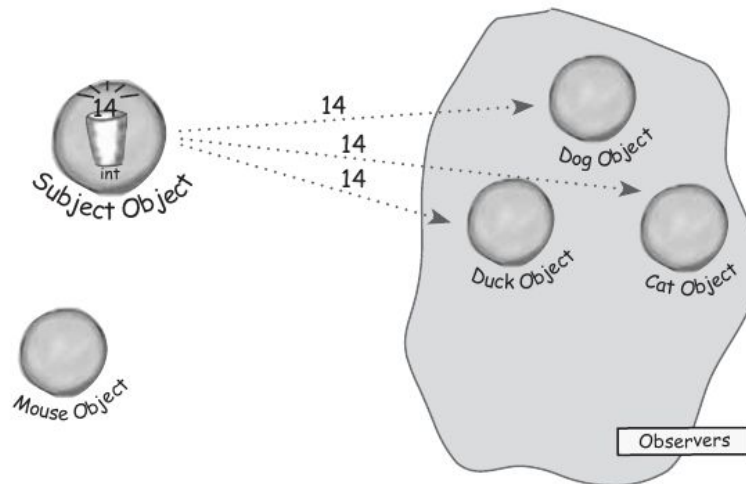
Mouse is outta here!

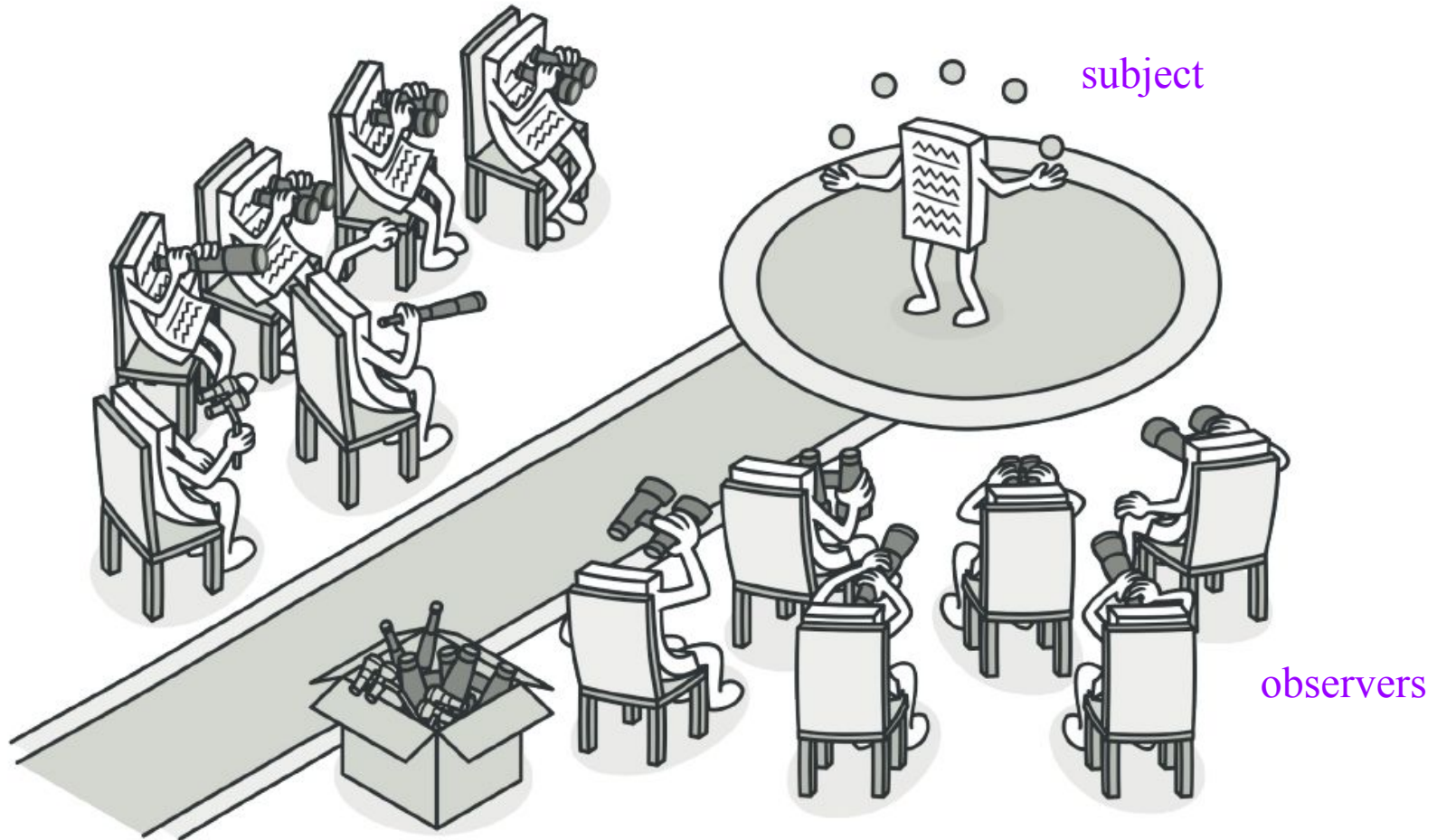
The Subject acknowledges the Mouse's request and removes it from the set of observers.



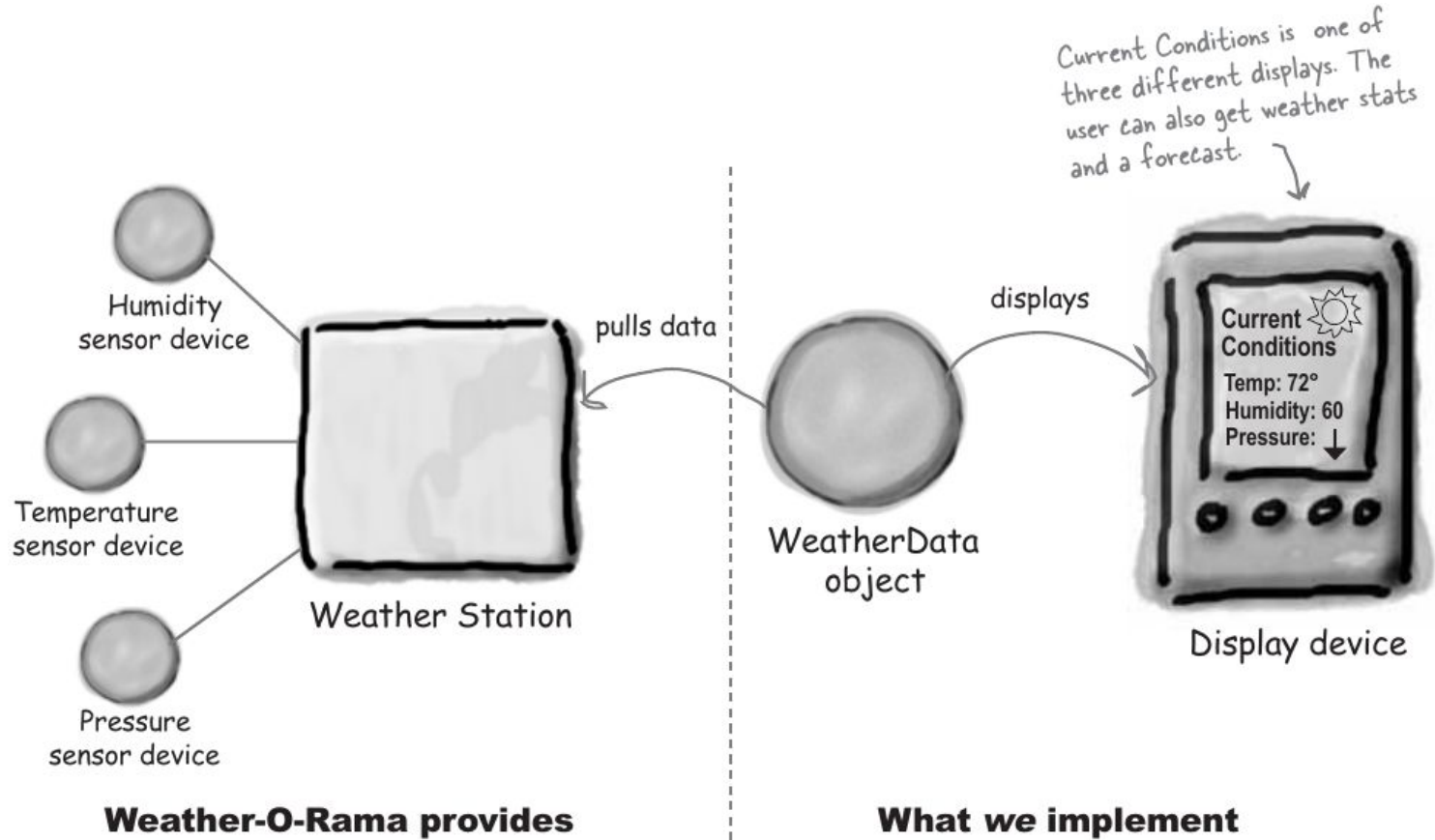
The Subject has another new int.

All the observers get another notification, except for the Mouse who is no longer included. Don't tell anyone, but the Mouse secretly misses those ints... maybe it'll ask to be an observer again some day.



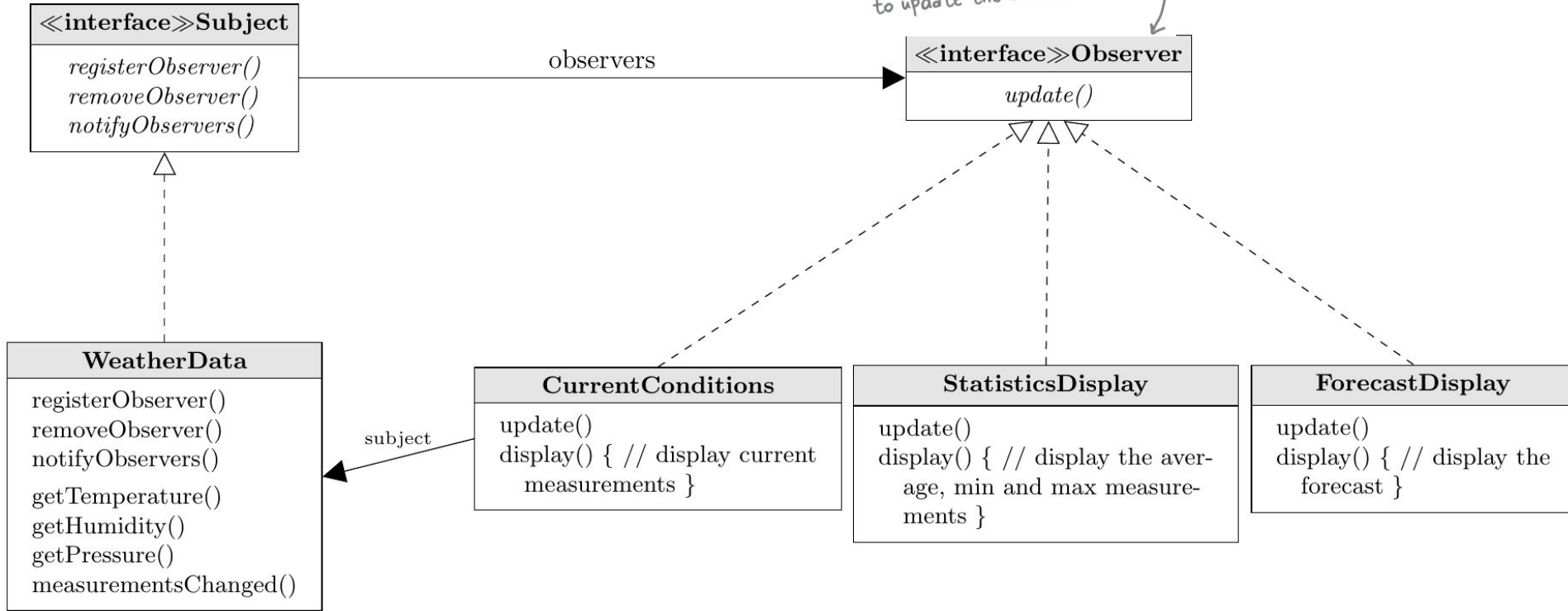


So in weather station...



Our new design

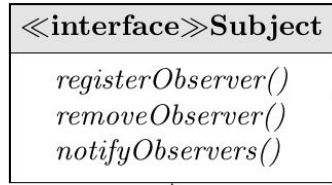
All our weather components implement the Observer interface. This gives the Subject a common interface to talk to when it comes time to update the observers.



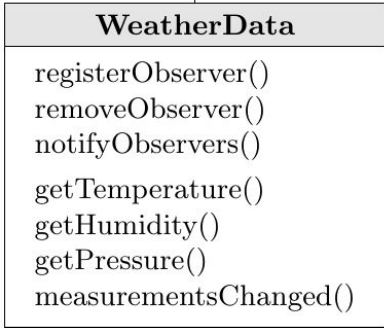
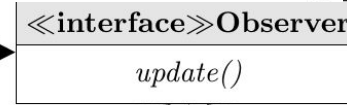
Our new design

list of observers

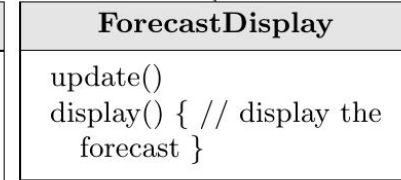
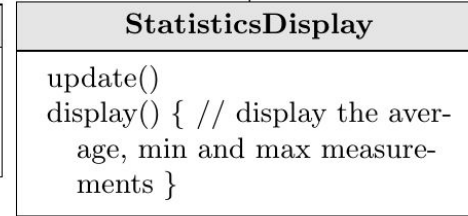
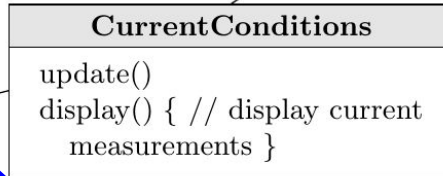
All our weather components implement the Observer interface. This gives the Subject a common interface to talk to when it comes time to update the observers.



observers



subject



Interface referencing WeatherData object

Let's see some code...

```
public interface Subject {  
    public void registerObserver(Observer o);  
    public void removeObserver(Observer o);  
    public void notifyObservers();  
}
```

Both of these methods take an Observer as an argument; that is, the Observer to be registered or removed.

This method is called to notify all observers when the Subject's state has changed.

```
public interface Observer {  
    public void update(float temp, float humidity, float pressure);  
}
```

These are the state values the Observers get from the Subject when a weather measurement changes

The Observer interface is implemented by all observers, so they all have to implement the update() method. Here we're following Mary and Sue's lead and passing the measurements to the observers.

Let's see some code...

```
public class WeatherData implements Subject {  
    private ArrayList observers;  
    private float temperature;  
    private float humidity;  
    private float pressure;
```

WeatherData now implements the Subject interface.

We've added an ArrayList to hold the Observers, and we create it in the constructor.

```
    public WeatherData() {  
        observers = new ArrayList();  
    }
```

Here we implement the Subject Interface.

```
    public void registerObserver(Observer o) {  
        observers.add(o);  
    }
```

When an observer registers, we just add it to the end of the list.

```
    public void removeObserver(Observer o) {  
        int i = observers.indexOf(o);  
        if (i >= 0) {  
            observers.remove(i);  
        }  
    }
```

Likewise, when an observer wants to un-register, we just take it off the list.

```
    public void notifyObservers() {  
        for (int i = 0; i < observers.size(); i++) {  
            Observer observer = (Observer)observers.get(i);  
            observer.update(temperature, humidity, pressure);  
        }  
    }
```

Here's the fun part; this is where we tell all the observers about the state. Because they are all Observers, we know they all implement update(), so we know how to notify them.

```
    public void measurementsChanged() {  
        notifyObservers();  
    }
```

We notify the Observers when we get updated measurements from the Weather Station.

Let's see some code...

```
public class WeatherData implements Subject {  
    private ArrayList observers;  
    private float temperature;  
    private float humidity;  
    private float pressure;  
  
    public WeatherData() {  
        observers = new ArrayList();  
    }  
  
    public void registerObserver(Observer o) {  
        observers.add(o);  
    }  
  
    public void removeObserver(Observer o) {  
        int i = observers.indexOf(o);  
        if (i >= 0) {  
            observers.remove(i);  
        }  
    }  
  
    public void notifyObservers() {  
        for (int i = 0; i < observers.size(); i++) {  
            Observer observer = (Observer)observers.get(i);  
            observer.update(temperature, humidity, pressure);  
        }  
    }  
  
    public void measurementsChanged() {  
        notifyObservers();  
    }  
}
```

WeatherData now implements the Subject interface.

We've added an ArrayList to hold the Observers, and we create it in the constructor.

When an observer registers, we just add it to the end of the list.

Likewise, when an observer wants to un-register, we just take it off the list.

Here's the fun part; this is where we tell all the observers about the state. Because they are all Observers, we know they all implement update(), so we know how to notify them.

We notify the Observers when we get updated measurements from the Weather Station.

Here we implement the Subject Interface.

Let's see some code...

```
public class WeatherData implements Subject {  
    private ArrayList observers;  
    private float temperature;  
    private float humidity;  
    private float pressure;  
  
    public WeatherData() {  
        observers = new ArrayList();  
    }  
  
    public void registerObserver(Observer o) {  
        observers.add(o);  
    }  
  
    public void removeObserver(Observer o) {  
        int i = observers.indexOf(o);  
        if (i >= 0) {  
            observers.remove(i);  
        }  
    }  
  
    public void notifyObservers() {  
        for (int i = 0; i < observers.size(); i++) {  
            Observer observer = (Observer)observers.get(i);  
            observer.update(temperature, humidity, pressure);  
        }  
    }  
  
    public void measurementsChanged() {  
        notifyObservers();  
    }  
}
```

WeatherData now implements the Subject interface.

We've added an ArrayList to hold the Observers, and we create it in the constructor.

When an observer registers, we just add it to the end of the list.

Likewise, when an observer wants to un-register, we just take it off the list.

Here's the fun part; this is where we tell all the observers about the state. Because they are all Observers, we know they all implement update(), so we know how to notify them.

We notify the Observers when we get updated measurements from the Weather Station.

Here we implement the Subject Interface.

Let's see some code...

```
public class WeatherData implements Subject {  
    private ArrayList observers;  
    private float temperature;  
    private float humidity;  
    private float pressure;  
  
    public WeatherData() {  
        observers = new ArrayList();  
    }  
  
    public void registerObserver(Observer o) {  
        observers.add(o);  
    }  
  
    public void removeObserver(Observer o) {  
        int i = observers.indexOf(o);  
        if (i >= 0) {  
            observers.remove(i);  
        }  
    }  
  
    public void notifyObservers() {  
        for (int i = 0; i < observers.size(); i++) {  
            Observer observer = (Observer)observers.get(i);  
            observer.update(temperature, humidity, pressure);  
        }  
    }  
  
    public void measurementsChanged() {  
        notifyObservers();  
    }  
}
```

WeatherData now implements the Subject interface.

We've added an ArrayList to hold the Observers, and we create it in the constructor.

When an observer registers, we just add it to the end of the list.

Likewise, when an observer wants to un-register, we just take it off the list.

Here's the fun part; this is where we tell all the observers about the state. Because they are all Observers, we know they all implement update(), so we know how to notify them.

We notify the Observers when we get updated measurements from the Weather Station.

Here we implement the Subject Interface.

Let's see some code...

*This display implements Observer
so it can get changes from the
WeatherData object.*



```
public class CurrentConditionsDisplay implements Observer  
    private float temperature;  
    private float humidity;  
    private Subject weatherData;
```

```
    public CurrentConditionsDisplay(Subject weatherData) {  
        this.weatherData = weatherData;  
        weatherData.registerObserver(this);  
    }
```

*The constructor is passed the
weatherData object (the Subject)
and we use it to register the
display as an observer.*



```
    public void update(float temperature, float humidity, float pressure) {  
        this.temperature = temperature;  
        this.humidity = humidity;  
        display();  
    }
```

*When update() is called, we
save the temp and humidity
and call display().*



```
    public void display() {  
        System.out.println("Current conditions: " + temperature  
            + "F degrees and " + humidity + "% humidity");  
    }  
}
```

*The display() method
just prints out the most
recent temp and humidity.*



Let's see some code...

*This display implements Observer
so it can get changes from the
WeatherData object.*

```
public class CurrentConditionsDisplay implements Observer  
    private float temperature;  
    private float humidity;  
    private Subject weatherData;
```

```
    public CurrentConditionsDisplay(Subject weatherData) {  
        this.weatherData = weatherData;  
        weatherData.registerObserver(this);  
    }
```

*The constructor is passed the
weatherData object (the Subject)
and we use it to register the
display as an observer.*

```
    public void update(float temperature, float humidity, float pressure) {  
        this.temperature = temperature;  
        this.humidity = humidity;  
        display();  
    }
```

passed in constructor

*When update() is called, we
save the temp and humidity
and call display().*

```
    public void display() {  
        System.out.println("Current conditions: " + temperature  
            + "F degrees and " + humidity + "% humidity");  
    }  
}
```

*The display() method
just prints out the most
recent temp and humidity.*

Let's see some code...

*This display implements Observer
so it can get changes from the
WeatherData object.*

```
public class CurrentConditionsDisplay implements Observer  
    private float temperature;  
    private float humidity;  
    private Subject weatherData;
```

```
    public CurrentConditionsDisplay(Subject weatherData) {  
        this.weatherData = weatherData;  
        weatherData.registerObserver(this);  
    }
```

*The constructor is passed the
weatherData object (the Subject)
and we use it to register the
display as an observer.*

```
    public void update(float temperature, float humidity, float pressure) {  
        this.temperature = temperature;  
        this.humidity = humidity;  
        display();  
    }
```

*When update() is called, we
save the temp and humidity
and call display().*

```
    public void display() {  
        System.out.println("Current conditions: " + temperature  
            + "F degrees and " + humidity + "% humidity");  
    }
```

*The display() method
just prints out the most
recent temp and humidity.*

How it registers itself

Let's see some code...

*This display implements Observer
so it can get changes from the
WeatherData object.*

```
public class CurrentConditionsDisplay implements Observer {
    private float temperature;
    private float humidity;
    private Subject weatherData;
```

```
    public CurrentConditionsDisplay(Subject weatherData) {
        this.weatherData = weatherData;
        weatherData.registerObserver(this);
    }
```

*The constructor is passed the
weatherData object (the Subject)
and we use it to register the
display as an observer.*

```
    public void update(float temperature, float humidity, float pressure) {
        this.temperature = temperature;
        this.humidity = humidity;
        display();
    }
```

display() called

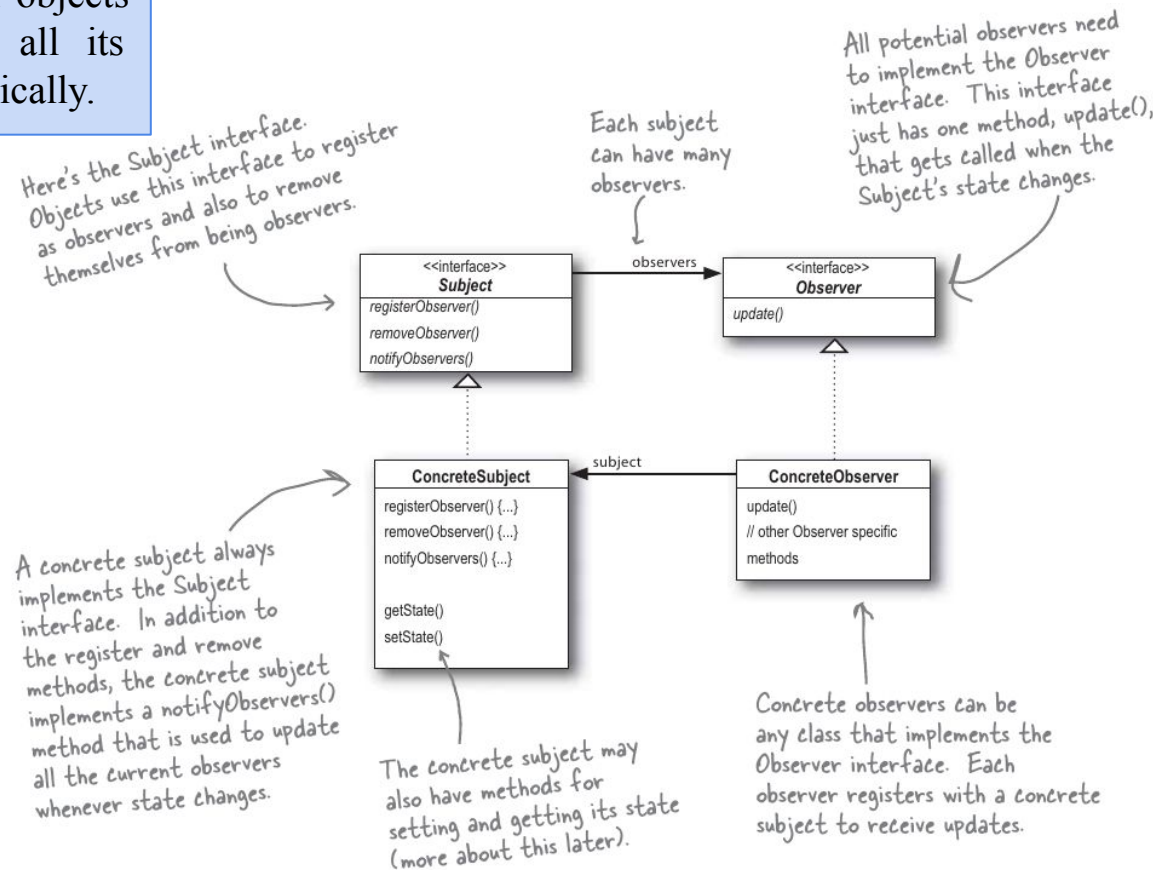
*When update() is called, we
save the temp and humidity
and call display().*

```
    public void display() {
        System.out.println("Current conditions: " + temperature
            + "F degrees and " + humidity + "% humidity");
    }
}
```

*The display() method
just prints out the most
recent temp and humidity.*

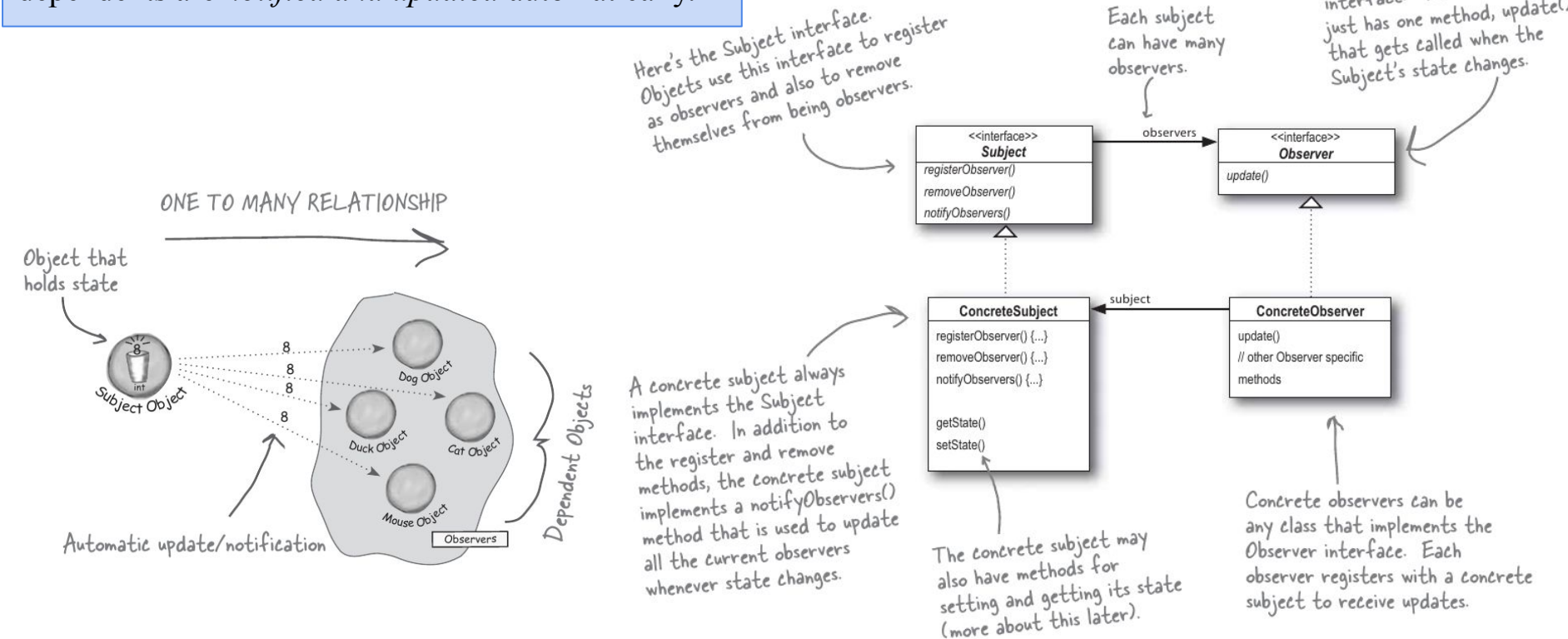
Observer Pattern Defined

Define a **one-to-many** dependency between objects so that when one object **changes state**, all its dependents are *notified and updated* automatically.

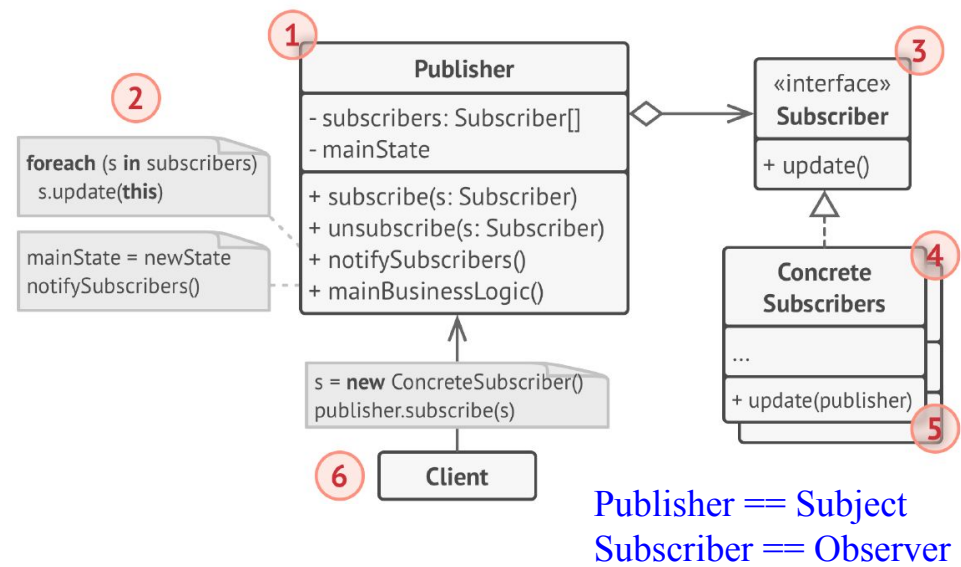


Observer Pattern Defined

Define a **one-to-many** dependency between objects so that when one object **changes state**, all its dependents are *notified and updated* automatically.

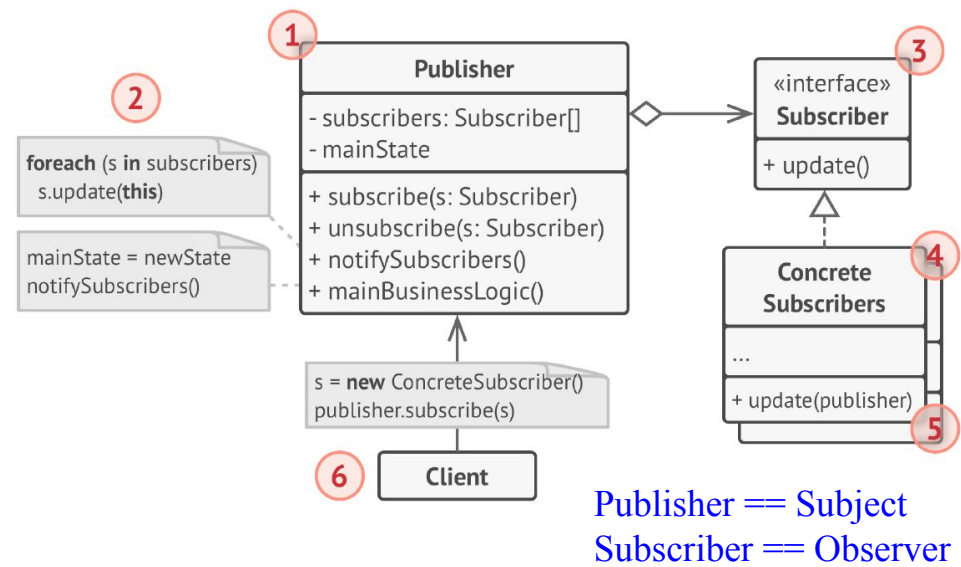


Observer Pattern Defined



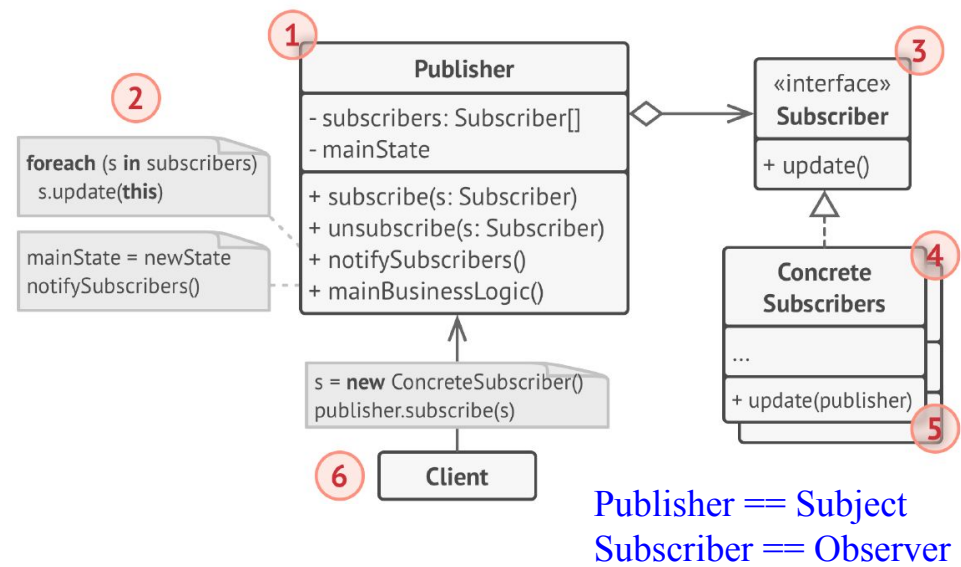
Observer Pattern Defined

- 1. The **Publisher** issues events of interest to other objects. These events occur when the publisher changes its state or executes some behaviors. Publishers contain a subscription infrastructure that lets new subscribers join and current subscribers leave the list.
- 2. When a new event happens, the publisher goes over the subscription list and calls the notification method declared in the subscriber interface on each subscriber object.
- 3. The **Subscriber** interface declares the notification interface. In most cases, it consists of a single `update` method. The method may have several parameters that let the publisher pass some event details along with the update.



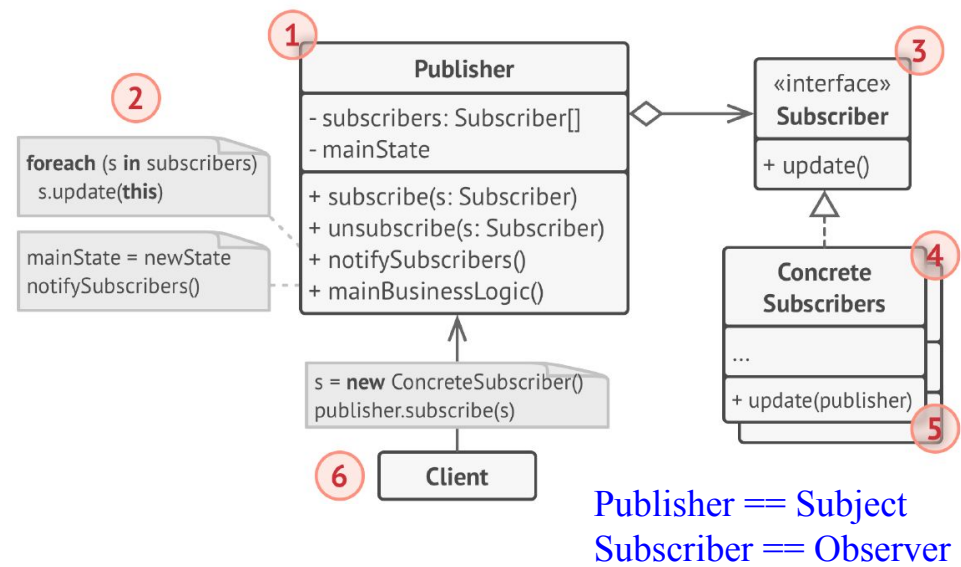
Observer Pattern Defined

- 1. The **Publisher** issues events of interest to other objects. These events occur when the publisher changes its state or executes some behaviors. Publishers contain a subscription infrastructure that lets new subscribers join and current subscribers leave the list.
- 2. When a new event happens, the publisher goes over the subscription list and calls the notification method declared in the subscriber interface on each subscriber object.
- 3. The **Subscriber** interface declares the notification interface. In most cases, it consists of a single `update` method. The method may have several parameters that let the publisher pass some event details along with the update.



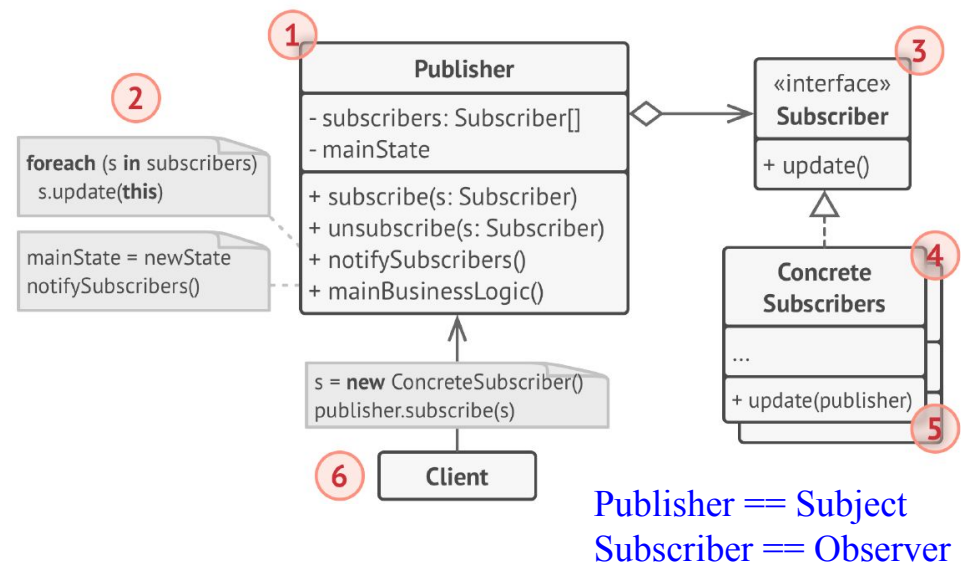
Observer Pattern Defined

- 1. The **Publisher** issues events of interest to other objects. These events occur when the publisher changes its state or executes some behaviors. Publishers contain a subscription infrastructure that lets new subscribers join and current subscribers leave the list.
- 2. When a new event happens, the publisher goes over the subscription list and calls the notification method declared in the subscriber interface on each subscriber object.
- 3. The **Subscriber** interface declares the notification interface. In most cases, it consists of a single `update` method. The method may have several parameters that let the publisher pass some event details along with the update.



Observer Pattern Defined

- 4. **Concrete Subscribers** perform some actions in response to notifications issued by the publisher. All of these classes must implement the same interface so the publisher isn't coupled to concrete classes.
- 5. Usually, subscribers need some contextual information to handle the update correctly. For this reason, publishers often pass some context data as arguments of the notification method. The publisher can pass itself as an argument, letting subscriber fetch any required data directly.
- 6. The **Client** creates publisher and subscriber objects separately and then registers subscribers for publisher updates.

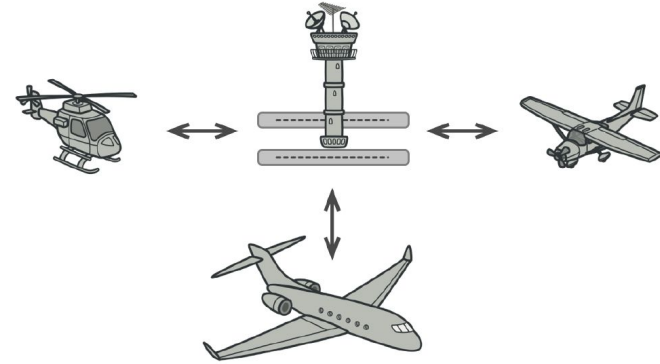




Air traffic control

During the landing of aircrafts -

- ❑ Aircraft pilots **don't** talk to each other directly
- ❑ All communication goes through the **control tower**.



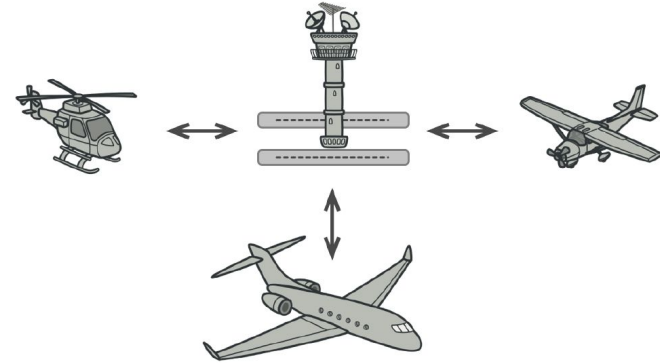
Air traffic control

During the landing of aircrafts -

- ❑ Aircraft pilots **don't** talk to each other directly
- ❑ All communication goes through the **control tower**.

Without control tower -

- ❖ Pilots need to be aware of *every other* aircrafts
- ❖ Airplane crash statistics would probably *skyrocket*



Now let's discuss UI design for a customer profile management

A dialog for creating and editing customer profiles.

Edit User Profile

Account Type

☒ Personal ☐ Business

Company:

Select Business

* Enabled when Business is selected

Personal Info

Full Name:

John Doe

 ✓

Email:

john@email

 ✗

* user@domain.com

Phone:

(555) 123-4567

 ✓

Gender:

☒ Male ☐ Female ☐ Other

Birth Date:

03/15/1985

 ✓

MM/DD/YYYY

Preferences

☒ Subscribe to newsletter ☐ Dark mode

☒ Show advanced options

Advanced

API Key:

sk-xxxx-xxxx

 ✓

Timeout:

30

 seconds ☒ Enable logging

Address

Street:

123 Main St

 ✓

City:

Springfield

 State:

IL

ZIP:

6270

 ✗

5 or 9 digits

☒ Use different shipping address

Shipping Address

Street:

456 Oak Avenue

City:

Chicago

 State:

IL

 ZIP:

60601

Notifications

☒ Enable notifications

Notify via:

☒ Email ☐ SMS ☐ Both

Frequency:

Daily

☒ Updates ☒ Promos ☐ Security ☒ News

Two-Factor Auth

☐ Enable 2FA

Method:

☐ SMS ☐ Auth App ☐ Email

✗ 3 validation errors. Please fix before submitting.

Help

OK

Cancel

Apply

Generated using LaTeX. Thanks to Claude

Now let's discuss UI design for a customer profile management

A dialog for creating and editing customer profiles.

- Various form controls such as text fields, checkboxes, buttons etc.
- Some interact with others

Edit User Profile

Account Type

☒ Personal

☐ Business

Company:

Select Business

* Enabled when Business is selected

Personal Info

Full Name:

John Doe

 ✓

Email:

john@email

 ✗

* user@domain.com

Phone:

(555) 123-4567

 ✓

Gender:

☒ Male

☐ Female

☐ Other

Birth Date:

03/15/1985

 ✓

MM/DD/YYYY

Preferences

☒ Subscribe to newsletter

☐ Dark mode

☒ Show advanced options

Advanced

API Key:

sk-xxxx-xxxx

 ✓

Timeout:

30

 seconds ☒ Enable logging

Address

Street:

123 Main St

 ✓

City:

Springfield

 State:

IL

ZIP:

6270

 ✗

5 or 9 digits

☒ Use different shipping address

Shipping Address

Street:

456 Oak Avenue

City:

Chicago

 State:

IL

 ZIP:

60601

Notifications

☒ Enable notifications

Notify via:

☒ Email

☐ SMS

☐ Both

Frequency:

Daily

☒ Updates

☒ Promos

☐ Security

☒ News

Two-Factor Auth

☐ Enable 2FA

Method:

☐ SMS

☐ Auth App

☐ Email

✗ 3 validation errors. Please fix before submitting.

Help

OK

Cancel

Apply

Generated using LaTeX. Thanks to Claude

Now let's discuss UI design for a customer profile management

A dialog for creating and editing customer profiles.

- Various form controls such as text fields, checkboxes, buttons etc.
- Some interact with others
- Ex: selecting “Business” enables company name

Edit User Profile

Account Type

☒ Personal

☐ Business

Company:

Select Business

* Enabled when Business is selected

Personal Info

Full Name:

John Doe

✓

Email:

john@email

✗

* user@domain.com

Phone:

(555) 123-4567

✓

Gender:

☒ Male

☐ Female

☐ Other

Birth Date:

03/15/1985

✓

MM/DD/YYYY

Preferences

☒ Subscribe to newsletter

☐ Dark mode

☒ Show advanced options

Advanced

API Key:

sk-xxxx-xxxx

✓

Timeout:

30

seconds

☒ Enable logging

Address

Street:

123 Main St

✓

City:

Springfield

State:

IL

ZIP:

6270

✗ 5 or 9 digits

☒ Use different shipping address

Shipping Address

Street:

456 Oak Avenue

City:

Chicago

State:

IL

ZIP:

60601

Notifications

☒ Enable notifications

Notify via:

☒ Email

☐ SMS

☐ Both

Frequency:

Daily

☒ Updates

☒ Promos

☐ Security

☒ News

Two-Factor Auth

☐ Enable 2FA

Method:

☐ SMS

☐ Auth App

☐ Email

✗ 3 validation errors. Please fix before submitting.

Help

OK

Cancel

Apply

Generated using LaTeX. Thanks to Claude

Now let's discuss UI design for a customer profile management

A dialog for creating and editing customer profiles.

- Various form controls such as text fields, checkboxes, buttons etc.
- Some interact with others
- Ex: “Apply” validates everything

Edit User Profile

Account Type

☒ Personal ☐ Business

Company:

* Enabled when Business is selected

Personal Info

Full Name: ✓

Email: ✗
* user@domain.com

Phone: ✓

Gender: ☒ Male ☐ Female ☐ Other

Birth Date: ✓ MM/DD/YYYY

Preferences

☒ Subscribe to newsletter ☐ Dark mode

☒ Show advanced options

Advanced

API Key: ✓

Timeout: seconds ☒ Enable logging

Address

Street: ✓

City: State:

ZIP: ✗ 5 or 9 digits

☒ Use different shipping address

Shipping Address

Street:

City: State: ZIP:

Notifications

☒ Enable notifications

Notify via: ☒ Email ☐ SMS ☐ Both

Frequency:

☒ Updates ☒ Promos ☐ Security ☒ News

Two-Factor Auth

☐ Enable 2FA

Method: ☐ SMS ☐ Auth App ☐ Email

✗ 3 validation errors. Please fix before submitting.

Help

OK

Cancel

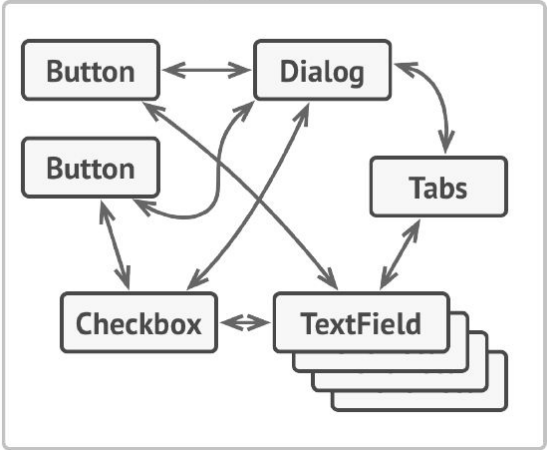
Apply

Now let's discuss UI design for a customer profile management

A dialog for creating and editing customer profiles.

- Various form controls such as text fields, checkboxes, buttons etc.
- Some interact with others

 Profile Dialog



Edit User Profile

Account Type

☒ Personal ☐ Business

Company:

* Enabled when Business is selected

Personal Info

Full Name: ✓

Email: ✗
* user@domain.com

Phone: ✓

Gender: ☒ Male ☐ Female ☐ Other

Birth Date: ✓ MM/DD/YYYY

Preferences

☒ Subscribe to newsletter ☐ Dark mode

☒ Show advanced options

Advanced

API Key: ✓

Timeout: seconds ☒ Enable logging

Address

Street: ✓

City: State:

ZIP: ✗ 5 or 9 digits

☒ Use different shipping address

Shipping Address

Street:

City: State: ZIP:

Notifications

☒ Enable notifications

Notify via: ☒ Email ☐ SMS ☐ Both

Frequency:

☒ Updates ☒ Promos ☐ Security ☒ News

Two-Factor Auth

☐ Enable 2FA

Method: ☐ SMS ☐ Auth App ☐ Email

✗ 3 validation errors. Please fix before submitting.

Help

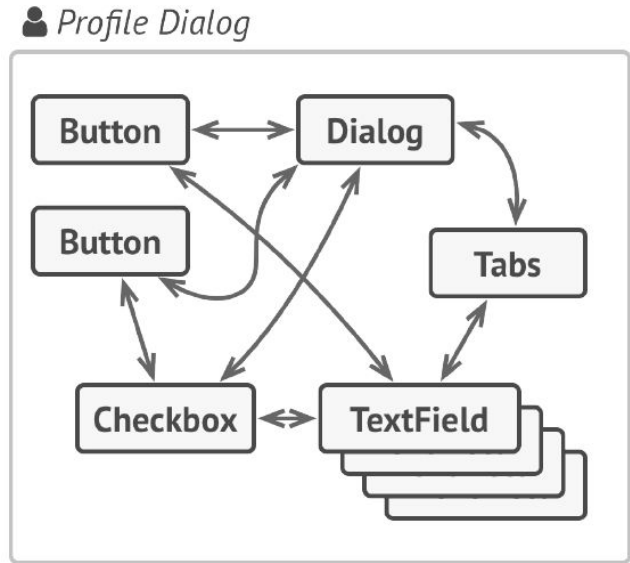
OK

Cancel

Apply

What we may think of doing initially...

Inside the elements (i.e in their class) implement interaction the logics directly.



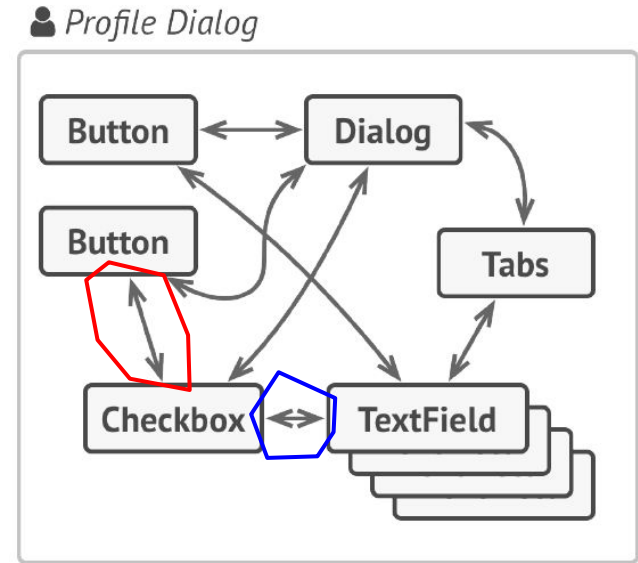
What we may think of doing initially...

Inside the elements (i.e in their class) implement interaction the logics directly.

Example:

In the checkbox -

- Logic 1 for interaction with the **button**
- Logic 2 for interaction with the **textfield**



What we may think of doing initially...

Inside the elements (i.e in their class) implement interaction the logics directly.

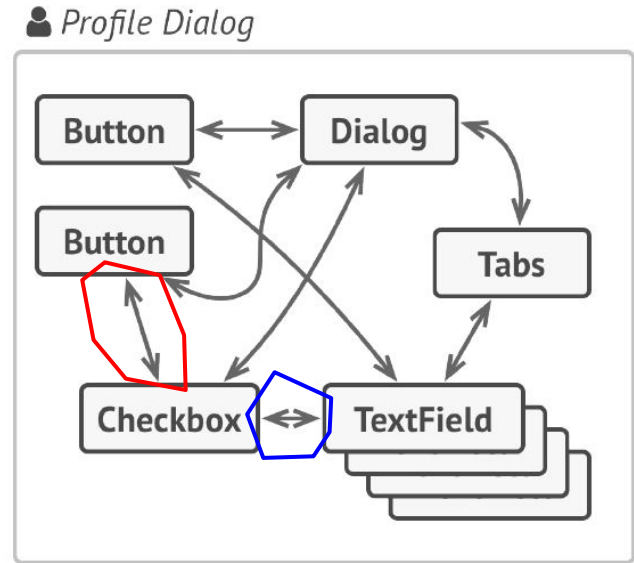
Example:

In the checkbox -

- Logic 1 for interaction with the **button**
- Logic 2 for interaction with the **textfield**

Problem -

Can we use the checkbox class in another **form**?



What we may think of doing initially...

Inside the elements (i.e in their class) implement interaction the logics directly.

Example:

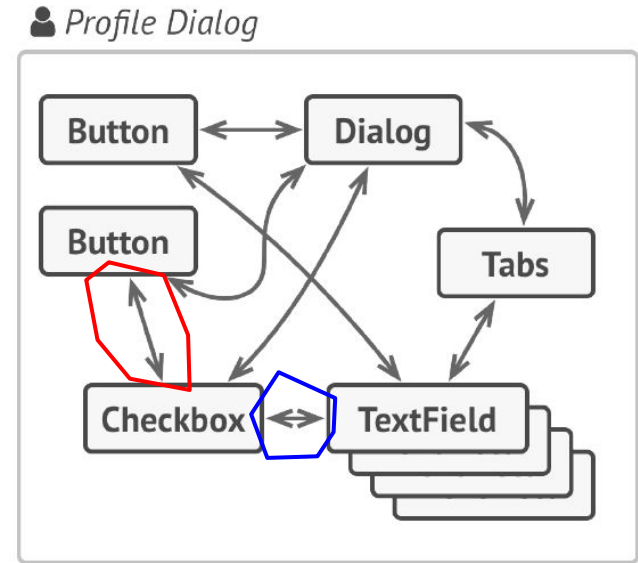
In the checkbox -

- Logic 1 for interaction with the **button**
- Logic 2 for interaction with the **textfield**

Problem -

Can we use the checkbox class in another **form**?

- **NO**. As it uses 1 button, 1 textfield, 1 dialogue



What we may think of doing initially...

Inside the elements (i.e in their class) implement interaction the logics directly.

Example:

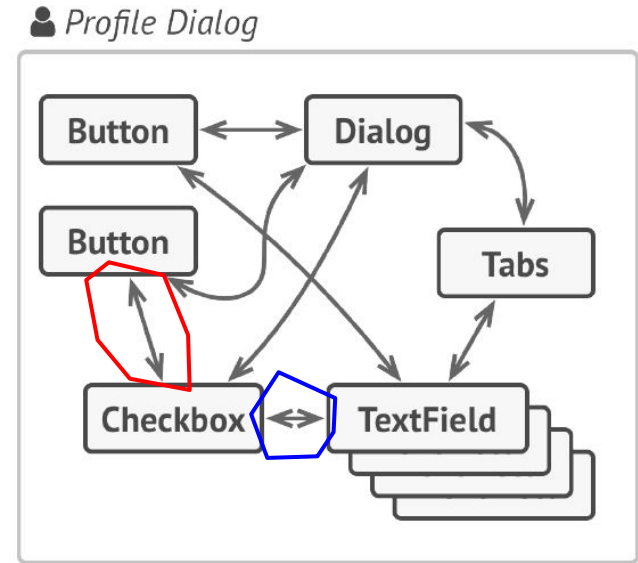
In the checkbox -

- Logic 1 for interaction with the **button**
- Logic 2 for interaction with the **textfield**

Problem -

Can we use the checkbox class in another **form**?

- **NO**. As it uses 1 button, 1 textfield, 1 dialogue
(**coupling**)



Solution?

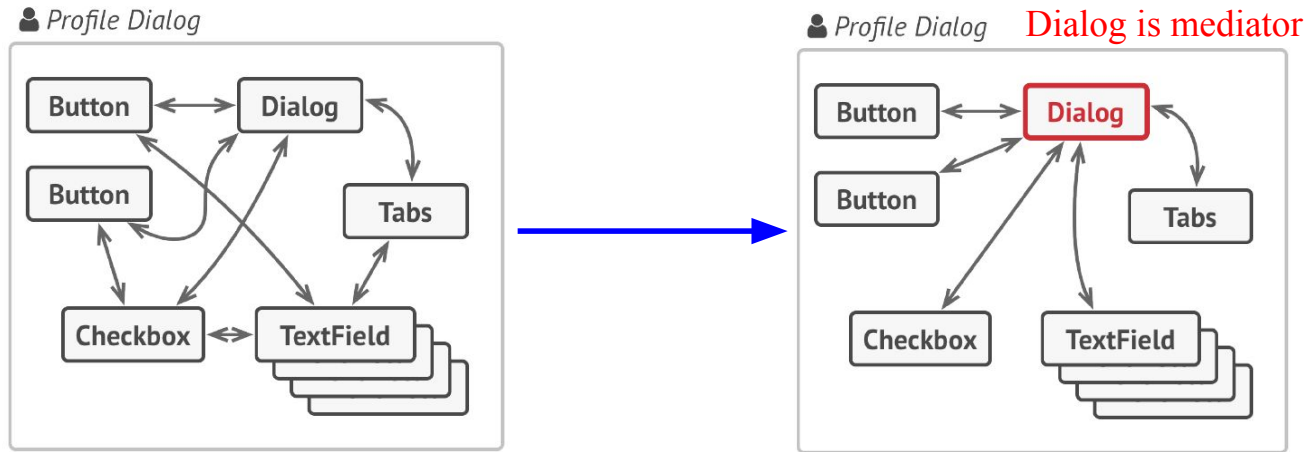
- ❑ Cease all direct communication between the components

Solution?

- ❑ **Cease** all **direct** communication between the components
- ❑ Must collaborate indirectly, through a **mediator**, who redirects call to appropriate components

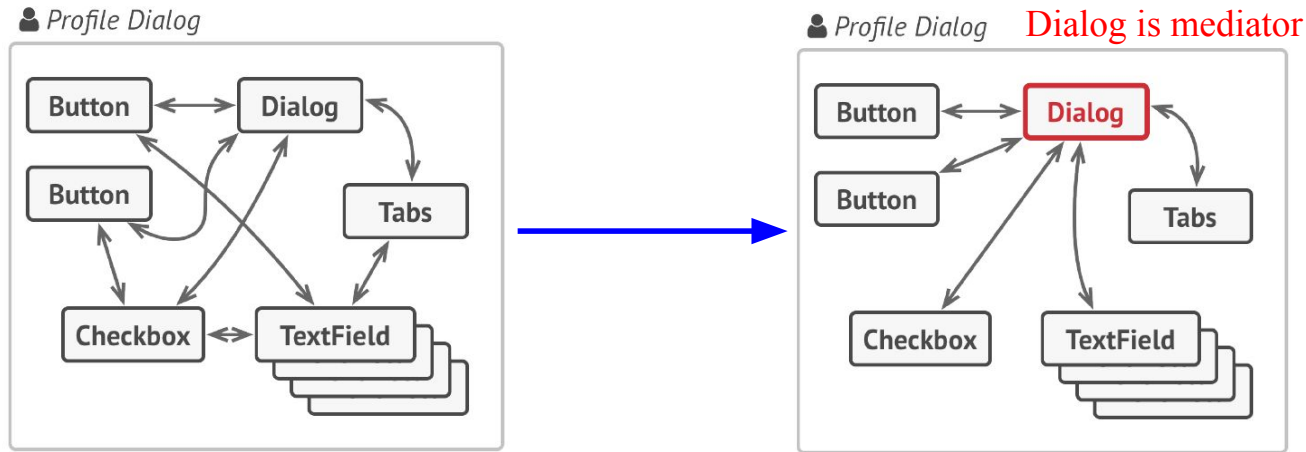
Solution?

- ❑ **Cease** all **direct** communication between the components
- ❑ Must collaborate indirectly, through a **mediator**, who redirects call to appropriate components



Solution?

- ❑ **Cease** all **direct** communication between the components
- ❑ Must collaborate indirectly, through a **mediator**, who redirects call to appropriate components



Components depend only on a **single mediator** class instead of being coupled to **dozens** of their **colleagues**.

So, in UI design...

- ❑ Dialogue class is the mediator.

Edit User Profile

Account Type

☒ Personal ☐ Business

Company: * Enabled when Business is selected

Personal Info

Full Name: ✓

Email: ✗ * user@domain.com

Phone: ✓

Gender: ☒ Male ☐ Female ☐ Other

Birth Date: ✓ MM/DD/YYYY

Address

Street: ✓

City: State:

ZIP: ✗ 5 or 9 digits

☒ Use different shipping address

Shipping Address

Street:

City: State: ZIP:

Notifications

☒ Enable notifications

Notify via: ☒ Email ☐ SMS ☐ Both

Frequency:

☒ Updates ☒ Promos ☐ Security ☒ News

Two-Factor Auth

☐ Enable 2FA

Method: ☐ SMS ☐ Auth App ☐ Email

Preferences

☒ Subscribe to newsletter ☐ Dark mode

☒ Show advanced options

Advanced

API Key: ✓

Timeout: seconds ☒ Enable logging

✗ 3 validation errors. Please fix before submitting.

Help OK Cancel Apply

So, in UI design...

- ❑ Dialogue class is the mediator.
- ❑ Dialogue should know all of its sub-elements (components)

Edit User Profile

Account Type

☒ Personal ☐ Business

Company: * Enabled when Business is selected

Personal Info

Full Name: ✓

Email: ✗ * user@domain.com

Phone: ✓

Gender: ☒ Male ☐ Female ☐ Other

Birth Date: ✓ MM/DD/YYYY

Preferences

☒ Subscribe to newsletter ☐ Dark mode

☒ Show advanced options

Advanced

API Key: ✓

Timeout: seconds ☒ Enable logging

Address

Street: ✓

City: State:

ZIP: ✗ 5 or 9 digits

☒ Use different shipping address

Shipping Address

Street:

City: State: ZIP:

Notifications

☒ Enable notifications

Notify via: ☒ Email ☐ SMS ☐ Both

Frequency:

☒ Updates ☒ Promos ☐ Security ☒ News

Two-Factor Auth

☐ Enable 2FA

Method: ☐ SMS ☐ Auth App ☐ Email

✗ 3 validation errors. Please fix before submitting.

Help OK Cancel Apply

So, in UI design...

- ❑ Dialogue class is the mediator.
- ❑ Dialogue should know all of its sub-elements (components)
 - No new dependency

The screenshot shows a 'Edit User Profile' dialog box with a blue title bar and standard window controls. The form is organized into several sections:

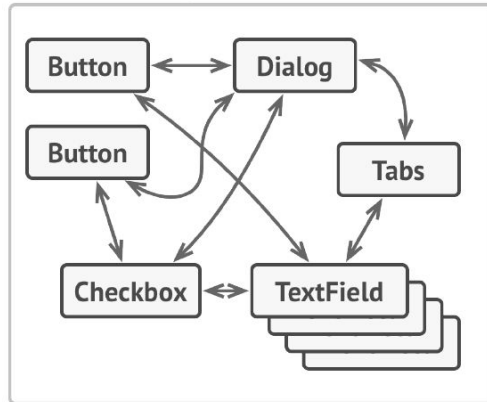
- Account Type:** Radio buttons for 'Personal' (selected) and 'Business'. A 'Company:' field contains 'Select Business' with a note '* Enabled when Business is selected'.
- Personal Info:** Fields for 'Full Name' (John Doe, ✓), 'Email' (john@email, ✗, note '* user@domain.com'), 'Phone' ((555) 123-4567, ✓), 'Gender' (Male, Female, Other), and 'Birth Date' (03/15/1985, ✓, MM/DD/YYYY).
- Address:** Fields for 'Street' (123 Main St, ✓), 'City' (Springfield), 'State' (IL), and 'ZIP' (6270, ✗, note '5 or 9 digits'). A checkbox 'Use different shipping address' is checked.
- Shipping Address:** Fields for 'Street' (456 Oak Avenue), 'City' (Chicago), 'State' (IL), and 'ZIP' (60601).
- Notifications:** A checked checkbox 'Enable notifications', 'Notify via' radio buttons (Email selected, SMS, Both), 'Frequency' dropdown (Daily), and checkboxes for 'Updates', 'Promos', 'Security', and 'News'.
- Two-Factor Auth:** An unchecked checkbox 'Enable 2FA' and 'Method' radio buttons (SMS, Auth App, Email).
- Preferences:** Checkboxes for 'Subscribe to newsletter' and 'Dark mode'. A checked checkbox 'Show advanced options' reveals an 'Advanced' section with an 'API Key' field (sk-xxxx-xxxx, ✓), a 'Timeout' field (30 seconds), and a checked checkbox 'Enable logging'.

At the bottom, a red error message states: '✗ 3 validation errors. Please fix before submitting.' The bottom right contains buttons for 'Help', 'OK', 'Cancel', and 'Apply'.

So, in UI design...

- ❑ Dialogue class is the mediator.
- ❑ Dialogue should know all of its sub-elements (components)
 - No new dependency
- ❑ Previously, “apply” button had to know all the other elements.

Profile Dialog



The screenshot shows a 'Edit User Profile' dialog box with the following sections:

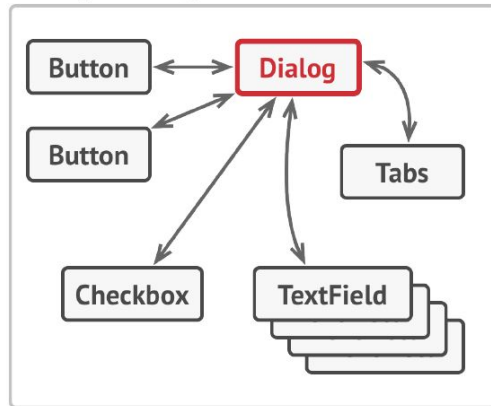
- Account Type:** Radio buttons for 'Personal' (selected) and 'Business'. A 'Company' field is disabled with the text 'Select Business' and a note '* Enabled when Business is selected'.
- Personal Info:** Fields for 'Full Name' (John Doe), 'Email' (john@email), 'Phone' ((555) 123-4567), 'Gender' (Male selected), and 'Birth Date' (03/15/1985).
- Address:** Fields for 'Street' (123 Main St), 'City' (Springfield), 'State' (IL), and 'ZIP' (6270). A checkbox 'Use different shipping address' is checked, showing a 'Shipping Address' section with fields for 'Street' (456 Oak Avenue), 'City' (Chicago), 'State' (IL), and 'ZIP' (60601).
- Notifications:** A checkbox 'Enable notifications' is checked. 'Notify via' has radio buttons for 'Email' (selected), 'SMS', and 'Both'. 'Frequency' is set to 'Daily'. There are checkboxes for 'Updates', 'Promos', 'Security', and 'News'.
- Two-Factor Auth:** A checkbox 'Enable 2FA' is unchecked. 'Method' has radio buttons for 'SMS', 'Auth App', and 'Email'.
- Advanced:** A section with a 'API Key' field (sk-xxxx-xxxx), a 'Timeout' field (30 seconds), and a checked 'Enable logging' checkbox.

At the bottom, there is a red error message: 'X 3 validation errors. Please fix before submitting.' and buttons for 'Help', 'OK', 'Cancel', and 'Apply'.

So, in UI design...

- ❑ Dialogue class is the mediator.
- ❑ Dialogue should know all of its sub-elements (components)
 - No new dependency
- ❑ Now, “apply” just notifies mediator (i.e the dialogue box)

Profile Dialog

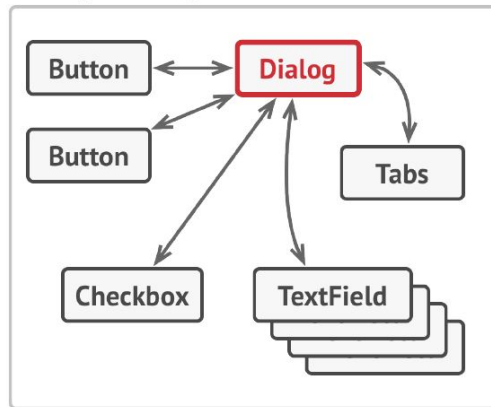


A screenshot of a 'Edit User Profile' dialog box. The dialog has a title bar with standard window controls. It contains several sections: 'Account Type' with radio buttons for 'Personal' (selected) and 'Business'; 'Address' with fields for 'Street', 'City', 'State', and 'ZIP'; 'Personal Info' with fields for 'Full Name', 'Email', 'Phone', 'Gender', and 'Birth Date'; 'Preferences' with checkboxes for 'Subscribe to newsletter', 'Show advanced options', and 'Dark mode'; 'Notifications' with a checkbox for 'Enable notifications' and options for 'Notify via' (Email, SMS, Both) and 'Frequency'; and 'Two-Factor Auth' with a checkbox for 'Enable 2FA' and options for 'Method' (SMS, Auth App, Email). At the bottom, there are buttons for 'Help', 'OK', 'Cancel', and 'Apply'. A red arrow points from the 'Dialogue class is the mediator' text to the 'Dialog' box in the diagram above. A red error message at the bottom states: '3 validation errors. Please fix before submitting.'

So, in UI design...

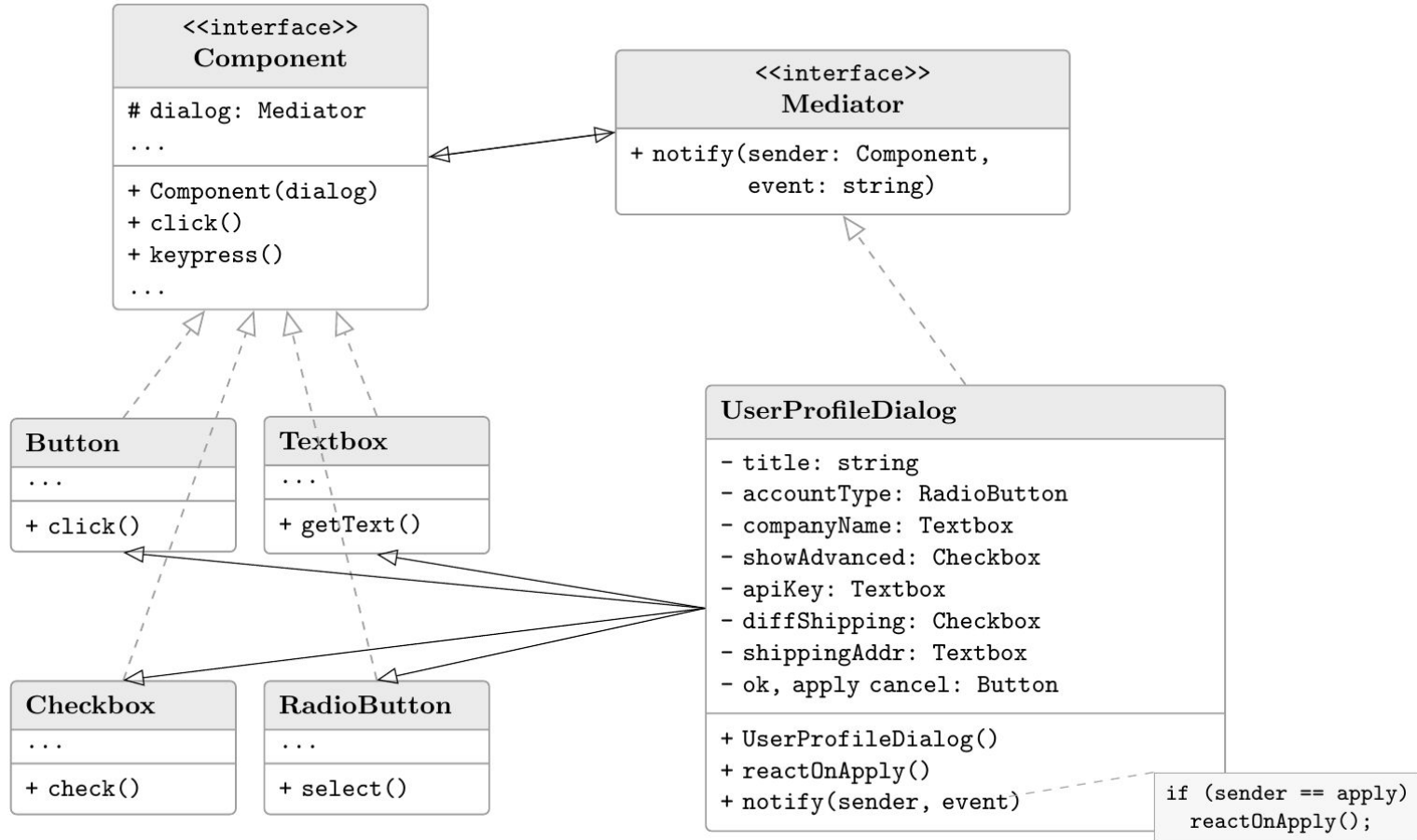
- ❑ Dialogue class is the mediator.
- ❑ Dialogue should know all of its sub-elements (components)
 - No new dependency
- ❑ Now, “apply” just notifies mediator (i.e the dialogue box). “Apply” needs to know who it is notifying...

Profile Dialog

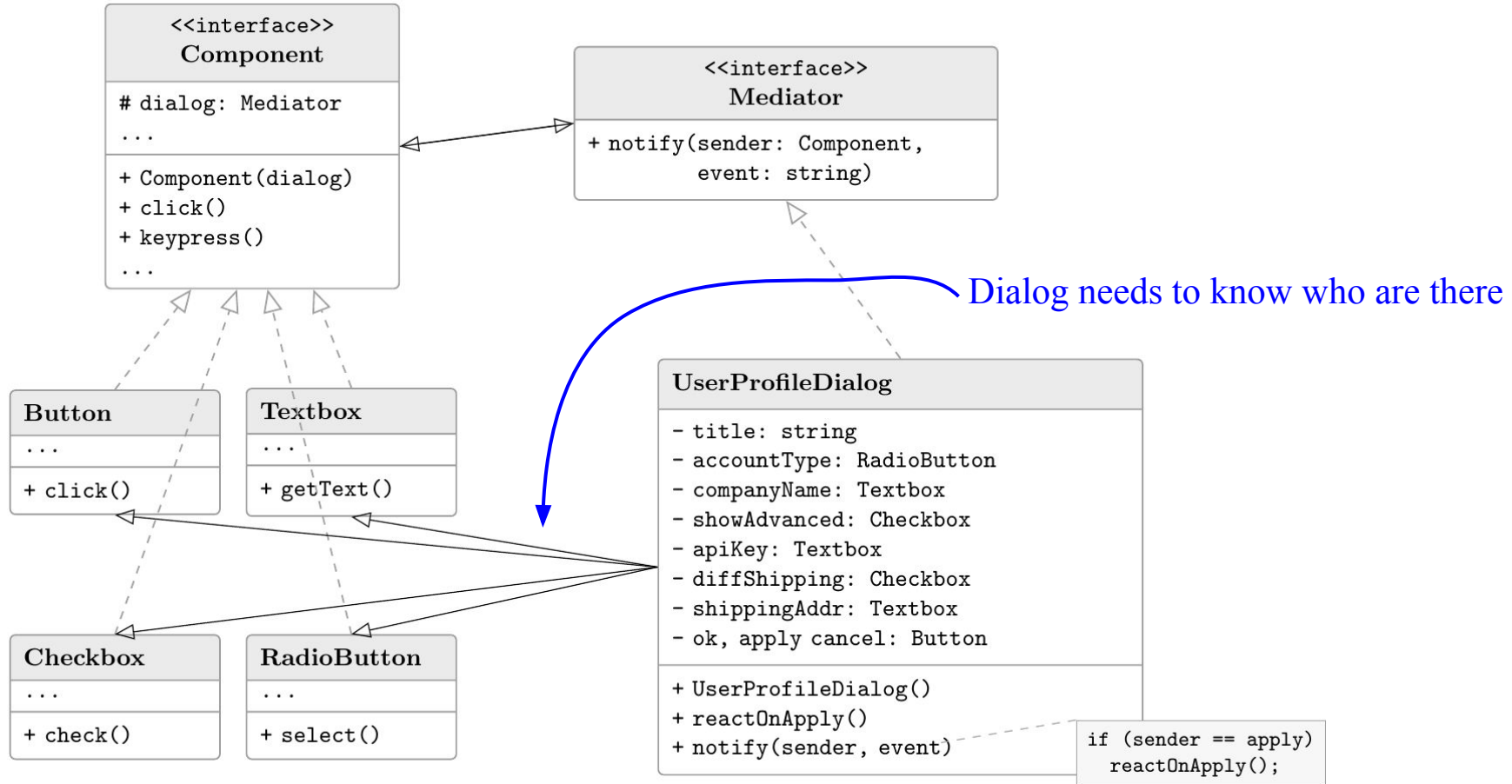


A screenshot of a 'Edit User Profile' dialog box. The dialog has a title bar with standard window controls. It is divided into several sections: 'Account Type' with radio buttons for 'Personal' (selected) and 'Business'; 'Address' with fields for 'Street' (123 Main St), 'City' (Springfield), 'State' (IL), and 'ZIP' (6270); 'Personal Info' with fields for 'Full Name' (John Doe), 'Email' (john@email), 'Phone' ((555) 123-4567), 'Gender' (Male selected), and 'Birth Date' (03/15/1985); 'Preferences' with checkboxes for 'Subscribe to newsletter', 'Show advanced options', and 'Dark mode'; 'Notifications' with a checked 'Enable notifications' checkbox and options for 'Notify via' (Email selected) and 'Frequency' (Daily); and 'Two-Factor Auth' with a unchecked 'Enable 2FA' checkbox. At the bottom, there is a red error message: '3 validation errors. Please fix before submitting.' and four buttons: 'Help', 'OK', 'Cancel', and 'Apply'. A red arrow points from the text 'Dialogue class is the mediator.' to the 'Apply' button.

Birds Eye View



Birds Eye View



Let's see some code...

```
interface Mediator {  
    notify(sender: Component, event: string);  
}
```

Let's see some code...

```
interface Mediator {  
    notify(sender: Component, event: string);  
}
```

```
class AuthenticationDialog implements Mediator {  
    private Button okBtn, cancelBtn;  
  
    public AuthenticationDialog() {  
        // Create all component objects and pass the current  
        // mediator into their constructors to establish links.  
        // use "this" (e.g okButton = new Button(this));  
    }  
  
    void notify(sender, event) {  
        if (sender == okBtn && event == "click") {  
            // do ok button clicking actions  
        }  
        else if(...){...}  
        ...  
        else {}  
    }  
}
```

Let's see some code...

```
interface Mediator {  
    notify(sender: Component, event: string);  
}
```

```
class AuthenticationDialog implements Mediator {  
    private Button okBtn, cancelBtn;  
  
    public AuthenticationDialog() {  
        // Create all component objects and pass the current  
        // mediator into their constructors to establish links.  
        // use "this" (e.g okButton = new Button(this));  
    }  
  
    void notify(sender, event) {  
        if (sender == okBtn && event == "click") {  
            // do ok button clicking actions  
        }  
        else if(...){...}  
        ...  
        else {}  
    }  
}
```

```
class Component {  
    private Mediator dialog;  
  
    Component(Mediator dialog) {  
        this.dialog = dialog  
    }  
  
    public void click() {  
        dialog.notify(this, "click");  
    }  
    ...  
}
```

Let's see some code...

```
interface Mediator {  
    notify(sender: Component, event: string);  
}
```

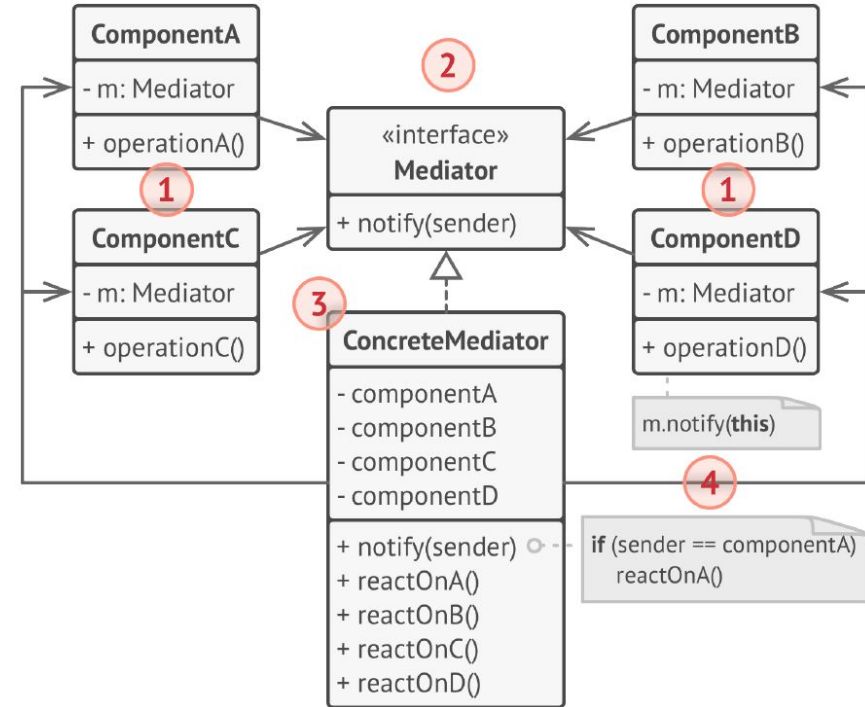
```
class AuthenticationDialog implements Mediator {  
    private Button okBtn, cancelBtn;  
  
    public AuthenticationDialog() {  
        // Create all component objects and pass the current  
        // mediator into their constructors to establish links.  
        // use "this" (e.g okButton = new Button(this));  
    }  
  
    void notify(sender, event) {  
        if (sender == okBtn && event == "click") {  
            // do ok button clicking actions  
        }  
        else if(...){...}  
        ...  
        else {}  
    }  
}
```

```
class Component {  
    private Mediator dialog;  
  
    Component(Mediator dialog) {  
        this.dialog = dialog  
    }  
  
    public void click() {  
        dialog.notify(this, "click");  
    }  
    ...  
}
```

```
class Button implements Component {  
    ...  
}
```

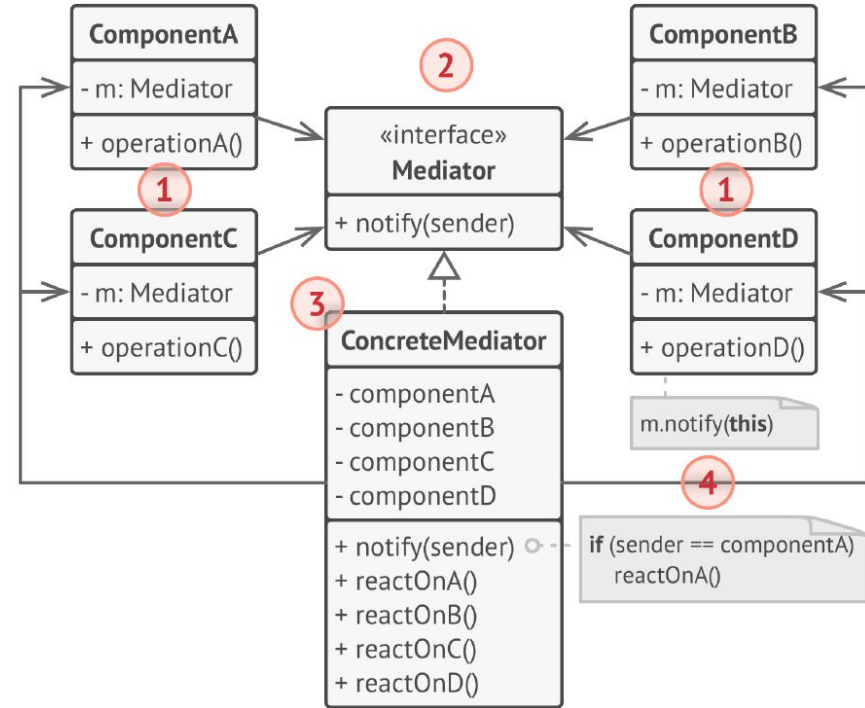
The Mediator Pattern Defined

Mediator pattern **restricts direct** communications between objects and forces them to **collaborate** only via a mediator. Lets you reduce **chaotic dependencies** between objects.



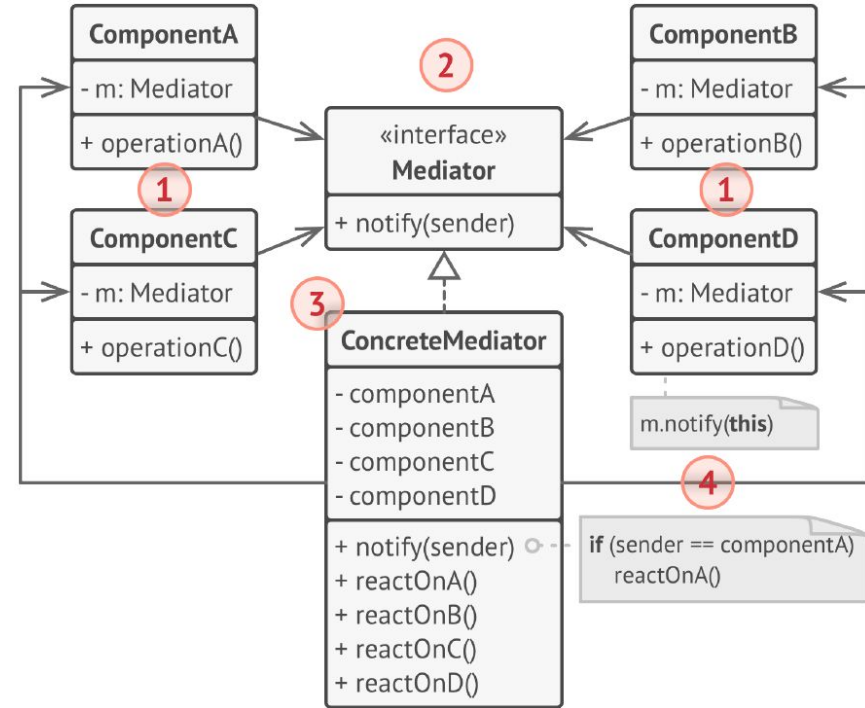
The Mediator Pattern Defined

1. **Components** are various classes that contain some business logic. Each component has a reference to a mediator, declared with the type of the mediator interface. The component isn't aware of the actual class of the mediator, so you can reuse the component in other programs by linking it to a different mediator.
2. The **Mediator** interface declares methods of communication with components, which usually include just a single notification method. Components may pass any context as arguments of this method, including their own objects, but only in such a way that no coupling occurs between a receiving component and the sender's class.



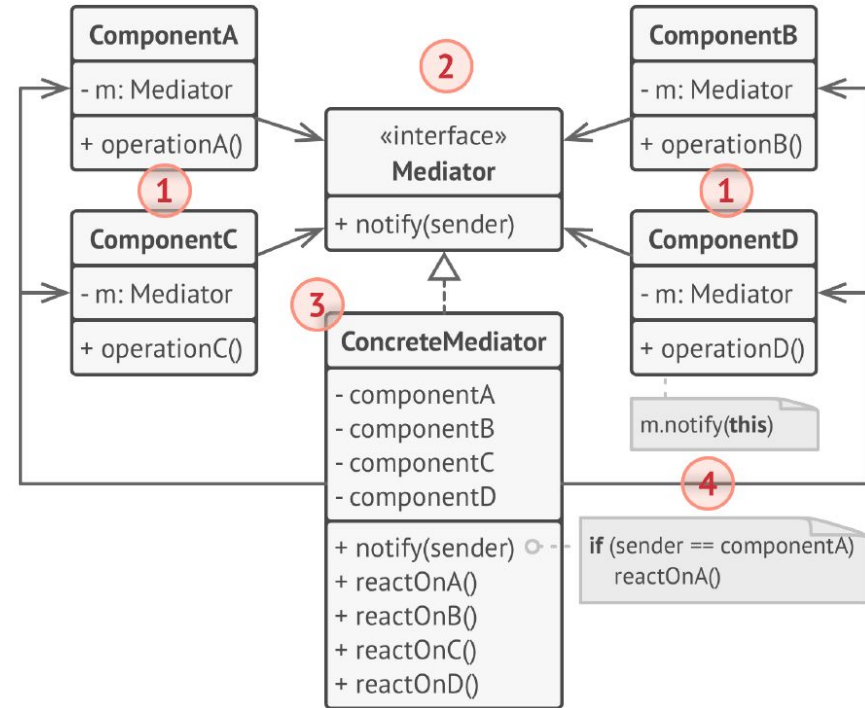
The Mediator Pattern Defined

1. **Components** are various classes that contain some business logic. Each component has a reference to a mediator, declared with the type of the mediator interface. The component isn't aware of the actual class of the mediator, so you can reuse the component in other programs by linking it to a different mediator.
2. The **Mediator** interface declares methods of communication with components, which usually include just a single notification method. Components may pass any context as arguments of this method, including their own objects, but only in such a way that no coupling occurs between a receiving component and the sender's class.



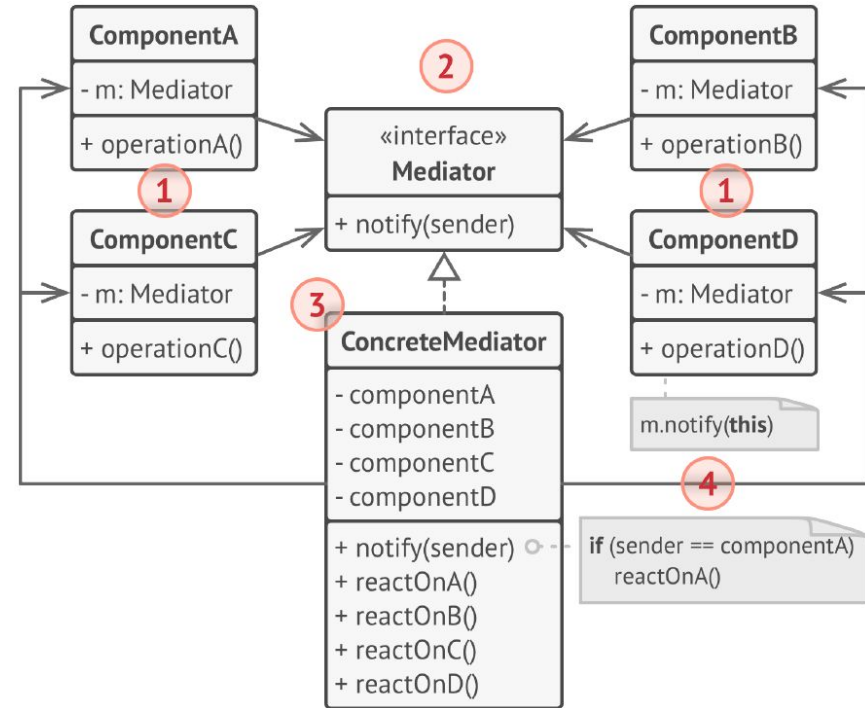
The Mediator Pattern Defined

1. **Components** are various classes that contain some business logic. Each component has a reference to a mediator, declared with the type of the mediator interface. The component isn't aware of the actual class of the mediator, so you can reuse the component in other programs by linking it to a different mediator.
2. The **Mediator** interface declares methods of communication with components, which usually include just a single notification method. Components may pass any context as arguments of this method, including their own objects, but only in such a way that no coupling occurs between a receiving component and the sender's class.



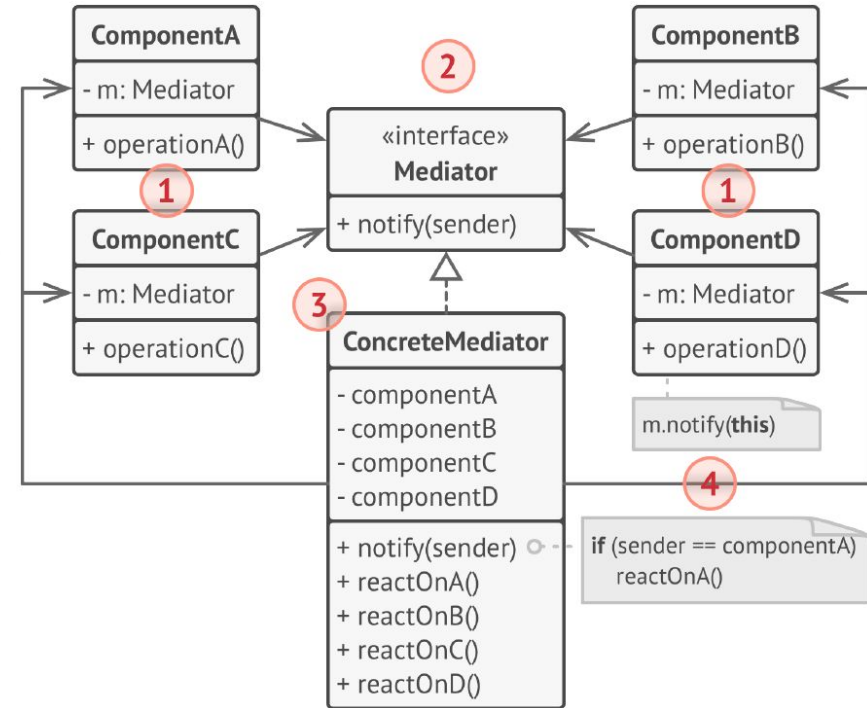
The Mediator Pattern Defined

1. **Components** are various classes that contain some business logic. Each component has a reference to a mediator, declared with the type of the mediator interface. The component isn't aware of the actual class of the mediator, so you can reuse the component in other programs by linking it to a different mediator.
2. The **Mediator** interface declares methods of communication with components, which usually include just a single notification method. Components may pass any context as arguments of this method, including their own objects, but only in such a way that no coupling occurs between a receiving component and the sender's class.



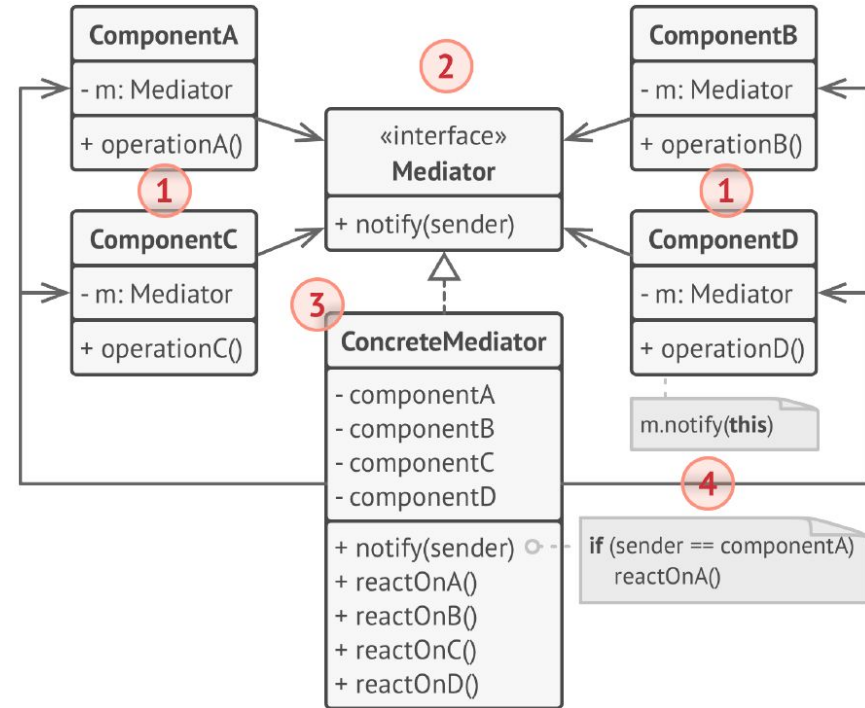
The Mediator Pattern Defined

1. **Components** are various classes that contain some business logic. Each component has a reference to a mediator, declared with the type of the mediator interface. The component isn't aware of the actual class of the mediator, so you can reuse the component in other programs by linking it to a different mediator.
2. The **Mediator** interface declares methods of communication with components, which usually include just a single notification method. Components may pass any context as arguments of this method, including their own objects, but only in such a way that no coupling occurs between a receiving component and the sender's class.



The Mediator Pattern Defined

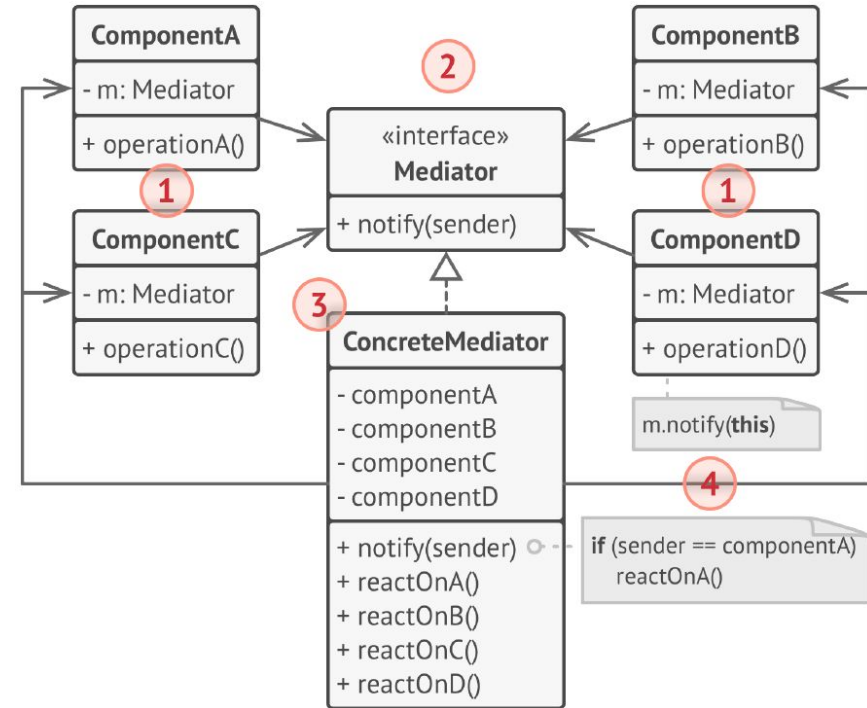
1. **Components** are various classes that contain some business logic. Each component has a reference to a mediator, declared with the type of the mediator interface. The component isn't aware of the actual class of the mediator, so you can reuse the component in other programs by linking it to a different mediator.
2. The **Mediator** interface declares methods of communication with components, which usually include just a single notification method. Components may pass any context as arguments of this method, including their own objects, but only in such a way that no coupling occurs between a receiving component and the sender's class.



The Mediator Pattern Defined

3. **Concrete Mediators** encapsulate relations between various components. Concrete mediators often keep references to all components they manage and sometimes even manage their lifecycle.
4. Components must not be aware of other components. If something important happens within or to a component, it must only notify the mediator. When the mediator receives the notification, it can easily identify the sender, which might be just enough to decide what component should be triggered in return.

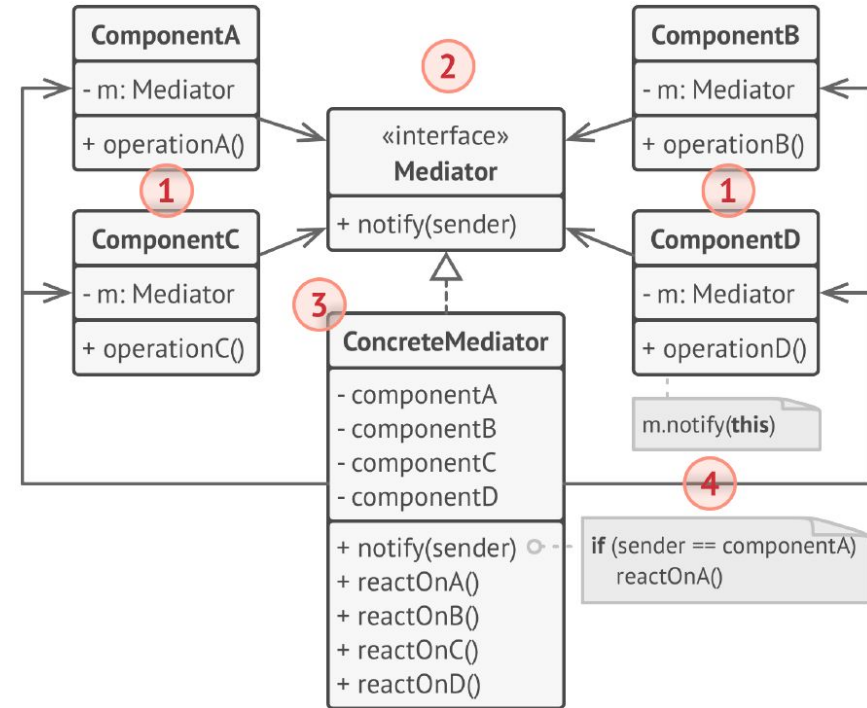
From a component's perspective, it all looks like a total black box. The sender doesn't know who'll end up handling its request, and the receiver doesn't know who sent the request in the first place.



The Mediator Pattern Defined

3. **Concrete Mediators** encapsulate relations between various components. Concrete mediators often keep references to all components they manage and sometimes even manage their lifecycle.
4. Components must not be aware of other components. If something important happens within or to a component, it must only notify the mediator. When the mediator receives the notification, it can easily identify the sender, which might be just enough to decide what component should be triggered in return.

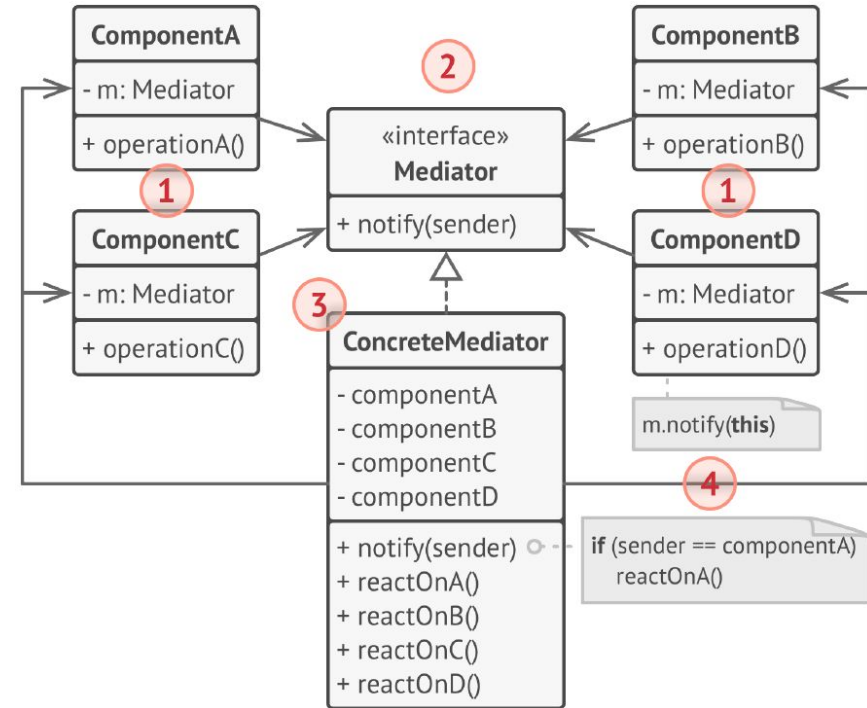
From a component's perspective, it all looks like a total black box. The sender doesn't know who'll end up handling its request, and the receiver doesn't know who sent the request in the first place.



The Mediator Pattern Defined

3. **Concrete Mediators** encapsulate relations between various components. Concrete mediators often keep references to all components they manage and sometimes even manage their lifecycle.
4. Components must not be aware of other components. If something important happens within or to a component, it must only notify the mediator. When the mediator receives the notification, it can easily identify the sender, which might be just enough to decide what component should be triggered in return.

From a component's perspective, it all looks like a total black box. The sender doesn't know who'll end up handling its request, and the receiver doesn't know who sent the request in the first place.

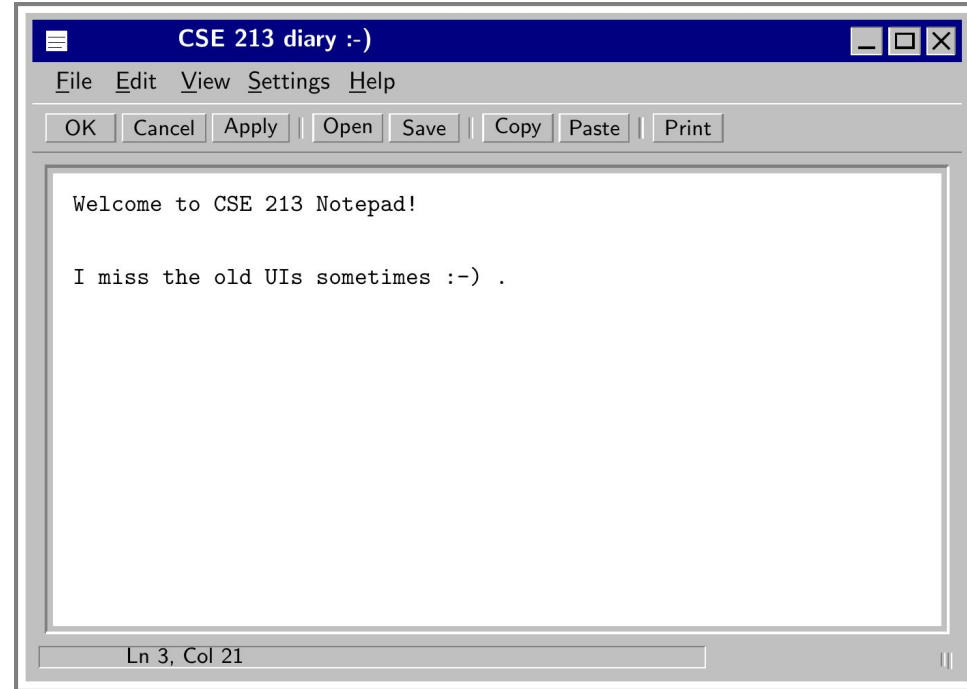


Difference between Mediator and Observer

Primary goal:

- Mediator eliminates **mutual dependencies**. Instead, objects depend on **single** mediator object.
- Observer establishes **dynamic** (can subscribe, unsubscribe) **one-way one-to-many** connection between objects.

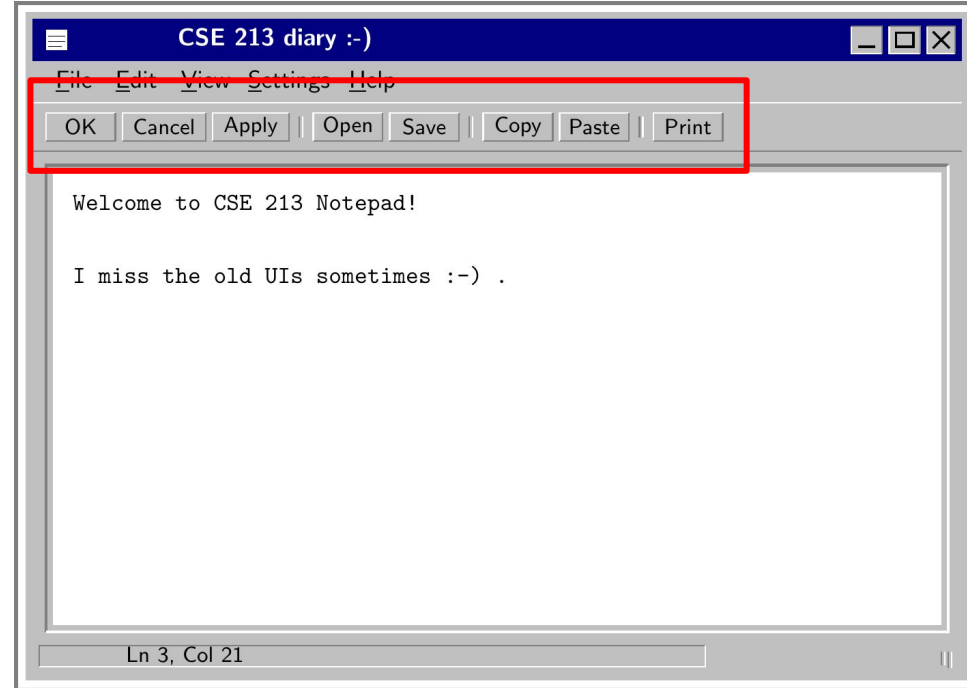
Let's design a simple text-editor



Generated using LaTeX. Thanks to Claude

Let's design a simple text-editor

Create a toolbar with a **bunch of buttons** for various operations of the editor

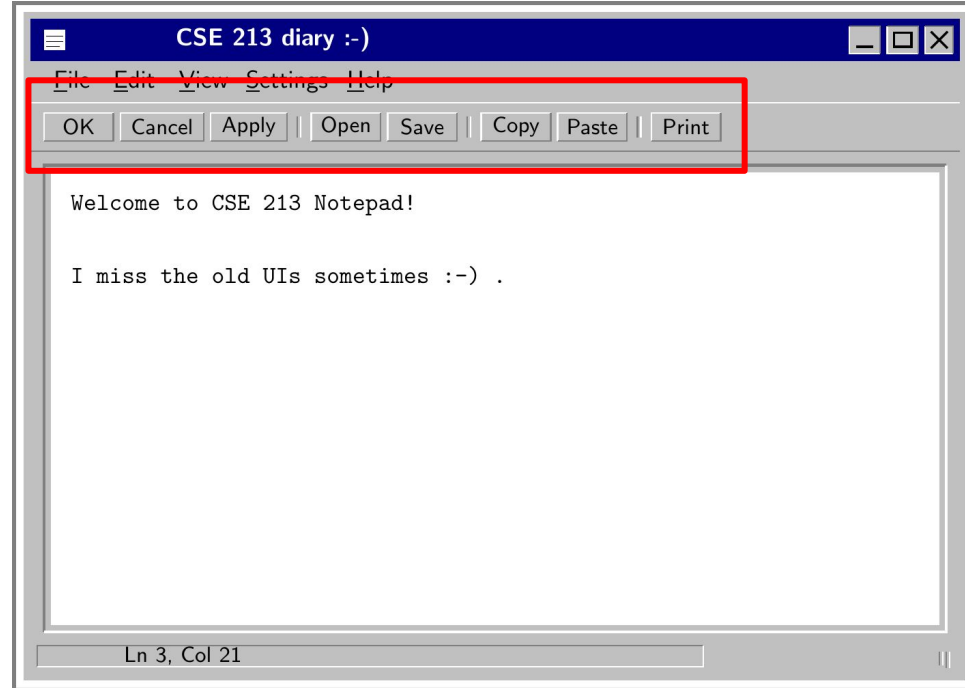


Generated using LaTeX. Thanks to Claude

Let's design a simple text-editor

Create a toolbar with a **bunch of buttons** for various operations of the editor

How to design buttons?

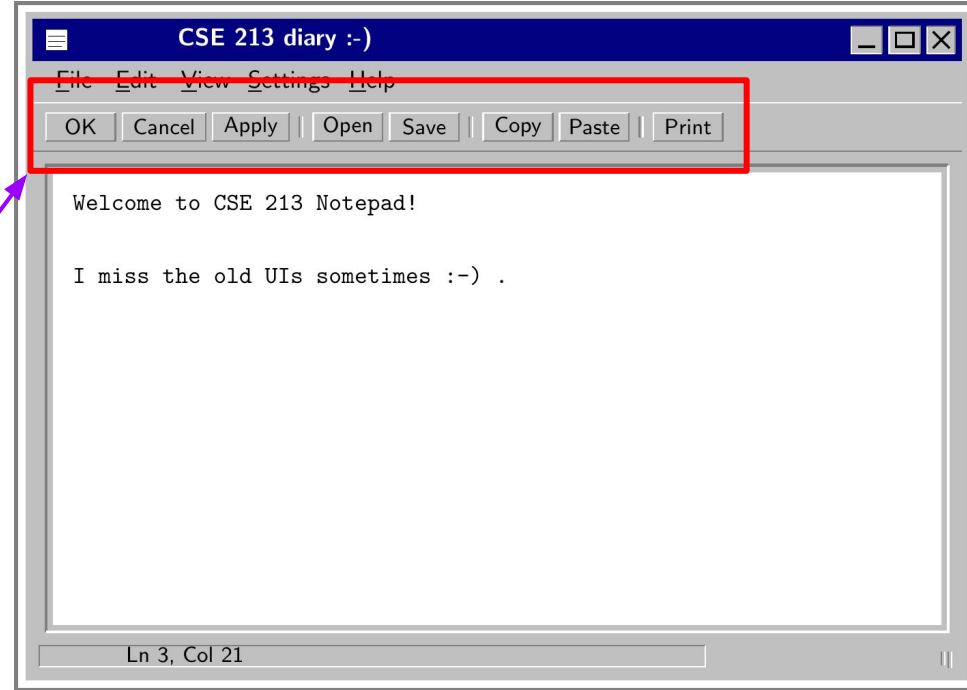
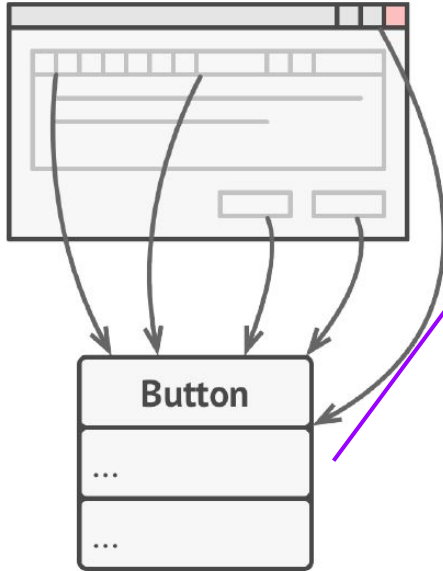


Generated using LaTeX. Thanks to Claude

Let's design a simple text-editor

Create a toolbar with a **bunch of buttons** for various operations of the editor

A neat button class used for **toolbar buttons**? [look same]

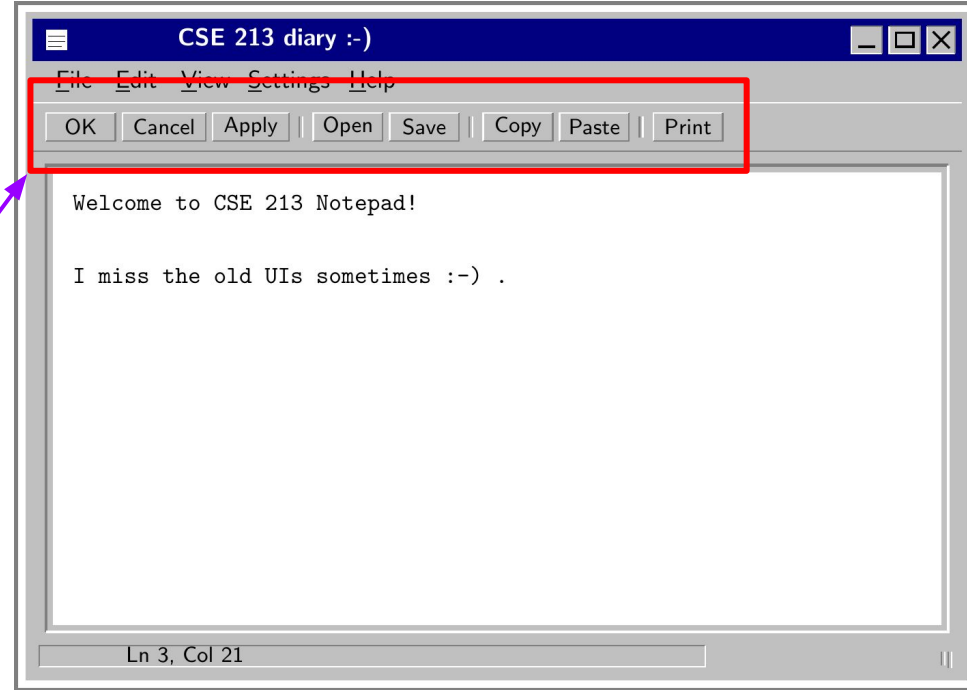
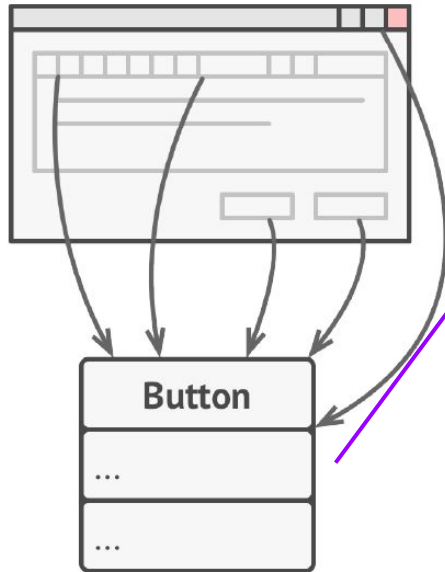


Generated using LaTeX. Thanks to Claude

Let's design a simple text-editor

Create a toolbar with a **bunch of buttons** for various operations of the editor

But they do different things...



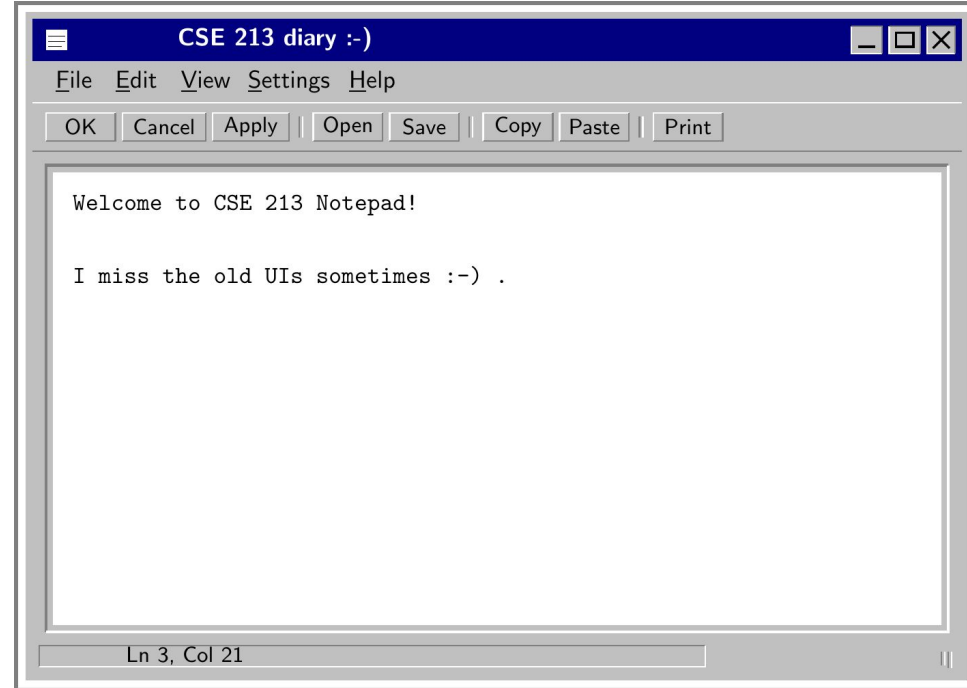
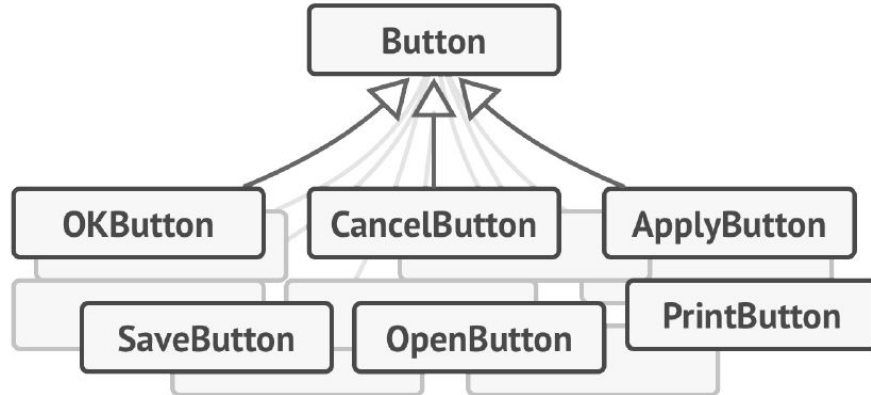
Generated using LaTeX. Thanks to Claude

Let's design a simple text-editor

Create a toolbar with a **bunch of buttons** for various operations of the editor

But they do different things...

Subclasses to do different things when **clicked**?

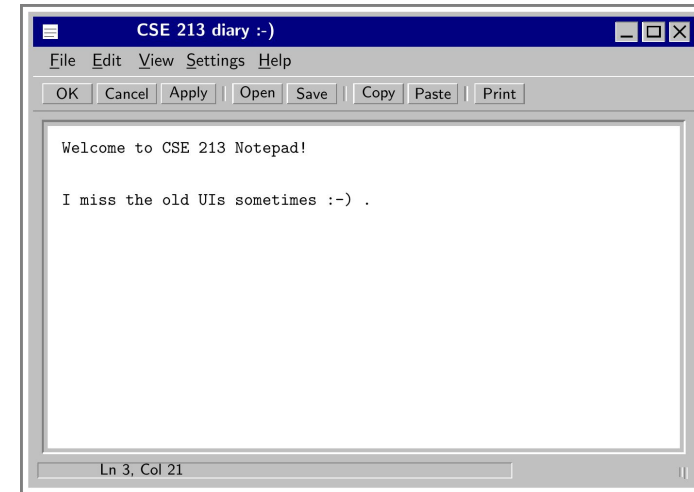
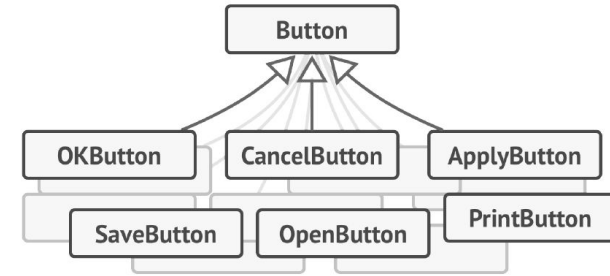


Generated using LaTeX. Thanks to Claude

Subclasses Issues...

Enormous number of subclasses

- Modifying **Button** class risks breaking code!



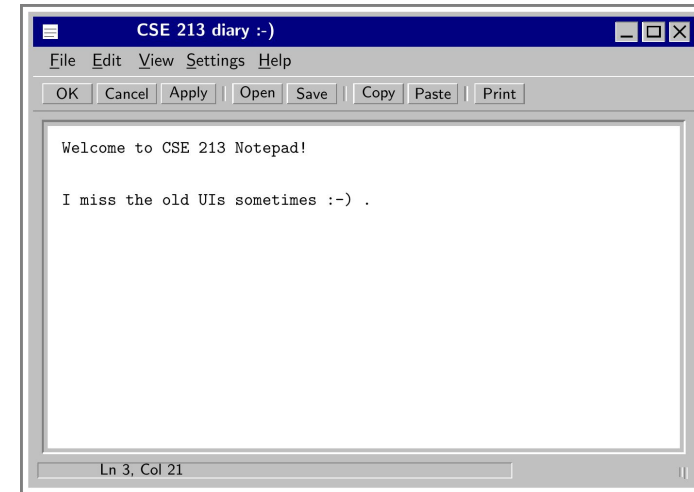
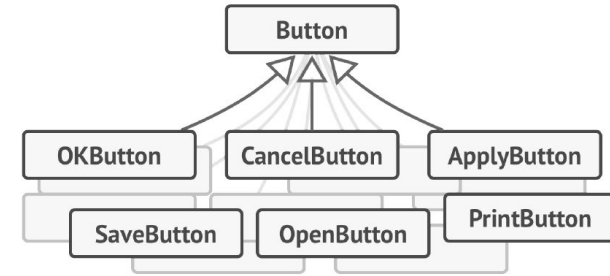
Subclasses Issues...

Enormous number of subclasses

- Modifying **Button** class risks breaking code!

Ugliest part

- Save can be done by **Ctrl+S** (shortcut) too.
- From menu (File -> Save) also.



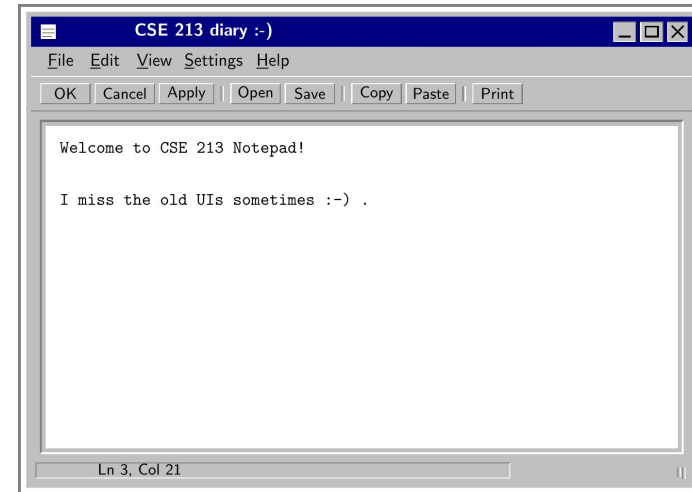
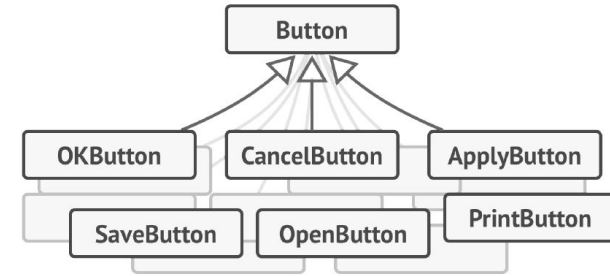
Subclasses Issues...

Enormous number of subclasses

- Modifying **Button** class risks breaking code!

Ugliest part

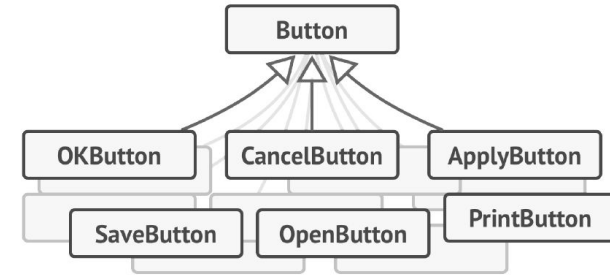
- Save can be done by **Ctrl+S** (shortcut) too.
- From menu (File -> Save) also.
- **Code duplication!!** (write same operations multiple times)



Subclasses Issues...

Enormous number of subclasses

- Modifying **Button** class risks breaking code!

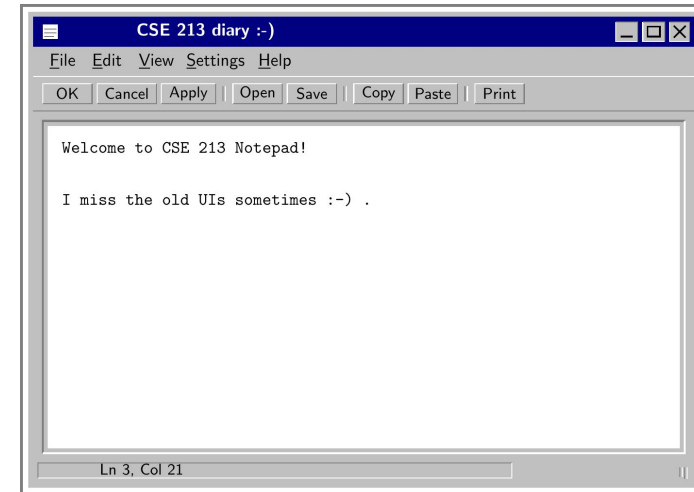


Ugliest part

- Save can be done by **Ctrl+S** (shortcut) too.
- From menu (File -> Save) also.
- **Code duplication!!** (write same operations multiple times)

Solution?

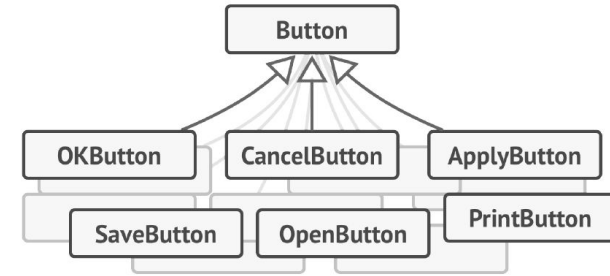
- Invoke button *click handler* from *shortcut or save menu*?



Subclasses Issues...

Enormous number of subclasses

- Modifying **Button** class risks breaking code!

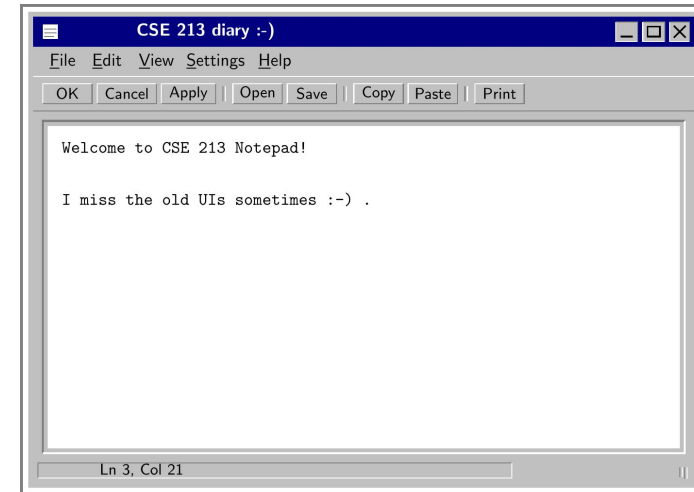


Ugliest part

- Save can be done by **Ctrl+S** (shortcut) too.
- From menu (File -> Save) also.
- **Code duplication!!** (write same operations multiple times)

Solution?

- Invoke button *click handler* from *shortcut or save menu*?
- Menu (& shortcut) **dependent** on buttons. **Too ugly**



Separation of concerns...

A layer for the **GUI** and another layer for the **business logic**.

Separation of concerns...

A layer for the **GUI** and another layer for the **business logic**.

- GUI does the job of rendering pictures, capturing input

Separation of concerns...

A layer for the **GUI** and another layer for the **business logic**.

- GUI does the job of rendering pictures, capturing input
- Calculating, data loading stuffs done by underlying business logic

Separation of concerns...

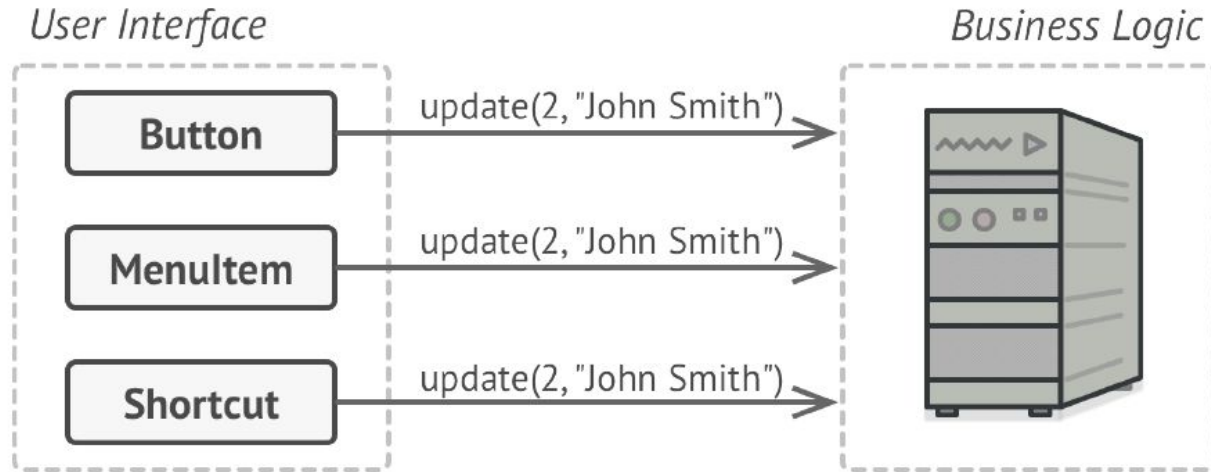
A layer for the **GUI** and another layer for the **business logic**.

- GUI does the job of rendering pictures, capturing input
- Calculating, data loading stuffs done by underlying business logic
- GUI delegates by *requesting* with arguments

Separation of concerns...

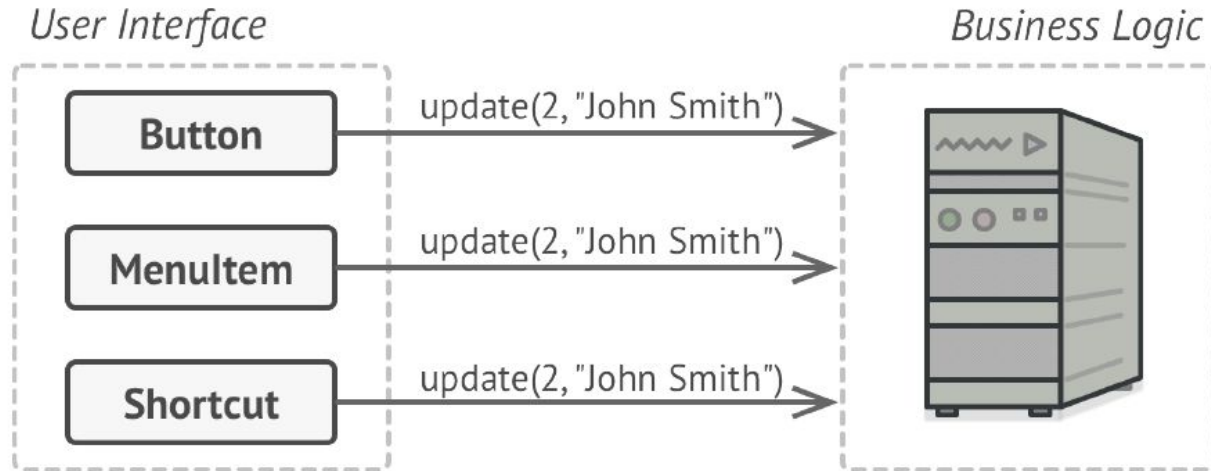
A layer for the **GUI** and another layer for the **business logic**.

- GUI does the job of rendering pictures, capturing input
- Calculating, data loading stuffs done by underlying business logic
- GUI delegates by *requesting* with arguments



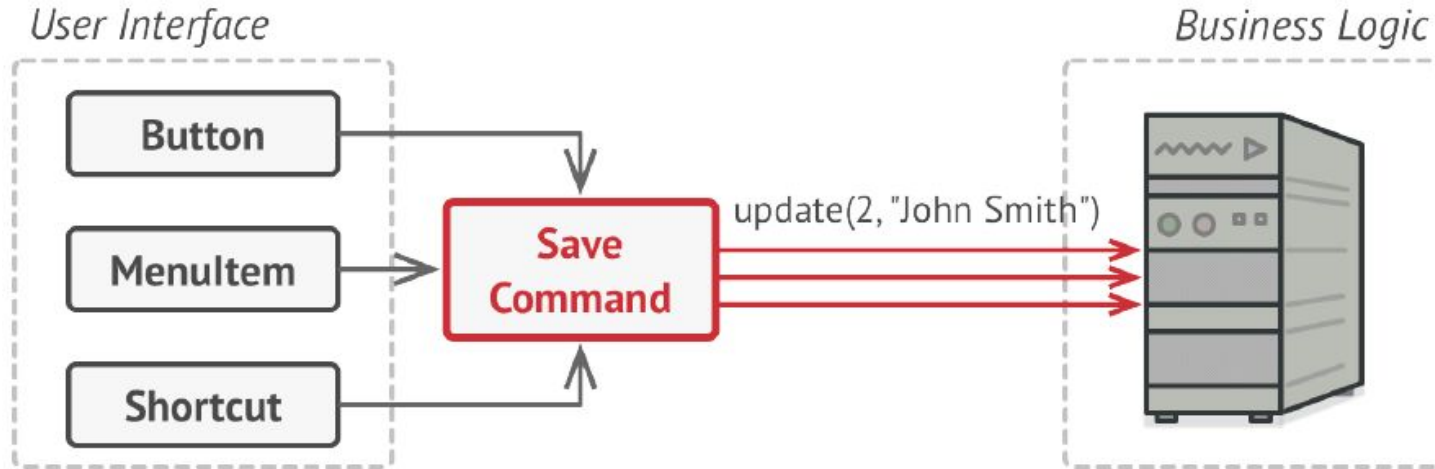
Separation of concerns...

- GUI objects **shouldn't send** these requests **directly either**



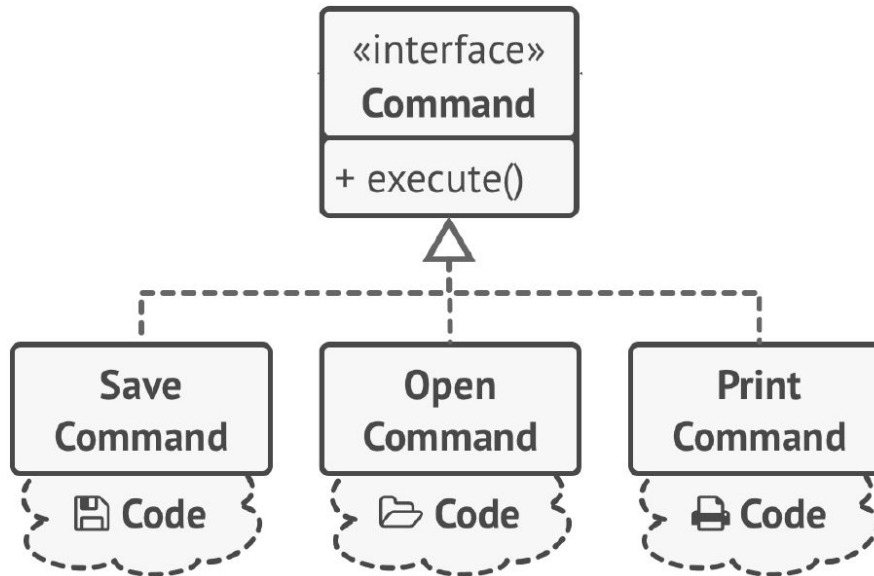
Separation of concerns...

- GUI objects **shouldn't send** these requests **directly either**
- Rather the save operation should be in a single “Command” class



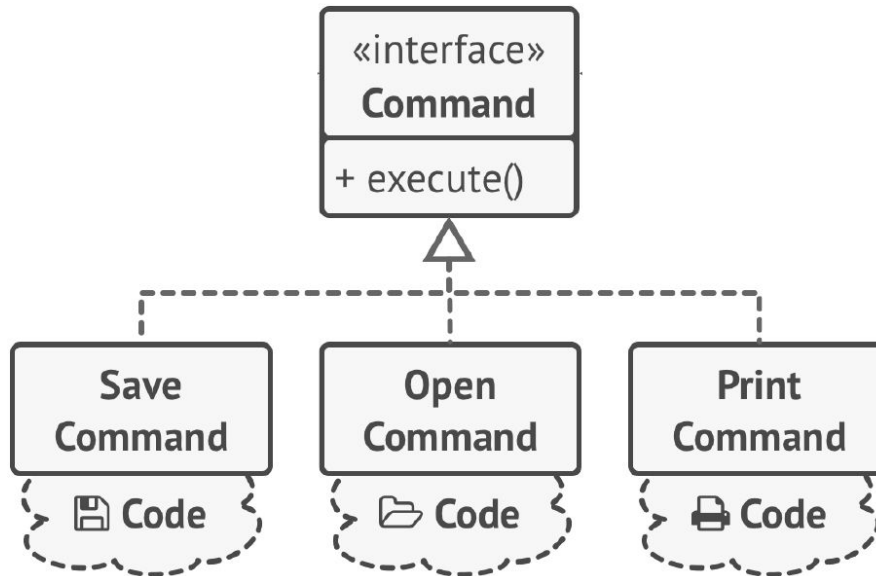
Again that “Code to interface...”

- Commands implement the same *interface*
- A single **execute()** method



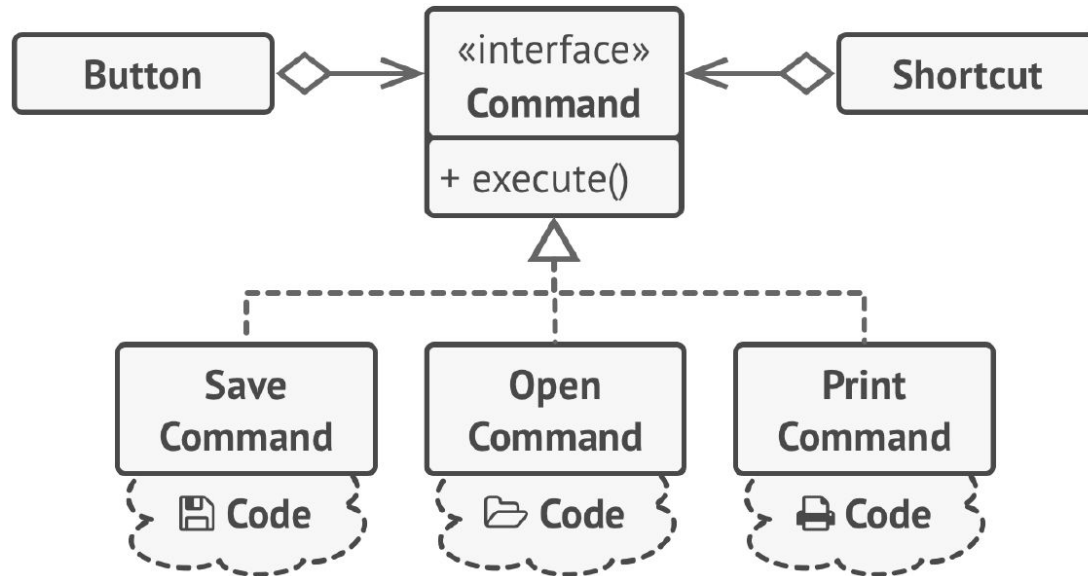
Again that “Code to interface...”

- Commands implement the same *interface*
- A single **execute()** method
- Command parameters? Should be pre-configured, they know what to do!

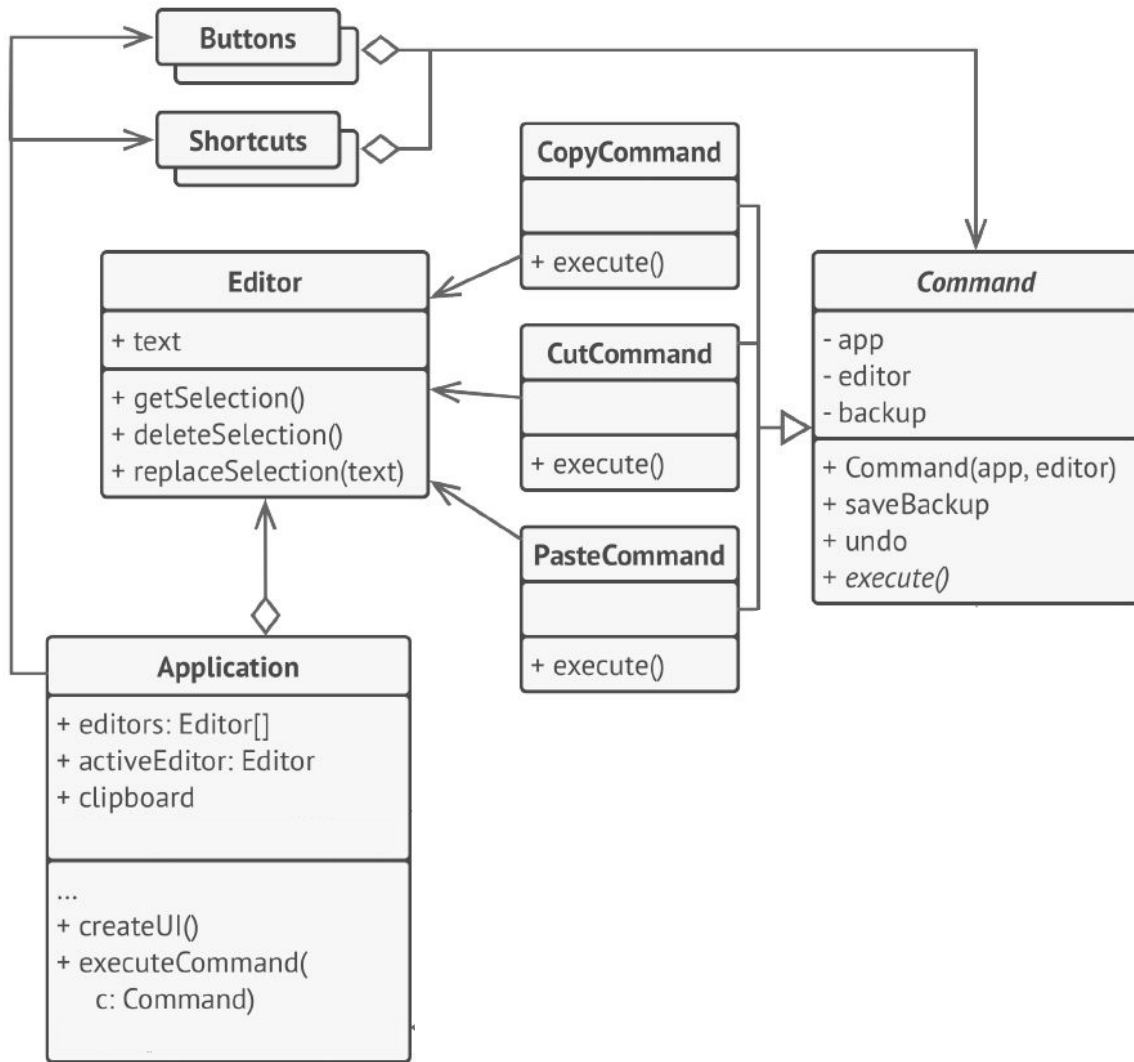


Again that “Code to interface...”

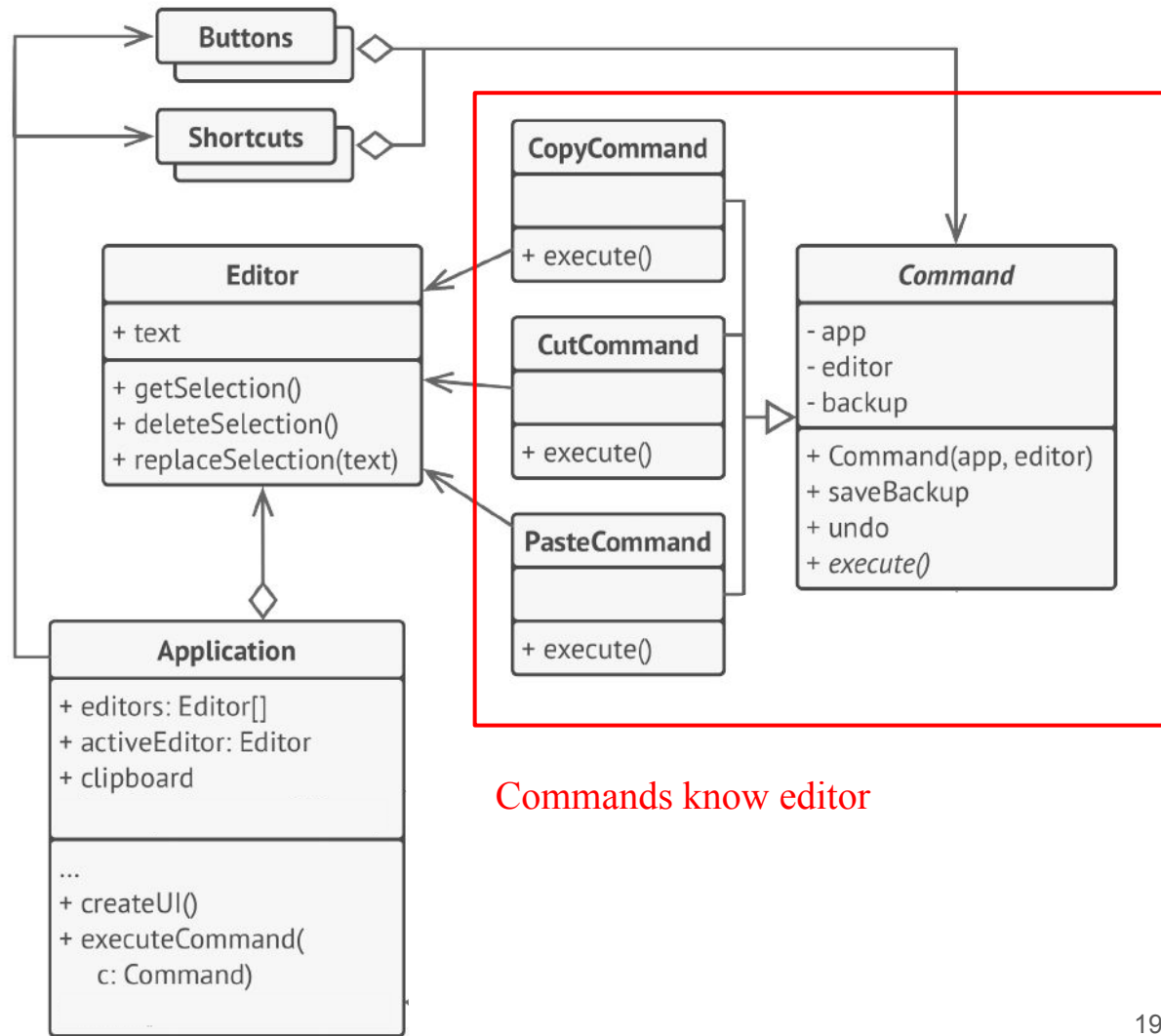
- Commands implement the same *interface*
- A single **execute()** method
- Command parameters? Should be pre-configured, they know what to do!



Birds eye view



Birds eye view



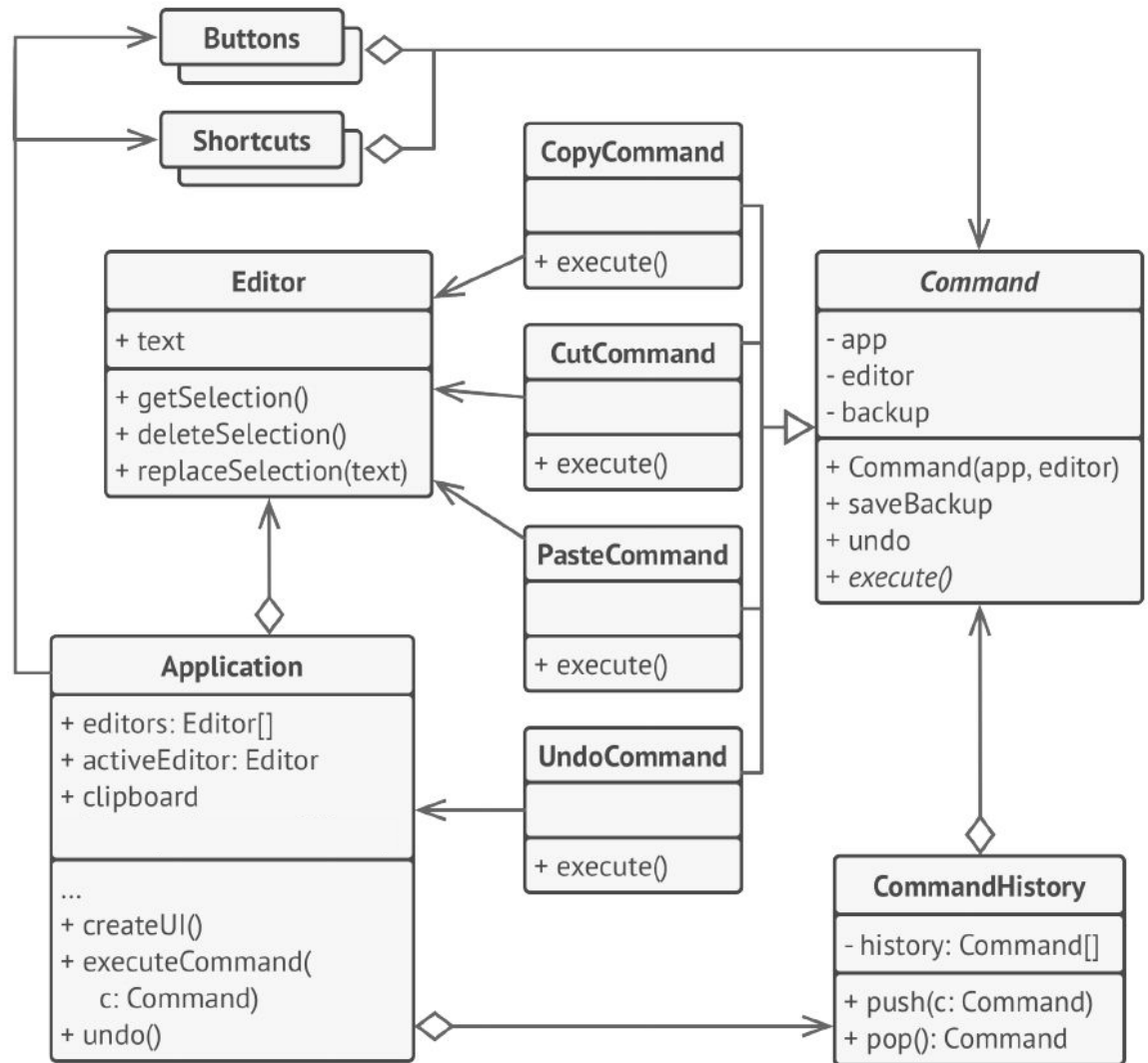
Commands know editor

Birds eye view

Undo operation:

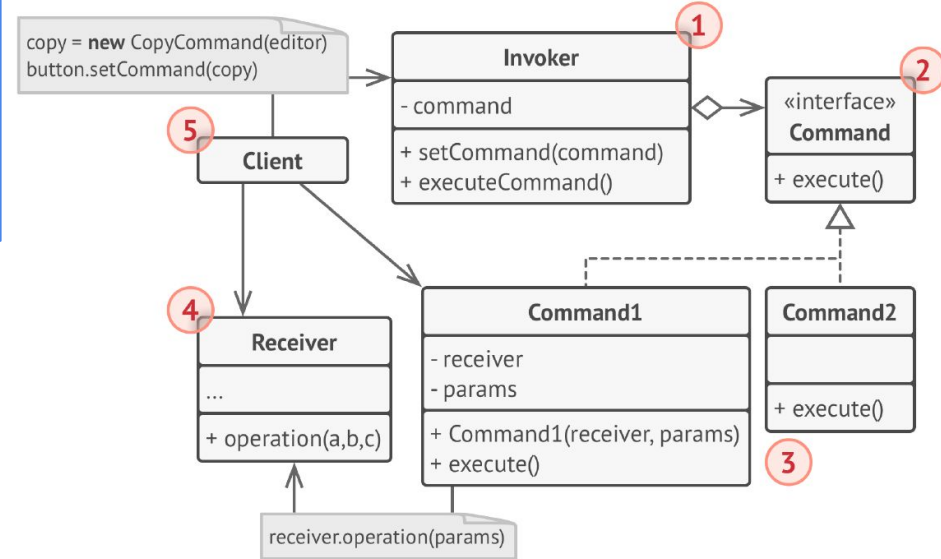
Commands that change editor state

- Push a **backup** copy in stack
- Undo **pops** the state and restore it



Command Pattern Defined

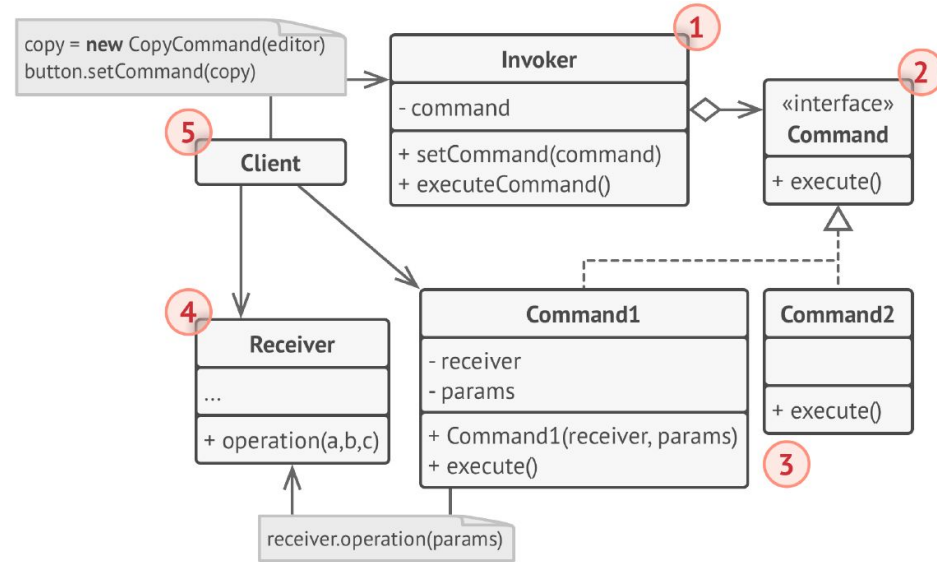
Command pattern that turns a request into a stand-alone **object** that contains **all information** about the request. This transformation lets you *parameterize methods* with different requests, **delay or queue** a request's execution, and support **undoable** operations.



Command Pattern Defined

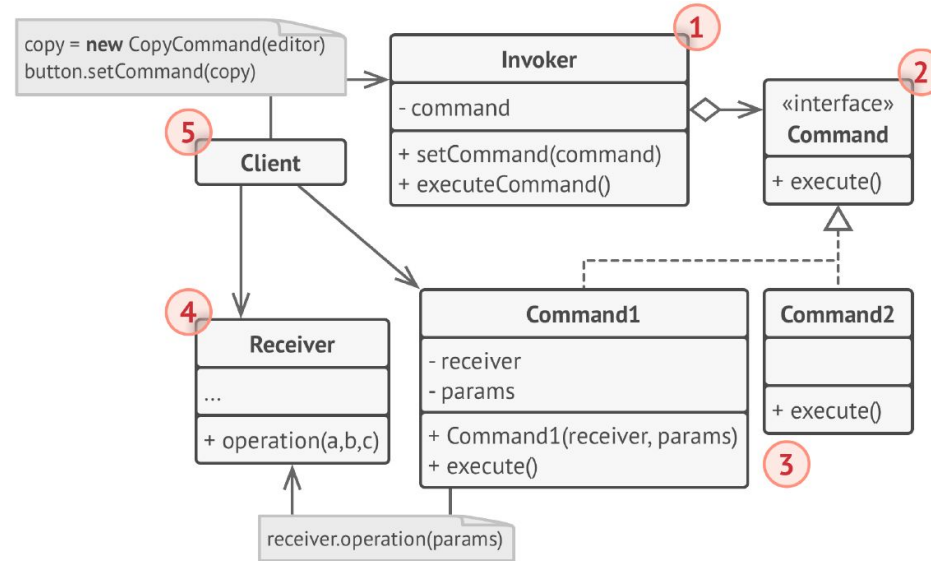
Command pattern that turns a request into a stand-alone **object** that contains **all information** about the request. This transformation lets you *parameterize methods* with different requests, **delay or queue** a request's execution, and support **undoable** operations.

passing a request as an argument



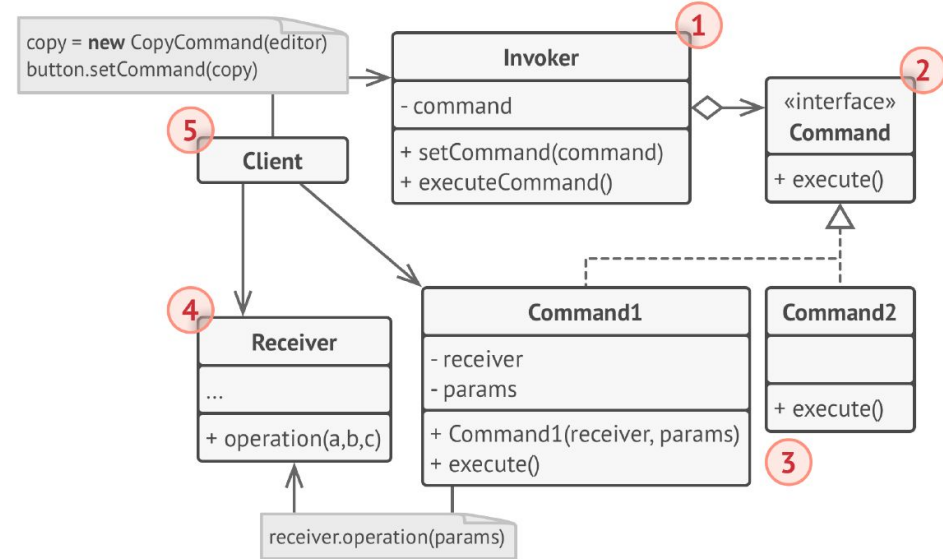
Command Pattern Defined

1. The **Sender** class (aka *invoker*) is responsible for initiating requests. This class must have a field for storing a reference to a command object. The sender triggers that command instead of sending the request directly to the receiver. Note that the sender isn't responsible for creating the command object. Usually, it gets a pre-created command from the client via the constructor.
2. The **Command** interface usually declares just a single method for executing the command.



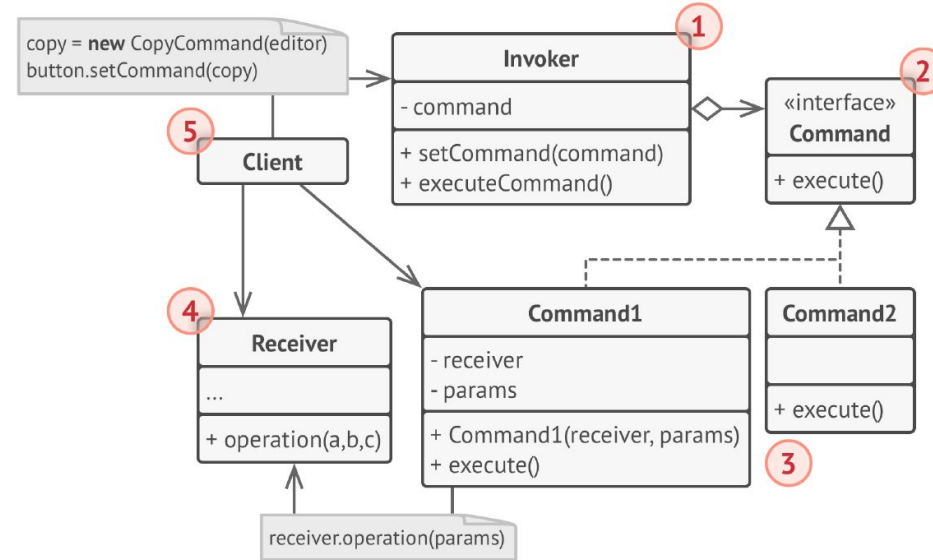
Command Pattern Defined

1. The **Sender** class (aka *invoker*) is responsible for initiating requests. This class must have a field for storing a reference to a command object. The sender triggers that command instead of sending the request directly to the receiver. Note that the sender isn't responsible for creating the command object. Usually, it gets a pre-created command from the client via the constructor.
2. The **Command** interface usually declares just a single method for executing the command.



Command Pattern Defined

1. The **Sender** class (aka *invoker*) is responsible for initiating requests. This class must have a field for storing a reference to a command object. The sender triggers that command instead of sending the request directly to the receiver. Note that the sender isn't responsible for creating the command object. Usually, it gets a pre-created command from the client via the constructor.
2. The **Command** interface usually declares just a single method for executing the command.

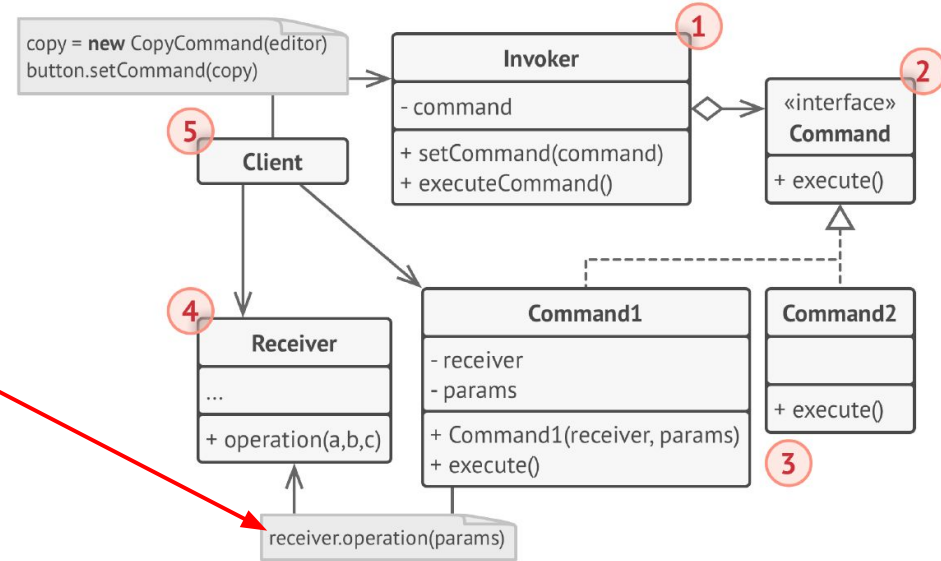


Command Pattern Defined

3. **Concrete Commands** implement various kinds of requests. A concrete command isn't supposed to perform the work on its own, but rather to pass the call to one of the business logic objects. However, for the sake of simplifying the code, these classes can be merged.

Parameters required to execute a method on a receiving object can be declared as fields in the concrete command. You can make command objects immutable by only allowing the initialization of these fields via the constructor.

4. The **Receiver** class contains some business logic. Almost any object may act as a receiver. Most commands only handle the details of how a request is passed to the receiver, while the receiver itself does the actual work.

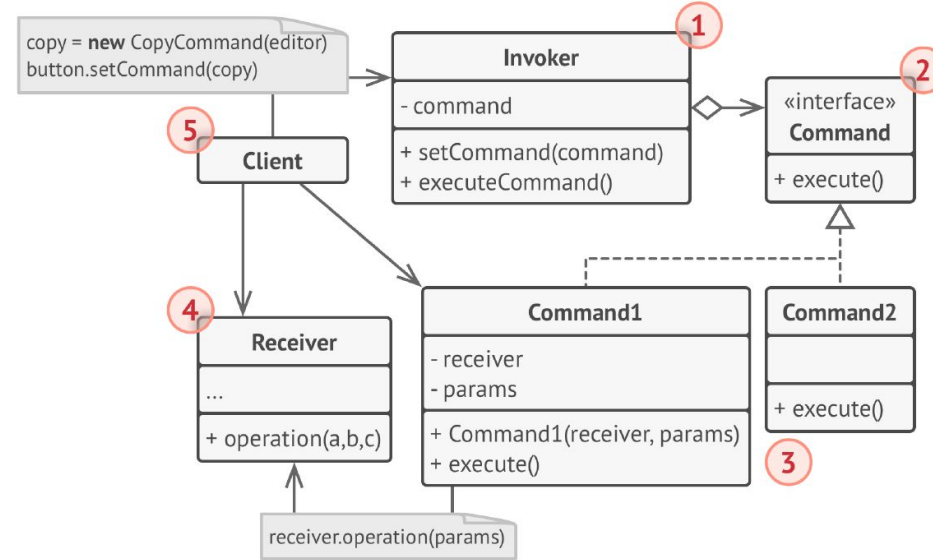


Command Pattern Defined

3. **Concrete Commands** implement various kinds of requests. A concrete command isn't supposed to perform the work on its own, but rather to pass the call to one of the business logic objects. However, for the sake of simplifying the code, these classes can be merged.

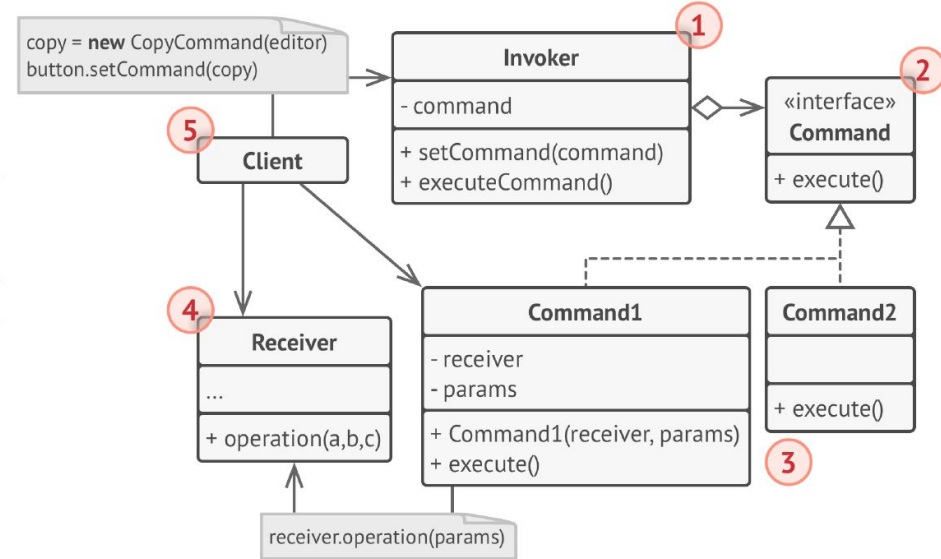
Parameters required to execute a method on a receiving object can be declared as fields in the concrete command. You can make command objects immutable by only allowing the initialization of these fields via the constructor.

4. The **Receiver** class contains some business logic. Almost any object may act as a receiver. Most commands only handle the details of how a request is passed to the receiver, while the receiver itself does the actual work. *operating system?*



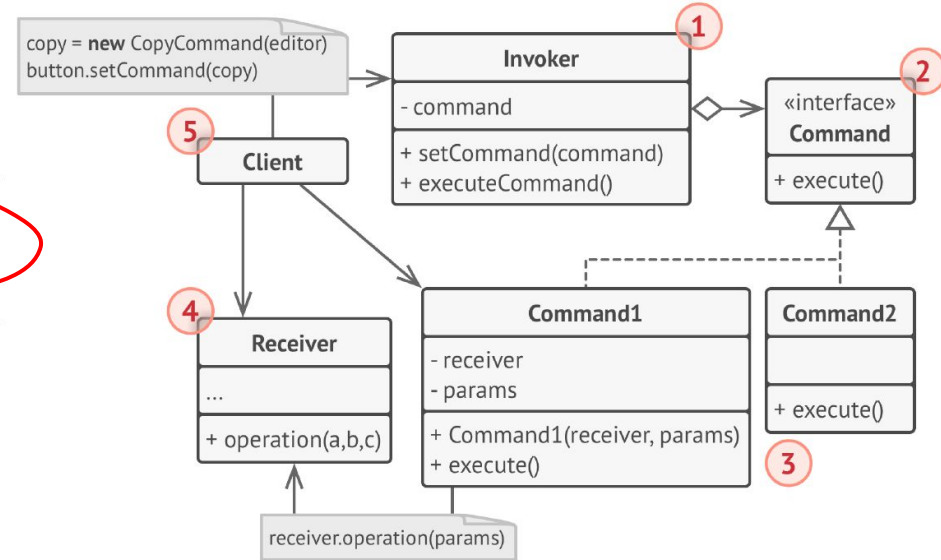
Command Pattern Defined

5. The **Client** creates and configures concrete command objects. The client must pass all of the request parameters, including a receiver instance, into the command's constructor. After that, the resulting command may be associated with one or multiple senders.

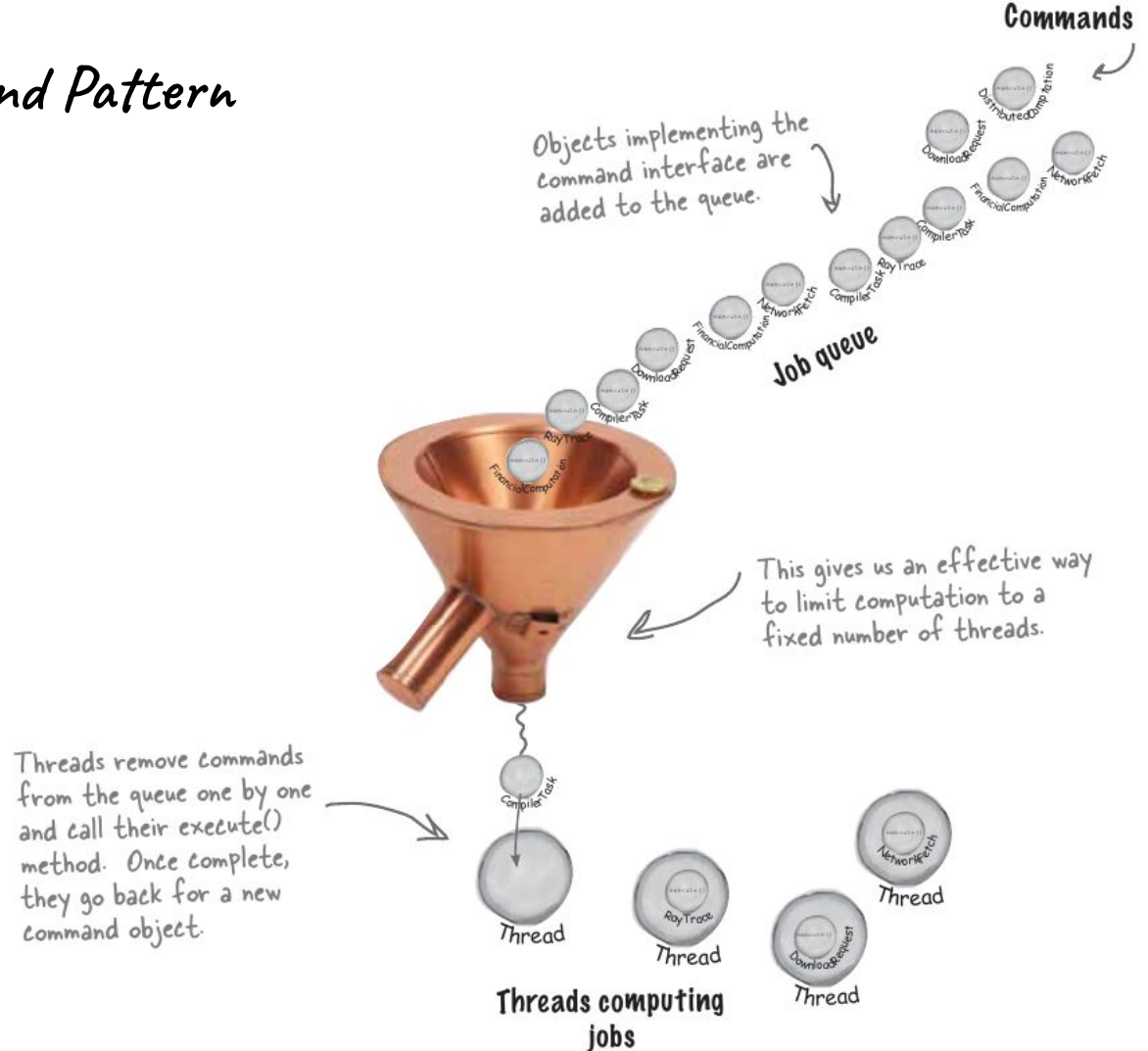


Command Pattern Defined

5. The **Client** creates and configures concrete command objects. The client must pass all of the request parameters, including a receiver instance, into the command's constructor. After that, the resulting command may be associated with one or multiple senders.



Queuing request in Command Pattern



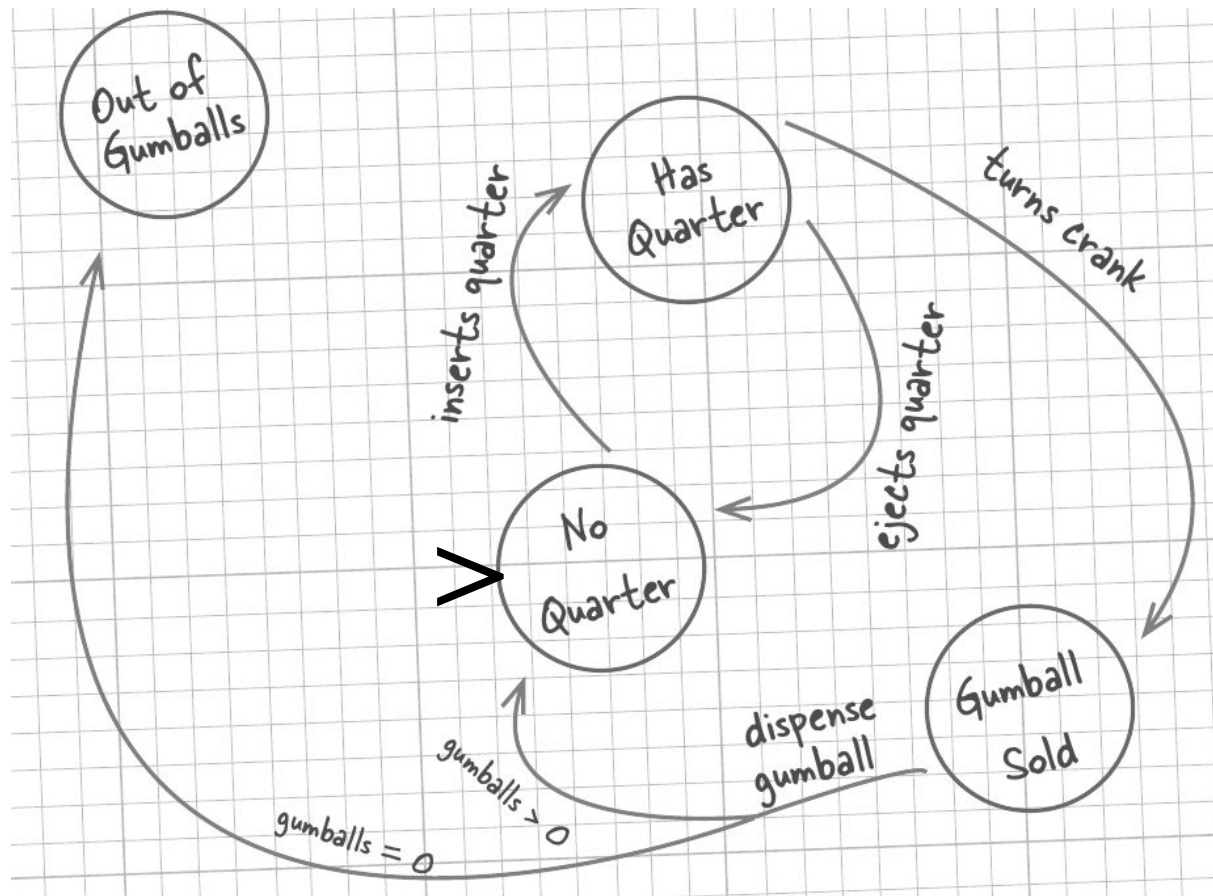
Difference Between Command and Mediator

- Command establishes **unidirectional** connections between senders and receivers.
- Mediator eliminates **direct connections** between senders and receivers, forcing them to communicate indirectly via a **mediator** object.

The Gumball Machine

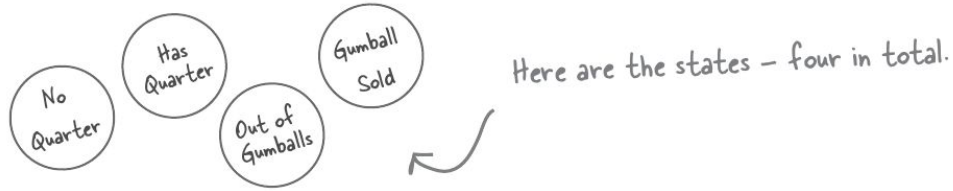


The Gumball Machine Workflow



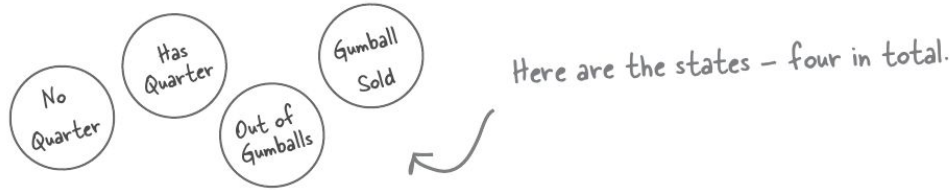
Let's try to write code...

- 1 First, gather up your states:



Let's try to write code...

- 1 First, gather up your states:



- 2 Next, create an instance variable to hold the current state, and define values for each of the states:

Let's just call "Out of Gumballs"
"Sold Out" for short.

```
final static int SOLD_OUT = 0;  
final static int NO_QUARTER = 1;  
final static int HAS_QUARTER = 2;  
final static int SOLD = 3;
```

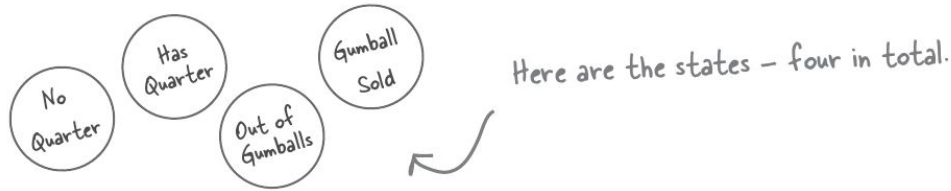
```
int state = SOLD_OUT;
```

Here's each state represented
as a unique integer...

...and here's an instance variable that holds the
current state. We'll go ahead and set it to
"Sold Out" since the machine will be unfilled when
it's first taken out of its box and turned on.

Let's try to write code...

1 First, gather up your states:



2 Next, create an instance variable to hold the current state, and define values for each of the states:

Let's just call "Out of Gumballs"
"Sold Out" for short.

```
final static int SOLD_OUT = 0;  
final static int NO_QUARTER = 1;  
final static int HAS_QUARTER = 2;  
final static int SOLD = 3;
```

```
int state = SOLD_OUT;
```

Here's each state represented
as a unique integer...

...and here's an instance variable that holds the
current state. We'll go ahead and set it to
"Sold Out" since the machine will be unfilled when
it's first taken out of its box and turned on.

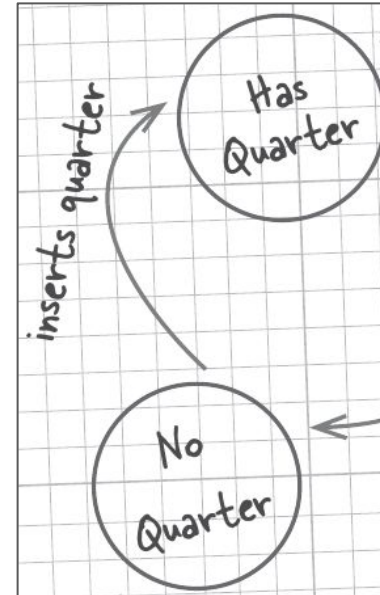
Current State

Let's try to write code...

```
public void insertQuarter() {  
    if (state == HAS_QUARTER) {  
        System.out.println("You can't insert another quarter");  
    } else if (state == SOLD_OUT) {  
        System.out.println("You can't insert a quarter, the machine is sold out");  
    } else if (state == SOLD) {  
        System.out.println("Please wait, we're already giving you a gumball");  
    } else if (state == NO_QUARTER) {  
        state = HAS_QUARTER;  
        System.out.println("You inserted a quarter");  
    }  
}
```

Each possible state is checked with a conditional statement...

...and exhibits the appropriate behavior for each possible state...



...but can also transition to other states, just as depicted in the diagram.

The Gumball Machine, how states are managed?

```
public class GumballMachine {  
  
    final static int SOLD_OUT = 0;  
    final static int NO_QUARTER = 1;  
    final static int HAS_QUARTER = 2;  
    final static int SOLD = 3;  
  
    int state = SOLD_OUT;  
    int count = 0;  
  
    public GumballMachine(int count) {  
        this.count = count;  
        if (count > 0) {  
            state = NO_QUARTER;  
        }  
    }  
}
```

Here are the four states; they match the states in Mighty Gumball's state diagram.

Here's the instance variable that is going to keep track of the current state we're in. We start in the `SOLD_OUT` state.

We have a second instance variable that keeps track of the number of gumballs in the machine.

The constructor takes an initial inventory of gumballs. If the inventory isn't zero, the machine enters state `NO_QUARTER`, meaning it is waiting for someone to insert a quarter, otherwise it stays in the `SOLD_OUT` state.

The Gumball Machine, how states are managed?

```
public class GumballMachine {
```

```
    final static int SOLD_OUT = 0;  
    final static int NO_QUARTER = 1;  
    final static int HAS_QUARTER = 2;  
    final static int SOLD = 3;
```

```
    int state = SOLD_OUT;  
    int count = 0;
```

```
    public GumballMachine(int count) {  
        this.count = count;  
        if (count > 0) {  
            state = NO_QUARTER;  
        }  
    }
```

Start at no quarter state

Here are the four states; they match the states in Mighty Gumball's state diagram.

Here's the instance variable that is going to keep track of the current state we're in. We start in the `SOLD_OUT` state.

We have a second instance variable that keeps track of the number of gumballs in the machine.

The constructor takes an initial inventory of gumballs. If the inventory isn't zero, the machine enters state `NO_QUARTER`, meaning it is waiting for someone to insert a quarter, otherwise it stays in the `SOLD_OUT` state.

The Gumball Machine, action codes...

```
public void ejectQuarter() {  
    if (state == HAS_QUARTER) {  
        System.out.println("Quarter returned");  
        state = NO_QUARTER;  
    } else if (state == NO_QUARTER) {  
        System.out.println("You haven't inserted a quarter");  
    } else if (state == SOLD) {  
        System.out.println("Sorry, you already turned the crank");  
    } else if (state == SOLD_OUT) {  
        System.out.println("You can't eject, you haven't inserted a quarter yet");  
    }  
}
```

Now, if the customer tries to remove the quarter...

If there is a quarter, we return it and go back to the NO_QUARTER state.

Otherwise, if there isn't one we can't give it back.

You can't eject if the machine is sold out, it doesn't accept quarters!

If the customer just turned the crank, we can't give a refund; he already has the gumball!

```
public void turnCrank() {  
    if (state == SOLD) {  
        System.out.println("Turning twice doesn't get you another gumball!");  
    } else if (state == NO_QUARTER) {  
        System.out.println("You turned but there's no quarter");  
    } else if (state == SOLD_OUT) {  
        System.out.println("You turned, but there are no gumballs");  
    } else if (state == HAS_QUARTER) {  
        System.out.println("You turned...");  
        state = SOLD;  
        dispense();  
    }  
}
```

The customer tries to turn the crank...

Someone's trying to cheat the machine.

We need a quarter first.

We can't deliver gumballs; there are none.

Success! They get a gumball. Change the state to SOLD and call the machine's dispense() method.

The Gumball Machine, action codes...

Problems?

```
public void ejectQuarter() {  
    if (state == HAS_QUARTER) {  
        System.out.println("Quarter returned");  
        state = NO_QUARTER;  
    } else if (state == NO_QUARTER) {  
        System.out.println("You haven't inserted a quarter");  
    } else if (state == SOLD) {  
        System.out.println("Sorry, you already turned the crank");  
    } else if (state == SOLD_OUT) {  
        System.out.println("You can't eject, you haven't inserted a quarter yet");  
    }  
}
```

Now, if the customer tries to remove the quarter...

If there is a quarter, we return it and go back to the NO_QUARTER state.

Otherwise, if there isn't one we can't give it back.

You can't eject if the machine is sold out, it doesn't accept quarters!

If the customer just turned the crank, we can't give a refund; he already has the gumball!

```
public void turnCrank() {  
    if (state == SOLD) {  
        System.out.println("Turning twice doesn't get you another gumball!");  
    } else if (state == NO_QUARTER) {  
        System.out.println("You turned but there's no quarter");  
    } else if (state == SOLD_OUT) {  
        System.out.println("You turned, but there are no gumballs");  
    } else if (state == HAS_QUARTER) {  
        System.out.println("You turned...");  
        state = SOLD;  
        dispense();  
    }  
}
```

The customer tries to turn the crank...

Someone's trying to cheat the machine.

We need a quarter first.

We can't deliver gumballs; there are none.

Success! They get a gumball. Change the state to SOLD and call the machine's dispense() method.

The Gumball Machine, action codes...

```
public void ejectQuarter() {  
    if (state == HAS_QUARTER) {  
        System.out.println("Quarter returned");  
        state = NO_QUARTER;  
    } else if (state == NO_QUARTER) {  
        System.out.println("You haven't inserted a quarter");  
    } else if (state == SOLD) {  
        System.out.println("Sorry, you already turned the crank");  
    } else if (state == SOLD_OUT) {  
        System.out.println("You can't eject, you haven't inserted a quarter yet");  
    }  
}  
  
public void turnCrank() {  
    if (state == SOLD) {  
        System.out.println("Turning twice doesn't get you another gumball!");  
    } else if (state == NO_QUARTER) {  
        System.out.println("You turned but there's no quarter");  
    } else if (state == SOLD_OUT) {  
        System.out.println("You turned, but there are no gumballs");  
    } else if (state == HAS_QUARTER) {  
        System.out.println("You turned...");  
        state = SOLD;  
        dispense();  
    }  
}
```

Now, if the customer tries to remove the quarter...

If there is a quarter, we return it and go back to the NO_QUARTER state.

Otherwise, if there isn't one we can't give it back.

You can't eject if the machine is sold out, it doesn't accept quarters!

The customer tries to turn the crank...

Someone's trying to cheat the machine.

We need a quarter first.

We can't deliver gumballs; there are none.

Success! They get a gumball. Change the state to SOLD and call the machine's dispense() method.

Problems?

The nightmare of change!!

The Gumball Machine, action codes...

```
public void ejectQuarter() {  
    if (state == HAS_QUARTER) {  
        System.out.println("Quarter returned");  
        state = NO_QUARTER;  
    } else if (state == NO_QUARTER) {  
        System.out.println("You haven't inserted a quarter");  
    } else if (state == SOLD) {  
        System.out.println("Sorry, you already turned the crank");  
    } else if (state == SOLD_OUT) {  
        System.out.println("You can't eject, you haven't inserted a quarter yet");  
    }  
}  
  
public void turnCrank() {  
    if (state == SOLD) {  
        System.out.println("Turning twice doesn't get you another gumball!");  
    } else if (state == NO_QUARTER) {  
        System.out.println("You turned but there's no quarter");  
    } else if (state == SOLD_OUT) {  
        System.out.println("You turned, but there are no gumballs");  
    } else if (state == HAS_QUARTER) {  
        System.out.println("You turned...");  
        state = SOLD;  
        dispense();  
    }  
}
```

Now, if the customer tries to remove the quarter...

If there is a quarter, we return it and go back to the NO_QUARTER state.

Otherwise, if there isn't one we can't give it back.

You can't eject if the machine is sold out, it doesn't accept quarters!

The customer tries to turn the crank...

Someone's trying to cheat the machine.

We need a quarter first.

We can't deliver gumballs; there are none.

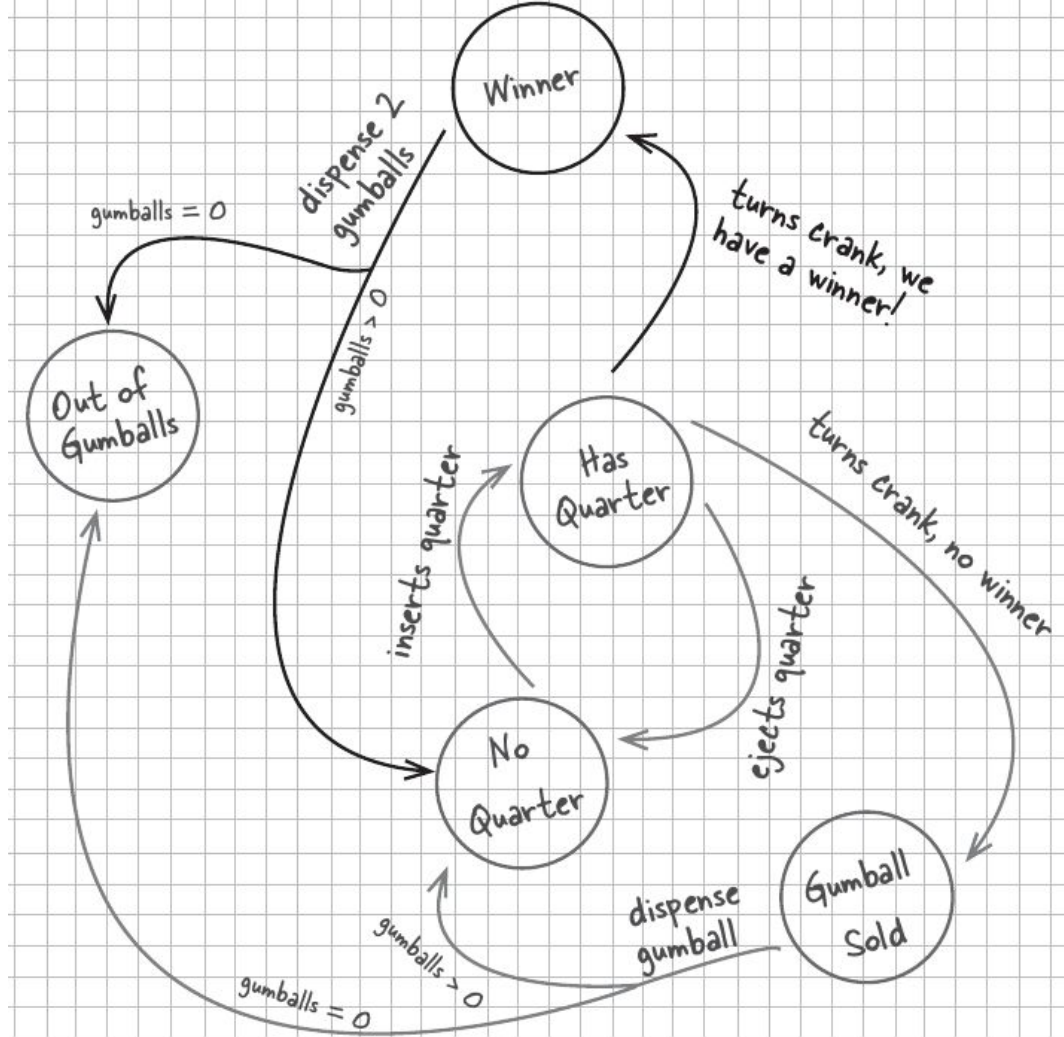
Success! They get a gumball. Change the state to SOLD and call the machine's dispense() method.

Problems?

The nightmare of change!!

To scale the sales, new *policy*:
10% of the time, customer wins a **free** gumball! (*Buy one, get one!*)

Updated State Diagram



The Gumball Machine, action codes...

```
public void ejectQuarter() {  
    if (state == HAS_QUARTER) {  
        System.out.println("Quarter returned");  
        state = NO_QUARTER;  
    } else if (state == NO_QUARTER) {  
        System.out.println("You haven't inserted a quarter");  
    } else if (state == SOLD) {  
        System.out.println("Sorry, you already turned the crank");  
    } else if (state == SOLD_OUT) {  
        System.out.println("You can't eject, you haven't inserted a quarter yet");  
    }  
}  
  
public void turnCrank() {  
    if (state == SOLD) {  
        System.out.println("Turning twice doesn't get you another gumball!");  
    } else if (state == NO_QUARTER) {  
        System.out.println("You turned but there's no quarter");  
    } else if (state == SOLD_OUT) {  
        System.out.println("You turned, but there are no gumballs");  
    } else if (state == HAS_QUARTER) {  
        System.out.println("You turned...");  
        state = SOLD;  
        dispense();  
    }  
}
```

Now, if the customer tries to remove the quarter...

If there is a quarter, we return it and go back to the NO_QUARTER state.

Otherwise, if there isn't one we can't give it back.

You can't eject if the machine is sold out, it doesn't accept quarters!

If the customer just turned the crank, we can't give a refund; he already has the gumball!

The customer tries to turn the crank...

Someone's trying to cheat the machine.

We need a quarter first.

We can't deliver gumballs; there are none.

Success! They get a gumball. Change the state to SOLD and call the machine's dispense() method.

Problems?

The nightmare of change!!

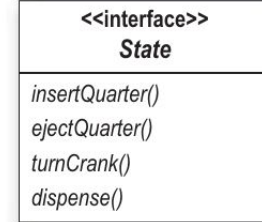
To scale the sales, new *policy*:
10% of the time, customer wins a **free** gumball! (*Buy one, get one!*)

Rewrite all action functions?

```
insertQuarter()  
ejectQuarter()  
turnCrank()  
dispense()
```

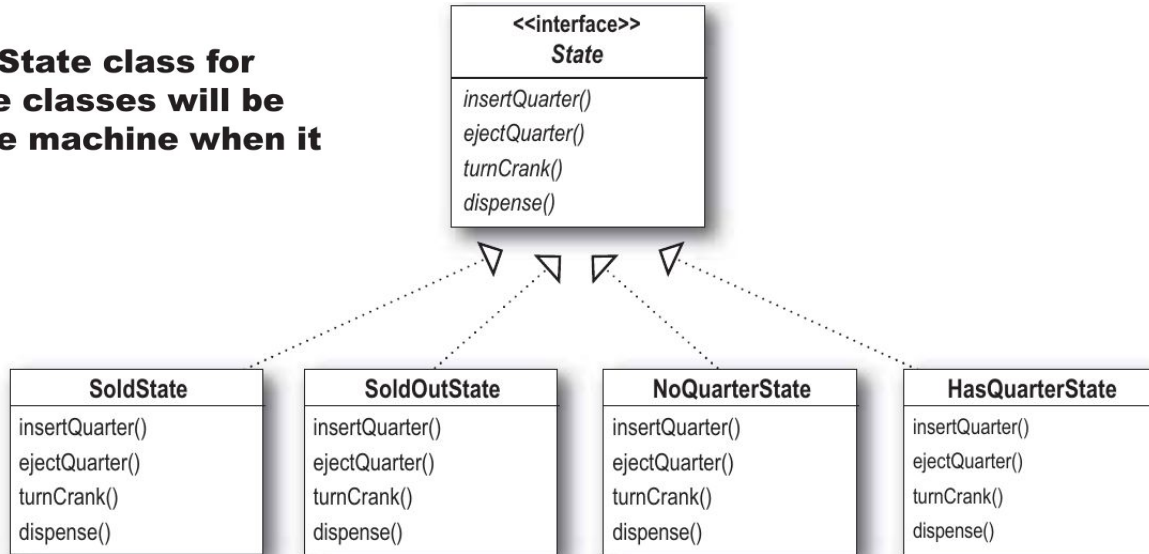
The new design

- 1 First, we're going to define a **State** interface that contains a method for every action in the **Gumball Machine**.



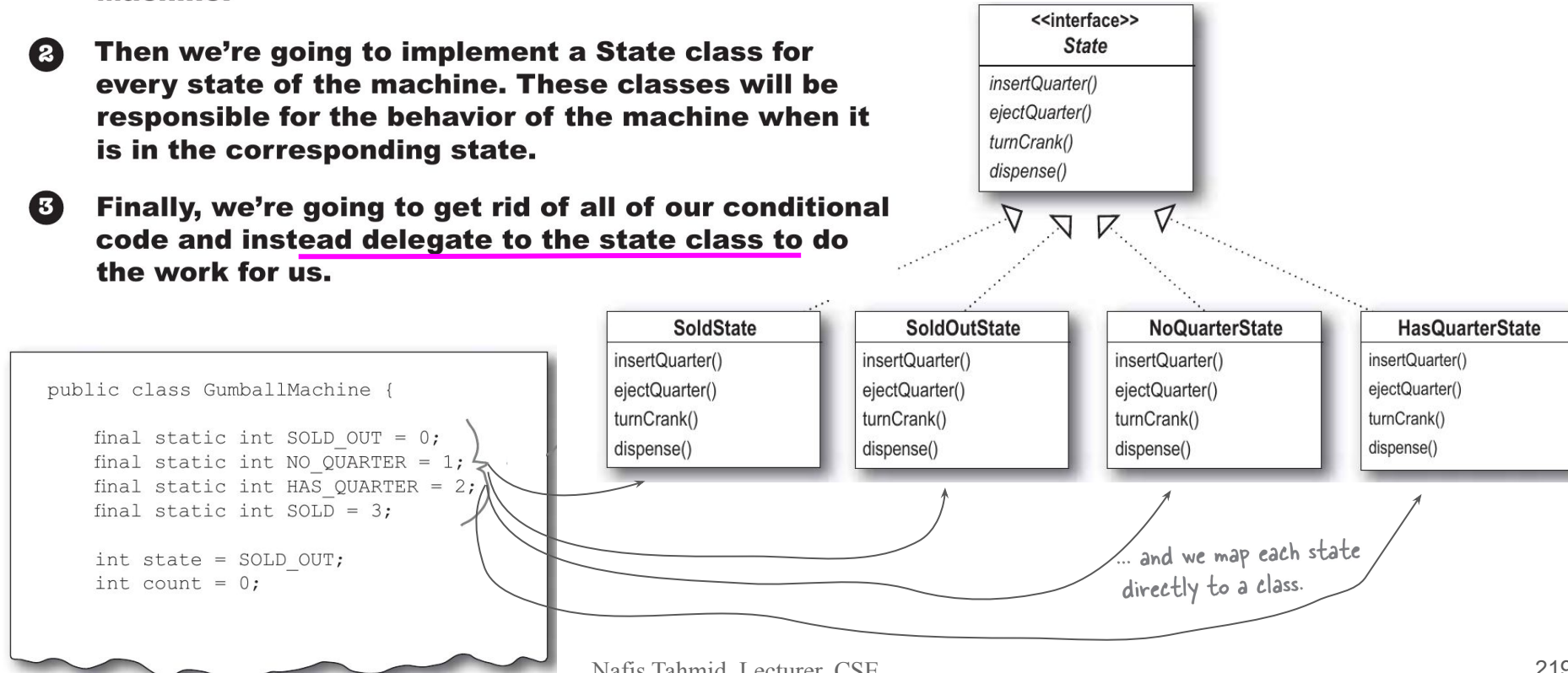
The new design

- 1 First, we're going to define a **State** interface that contains a method for every action in the **Gumball Machine**.
- 2 Then we're going to implement a **State** class for every state of the machine. These classes will be responsible for the behavior of the machine when it is in the corresponding state.



The new design

- 1 First, we're going to define a **State** interface that contains a method for every action in the **Gumball Machine**.
- 2 Then we're going to implement a **State** class for every state of the machine. These classes will be responsible for the behavior of the machine when it is in the corresponding state.
- 3 Finally, we're going to get rid of all of our conditional code and instead delegate to the state class to do the work for us.



State classes...

First we need to implement the State interface.

We get passed a reference to the Gumball Machine through the constructor. We're just going to stash this in an instance variable.

```
public class NoQuarterState implements State {  
    GumballMachine gumballMachine;
```

```
    public NoQuarterState(GumballMachine gumballMachine) {  
        this.gumballMachine = gumballMachine;  
    }
```

```
    public void insertQuarter() {  
        System.out.println("You inserted a quarter");  
        gumballMachine.setState(gumballMachine.getHasQuarterState());  
    }
```

If someone inserts a quarter, we print a message saying the quarter was accepted and then change the machine's state to the HasQuarterState.

You'll see how these work in just a sec...

```
    public void ejectQuarter() {  
        System.out.println("You haven't inserted a quarter");  
    }
```

You can't get money back if you never gave it to us!

```
    public void turnCrank() {  
        System.out.println("You turned, but there's no quarter");  
    }
```

```
    public void dispense() {  
        System.out.println("You need to pay first");  
    }  
}
```

And, you can't get a gumball if you don't pay us.

We can't be dispensing gumballs without payment.

State classes...

First we need to implement the State interface.

```
public class NoQuarterState implements State {
    GumballMachine gumballMachine;

    public NoQuarterState(GumballMachine gumballMachine) {
        this.gumballMachine = gumballMachine;
    }

    public void insertQuarter() {
        System.out.println("You inserted a quarter");
        gumballMachine.setState(gumballMachine.getHasQuarterState());
    }

    public void ejectQuarter() {
        System.out.println("You haven't inserted a quarter");
    }

    public void turnCrank() {
        System.out.println("You turned, but there's no quarter");
    }

    public void dispense() {
        System.out.println("You need to pay first");
    }
}
```

We get passed a reference to the Gumball Machine through the constructor. We're just going to stash this in an instance variable.

If someone inserts a quarter, we print a message saying the quarter was accepted and then change the machine's state to the HasQuarterState.

You'll see how these work in just a sec...

You can't get money back if you never gave it to us!

And, you can't get a gumball if you don't pay us.

We can't be dispensing gumballs without payment.

State classes...

First we need to implement the State interface.

We get passed a reference to the Gumball Machine through the constructor. We're just going to stash this in an instance variable.

```
public class NoQuarterState implements State {  
    GumballMachine gumballMachine;
```

```
    public NoQuarterState(GumballMachine gumballMachine) {  
        this.gumballMachine = gumballMachine;  
    }
```

```
    public void insertQuarter() {  
        System.out.println("You inserted a quarter");  
        gumballMachine.setState(gumballMachine.getHasQuarterState());  
    }
```

```
    public void ejectQuarter() {  
        System.out.println("You haven't inserted a quarter");  
    }
```

```
    public void turnCrank() {  
        System.out.println("You turned, but there's no quarter");  
    }
```

```
    public void dispense() {  
        System.out.println("You need to pay first");  
    }
```

```
}
```

If someone inserts a quarter, we print a message saying the quarter was accepted and then change the machine's state to the HasQuarterState.

You'll see how these work in just a sec...

You can't get money back if you never gave it to us!

And, you can't get a gumball if you don't pay us.

We can't be dispensing gumballs without payment.

Machine ref. passed

State classes...

First we need to implement the State interface.

We get passed a reference to the Gumball Machine through the constructor. We're just going to stash this in an instance variable.

```
public class NoQuarterState implements State {  
    GumballMachine gumballMachine;
```

```
    public NoQuarterState(GumballMachine gumballMachine) {  
        this.gumballMachine = gumballMachine;  
    }
```

```
    public void insertQuarter() {  
        System.out.println("You inserted a quarter");  
        gumballMachine.setState(gumballMachine.getHasQuarterState());  
    }
```

If someone inserts a quarter, we print a message saying the quarter was accepted and then change the machine's state to the HasQuarterState.

You'll see how these work in just a sec...

```
    public void ejectQuarter() {  
        System.out.println("You haven't inserted a quarter");  
    }
```

You can't get money back if you never gave it to us!

```
    public void turnCrank() {  
        System.out.println("You turned, but there's no quarter");  
    }
```

```
    public void dispense() {  
        System.out.println("You need to pay first");  
    }
```

And, you can't get a gumball if you don't pay us.

We can't be dispensing gumballs without payment.

```
}
```

Knows who's next

State classes...

First we need to implement the State interface.

We get passed a reference to the Gumball Machine through the constructor. We're just going to stash this in an instance variable.

```
public class NoQuarterState implements State {  
    GumballMachine gumballMachine;
```

```
    public NoQuarterState(GumballMachine gumballMachine) {  
        this.gumballMachine = gumballMachine;  
    }
```

```
    public void insertQuarter() {  
        System.out.println("You inserted a quarter");  
        gumballMachine.setState(gumballMachine.getHasQuarterState());  
    }
```

If someone inserts a quarter, we print a message saying the quarter was accepted and then change the machine's state to the HasQuarterState.

You'll see how these work in just a sec...

```
    public void ejectQuarter() {  
        System.out.println("You haven't inserted a quarter");  
    }
```

You can't get money back if you never gave it to us!

```
    public void turnCrank() {  
        System.out.println("You turned, but there's no quarter");  
    }
```

```
    public void dispense() {  
        System.out.println("You need to pay first");  
    }
```

And, you can't get a gumball if you don't pay us.

We can't be dispensing gumballs without payment.

Invalid Ops.

The change in the machine

```
public class GumballMachine {
```

```
    final static int SOLD_OUT = 0;  
    final static int NO_QUARTER = 1;  
    final static int HAS_QUARTER = 2;  
    final static int SOLD = 3;
```

```
    int state = SOLD_OUT;  
    int count = 0;
```

In the GumballMachine, we update the code to use the new classes rather than the static integers. The code is quite similar, except that in one class we have integers and in the other objects...

Old code

```
public class GumballMachine {
```

```
    State soldOutState;  
    State noQuarterState;  
    State hasQuarterState;  
    State soldState;
```

```
    State state = soldOutState;  
    int count = 0;
```

New code

All the State objects are created and assigned in the constructor.

This now holds a State object, not an integer.

The change in the machine

```
public class GumballMachine {
```

```
    State soldOutState;  
    State noQuarterState;  
    State hasQuarterState;  
    State soldState;
```

```
    State state = soldOutState;  
    int count = 0;
```

```
    public GumballMachine(int numberGumballs) {  
        soldOutState = new SoldOutState(this);  
        noQuarterState = new NoQuarterState(this);  
        hasQuarterState = new HasQuarterState(this);  
        soldState = new SoldState(this);  
        this.count = numberGumballs;  
        if (numberGumballs > 0) {  
            state = noQuarterState;  
        }  
    }
```

Here are all the States again...

...and the State instance variable.

The count instance variable holds the count of gumballs – initially the machine is empty.

Our constructor takes the initial number of gumballs and stores it in an instance variable.

It also creates the State instances, one of each.

If there are more than 0 gumballs we set the state to the NoQuarterState.

The change in the machine

```
public class GumballMachine {
```

```
    State soldOutState;  
    State noQuarterState;  
    State hasQuarterState;  
    State soldState;
```

```
    State state = soldOutState;  
    int count = 0;
```

```
    public GumballMachine(int numberGumballs) {
```

```
        soldOutState = new SoldOutState(this);  
        noQuarterState = new NoQuarterState(this);  
        hasQuarterState = new HasQuarterState(this);  
        soldState = new SoldState(this);  
        this.count = numberGumballs;  
        if (numberGumballs > 0) {  
            state = noQuarterState;  
        }  
    }
```

```
}
```

Here are all the States again...

...and the State instance variable.

The count instance variable holds the count of gumballs – initially the machine is empty.

Our constructor takes the initial number of gumballs and stores it in an instance variable.

It also creates the State instances, one of each.

If there are more than 0 gumballs we set the state to the NoQuarterState.

The change in the machine action functions...

```
public void insertQuarter() {  
    state.insertQuarter();  
}
```

```
public void ejectQuarter() {  
    state.ejectQuarter();  
}
```

```
public void turnCrank() {  
    state.turnCrank();  
    state.dispense();  
}
```

```
void setState(State state) {  
    this.state = state;  
}
```

```
void releaseBall() {  
    System.out.println("A gumball comes rolling out the slot...");  
    if (count != 0) {  
        count = count - 1;  
    }  
}
```

Now for the actions. These are **VERY EASY** to implement now. We just delegate to the current state.

Note that we don't need an action method for `dispense()` in `GumballMachine` because it's just an internal action; a user can't ask the machine to dispense directly. But we do call `dispense()` on the `State` object from the `turnCrank()` method.

This method allows other objects (like our `State` objects) to transition the machine to a different state.

The machine supports a `releaseBall()` helper method that releases the ball and decrements the `count` instance variable.

The change in the machine action functions...

```
public void insertQuarter() {  
    state.insertQuarter();  
}  
  
public void ejectQuarter() {  
    state.ejectQuarter();  
}  
  
public void turnCrank() {  
    state.turnCrank();  
    state.dispense();  
}  
  
void setState(State state) {  
    this.state = state;  
}
```

```
void releaseBall() {  
    System.out.println("A gumball comes rolling out the slot...");  
    if (count != 0) {  
        count = count - 1;  
    }  
}
```



Now for the actions. These are
VERY EASY to implement now. We
just delegate to the current state.



Note that we don't need an
action method for dispense() in
GumballMachine because it's just an
internal action; a user can't ask the
machine to dispense directly. But we
do call dispense() on the State object
from the turnCrank() method.

No change when new
state is added

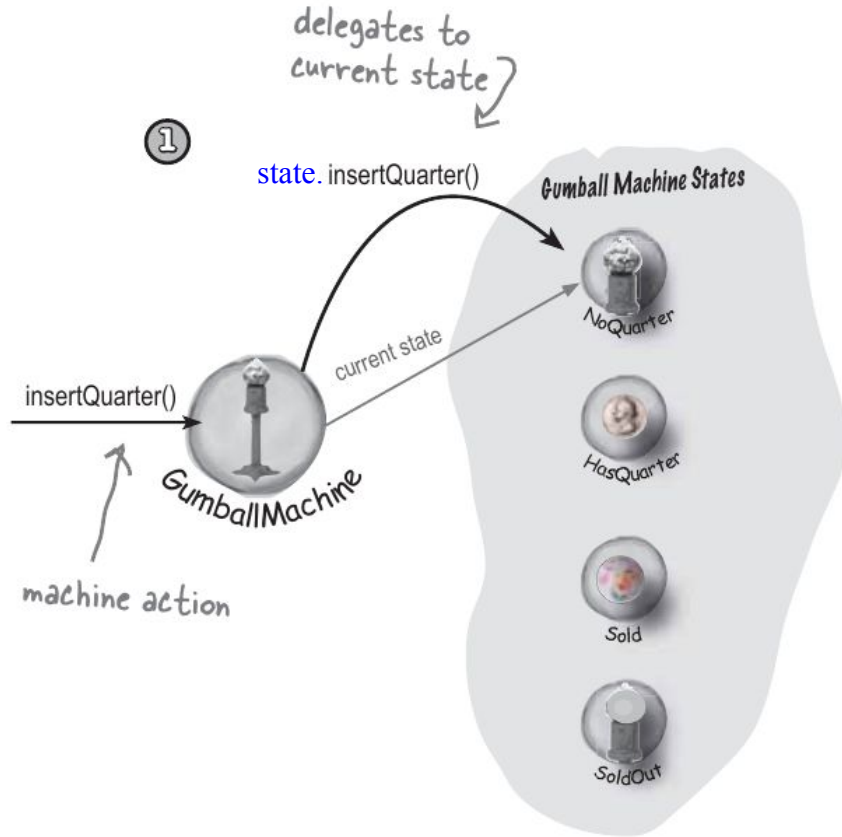


This method allows other objects (like
our State objects) to transition the
machine to a different state.

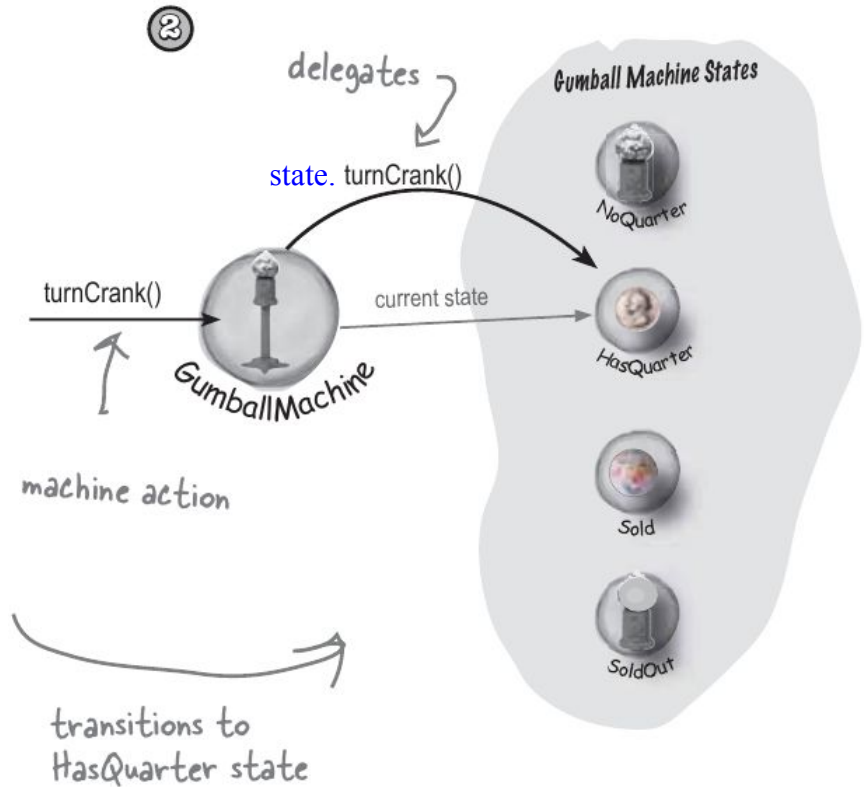
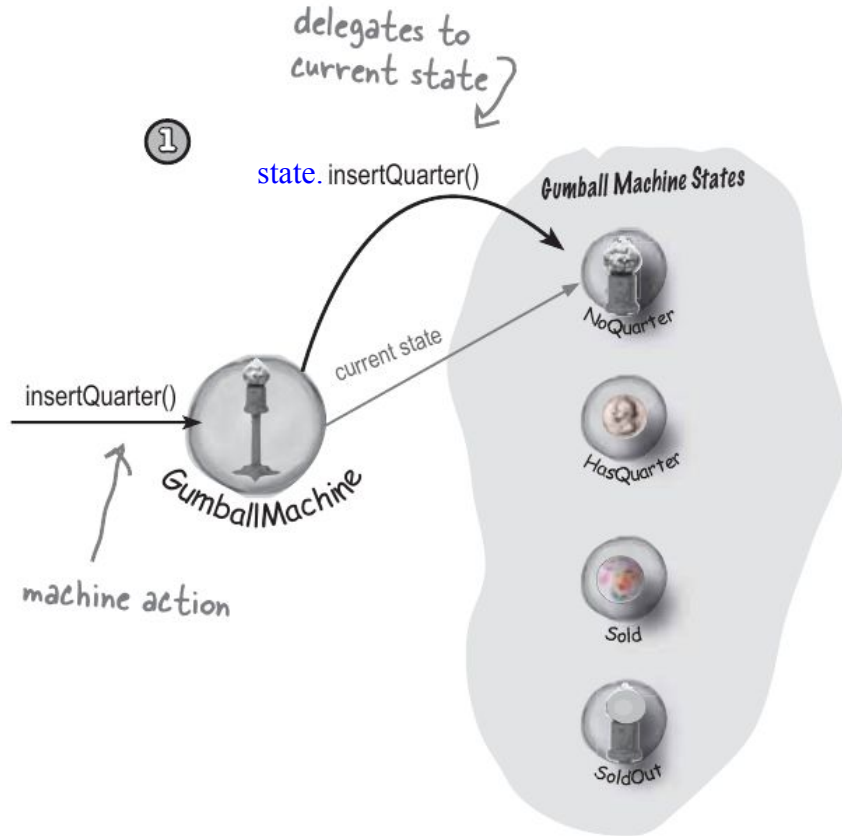


The machine supports a releaseBall()
helper method that releases the ball and
decrements the count instance variable.

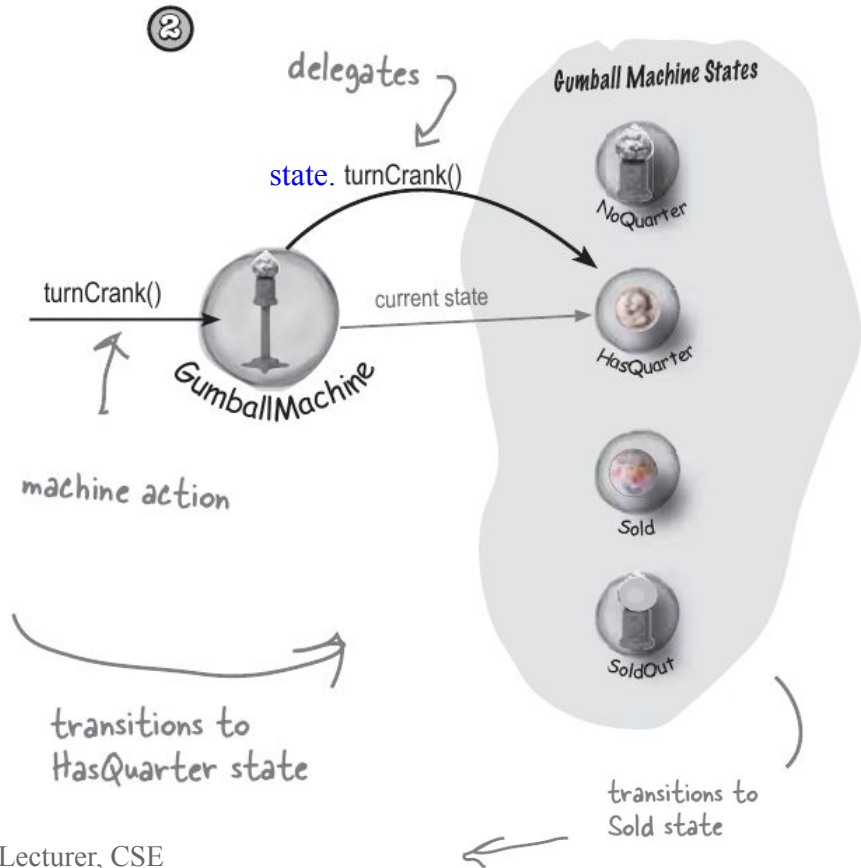
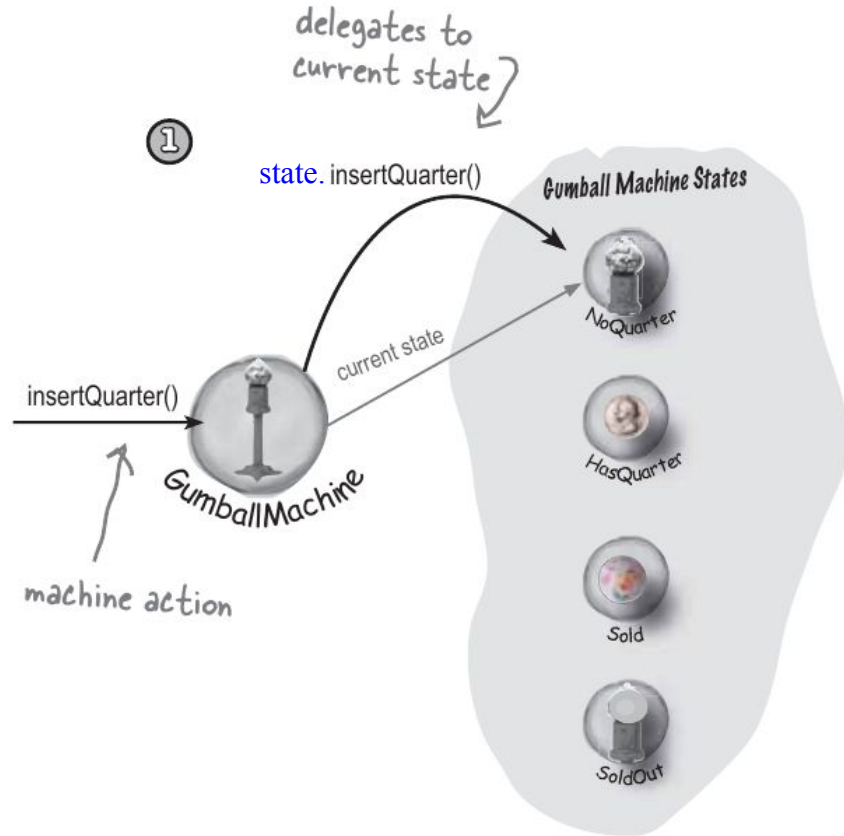
What we have done so far...



What we have done so far...

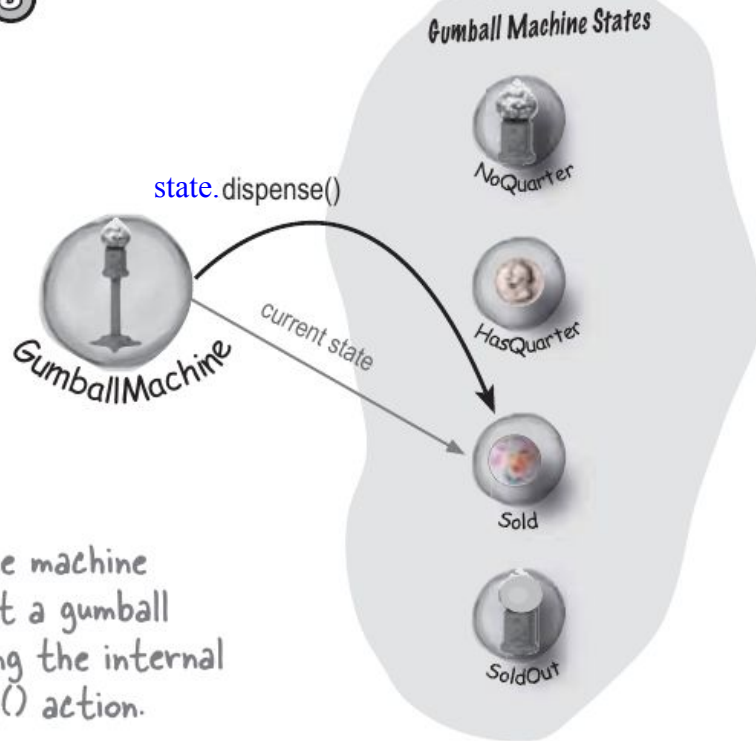


What we have done so far...



What we have done so far...

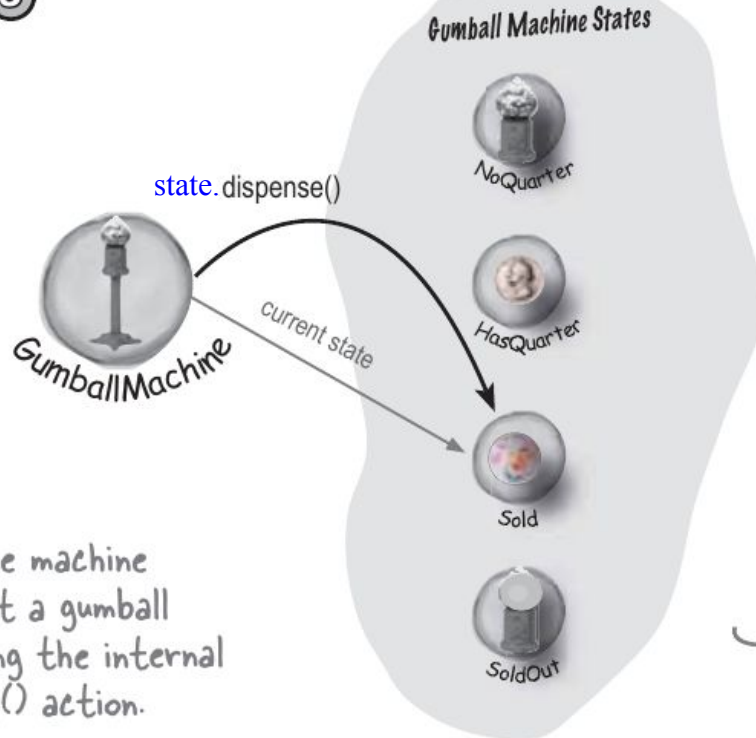
③



Here the machine gives out a gumball by calling the internal `dispense()` action.

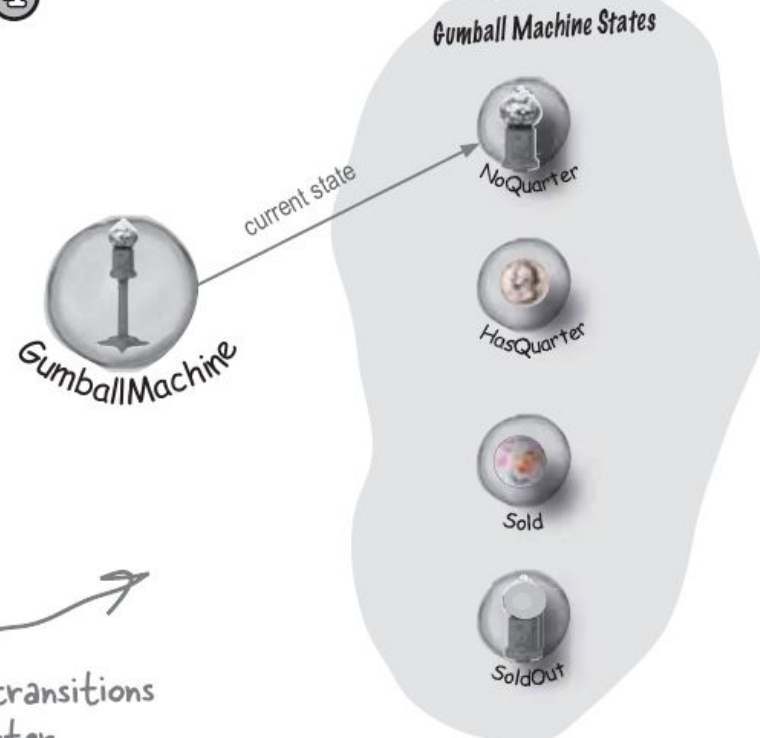
What we have done so far...

③



Here the machine gives out a gumball by calling the internal `dispense()` action.

④

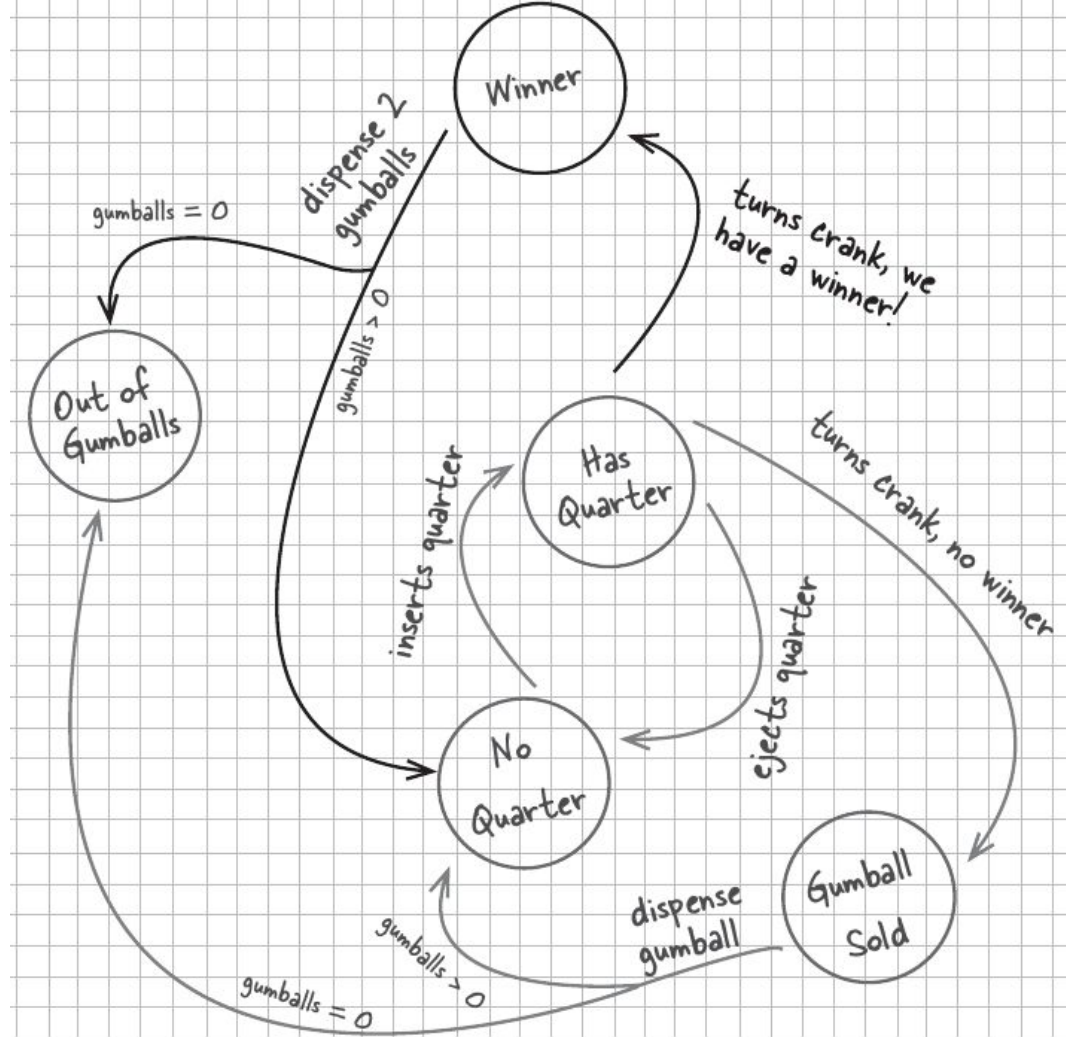


and then transitions to NoQuarter

What we have done so far...

- Localized the behavior of each state into its own class.
- Removed all the troublesome **if** statements that would have been difficult to maintain.
- Closed each state for modification, and yet left the Gumball Machine open to extension by adding new state classes (and we'll do this in a second).

Winner State...



Change in GumballMachine

```
public class GumballMachine {
```

```
    State soldOutState;
```

```
    State noQuarterState;
```

```
    State hasQuarterState;
```

```
    State soldState;
```

```
    State winnerState;
```

```
    State state = soldOutState;
```

```
    int count = 0;
```

```
}
```

All you need to add here is the new WinnerState and initialize it in the constructor.



Winner State

```
public class WinnerState implements State {
```

```
    // instance variables and constructor
```

```
    // insertQuarter error message
```

```
    // ejectQuarter error message
```

```
    // turnCrank error message
```

Just like SoldState.

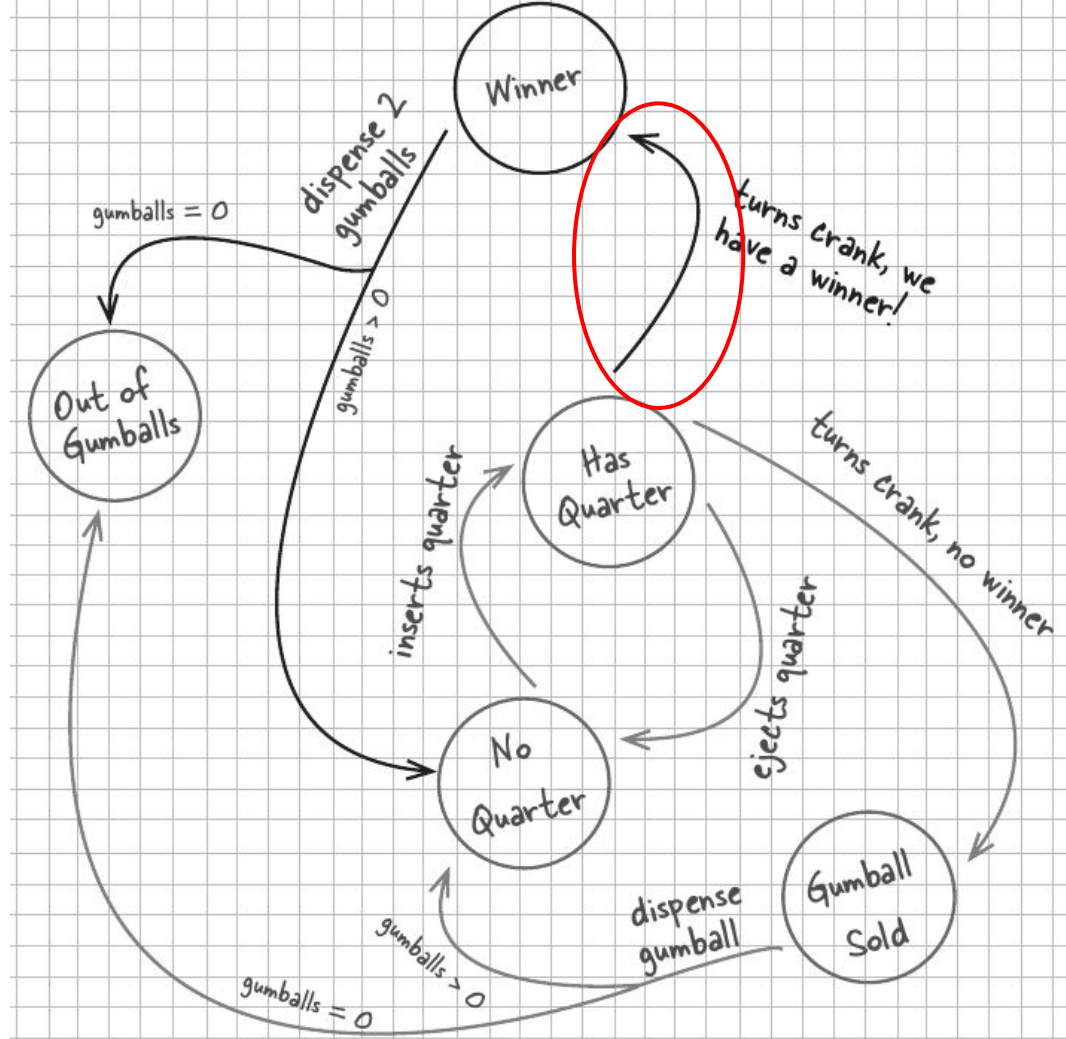
Here we release two gumballs and then either go to the NoQuarterState or the SoldOutState.

```
public void dispense() {
    System.out.println("YOU'RE A WINNER! You get two gumballs for your quarter");
    gumballMachine.releaseBall();
    if (gumballMachine.getCount() == 0) {
        gumballMachine.setState(gumballMachine.getSoldOutState());
    } else {
        gumballMachine.releaseBall();
        if (gumballMachine.getCount() > 0) {
            gumballMachine.setState(gumballMachine.getNoQuarterState());
        } else {
            System.out.println("Oops, out of gumballs!");
            gumballMachine.setState(gumballMachine.getSoldOutState());
        }
    }
}
```

As long as we have a second gumball we release it.

```
}
```

Winner State...



Winner State

```
public class HasQuarterState implements State {  
    Random randomWinner = new Random(System.currentTimeMillis());  
    GumballMachine gumballMachine;  
  
    public HasQuarterState(GumballMachine gumballMachine) {  
        this.gumballMachine = gumballMachine;  
    }  
  
    public void turnCrank() {  
        System.out.println("You turned...");  
        int winner = randomWinner.nextInt(10);  
        if ((winner == 0) && (gumballMachine.getCount() > 1)) {  
            gumballMachine.setState(gumballMachine.getWinnerState());  
        } else {  
            gumballMachine.setState(gumballMachine.getSoldState());  
        }  
    }  
}
```



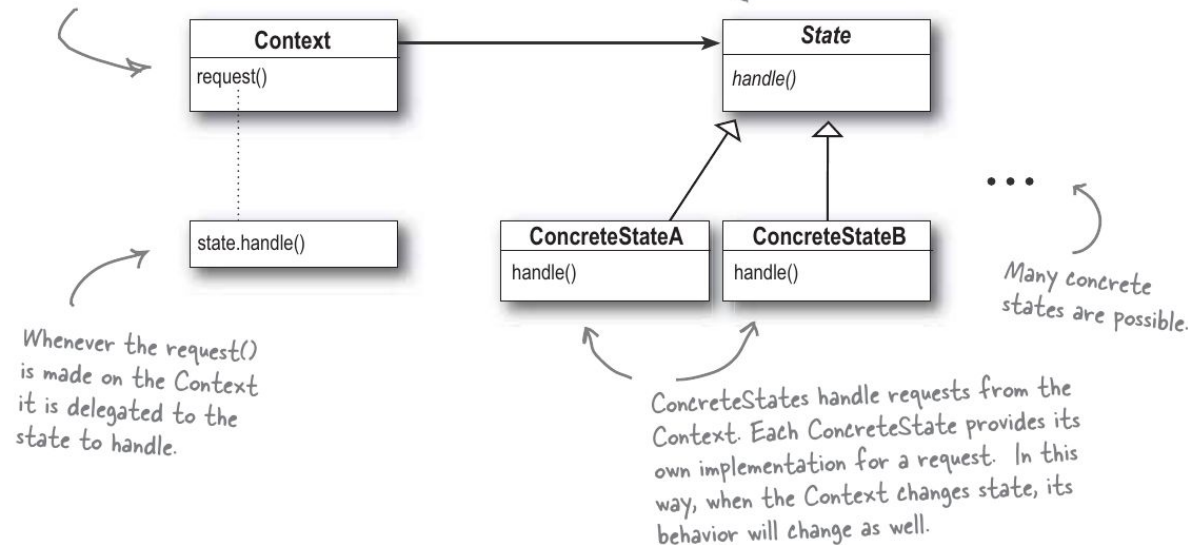
First we add a random number generator to generate the 10% chance of winning...

The State Pattern Defined

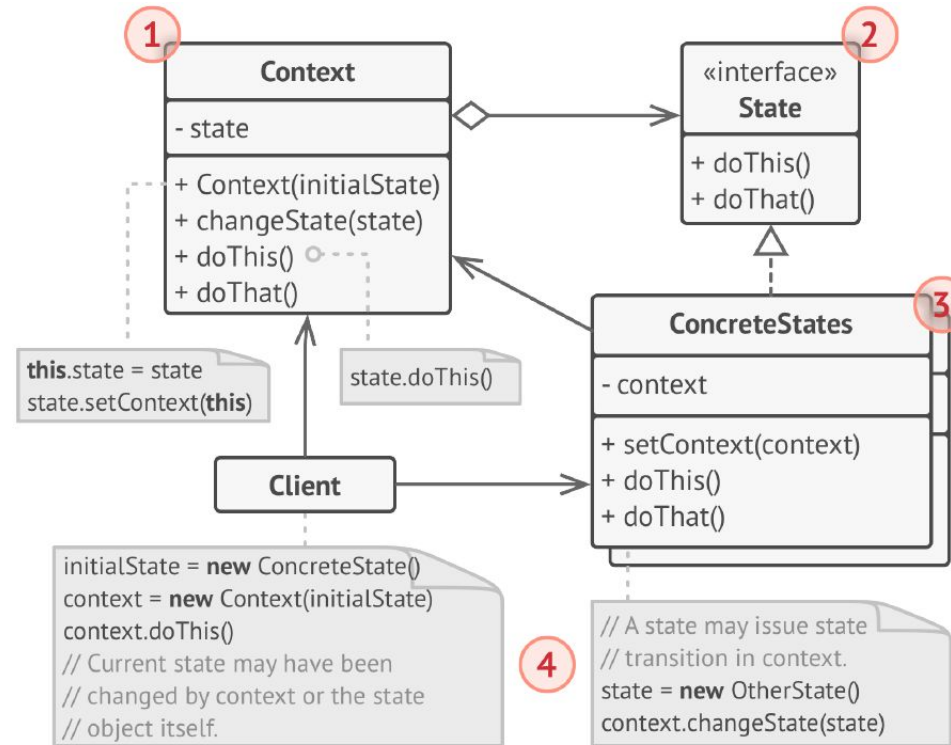
Allow an object to **alter** its behavior when its **internal state** changes. The object will appear to change its **class**.

The Context is the class that can have a number of internal states. In our example, the GumballMachine is the Context.

The State interface defines a common interface for all concrete states; the states all implement the same interface, so they are interchangeable.



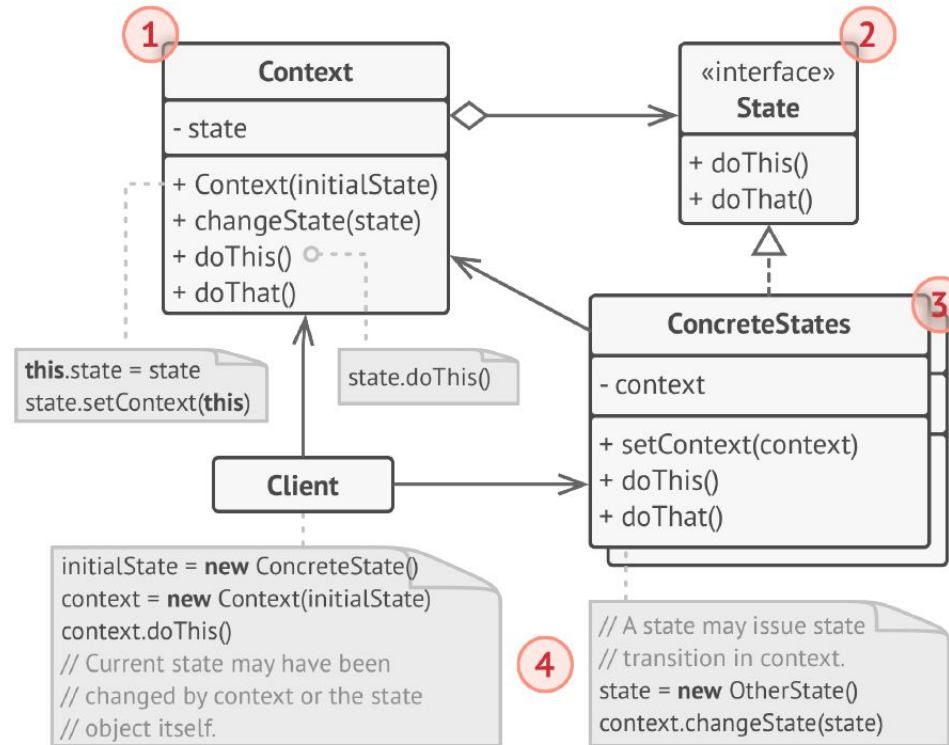
The State Pattern Defined



The State Pattern Defined

1. **Context** stores a reference to one of the concrete state objects and delegates to it all state-specific work. The context communicates with the state object via the state interface. The context exposes a setter for passing it a new state object.
2. The **State** interface declares the state-specific methods. These methods should make sense for all concrete states because you don't want some of your states to have useless methods that will never be called.
3. **Concrete States** provide their own implementations for the state-specific methods. To avoid duplication of similar code across multiple states, you may provide intermediate abstract classes that encapsulate some common behavior.

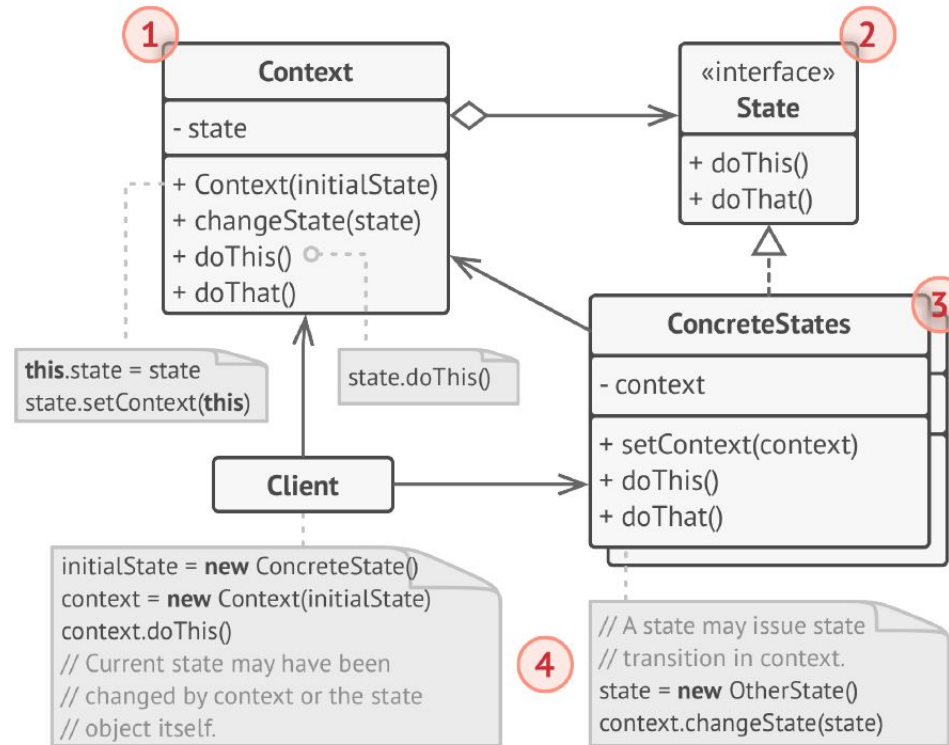
State objects may store a backreference to the context object. Through this reference, the state can fetch any required info from the context object, as well as initiate state transitions.



The State Pattern Defined

1. **Context** stores a reference to one of the concrete state objects and delegates to it all state-specific work. The context communicates with the state object via the state interface. The context exposes a setter for passing it a new state object.
2. The **State** interface declares the state-specific methods. These methods should make sense for all concrete states because you don't want some of your states to have useless methods that will never be called.
3. **Concrete States** provide their own implementations for the state-specific methods. To avoid duplication of similar code across multiple states, you may provide intermediate abstract classes that encapsulate some common behavior.

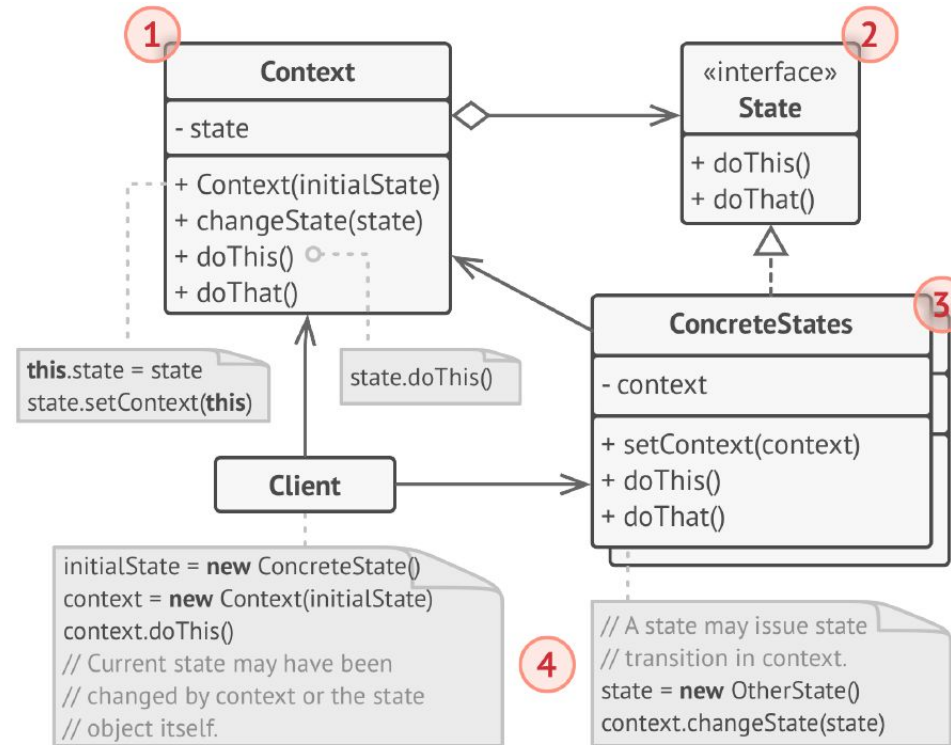
State objects may store a backreference to the context object. Through this reference, the state can fetch any required info from the context object, as well as initiate state transitions.



The State Pattern Defined

1. **Context** stores a reference to one of the concrete state objects and delegates to it all state-specific work. The context communicates with the state object via the state interface. The context exposes a setter for passing it a new state object.
2. The **State** interface declares the state-specific methods. These methods should make sense for all concrete states because you don't want some of your states to have useless methods that will never be called.
3. **Concrete States** provide their own implementations for the state-specific methods. To avoid duplication of similar code across multiple states, you may provide intermediate abstract classes that encapsulate some common behavior.

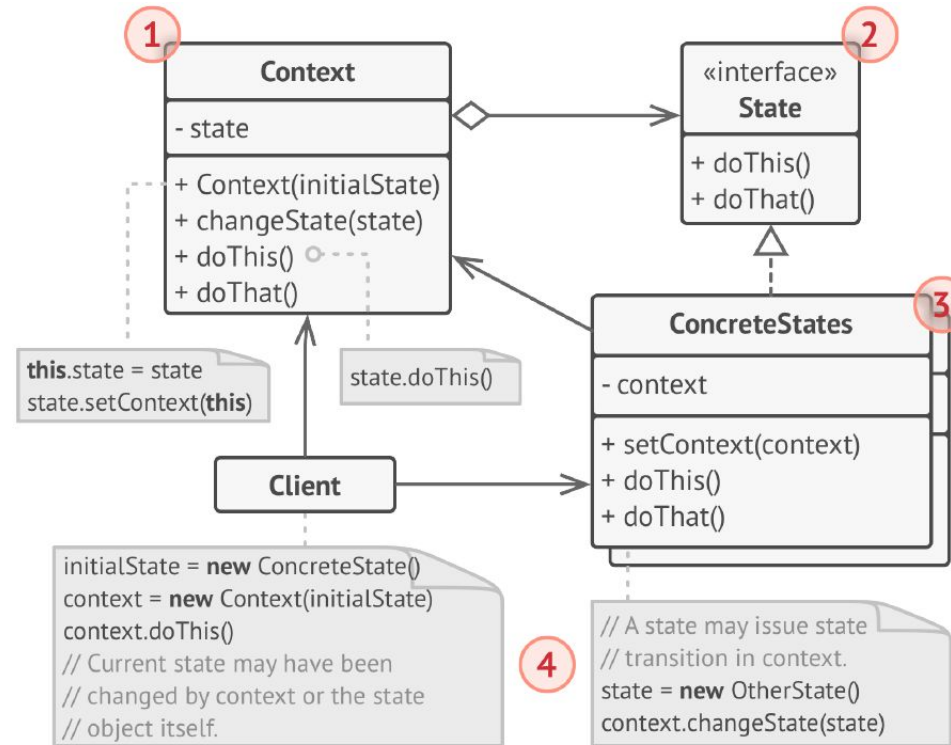
State objects may store a backreference to the context object. Through this reference, the state can fetch any required info from the context object, as well as initiate state transitions.



The State Pattern Defined

1. **Context** stores a reference to one of the concrete state objects and delegates to it all state-specific work. The context communicates with the state object via the state interface. The context exposes a setter for passing it a new state object.
2. The **State** interface declares the state-specific methods. These methods should make sense for all concrete states because you don't want some of your states to have useless methods that will never be called.
3. **Concrete States** provide their own implementations for the state-specific methods. To avoid duplication of similar code across multiple states, you may provide intermediate abstract classes that encapsulate some common behavior.

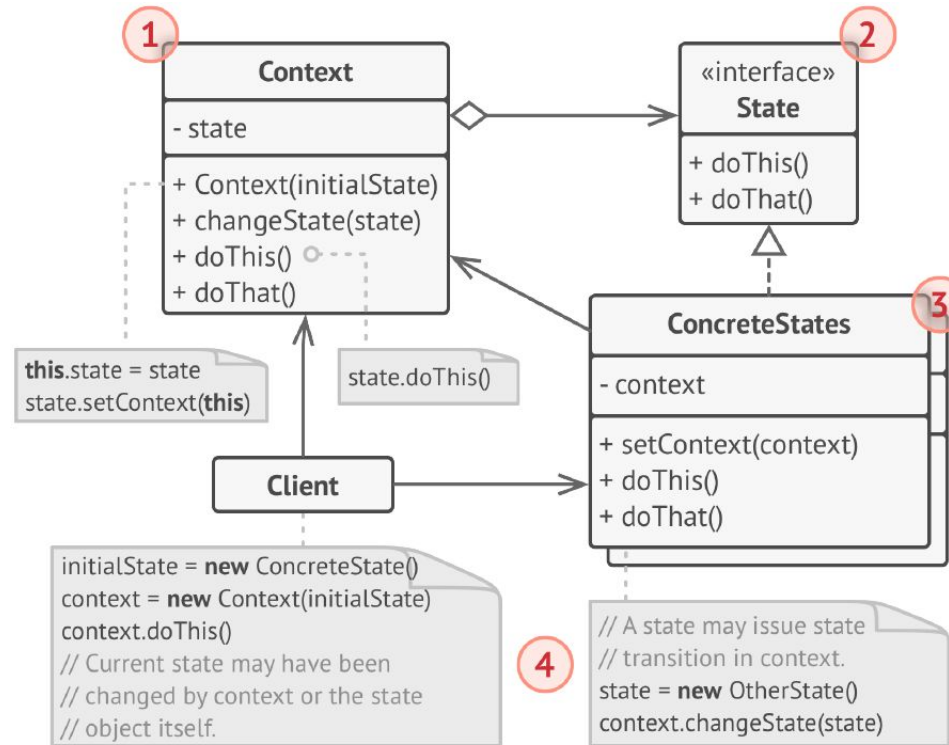
State objects may store a backreference to the context object. Through this reference, the state can fetch any required info from the context object, as well as initiate state transitions.



The State Pattern Defined

1. **Context** stores a reference to one of the concrete state objects and delegates to it all state-specific work. The context communicates with the state object via the state interface. The context exposes a setter for passing it a new state object.
2. The **State** interface declares the state-specific methods. These methods should make sense for all concrete states because you don't want some of your states to have useless methods that will never be called.
3. **Concrete States** provide their own implementations for the state-specific methods. To avoid duplication of similar code across multiple states, you may provide intermediate abstract classes that encapsulate some common behavior.

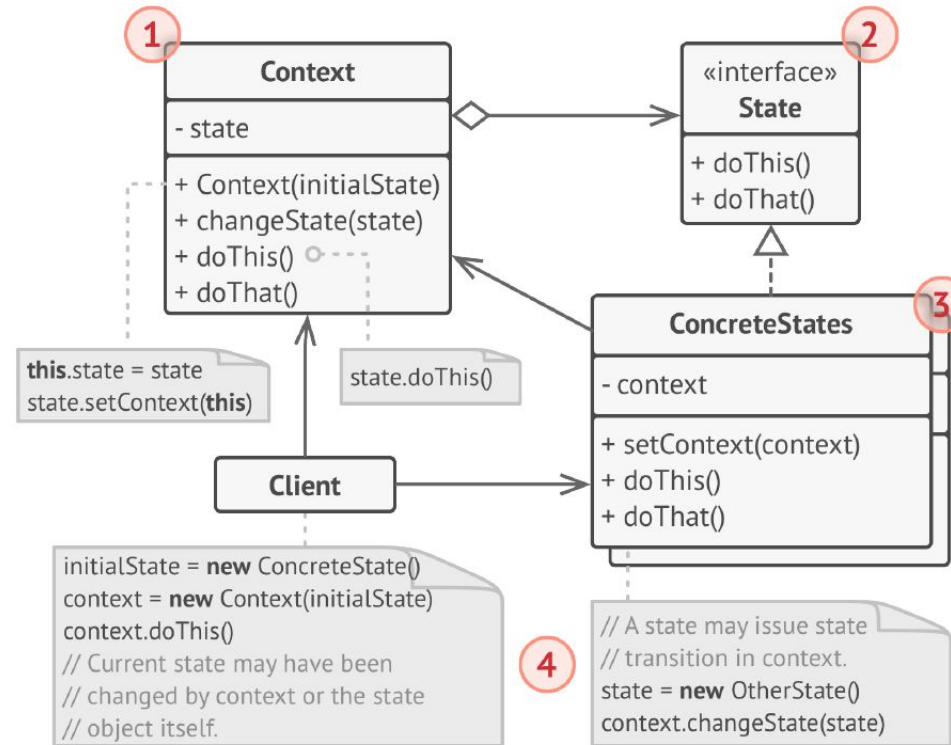
State objects may store a backreference to the context object. Through this reference, the state can fetch any required info from the context object, as well as initiate state transitions.



The State Pattern Defined

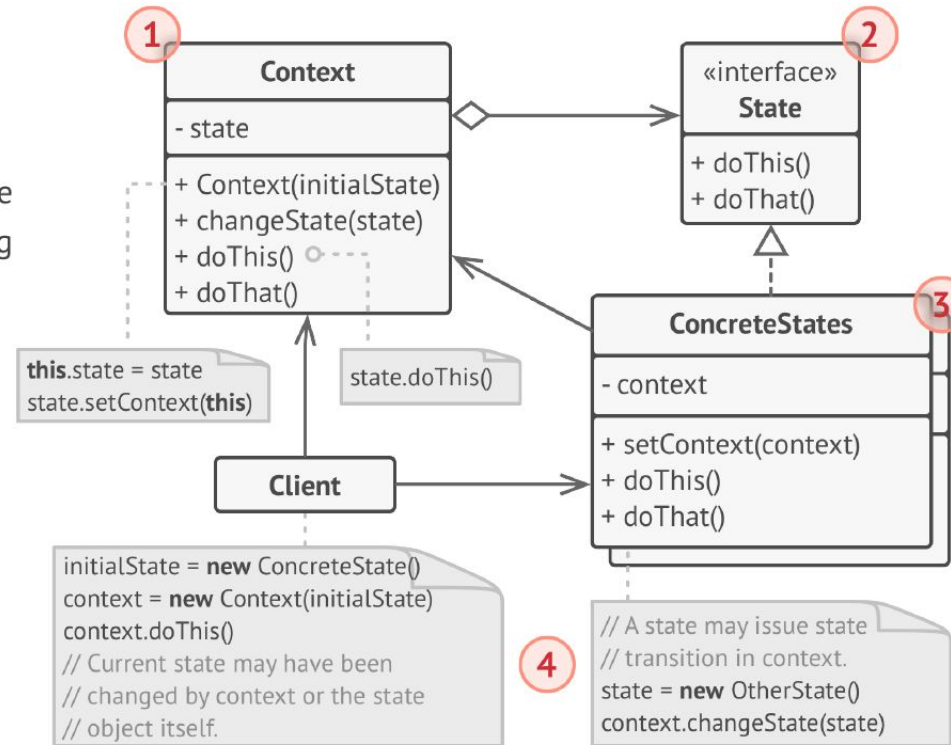
1. **Context** stores a reference to one of the concrete state objects and delegates to it all state-specific work. The context communicates with the state object via the state interface. The context exposes a setter for passing it a new state object.
2. The **State** interface declares the state-specific methods. These methods should make sense for all concrete states because you don't want some of your states to have useless methods that will never be called.
3. **Concrete States** provide their own implementations for the state-specific methods. To avoid duplication of similar code across multiple states, you may provide intermediate abstract classes that encapsulate some common behavior.

State objects may store a backreference to the context object. Through this reference, the state can fetch any required info from the context object, as well as initiate state transitions.



The State Pattern Defined

4. Both context and concrete states can set the next state of the context and perform the actual state transition by replacing the state object linked to the context.



There are no DUMB Questions

Q: In the GumballMachine, the states decide what the next state should be. Do the ConcreteStates always decide what state to go to next?

A: No, not always. The alternative is to let the Context decide on the flow of state transitions.

As a general guideline, when the state transitions are fixed they are appropriate for putting in the Context; however, when the transitions are more dynamic, they are typically placed in the state classes themselves (for instance, in the GumballMachine the choice of the transition to NoQuarter or SoldOut depended on the runtime count of gumballs).

The disadvantage of having state transitions in the state classes is that we create dependencies between the state classes. In our implementation of the GumballMachine we tried to minimize this by using getter methods on the Context, rather than hardcoding explicit concrete state classes.

Notice that by making this decision, you are making a decision as to which classes are closed for modification – the Context or the state classes – as the system evolves.

Q: Do clients ever interact directly with the states?

A: No. The states are used by the Context to represent its internal state and behavior, so all requests to the states come from the Context. Clients don't directly change the state of the Context. It is the Context's job to oversee its state, and you don't usually want a client changing the state of a Context without that Context's knowledge.

Q: If I have lots of instances of the Context in my application, is it possible to share the state objects across them?

A: Yes, absolutely, and in fact this is a very common scenario. The only requirement is that your state objects do not keep their own internal state; otherwise, you'd need a unique instance per context.

To share your states, you'll typically assign each state to a static instance variable. If your state needs to make use of methods or instance variables in your Context, you'll also have to give it a reference to the Context in each handler() method.

Q: It seems like using the State Pattern always increases the number of classes in our designs. Look how many more classes our GumballMachine had than the original design!

A: You're right, by encapsulating state behavior into separate state classes, you'll always end up with more classes in your design. That's often the price you pay for flexibility. Unless your code is some "one off" implementation you're going to throw away (yeah, right), consider building it with the additional classes and you'll probably thank yourself down the road. Note that often what is important is the number of classes that you expose to your clients, and there are ways to hide these extra classes from your clients (say, by declaring them package visible).

Also, consider the alternative: if you have an application that has a lot of state and you decide not to use separate objects, you'll instead end up with very large, monolithic conditional statements. This makes your code hard to maintain and understand. By using objects, you make states explicit and reduce the effort needed to understand and maintain your code.

There are no DUMB Questions

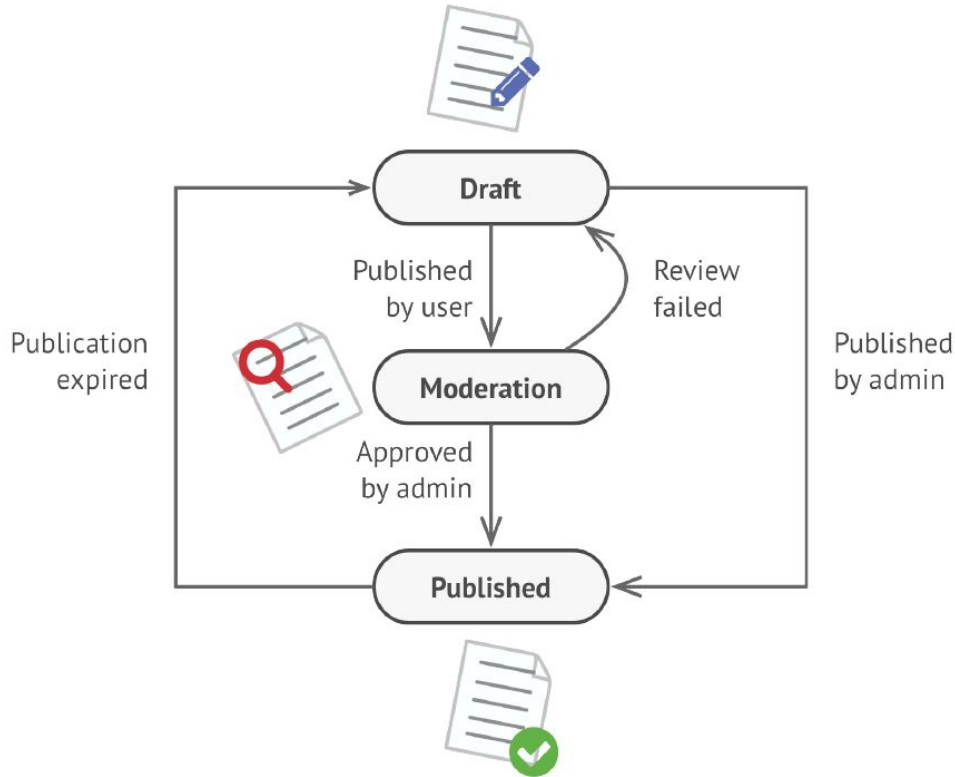
Q: The State Pattern class diagram shows that **State** is an abstract class. But didn't you use an interface in the implementation of the gumball machine's state?

A: Yes. Given we had no common functionality to put into an abstract class, we went with an interface. In your own implementation, you might want to consider an abstract class. Doing so has the benefit of allowing you to add methods to the abstract class later, without breaking the concrete state implementations.

Q: Why do we need the **WinnerState**? Couldn't we just have the **SoldState** dispense two gumballs?

A: That's a great question. **SoldState** and **WinnerState** are almost identical, except that **WinnerState** dispenses two gumballs instead of one. You certainly could put the code to dispense two gumballs into the **SoldState**. The downside is, of course, that now you've got TWO states represented in one **State** class: the state in which you're a winner, and the state in which you're not. So you are sacrificing clarity in your **State** class to reduce code duplication. Another thing to consider is the principle you learned in the previous chapter: One class, One responsibility. By putting the **WinnerState** responsibility into the **SoldState**, you've just given the **SoldState** TWO responsibilities. What happens when the promotion ends? Or the stakes of the contest change? So, it's a tradeoff and comes down to a design decision.

Yet another example

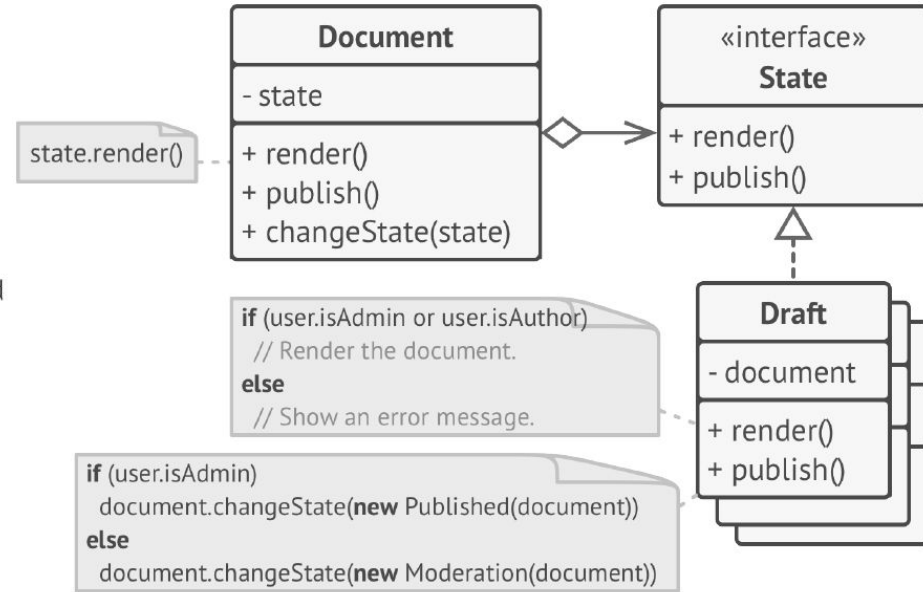
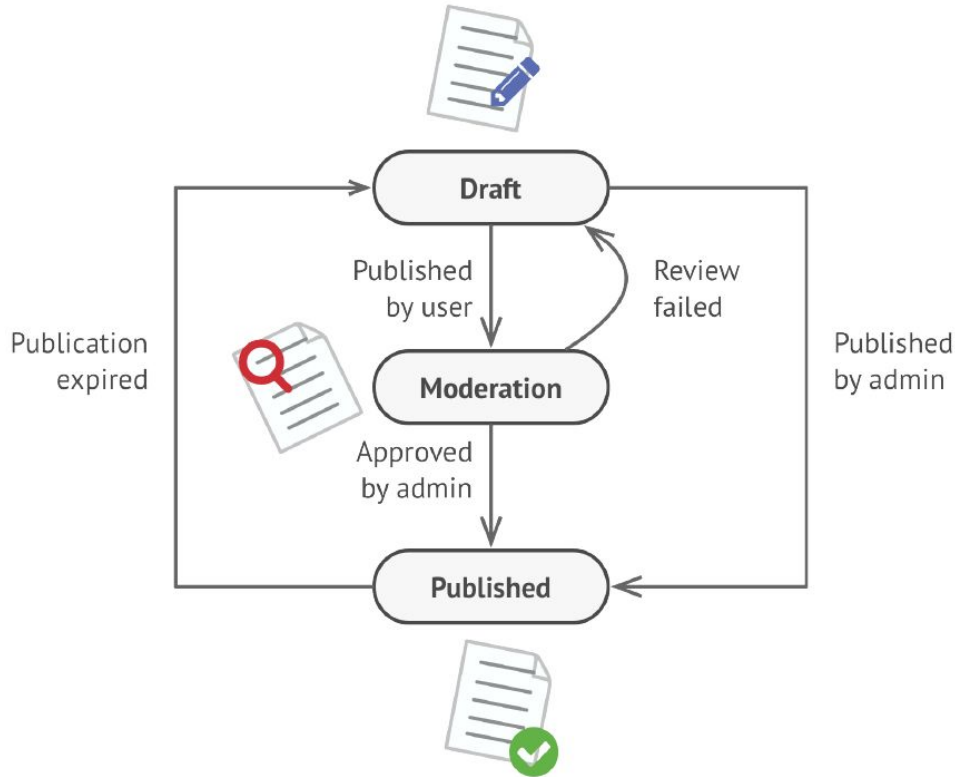


The **publish** method of the document works a little bit differently in each state

- In **Draft**, it moves the document to moderation.
- In **Moderation**, it makes the document public, but only if the current user is an administrator.
- In **Published**, it doesn't do anything at all.

Possible states and transitions of a document object.

Yet another example



Document delegates the work to a state object.

Possible states and transitions of a document object.

Difference between State and Strategy

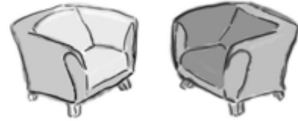
- They are twins separated at birth
- Both based on composition
- Change the behavior of the context by delegating some work to helper objects.

Difference between State and Strategy

- They are twins separated at birth
- Both based on composition
- Change the behavior of the context by delegating some work to helper objects.
- Strategy makes these objects **completely independent and unaware** of each other.

Difference between State and Strategy

- They are twins separated at birth
- Both based on composition
- Change the behavior of the context by delegating some work to helper objects.
- Strategy makes these objects **completely independent and unaware** of each other.
- State *doesn't restrict dependencies between concrete states*, letting them alter the state of the context at will.



Strategy

Hey bro. Did you hear I was in Chapter 1?

I was just over giving the Template Method guys a hand – they needed me to help them finish off their chapter. So, anyway, what is my noble brother up to?

I don't know, you always sound like you've just copied what I do and you're using different words to describe it. Think about it: I allow objects to incorporate different behaviors or algorithms through composition and delegation. You're just copying me.

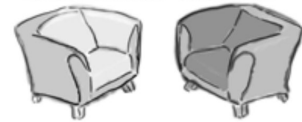
Oh yeah? How so? I don't get it.

State

Yeah, word is definitely getting around.

Same as always – helping classes to exhibit different behaviors in different states.

I admit that what we do is definitely related, but my intent is totally different than yours. And, the way I teach my clients to use composition and delegation is totally different.



Strategy

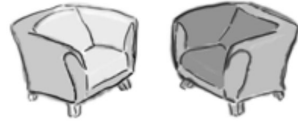
Yeah, that was some *fine* work... and I'm sure you can see how that's more powerful than inheriting your behavior, right?

Sorry, you're going to have to explain that.

State

Well if you spent a little more time thinking about something other than *yourself*, you might. Anyway, think about how you work: you have a class you're instantiating and you usually give it a strategy object that implements some behavior. Like, in Chapter 1 you were handing out quack behaviors, right? Real ducks got a real quack, rubber ducks got a quack that squeaked.

Yes, of course. Now, think about how I work; it's totally different.



Strategy

Hey, come on, I can change behavior at runtime too; that's what composition is all about!

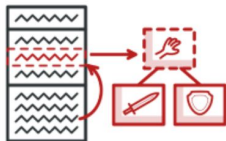
Well, I admit, I don't encourage my objects to have a well-defined set of transitions between states. In fact, I typically like to control what strategy my objects are using.

State

Okay, when my Context objects get created, I may tell them the state to start in, but then they change their own state over time.

Sure you can, but the way I work is built around discrete states; my Context objects change state over time according to some well defined state transitions. In other words, changing behavior is built in to my scheme – it's how I work!

Look, we've already said we're alike in structure, but what we do is quite different in intent. Face it, the world has uses for both of us.



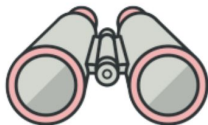
Strategy

Lets you define a family of algorithms, put each of them into a separate class, and make their objects interchangeable.



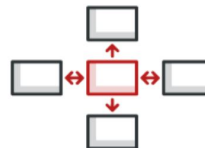
Template Method

Defines the skeleton of an algorithm in the superclass but lets subclasses override specific steps of the algorithm without changing its structure.



Observer

Lets you define a subscription mechanism to notify multiple objects about any events that happen to the object they're observing.



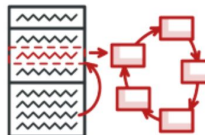
Mediator

Lets you reduce chaotic dependencies between objects. The pattern restricts direct communications between the objects and forces them to collaborate only via a mediator object.



Command

Turns a request into a stand-alone object that contains all information about the request. This transformation lets you parameterize methods with different requests, delay or queue a request's execution, and support undoable operations.



State

Lets an object alter its behavior when its internal state changes. It appears as if the object changed its class.

Acknowledgement

- ❑ Dive Into Design Patterns - Alexander Shvets
- ❑ Head First Design Patterns, 2nd Edition - Eric Freeman, Elisabeth Robson